



GALALA UNIVERSITY

Powered by
Arizona State University®

Faculty of Computer Science and Engineering

Personalized AI Talent Hiring System



Submitted By

Abdelrahman Osheba	222100946
Ahmed Abouelela	222100032
Ahmed Abdelghany	222100123
Osama Khalil	222100211
Youssef Abousrewa	221101030

Supervised By

Associate Professor. Shaker El-Sappagh

Assistant Professor. Amr Hefny

2025 - 2026

Graduation Project

Personalized AI Talent Hiring System

A Project submitted to the Faculty of Computer Science and Engineering, Galala University, in partial fulfillment of the requirements for the Bachelor Degree in **Artificial Intelligence Science and Artificial Intelligence Engineering**

Submitted By

Abdelrahman Osheba	222100946
Ahmed Abouelela	222100032
Ahmed Abdelghany	222100123
Osama Khalil	222100211
Youssef Abousrewa	221101030

Project Supervisor

Supervisor	Associate Professor. Shaker El-Sappagh
Co-Supervisor	Assistant Professor. Amr Hefny

Approved by the discussion committee:

		Signature
Committee Chair Faculty of Computer Science and Engineering Galala University.	Chair	
Committee Member 1 Faculty of Computer Science and Engineering Galala University.	Member	
Committee Member 2 Faculty of Computer Science and Engineering Galala University.	Member	

Cairo

June - 2026

Graduation Project Declaration Form

Project Title: Personalized AI Talent Hiring System

Academic Year: 2025/2026 **Program:** Artificial Intelligence Science and Artificial Intelligence Engineering

Supervisor: Associate Professor. Shaker El-Sappagh, Assistant Professor. Amr Hefny

Student Declaration

We, the undersigned students, hereby declare that:

1. This graduation project report and all submitted deliverables are our own original work, except where proper acknowledgment and references have been provided.
2. We have complied with the university regulations regarding academic integrity, plagiarism, copyright, and ethical conduct.
3. All external sources, including books, articles, websites, software libraries, datasets, and AI-assisted tools, have been appropriately cited and referenced.
4. We understand that plagiarism, fabrication, falsification of data, or unauthorized copying of work may result in disciplinary action according to university regulations.
5. Any AI tools used during the project served only as supporting tools. We remain fully responsible for the accuracy, originality, implementation, results, and conclusions presented in this project.
6. We certify that the submitted software, designs, models, datasets, experiments, results, and documentation accurately reflect the work performed by our team.
7. We acknowledge that the university may use similarity-checking and plagiarism-detection tools to verify the originality of our submission.

Intellectual Property and Ownership Declaration

We further acknowledge and agree that:

1. Intellectual property developed as part of this project shall be governed by applicable university policies, agreements, and intellectual property laws.
2. Students shall be recognized as the creators of their original contributions to the project.
3. Supervisors shall receive appropriate recognition for substantial academic or intellectual contributions, including authorship or inventorship where applicable.
4. Galala University, through supervisors, retains the right to archive, evaluate, and use the project and its deliverables for academic, research, accreditation, and educational purposes, with proper attribution to the authors.
5. Any publication, patent application, commercialization, or public dissemination arising from this project shall appropriately recognize the contributions of students, supervisors, the university, and any external collaborators in accordance with applicable regulations.
6. No team member may represent the project team, make commitments, disclose project-related information, submit project materials, or enter into agreements with any external party on behalf of the team without the prior written approval of the team members and the project supervisor.

Cairo

June - 2026

Individual Contribution Statement

Each team member declares that the percentage shown below reasonably reflects their contribution to the project.

Student Name	Student ID	Contribution (%)	Signature
Abdelrahman Osheba	222100946	20 %	
Ahmed Abouelela	222100032	20 %	
Ahmed Abdelghany	222100123	20 %	
Osama Khalil	222100211	20 %	
Youssef Abousrewa	221101030	20 %	

AI Usage Disclosure (If Applicable)

Please indicate any AI-assisted tools used during the project:

Tool name	Purpose of use
ChatGPT	Language improvement and proofreading
Claude	Code debugging assistance
Gemini	Literature search support

Examples of acceptable use may include language improvement and proofreading, code debugging assistance, literature search support, documentation formatting.

Students remain fully responsible for verifying the accuracy, originality, and validity of all generated content.

Supervisor Verification

I have reviewed the submitted graduation project and confirm that, to the best of my knowledge, it has been conducted under my supervision and satisfies the academic requirements of the program.

Supervisor Name:

Signature:

Date: / /

Cairo

June - 2026

Student Signatures

By signing below, we confirm that we have read and understood this declaration and agree to its contents.

Student Name	Student ID	Signature
Abdelrahman Osheba	222100946	
Ahmed Abouelela	222100032	
Ahmed Abdelghany	222100123	
Osama Khalil	222100211	
Youssef Abousrewa	221101030	

Date: / /

For Committee Use Only

- Similarity Index: _____ %
- Declaration Accepted: ☐ Yes ☐ No
- Comments:

.....

.....

.....

.....

.....

Committee Chair Signature: _____

Date: / /

Cairo
June - 2026

Abstract

The PATHS (Personalized AI Talent Hiring System) has been designed to utilize Artificial Intelligence (AI) technology to provide a standardized hiring platform that resolves the gaps currently occurring within most hiring workflows - the disjointedness of the tools utilized, gaps in the passive talent market, ineffectiveness of our outreach efforts, failure to consistently evaluate candidates, and lack of alignment between the decisions made by hiring managers and the growth and development of employees. PATHS has combined all of these phases into a single end-to-end solution.

The underlying architecture of PATHS is a modular, agent-based system that assimilates several forms of data related to a candidate (CV, LinkedIn profile, GitHub account, portfolio, and ATS export) to produce a master profile and create a standardized form of evidence that describes the skills and experience of the candidates. The PATHS system anonymizes candidates before the evaluation process to minimize bias, then generates a robust Top-K shortlist of candidates with rationale and confidence ratings. Additionally, for technical positions, the PATHS system performs an assessment stage prior to scheduling a technical interview and provides pass/fail rationale for each assessment. The PATHS team has included a checkpoint for human approval to avoid the risk of the system automatically rejecting candidates. Using Rigorous Agility Governance (RAG), the HR Agents and Technical Assessors generate structured rubric-aligned interview questions that have been configured to match the organizations' knowledge base, while retaining the ability to generate summaries and transcripts of the interview process for future audits.

Finally, PATHS converts the results of its assessments into decisions: It gives hiring supervisors a package to send with their hiring decision and can produce an Individual Development Plan (IDP) based on the identified development needs for all applicants, including those who are not hired. All applicants have the opportunity for meaningful developmental feedback and to be considered for future employment opportunities. The system includes metrics to evaluate success factors, such as reduced time to hire, decreased cost per hire, improved response rates for outreach, unbiased evaluation of candidates and improved retention of new hires within the first several weeks of employment all while providing hiring authorities with the same level of transparency and compliance that they would expect from traditional methods, as well as retaining the same level of personal interaction and human element that drives successful recruitment.

Date: 23 June 2026

Acknowledgements

In the name of Allah, the Most Gracious, the Most Merciful. All praise is due to Allah Almighty, who granted us the strength, patience, and guidance to complete this work. Without His blessings, this research and its successful completion would not have been possible.

We would like to express our deepest gratitude and sincere appreciation to our advisors, Associate Professor Shaker El-Sappagh and Assistant Professor Amr Hefny, Faculty of Computer Science and Engineering. Their expertise, continuous support, and constructive feedback shaped this project at every stage from the earliest problem definition to the final refinement of our results. We are truly thankful for their time, mentorship, and encouragement, and for the high standards they inspired us to meet. Their guidance was not only academic, but also a source of motivation and confidence whenever we faced difficulties.

We also extend our heartfelt thanks to our instructors and colleagues for the discussions, suggestions, and academic environment that enriched our thinking and strengthened this research.

Finally, we owe our greatest appreciation to our families. Their endless patience, prayers, and emotional support carried us through the demanding moments of this journey. Their observing belief in us gave meaning to every milestone and provided the determination to continue. This work is dedicated to everyone who stood beside us, supported our ambitions, and contributed directly or indirectly to making this achievement possible.

Table of Contents

Graduation Project Declaration Form	3
Abstract.....	i
Acknowledgements	ii
Table of Contents.....	iii
List of Figures.....	vi
List of Tables	x
1 Chapter 1 Introduction	xii
1.1 Overview.....	13
1.2 Problem Statement.....	15
1.3 Motivation.....	17
1.4 Work Plan	17
2 Chapter 2 State-of-The-Art.....	20
2.1 Background.....	21
2.2 Related Work	24
2.2.1 Production and Market Benchmark	24
2.2.2 Literature Review.....	43
3 Chapter 3 Proposed solution analysis and design	49
3.1 Solution Methodology	50
3.2 System analysis.....	52
3.2.1 Functional/ Non-functional Requirements.....	110
3.2.2 Functional/ Non-functional Requirements.....	111
4 Chapter 4 Implementation	117
4.1 Implementation Overview	118
4.2 Tools and Libraries	120
4.2.1 Front-end (Web Application)	120
4.2.2 Backend Server Stack	124
4.2.3 Data and Model-Inference Services.....	129
4.2.4 Host Requirements.....	131
4.3 Environment and Platform.....	134
4.3.1 Container-First Runtime Topology	134

4.3.2 Configuration and Environment Variables.....	136
4.3.3 System Inputs and Outputs	136
4.4 Backend Implementation and Module Organization	137
4.5 Agentic AI Implementation	141
4.6 Knowledge Graph Synchronization.....	143
4.7 Vector Database Collections and Retrieval Use Cases	143
4.8 Recommendation System Implementation	143
4.8.1 Score Combination Formula	145
4.8.2 Embeddings and Content-Based Similarity	145
4.8.3 Graph-Based Matching Signals	146
4.8.4 Robustness and Fairness Controls	146
4.9 Model Context Protocol (MCP).....	147
4.9.1 LinkedIn MCP Sourcing Provider	149
4.9.2 Skill-Evidence MCP-Style Tools	151
4.9.3 Google Calendar and Gmail MCP Bridges.....	152
4.10 Security, Privacy, RBAC, and Auditability.....	153
4.11 System Testing	154
4.11.1 Testing Strategy	154
4.12 Deployment and Operational Considerations	156
4.13 Implementation Challenges	159
4.13.1 LLM Latency	160
4.13.2 Matching Threshold Tuning.....	160
4.13.3 Data Quality from CVs	161
4.13.4 Duplicate Candidates	162
4.13.5 Cross-Store Data Drift and ID Mismatch	162
4.13.6 Apache AGE and Cypher Query Handling	162
4.13.7 Authentication and Role Permissions	163
4.13.8 Tenant Isolation in Vector Search	163
4.13.9 Bias and Fairness	164
4.13.10 Deployment Issues	164
4.13.11 Google Calendar and Gmail OAuth Integration	165
4.13.12 Next.js 16 Frontend Breaking Changes	165
4.13.13 External Job Scraping	166
5 Chapter 5 Results	167

5.1	Chapter Overview	168
5.2	Evaluation Metric	168
5.2.1	Process Coverage	169
5.2.2	Functional Success Rate	178
5.2.3	Matching Accuracy Band-Hit	182
5.2.4	Matching Accuracy Relevance-Consistent	184
5.2.5	CV/Profile Extraction Accuracy	187
5.2.6	RAG Relevance	189
5.2.7	Outreach Quality	201
5.3	Testing Scenarios and Case Studies	203
5.3.1	Primary End-to-End Recruitment Scenario	203
5.3.2	Alternative Workflow Scenarios	204
5.3.3	Edge Cases and Failure Handling	204
6	Chapter 6 Discussion	205
6.1	Chapter Overview	206
6.2	Interpretation of the Evaluation Results	206
6.2.1	End-to-End Workflow Coverage and Functional Success	207
6.2.2	Candidate-Role Matching and Ranking Quality	207
6.2.3	Profile Extraction, RAG Grounding, and Generated Artifacts	208
6.3	Significance of the Findings	209
6.3.1	Alignment with the Project Objectives and Expected Outcomes	210
6.4	Functional Positioning Relative to Existing Recruitment Systems	211
6.5	PATHS as a Digital-Employee Strategy with Human Oversight	212
6.6	Maintaining Current Knowledge, Adoption, and Future Scalability	212
6.6.1	Maintaining Current Recruitment Knowledge	212
6.6.2	User Experience and Adoption	213
6.6.3	Scalability, Reliability, and Architectural Evolution	213
6.7	Limitations and Practical Constraints	214
7	Chapter 7 Conclusion	216
7.1	Summary and Conclusion	217
7.2	Future Work	218
7.2.1	Lessons Learned from the Implementation of PATHS	218
7.2.2	Future Development Roadmap	219
	References	221

List of Figures

Figure 2.1: Regular Recruitment process steps	21
Figure 2.2: High-Level Architecture of PATHS System	22
Figure 2.3: Illustration of Paradox’s “Olivia” conversational AI interface used for candidate screening, Q&A, and automated interview scheduling [23].	25
Figure 2.4: Illustration of Beamery’s talent pipeline / talent graph view for engaging passive candidates, ranking matches, and managing talent pools [24].	26
Figure 2.5: Demonstration of Elevatus’s recruiting dashboard for resume parsing, shortlisting, and tracking candidate progress across hiring stages [25].	27
Figure 2.6: HireVue’s structured video interview workflow and recruiter evaluation dashboard with response insights and scoring outputs [26].	29
Figure 2.7: Eightfold’s AI-driven candidate matching (skill inference and role-fit ranking) and/or internal talent marketplace recommendations [27].	30
Figure 2.8: SeekOut’s advanced talent search filters (skills, location, diversity) and/or Talent Insights dashboard for labor market analytics [28].	31
Figure 2.9: Pymetrics’ game-based assessment [29].	33
Figure 2.10: HackerRank’s coding used to test interface or evaluation report used to assess programming proficiency at scale [22].	34
Figure 2.11: Codility’s analytics view for technical hiring, including CodeCheck completion metrics, candidate funnel progression, and performance scoring summaries used to evaluate coding assessments at scale [30].	36
Figure 2.12: Greenhouse’s candidate matching (preferred/required keywords) and structured scorecard workflow, including an anonymized candidate view and agreement/disagreement summaries to standardize hiring decisions [17].	37
Figure 2.13: Ashby’s reporting dashboard with key hiring KPIs (hires, open roles, monthly trends), stage-level passthrough rates, and time-in-process breakdown by team to support data-driven recruiting optimization [31].	39
Figure 2.14: SmartRecruiters’ main dashboard with task lists, at-a-glance hiring activity (active jobs and candidate counts), approvals, and upcoming interviews which is supporting collaborative and workflow-driven recruitment operations [5].	41
Figure 3.1 System Context Diagram	51

Figure 3.2: Knowledge Graph for Recruitment Data Modeling.	52
Figure 3.3: Modular Multi-Agent Recruitment Architecture and Agents Communication.	53
Figure 3.4: Entity-Relationship Diagram (ERD) for PATHS core hiring workflow which linking organizations, jobs, candidates, skills, applications, assessments, and post-selection growth plans through normalized many-to-many mappings and audit-friendly records.	55
Figure 3.5: PATHS Stakeholder Use-Case and Role Responsibilities (RBAC View).	56
Figure 3.6: Candidate Shortlisting Orchestration with Bias Guardrails and Approval.	57
Figure 3.7: Layered modular architecture of PATHS showing the separation between business domain modules, shared infrastructure services, REST API controllers, AI agents, data repositories, and shared database models.	58
Figure 3.8: HR-side recruitment workflow in PATHS showing how recruiters manage dashboards, jobs, candidate sourcing, anonymized evaluation, interview preparation, assessments, and final hiring decisions with automated feedback and development plans.	61
Figure 3.9: HR-side recruitment workflow in PATHS showing how recruiters manage dashboards, jobs, candidate sourcing, anonymized evaluation, interview preparation, assessments, and final hiring decisions with automated feedback and development plans.	64
Figure 3.10: Human-in-the-loop approval and final hiring decision workflow, showing how HR actions, approval queues, de-anonymization requests, and decision-support outputs are controlled before candidate decision	67
Figure 3.11: Organization administration workflow in PATHS showing how the admin manages HR users, hiring managers, member invitations, role assignment, email communication, and authentication settings.	69
Figure 3.12: CV and document ingestion pipeline in PATHS showing how uploaded candidate documents are stored, loaded, extracted, normalized, and indexed across PostgreSQL, Apache AGE graph database, and Qdrant vector database.	70
Figure 3.13: External job sourcing and ingestion workflow in PATHS showing how jobs are collected from job portals, LinkedIn-style sources, and RemoteOK RSS, then scraped, normalized, deduplicated, stored, and indexed in PostgreSQL, Apache AGE, and Qdrant.	73
Figure 3.14: Candidate-job scoring pipeline in PATHS showing anonymization, relevance filtering, vector similarity, ReAct reasoning, weighted evaluation criteria, and the final explainable candidate-job score.	75
Figure 3.15: Candidate sourcing and shortlisting workflow in PATHS showing how HR imports candidates from multiple sources, creates candidate accounts, normalizes and deduplicates	

records, then applies filtering, scoring, and explanation before adding candidates to the sourced-candidates list.	77
Figure 3.16: Interview analysis and decision-support workflow in PATHS showing how interview transcripts, HR notes, job requirements, candidate data, and agentic analysis are assembled into HR and technical evaluations, weighted, and converted into a final decision-support packet.	79
Figure 3.17: RAG-based interview preparation workflow in PATHS showing how job data, candidate data, company knowledge, semantic retrieval, web search, GitHub evidence, and agentic reasoning are combined to generate pre-analysis, technical preparation, and HR/behavioral preparation outputs.	82
Figure 3.18: Assessment generation and grading workflow in PATHS showing how the assessment agent uses job requirements, HR instructions, and the knowledge base to create different assessment types, publish them after recruiter approval, receive candidate submissions, and generate rubric-based grading, gaps, and improvement tips.....	85
Figure 3.19: Candidate skill evidence workflow in PATHS showing how candidate skills are verified using CV evidence and GitHub evidence, then converted into a weighted proficiency score stored in the candidate skills table.	87
Figure 3.20: Candidate-job comparison and development recommendation workflow in PATHS showing how the system compares a candidate profile with a posted job description, identifies missing and matching skills, evaluates weak signals and alignment, calculates a fit.....	89
Figure 3.21: Candidate contact enrichment workflow in PATHS showing how interview-process candidates are reviewed for missing contact information, enriched from professional sources such as LinkedIn and GitHub, merged into a unified contact profile, and stored as candidate contact details.....	91
Figure 3.22: Personalized development recommendation workflow in PATHS showing how the candidate's target role, skill gaps, interests, and job requirements are weighted and processed by the Development Agent to generate roadmaps, recommended projects, required skills, best practices, and external learning links.	93
Figure 3.23: LinkedIn-based outbound sourcing workflow in PATHS showing how HR search queries are first checked against the internal candidate database, then expanded through LinkedIn MCP crawling, Open-to-Work filtering, profile enrichment, semantic search, deduplication, and imported candidate profile creation.....	95
Figure 3.24: Context-aware HR assistant workflow in PATHS showing how the floating chat interface detects the current page, sends the user query to the assistant API, retrieves page-	

specific data and conversation history, generates a grounded reply, and stores memory separately for each page context.	97
Figure 3.25: Isolated memory-thread design for the PATHS context-aware assistant, showing how each page context keeps a separate conversation history while storing all assistant messages in one structured database table.	100
Figure 3.26: Context-Aware PATHY Assistant Interaction Sequence for Retrieving Page-Specific Data, Maintaining Conversation History, and Generating Grounded Responses.	103
Figure 3.27: Candidate Scoring Pipeline with Evidence Gating, Bias Guardrails, and Explainable Ranking.	105
Figure 3.28: End-to-End Evaluation Orchestration with Assessment Gating, RAG Interviews, and HITL Decision Support.	108
Figure 3.29: Outreach-to-Scheduling Orchestration with RAG Personalization and Multi-Channel Tracking.	109
Figure 4.1: high-level implementation architecture showing the separation between the Next.js frontend, FastAPI backend, model services, and the polyglot persistence layer consisting of PostgreSQL, Apache AGE, and Qdrant [73], [78], [96], [97].	118
Figure 4.2: Hybrid Candidate-Job Recommendation Pipeline.....	144
Figure 4.3: MCP Streamable-HTTP handshake and tool call.	149
Figure 4.4: Source-candidate provider abstraction.	151
Figure 4.5. Layered testing pyramid combining backend unit/integration tests with full-stack Playwright end-to-end tests across four user personas.	155
Figure 5.1: Final PATHS recommendation scores for labelled candidates, grouped by expected fit band.....	177
Figure 5.2: Band-level prediction agreement of PATHS recommendation scores.....	177
Figure 5.3: Confusion matrix comparing expected and PATHS-predicted candidate fit bands..	178
Figure 5.4: End-to-End Tested PATHS Recruitment Workflow.	203

List of Tables

Table 1.1: Work plan for planning and implementation.....	19
Table 2.1: Competitive Landscape Comparison of PATHS vs. Leading Recruitment Platforms .	42
Table 2.2: Feature Matrix Comparison of PATHS vs. Related Research Systems.....	48
Table 4.1: Frontend technology stack and dependency versions used in the web application....	121
Table 4.2: Backend technology stack used in the web application.	124
Table 4.3: Data and inference services used in the system.	129
Table 4.4: Development Baseline, Current and Recommended Cloud Deployment and Requirements.	131
Table 4.5: Container Port Mapping.	134
Table 4.6: Key configuration variables	136
Table 4.7: Principal input categories	136
Table 4.8: Principal output categories.	136
Table 4.9: Backend modules and their implementation responsibilities.	137
Table 4.10: Agentic and Reasoning Agents, Data Stores, and Human-in-the-Loop Checkpoints.	142
Table 4.11: Qdrant collections and their retrieval use cases	143
Table 4.12: MCP usage across system.	148
Table 4.13: LinkedIn MCP tools used.....	150
Table 4.14: Skill-evidence tools.	151
Table 4.15: Security and governance controls implemented by design.	154
Table 4.16. Testing tools and runtime components used to validate the system.	155
Table 4.17. Observability and reliability mechanisms.	156
Table 4.18. Deployment Responsibilities and Operational Requirements.	158
Table 4.19. Summary of Implementation Challenges and Mitigation Strategies.....	159
Table 5.1: Evaluation metrics used for PATHS.	168
Table 5.2: System Process Coverage Scores Table for each stage.	174
Table 5.3: Candidate recommendation score bands and fit interpretations.....	175
Table 5.4: Candidate-level evaluation of system recommendation scores and predicted fit-band agreement.....	176
Table 5.5: Summary of automated evaluation suites.....	179
Table 5.6: End-to-end results.....	180
Table 5.7: Detailed end-to-end test results and duration.	180

Table 5.8: Minimum Scores Required for the Ranking Classification and Direction.....	185
Table 5.9: CV and profile extraction accuracy by field category.	188
Table 5.10: Example structured profile extracted from the sample CV.	188
Table 5.11: Retrieval Metrics	190
Table 5.12: RAG retrieval evaluation setup.	194
Table 5.13: RAG evaluation test cases.	194
Table 5.14: Per-query RAG retrieval results.	195
Table 5.15: Aggregate RAG retrieval results.	195
Table 5.16: Generation Evaluation metrics.	198
Table 5.17: Per-query RAG generation results.	199
Table 5.18: Rating Methodology.	202
Table 5.19: Final Rating and Qualitative results.	202
Table 5.20: Metric Results	203
Table 5.21. Alternative PATHS workflow scenarios and verified behaviour.	204
Table 5.22. Edge cases and observed system response.	204
Table 7.1: Strategic roadmap for future PATHS development.	220

Chapter 1

Introduction

1.1 Overview

Modern organizations face several persistent recruitment pain points abstracted from mainly eight key challenges in hiring workflows such as Slow and Expensive Hiring Cycles, Poor Access to Passive Talent, Low Outreach Response Rates, Inconsistent and Biased Evaluation and Data Fragmentation and System Inefficiency. That's why the organizations highly invest in a lot of software tools that can enhance their recruitment process, existing recruitment tools are fragmented point solutions that lack an integrated approach across the hiring lifecycle. Organizations must juggle separate systems for each stage like Applicant Tracking Systems (ATS) for applications, sourcing platforms for candidate search, assessment tools for interviews. none of which share data or intelligence. There is no ***unified platform*** connecting the whole process through sourcing, engagement, evaluation then development as one continuous workflow. This siloed landscape forces companies to stitch together 4 - 6 tools with manual handoffs, causing data silos and inefficiency, and ultimately perpetuating the very problems outlined above.

PATHS is proposed as an integrated AI talent acquisition system that addresses these gaps through a cohesive platform and intelligent automation:

- Ingesting fragmented candidate data from multiple sources. résumés/CVs, LinkedIn profiles, GitHub contributions and consolidates them. It begins by gathering both inbound applicant data and outbound-sourced profiles (including passive candidates), preparing a rich, unified dataset for each individual. This stage includes identity resolution to merge duplicate records and ensure each candidate has one cohesive profile.
- Employing agentic AI workflows that orchestrate tasks autonomously while keeping a human in the loop. The engine leverages Retrieval-Augmented Generation (RAG) for personalization meaning it retrieves relevant information about the role and candidate to generate context-aware content and advanced entity resolution algorithms to normalize and link data across sources. Together, these components enable the system to automatically identify passive talent across platforms, craft tailored outreach messages for each candidate, and intelligently process candidate information by matching skills and experiences even when described in different formats. The result is an AI-driven workflow that can execute sourcing, engagement, and analysis tasks at scale, under recruiter supervision.

- Producing actionable outputs at each stage of hiring. It creates unified candidate records with complete, up-to-date profiles for each person *resolving the data fragmentation issue*. It generates personalized outreach messages to candidates, increasing the likelihood of engagement by referencing the candidate's background and the role's context. During evaluation, the system provides structured interview and finally individual development plans or learning path recommendations by transferring interview findings into post-hire development objectives. This end-to-end output (from initial contact scripts to interview reports and development roadmaps) creates a continuous talent data thread from sourcing through hiring into onboarding.
- At every stage, PATHS is designed with recruiter and hiring manager oversight as a fundamental principle *it augments rather than replaces human decision-making*. Recruiters remain in control, they review and approve the AI-generated candidate shortlists, outreach content, interview questions, and hiring decisions. The system's agentic workflows operate under defined parameters, with humans providing guidance, handling exceptions, and making final decisions. This *human-in-the-loop* approach ensures that automation is applied ethically and intelligently: AI handles repetitive tasks and data analysis, while recruiters apply judgment on fit and ultimately build the relationship with the candidate. Such design not only maintains transparency and accountability in hiring decisions but also helps with compliance (meeting fairness, bias audit, and privacy requirements) by allowing human judgement to oversee AI recommendations.

The project aims to achieve measurable improvements in hiring efficiency, effectiveness, and retention. Key objectives and their success metrics include Time-to-Hire Reduction, Cost-per-Hire Reduction, Outreach Effectiveness, First-Year Retention and Recruiter Efficiency.

The launching of PATHS is expected to yield significant strategic and operational benefits for organizations, particularly in Egypt's fast-growing tech talent market. Strategically, by improving the quality and consistency of hiring decisions, PATHS will reduce costly mis-hires which is a critical benefit given that a single bad hire can from 16% up to 213% of annual salary depending on role/skill level [1]. Fewer mis-hires translate to substantial savings and a stronger overall workforce. Operationally, the system's automation and intelligence help enabling companies to scale their teams faster by cutting hiring cycle times in half and reaching the broad passive talent pool that was previously untapped. With faster, more targeted hiring, firms can seize market opportunities without the delays of traditional recruitment. Moreover, we can strengthen retention

of technical talent by linking recruitment to career development. New hires will come into roles with clear growth plans, addressing the prevalent issue of early turnover (currently 41% of new hires in Egypt's tech sector leave within 18 months due to lack of growth opportunities) [2]. By aiming for more than 90% for first 18 months retention through better fit and proactive development alignment, the system helps companies maintain expertise and reduce the disruption and cost of frequent re-hiring. In sum, PATHS transforms recruitment from a reactive, fragmented process into a strategic asset by reducing time and cost per hire, improving hire quality and engagement, and enabling organizations in Egypt to build high-performing teams that stay longer and drive innovation.

1.2 Problem Statement

The process of recruitment today is often executed through multiple disconnected tools such as ATS, sourcing platforms, email tools and assessments plus all of those a manual handoff, which creates fragmented workflows and data silos across the hiring lifecycle [3].

This fragmentation makes hiring slower, more expensive, and less consistent. Key decisions are often made with incomplete information, lots of manual follow-ups, and evaluation methods that aren't standardized across teams. Even though there are many tools that support parts of the recruitment workflow, they usually operate in isolation. They help with specific tasks, but because they aren't connected end-to-end, their overall impact on the hiring process remains limited [5].

There are 5 main symptoms for this problem starting with *Time-to-hire* which is too long as the global median time-to-hire is approximately 38 days or 6 weeks [5]. The out-reach-rates are comparably low with the number of the applicants that deserve the role using a large-scale recruiting email analysis shows overall reply rates around the low-20% range, and first-email reply rates can be single digits [6]. Evaluation in-consistency and can be bias prone as research consistently finds structured interviews are more reliable and valid than unstructured interviews (which are more vulnerable to noise and bias) [7]. Data fragmentation where HR systems are increasingly fragmented due to "best-of-breed" tools across recruitment, performance, payroll, engagement creating interoperability/data synchronization challenges [8]. Retention suffers due to no development link as reported by LinkedIn Learning reports where over 90% of employees today say that they will stay longer at their current companies if they invested in learning and development (L&D) [9] that is a major challenge as cited by Amazon a study summary notes that many younger workers plan to quite due to a lack of skills-building [9].

Discussing the costs and risks can be summarized in Money (direct hiring + replacement costs) Egypt gives companies access to a large, relatively cost-effective talent pool across Cairo,

Alexandria, and emerging tech hubs. The total hiring cost is driven by gross salary plus mandatory employer social insurance contributions and, for some employers, supplemental private benefits. Below are average monthly costs for popular roles in 2026, combining market salaries with typical employer social insurance contributions of roughly 18.75% of gross salary for most private-sector employers, up to the statutory wage ceilings. Figures are estimates for full-time roles in major cities and will vary by seniority, industry, and benefits. USD values assume an exchange rate of roughly EGP 50 = USD 1 and are rounded [10]. Hiring in Egypt carries real, measurable cost-per-hire about EGP 12,000 to 80,000+ depending on the role and level and it also includes mandatory social insurance/compliance costs that increase total employment cost [11]. On top of that, turnover/replacement creates additional loss; Egypt-focused business sources estimate replacement cost around 20–30% of annual salary in many formal firms (and it can be higher for more complex roles) [12].

The Operational impact shows that the delays in hiring directly slow team scaling and delivery estimated time-to-fill around about 38 days in a typical scenario [5], and Egypt trend reporting indicates time-to-hire improves when digital recruitment is adopted (reported as a 14% shortening with digital adoption) [11]. Also a local HR survey results show process strain/inefficiency: 92% of surveyed Egyptian HR professionals were dissatisfied with application quality, and 82% were seeking skills-matching algorithms by signalling, matching and screening inefficiency that slows hiring [13].

One of the major risk's specially in Egypt is between privacy and compliance candidate-data handling is regulated under the Personal Data Protection Law framework. Recent updates note enforceable requirements including: explicit consent for sensitive data processing, breach reporting within 72 hours, and data-subject rights including access, rectification, and erasure all directly relevant when storing CVs, scoring candidates, or using automated screening [14] and [15].

The tools impact on people where there is an operational burden shows up in tooling needs and turnover pressures. In an Egypt HR survey about 200 HR professionals/recruiters, 40% emphasized the importance of interview scheduling tools, which is a strong signal that admin coordination is a persistent pain point [13]. On the retention side, Egypt-based sources report from 20% to 30% annual churn as a market reality in many formal environments, and sector-specific churn can be even higher in practice [16].

The main Gap through recruitment process and recruiting tools they are largely optimize isolated steps, but the overall ecosystem remains fragmented as teams juggle multiple point solutions, causing silos, manual handoffs, and inconsistent end-to-end decision quality [3].

We are trying to answer the question of how can we design a unified AI-driven agentic system that reduces time-to-hire and cost-per-hire, improves outreach response, and supports fair, auditable, privacy-respecting hiring decisions?

1.3 Motivation

The recruitment has become a business-critical function, especially when being in high-skill markets where unfilled roles directly delay delivery and growth. Yet hiring workflows remain somehow slow and fragmented, forcing organizations to spend highly significant effort coordinating multiple tools and manual steps, which reduces decision quality and increases hiring cost. This impacts organizations that struggle to scale teams, recruiters who are spending a lot of time on low-value tasks instead of engaging talent, and candidates who experience inconsistent evaluation and generic outreach that feels transactional-wise rather than human-wise the urgency has increased as hiring becomes more global and competitive, while the majority of qualified candidates are passive and require personalized engagement. At the same time, recent advances in large language models (LLMs), retrieval-augmented generation (RAG), and agentic workflows make it feasible to automate repetitive recruitment tasks while still supporting human oversight and governance.

The main motivation is the need for a unified system that closes the gaps between sourcing, engagement, evaluation, and post-hire development, PATHS is designed to augment recruiters, automating coordination and personalization, while enabling structured, auditable evaluation and converting interview insights into development plans that support long-term retention. Therefore, this work aims to demonstrate that an integrated AI-driven hiring pipeline can improve hiring speed and quality without sacrificing fairness, transparency, or human control.

1.4 Work Plan

This proposed timeline presents a structured and realistic path to deliver the project from early planning to an integrated, deployable system within 28 weeks. It begins with Problem Definition and a Feasibility Study (Weeks 1-2), where the focus is to clearly define the hiring problem being addressed, validate the value of the proposed solution, and confirm that the scope is achievable given the available time, data, and resources. This phase ensures the project starts with a well-scoped direction and measurable objectives.

Next, the project proceeds to System Requirement Analysis (Weeks 3-4), where both functional requirements (e.g., sourcing, screening, assessments, interviews, scoring, reporting, and decision support) and non-functional requirements are documented in a structured form. Special

attention is given to critical qualities such as privacy, fairness and bias control, security, reliability, auditability, and explain-ability, ensuring that the system is not only effective but also responsible and defensible.

After requirements are finalized, the work transitions into the design stage. Data Acquisition and Preprocessing Design (Weeks 5-6) defines the data sources the system will rely on, how data will be collected legally and ethically, and how it will be cleaned, normalized, and transformed into consistent inputs for AI components. Then, System Architecture Design (Weeks 7-8) outlines the system's major modules and how they interact, covering agent orchestration, workflow sequencing, logging, monitoring, and where human approvals (HITL checkpoints) are enforced to prevent unsafe or fully automated decisions.

This is followed by Database Design and Data Modeling (Weeks 9-10), where the "single source of truth" is established for core entities such as candidates, job roles, evaluations, assessments, evidence, decisions, and audit logs. A strong database design at this stage ensures traceability, supports transparency, and prepares the system for scaling and integration.

With the system foundation defined, the plan moves into AI-specific design. AI Framework Design and Model Selection (Weeks 11-12) determines which modeling approaches are most suitable for each component (e.g., resume parsing, matching, ranking, and interview support), how the models will be evaluated, and how quality, robustness, and fairness will be monitored. After that, NLP Pipeline and Recommendation Logic Design (Week 13) formalizes how resumes and profiles will be interpreted, how evidence will be connected to job requirements, and how the system will produce defensible outputs that means recommendations that include evidence-backed explanations, not only raw scores.

Once the decision logic is clear, UI/UX and Interaction Design (Weeks 14-15) ensures the platform is usable for real stakeholders. This phase defines recruiter and hiring-manager workflows, dashboards, approval steps, and human-in-the-loop controls so that the system remains transparent, practical, and aligned with real hiring operations.

The longest and most critical phase is implementation. Technical Implementation of Agents (Weeks 16-24) is where the system is built end-to-end: backend services, database implementation, AI/NLP pipelines, interfaces, and agent workflows. This phase also includes iterative testing, bug fixing, performance tuning, and validating outputs (especially around fairness, reliability, and explain-ability). Finally, System Integration (Weeks 24-28) brings all modules together into a unified workflow, ensuring components communicate correctly, audit logging is consistent, and the full hiring pipeline runs smoothly from ingestion to reporting and decision support. This phase also supports final stabilization and deployment readiness.

Table 1.1: Work plan for planning and implementation

Task/Topic	Duration of Execution
Problem Definition & Feasibility Study	Weeks 1 to 2
System Requirement Analysis (Functional & Non-functional)	Weeks 3 to 4
Data Acquisition and Preprocessing Design	Weeks 5 to 6
System Architecture Design	Weeks 7 to 8
Database Design & Data Modeling	Weeks 9 to 10
AI Framework Design & Model Selection	Weeks 11 to 12
NLP Pipeline and Recommendation Logic Design	Week 13
User Interface (UI/UX) & Interaction Design	Weeks 14 to15
Technical Implementation of Agents	Weeks 16 to24
System Integration	Weeks 24 to 28

Chapter 2

State-of-The-Art

2.1 Background

The modern recruitment practices have undergone significant transformation over the past two decades evolving from traditional manual, paper-based processes which is paperwork-heavy approaches to highly digitized systems (digital platforms and AI-assisted tools). Yet, this evolution has been uneven, leading to a patchwork of systems rather than a cohesive solution. Like early online job boards and Applicant Tracking Systems (ATS) in the 2000s enabled basic electronic resume collection and tracking, but often in isolation. Over time, leading ATS platforms as (GreenHouse, ClickUp and Workable) consolidated functions like resume parsing and interview scheduling into single systems [17][18][19][20]. as shown in Figure [1].

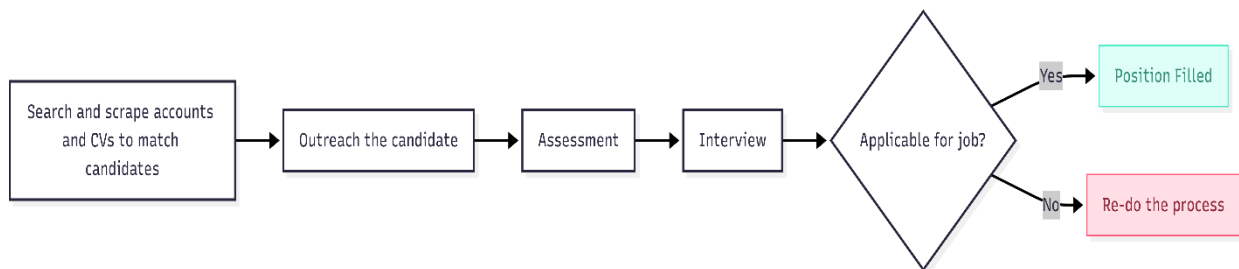


Figure 2.1: Regular Recruitment process steps

Organizations still found themselves stitching together multiple tools were one for sourcing candidates, another for assessments, others for onboarding that resulting in fragmented workflows. Early ATS digitized resume storage and tracking, helping organizations move away from filing cabinets. However, as companies adopted various hiring tools as said before one for sourcing candidates on LinkedIn [21], another for online assessments like HackerRank [22], others for video interviews like HireVue [23].

At the same time, AI in recruitment rose to prominence, promising to fill some of the major gaps of the old recruitment software's. AI chatbots began to handle candidate Q&A, algorithms screened resumes for keywords, and video analysis tried to assess soft skills. These innovations, however, were typically introduced as point solutions addressing a single stage of hiring. The lack of an integrated design meant that gains in one area (say, faster resume screening) could be undermined by bottlenecks or bias in another (like impersonal outreach emails or inconsistent interviews). In essence, the *evolution* of recruitment technology up to 2025 has solved specific old problems but created new ones which is notably a lack of cohesion and personal touch. This recognition sets the context for why a more unified approach is necessary. It's not merely about adding more AI into recruitment; it's about orchestrating the tools and data into a seamless, intelligent system. The limitations of the current evolution as fragmentation which shallows integration of AI, and minimal learning from outcomes as it strongly motivates a re-think that

naturally points toward a system like PATHS (Personalized AI Talent Hiring System), which is conceived to address these very pain points by design as shown in Figure 2.2.

The global recruitment technology market including applicant tracking systems (ATS),

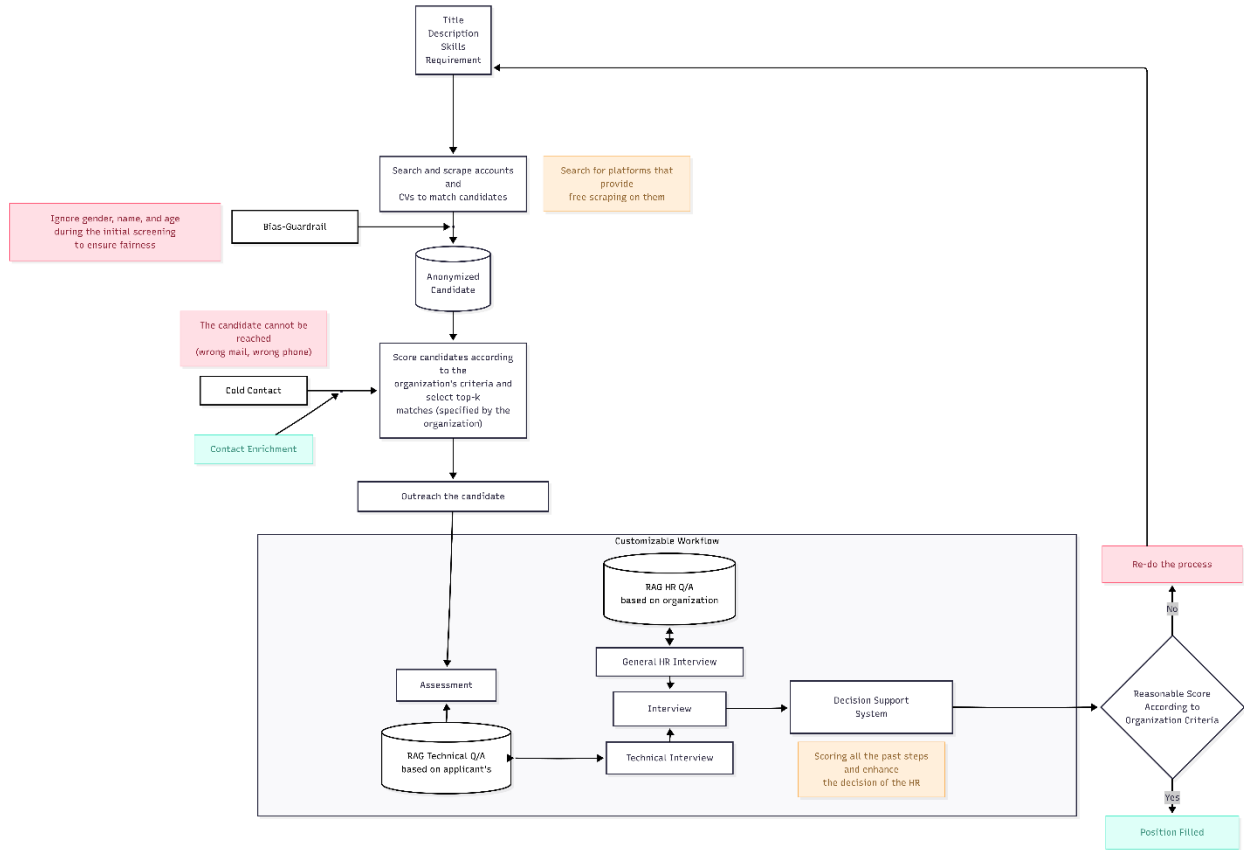


Figure 2.2: High-Level Architecture of PATHS System

recruitment platforms, HR software, and AI-enabled hiring tools is estimated at USD 30–35 billion in annual revenue, with a projected compound annual growth rate (CAGR) of approximately 7% to 9%. Growth is driven by persistent talent shortages, increased adoption of remote and global hiring models, rising cost-per-hire, and enterprise demand for automation, analytics, and skills-based evaluation.

Within this broader market, the AI-driven recruitment segment represents a smaller but rapidly expanding submarket, currently estimated at USD 5–7 billion, growing at a CAGR exceeding 20%. This growth is primarily fuelled by advancements in machine learning, natural language processing, and generative AI, as well as increasing enterprise acceptance of AI-assisted decision-making in hiring [32-35].

For an agentic recruitment system such as PATHS, the Serviceable Available Market (SAM) in Egypt and the Middle East is best defined as the portion of HR software spend that is directly tied to talent acquisition workflows and is actively shifting toward skills-based and automated hiring.

- Tech startups and scale-ups hiring engineers, data, and AI talent (fast cycles, high sourcing volume, heavy screening load).
- Large local enterprises + banks + telecom + FMCG running continuous hiring and needing standardization across business units.
- BPO/SSC and customer operations were high-volume roles where automation and consistent evaluation materially reduce time-to-hire [36].
- Organizations moving from credential-based to skills-based hiring to widen talent pools and reduce mismatch which is especially relevant in MENA labor markets where skill/education mismatch is repeatedly documented [37].
- GCC enterprises with high recruitment throughput (large projects, multi-country hiring, strong HR-tech budgets). The GCC HCM software market alone is estimated around USD 0.90B in 2024 and USD 0.99B in 2025, which signals a large “ready-to-buy” base for advanced TA automation [38].
- Regional and global organizations adopting skills-based hiring approaches to access a wider talent pool [39].
- Companies accelerating AI adoption and skills transformation [40].

In the near to mid-term, PATHS’ Serviceable Obtainable Market (SOM) is focused on early adopters, including tech-forward start-ups, AI-native companies, and organizations actively replacing fragmented recruitment stacks. Assuming a 1% to 3% penetration of the SAM over a 3-to-5-year horizon, the estimated SOM is USD 150 to 300 million.

This range is consistent with historical SaaS adoption benchmarks for new enterprise software categories [41].

There is a significant Market Gap despite significant investment in recruitment technology, the current hiring ecosystem remains highly fragmented across sourcing tools, screening platforms, interview solutions, and ATS systems [42]. Key gaps include:

- Limited deep reasoning during candidate evaluation, with most systems relying on keyword matching or rule-based screening
- Lack of end-to-end automation across the full hiring lifecycle
- Weak personalization of interviews based on role requirements and candidate context
- Insufficient human-in-the-loop control in AI-driven hiring systems

Also, there are major key barriers to entry in such a market include limited access to high-quality hiring and interview data, enterprise requirements for explain ability and compliance, the technical complexity of building advanced reasoning-based interview models, and long, relationship-driven enterprise sales cycles [43][44].

The Barriers of adoption include organizational risk aversion in hiring decisions, legal and compliance concerns surrounding AI usage, change-management challenges for recruiters and hiring managers, and the need for seamless integration with existing ATS and HR systems. Also, AI-driven hiring systems must comply with evolving regulatory and ethical frameworks, including the EU AI Act, US EEOC guidance on algorithmic fairness, and global data protection standards such as GDPR, SOC 2, and ISO 27001. PATHS' human-in-the-loop design and skills-based evaluation framework mitigate regulatory and ethical risks [40] [45-49].

2.2 Related Work

2.2.1 Production and Market Benchmark

In this section we are showing market competitors and their approach for how solving the problems in the recruitment processes

2.2.1.1 Olivia (Paradox.ai)

Paradox offers a conversational AI assistant named Olivia that automates front-end recruiting tasks like screening, Q&A, and interview scheduling via natural chat. Its main benefit is efficiency in high-volume hiring as Olivia engages candidates 24 Hours /7 Days through text or mobile chat, quickly moving them through applications and scheduling without human intervention. This always-on chatbot improves candidate experience (feels like texting) and frees recruiters from repetitive admin work [24]. As shown in Figure 2.3.

Strengths

- Excels in hourly and event-based hiring, where speed and scale are critical.
- Enables faster hiring cycles and higher candidate completion rates using its AI assistant, *Olivia*.
- Integrates with major ATS platforms, making it suitable for enterprise hiring workflows.

Limitations

- Designed for large enterprises; may be too complex or costly for small teams.
- Requires a steep implementation effort where typically from 2 to 3 months and IT involvement.

- Offers limited customization of chatbot behavior and only basic video interview features.
- High pricing model focused on organizations with large-scale hiring needs.



Figure 2.3: Illustration of Paradox’s “Olivia” conversational AI interface used for candidate screening, Q&A, and automated interview scheduling [23].

2.2.1.2 Beamery

Beamery is an AI-powered Talent Lifecycle Management suite that combines a recruiting CRM, talent marketing automation, and internal mobility tools in one platform. Its value proposition is managing the entire talent journey from attracting passive candidates in a CRM to rediscovering past applicants and growing internal talent which is underpinned by a rich “talent graph” of skills and AI recommendations. for enterprises with long-term talent strategies, but its breadth means higher complexity and cost [25]. As shown in Figure 2.4.

Strengths

- Known for proactive talent engagement and pipeline building through talent pools and nurturing campaigns.
- Enables recruiters to rank passive leads and rediscover “silver medalist” candidates using AI-based suggestions.
- Strong in enterprise compliance and diversity:
 - Includes GDPR consent management.
 - Undergoes external bias audits on its AI models.
- Offers advanced internal mobility features, such as:

- Role recommendations for current employees.
- Upskilling pathways to support employee retention.

Limitations

- Targeted at large, global enterprises as it is not designed for smaller organizations.
- Requires significant investment and integration efforts.
 - Full implementation often involves Beamery's team.
 - Annual costs can reach six to seven figures.
- Lacks off-the-shelf simplicity for SMBs.
- Offers slower time-to-value for lean or fast-moving teams.

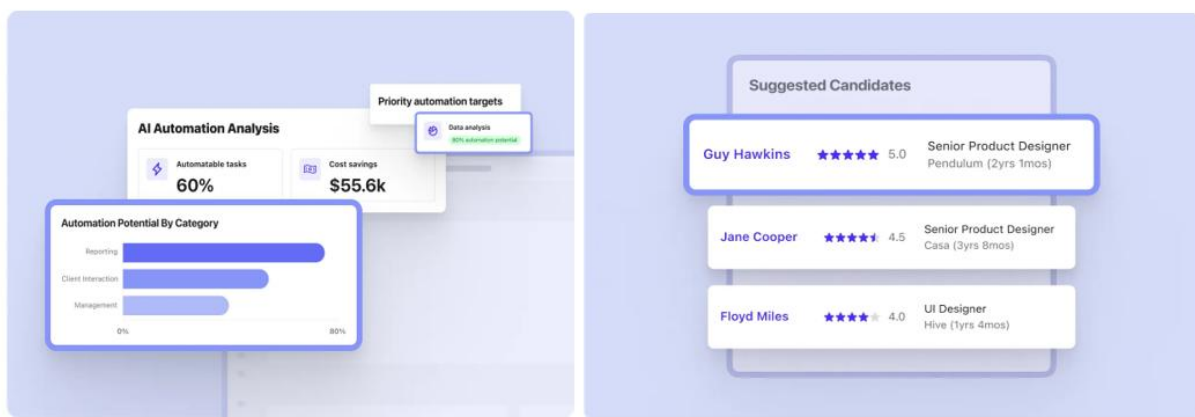


Figure 2.4: Illustration of Beamery's talent pipeline / talent graph view for engaging passive candidates, ranking matches, and managing talent pools [24].

2.2.1.3 Elevatus (EVA-REC)

Elevatus which is often branded as *EVA-REC* is an AI-powered recruitment platform/ATS that streamlines the hiring process from requisition to onboarding. Its main value lies in an intuitive, easy-to-use interface combined with AI tools for screening and matching, which helps teams efficiently manage and track candidates [26]. As shown in Figure 2.5.

Strengths

- Boosts recruiter productivity by automating key tasks like:
 - Resume parsing
 - Candidate shortlisting
 - Interview scheduling
- Supports customizable hiring workflows, allowing:
 - Collaboration among hiring managers
 - Profile reviews
 - Real-time tracking of pipeline progress

- Offers AI-driven candidate matching, which:
 - Quickly identifies best-fit applicants
 - Reduces manual screening time
- Known for efficiency and ease of adoption as it is suitable for teams looking for a streamlined, intuitive platform.

Limitations

- Some advanced features have a learning curve:
 - Analytics dashboards
 - Custom assessments
- Reporting capabilities are relatively basic:
 - Limited flexibility in BI dashboards
 - Users desire more dynamic, customizable reports
- As a newer entrant, some deep integrations and niche functionalities may be less mature than in older ATS products.
- Requires additional training to unlock the full power of advanced features.

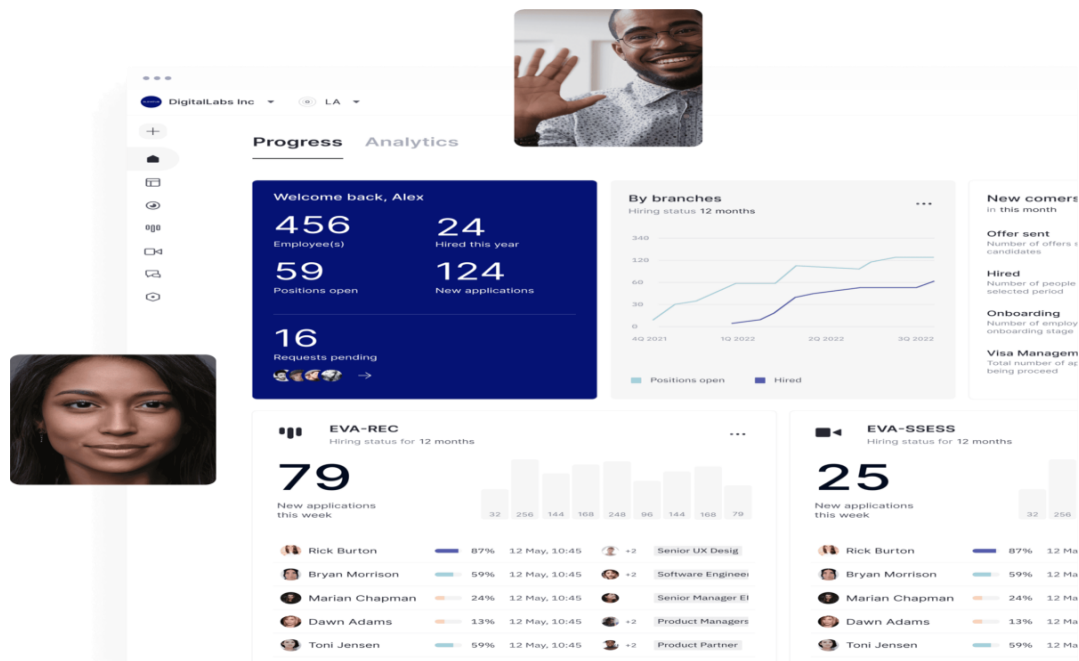


Figure 2.5: Demonstration of Elevatus’s recruiting dashboard for resume parsing, shortlisting, and tracking candidate progress across hiring stages [25].

2.2.1.4 HireVue

HireVue is a pioneer in on-demand video interviewing and AI-driven candidate assessment. Its core offering enables candidates to record video or audio responses to structured interview questions at their convenience, which recruiters can later review. The platform’s standout feature

is its AI analysis of these recordings as algorithms evaluate speech to score candidates on competencies, although the facial analysis aspect has sparked debate over bias. Additionally, HireVue provides game-based cognitive assessments and coding tests, making it a comprehensive early-stage screening suite beyond video [23]. As shown in Figure 2.6.

Strengths

- Ideal for high-volume applicant screening:
 - Automates early-stage interviews via on-demand video responses.
 - Saves recruiter time by eliminating the need for manual phone screens.
- Reported to reduce initial screening time by about 60%.
- Delivers consistency by asking all candidates the same structured questions.
- Provides flexibility for candidates to interview on their own schedule.
- Includes automatic transcription and response insights, improving review efficiency.
- Seamlessly integrates with major ATS platforms, making it compatible with enterprise tech stacks.

Limitations

- Raises ethical and compliance concerns related to:
 - AI scoring transparency
 - Potential bias from past use of facial/tone analysis
- Though facial analysis has been phased out, skepticism about bias in AI evaluation still exists.
- Enterprise-oriented pricing model:
 - High cost with no public pricing or free trial
 - Custom contracts often start in the tens of thousands USD per year
- May be overkill for small companies or limited hiring needs.
- Candidate experience varies:
 - Some find one-way video interviews impersonal or stressful, potentially affecting performance.

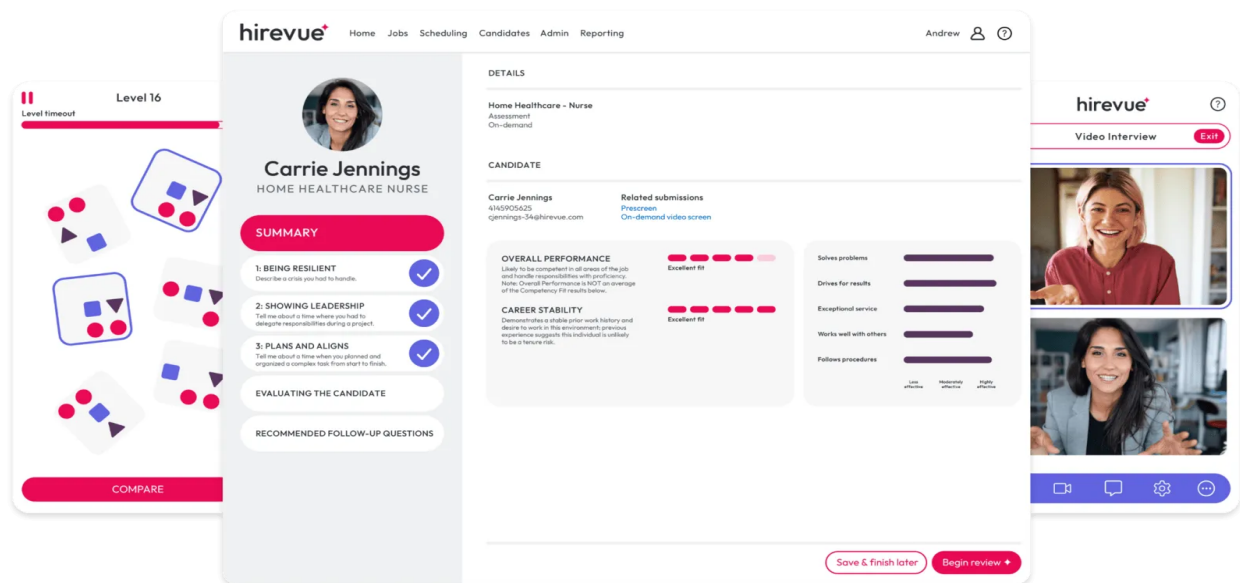


Figure 2.6: HireVue's structured video interview workflow and recruiter evaluation dashboard with response insights and scoring outputs [26].

2.2.1.5 Eightfold AI

Eightfold AI is a talent intelligence platform that uses advanced machine learning to power recruiting, internal mobility, and talent management. Its value proposition is leveraging a massive dataset (over 1.5 billion talent profiles by some reports) to infer skills and match candidates to roles with high accuracy. Eightfold can integrate with a company's ATS/HRIS and LinkedIn to analyze resumes, past applicants, and employees, then recommend candidates not only for open jobs but also for future roles (talent pipelining) and internal promotions [27]. As shown in Figure 2.7.

Strengths

- Known for advanced AI capabilities:
 - Goes beyond keyword matching by analyzing career paths and skill adjacencies.
 - Identifies non-obvious candidates with transferable skills.
- Improves diversity and quality-of-hire by surfacing unconventional but well-matched candidates.
- Offers a comprehensive, full-suite platform:
 - Includes ATS and CRM features
 - Provides diversity analytics
 - Supports an internal talent marketplace for mobility and retention
- Delivers predictive workforce insights, such as:

- Forecasting hard-to-fill roles
- Detecting employees at risk of attrition due to career stagnation

Limitations

- Enterprise-focused which is best suited for large organizations with complex, global needs.
- Heavy implementation effort:
 - Requires significant integration and change management
 - Onboarding may take time and internal coordination
- High cost:
 - Typically priced at \$7 to \$10 per employee/year
 - Can reach hundreds of thousands USD for large-scale deployments
- Limited transparency:
 - Algorithms are proprietary, creating a “black box” effect that can hinder trust
 - Some HR teams may prefer more explain-ability in AI decision-making
- May be too complex for mid-sized companies with simpler needs or limited data infrastructure.

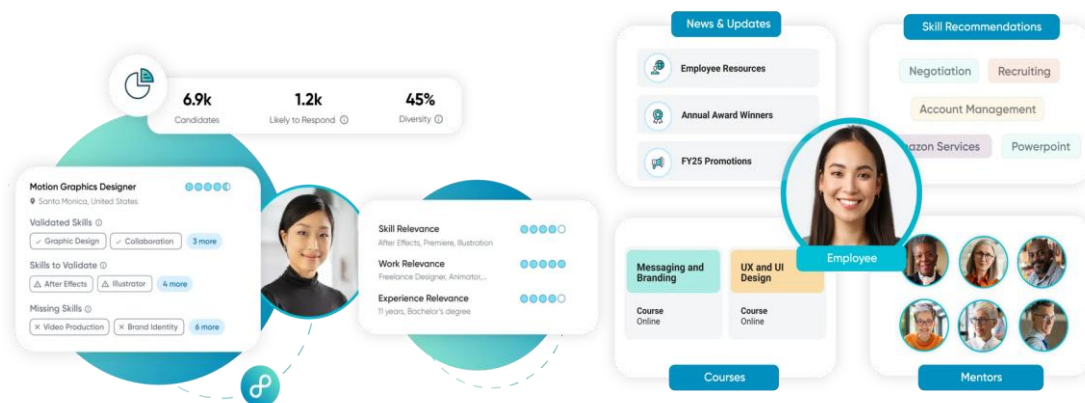


Figure 2.7: Eightfold’s AI-driven candidate matching (skill inference and role-fit ranking) and/or internal talent marketplace recommendations [27].

2.2.1.6 SeekOut

SeekOut is an AI-driven talent sourcing platform known for its ability to find “hard-to-find” candidates, including those outside of LinkedIn and those from underrepresented groups. Its main benefit is an extensive aggregated database (over 750 million profiles) drawn from diverse sources like GitHub, papers, patents, social media, LinkedIn which make it a combined with powerful search filters and AI assistance to pinpoint talent [28]. As shown in Figure 2.8.

Strengths

- Offers granular and intelligent search capabilities:

- Search by skills, titles, location, diversity tags, or natural language queries
 - Suggests related keywords/technologies to broaden searches
- Excels in diversity sourcing:
 - Filter candidates by demographics
 - Includes “bias reducer” mode that hides names and photos for objective review
- Includes a powerful Talent Insights dashboard:
 - Provides market data like candidate supply, diversity distribution, and employer presence by role/location
- Combines ease of use with advanced capabilities:
 - No Boolean skills required (but supported for power users)
 - Integrates with ATS/CRM to export or enrich candidate profiles

Limitations

- Not a full ATS:
 - Focused on sourcing and talent discovery
 - Does not manage full hiring workflow
- Data privacy considerations:
 - Uses publicly available candidate data
 - Organizations must ensure regulatory compliance
- Premium pricing model:
 - May be costly for smaller firms or low-volume hiring teams
 - Value depends on leveraging advanced features consistently
- Manual recruiter engagement required:
 - AI does not contact candidates autonomously (beyond basic email integrations)
 - Success depends on active recruiter usage within workflows

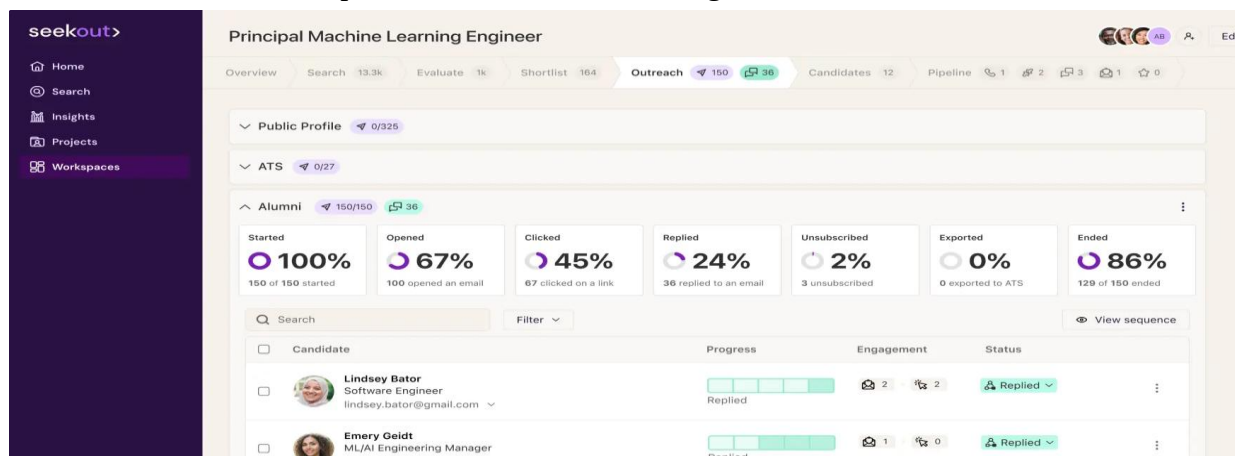


Figure 2.8: SeekOut’s advanced talent search filters (skills, location, diversity) and/or Talent Insights dashboard for labor market analytics [28].

2.2.1.7 Pymetrics

Pymetrics is a specialized platform using neuroscience-based games and AI to assess candidates' cognitive, social, and emotional traits. Instead of traditional tests or resumes, candidates play a series of short games (about 20 to 30 minutes total) that measure attributes like memory, risk-taking, attention, emotional recognition, and problem-solving style. The system then uses AI to compare a candidate's behavioral profile to success profiles (either top performers in a role or other benchmarks) to predict job fit beyond what a resume could show [29]. As shown in Figure 2.9.

Strengths

- Focuses on soft skills and potential using neuroscience-based games:
 - Quantifies traits like decision-making speed, attention, and emotional recognition
 - Helps employers evaluate candidates beyond resumes and hard credentials
- Enables discovery of high-potential, non-traditional candidates often missed by traditional filters
- Designed to reduce hiring bias:
 - Does not use demographic data in assessments
 - Audited externally to ensure fairness across gender, ethnicity, and other attributes
 - Algorithms are deployed only after passing rigorous bias checks
- Offers a gamified, engaging candidate experience, especially appealing for early-career or volume hiring

Limitations

- Validity concerns:
 - Independent studies suggest weak correlation between game results and actual job performance in some contexts
 - Predictive value may vary across roles and industries
- Risk of misinterpretation or over-reliance on soft-skill data:
 - Requires proper HR training to avoid misuse or oversimplification
- Candidate perception issues:
 - Some professionals may find the games non-serious or unrepresentative of their capabilities
 - Poor communication of purpose may lead to drop-off or disengagement
- Operates as a point solution:
 - Must be integrated into a broader hiring workflow

- Typically used early in the funnel, not as a full hiring system
- While compliant and transparent, bias mitigation is not automatic:
 - Employers still need to monitor and audit impact across demographics over time

Core Soft Skills

In just 20 - 25 minutes, users can partake in **12 core exercises** that capture cognitive, emotional and social traits. These behavioral assessments provide **rich feedback for recruiters and managers as well as the candidates themselves**...not to mention a 95%+ candidate satisfaction rating.

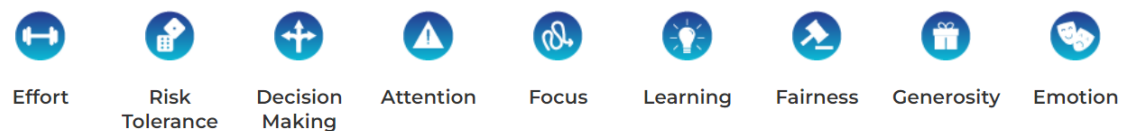
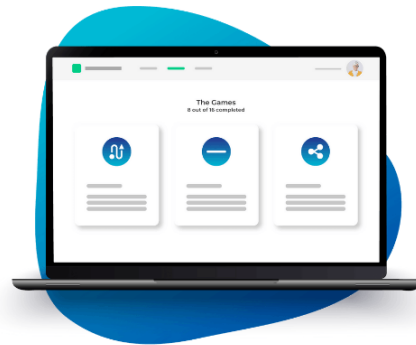


Figure 2.9: Pymetrics' game-based assessment [29].

2.2.1.8 HackerRank

HackerRank is a widely-used platform for technical hiring assessments, offering a large library of coding challenges and tests across dozens of programming languages. Its main benefit is providing a standardized, scalable way to evaluate coding and technical skills objectively, helping organizations screen and identify capable developers from a large applicant pool [22]. As shown in Figure 2.10.

Strengths

- Supports a wide variety of technical assessments, including:
 - Algorithmic coding challenges
 - Project-based tasks
 - Multiple-choice questions
 - Pair-programming interviews
- Offers an extensive question library and broad language support for diverse technical stacks
- Helps reduce time-to-hire by:
 - Automating technical screening
 - Ensuring a consistent baseline skill filter before interviews
- Highly user-friendly for both recruiters and candidates

- Integrates with ATS systems to streamline candidate flow
- Includes plagiarism detection, timed tests, and AI-based code quality analysis
 - Evaluates efficiency and style, not just correctness

Limitations

- Candidate experience concerns:
 - Timed algorithm challenges can feel artificial or stressful
 - May disadvantage strong developers who don't perform well under test-like conditions
- Risk of false negatives:
 - Practical coding skills may not be reflected in academic-style puzzles
- Cheating and fairness issues:
 - Some candidates use online solutions or exploit the system
 - Anti-cheating measures need improvement in certain areas
- Admin-side limitations:
 - Reporting and analytics could be more robust
 - Creating highly customized assessments can be time-consuming
- Integration challenges:
 - ATS integration not always seamless; some users report difficulties
- Pricing is enterprise-focused:
 - Plans may be too expensive for small teams or low-volume hiring
 - Often priced per seat or per assessment volume, with limited flexibility for smaller users

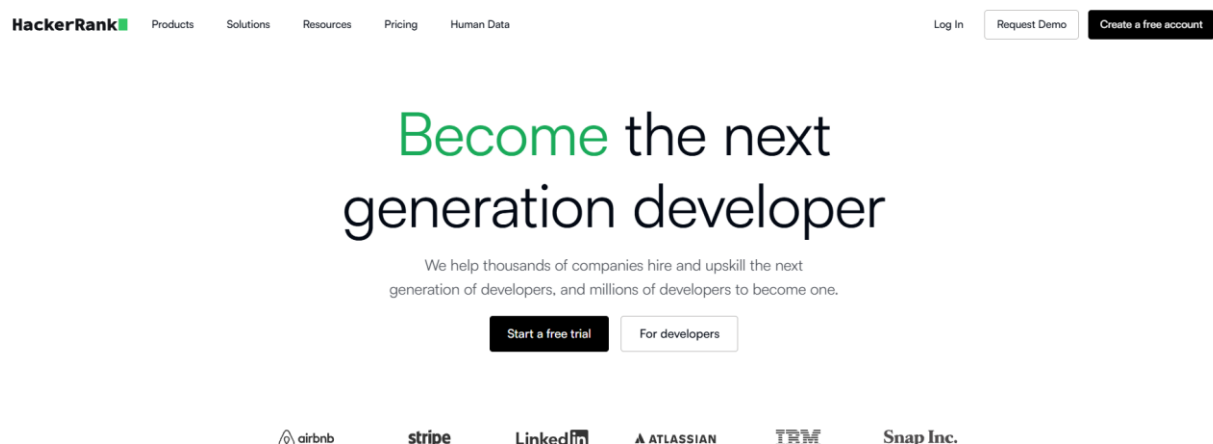


Figure 2.10: HackerRank's coding used to test interface or evaluation report used to assess programming proficiency at scale [22].

2.2.1.9 Codility

Codility is another popular technical assessment platform, focused on real-world coding tasks and automated evaluation. It provides online coding tests that measure not only whether code works, but also how well it's written (examining correctness, performance, and edge cases). Works best when complemented with interviews or additional assessments to get a full picture of candidate ability [30]. As shown in Figure 2.11.

Strengths

- Known for realistic, production-style coding challenges:
 - Emphasizes code efficiency, not just correctness
- Offers strong anti-cheating features:
 - Plagiarism detection
 - Proctoring tools
- Supports a wide range of programming languages and question types
- Generates detailed candidate reports, making evaluation easier
- Especially helpful for non-technical recruiters:
 - Provides clear scoring, rankings, and skill comparisons
 - Enables informed hiring decisions without deep technical background

Limitations

- User interface is considered dated and less intuitive compared to newer platforms
 - May negatively impact candidate experience
- Test creation can be time-consuming:
 - Requires manual selection of questions to match specific role needs
- Focus on algorithmic challenges may not reflect real-world coding tasks
 - Similar downsides to HackerRank
- Limited candidate feedback:
 - Candidates often don't receive insights or results, which may hurt brand perception
- Enterprise pricing model:
 - No public pricing available
 - Typically requires direct sales contact
 - May be too costly for small-scale or occasional hiring

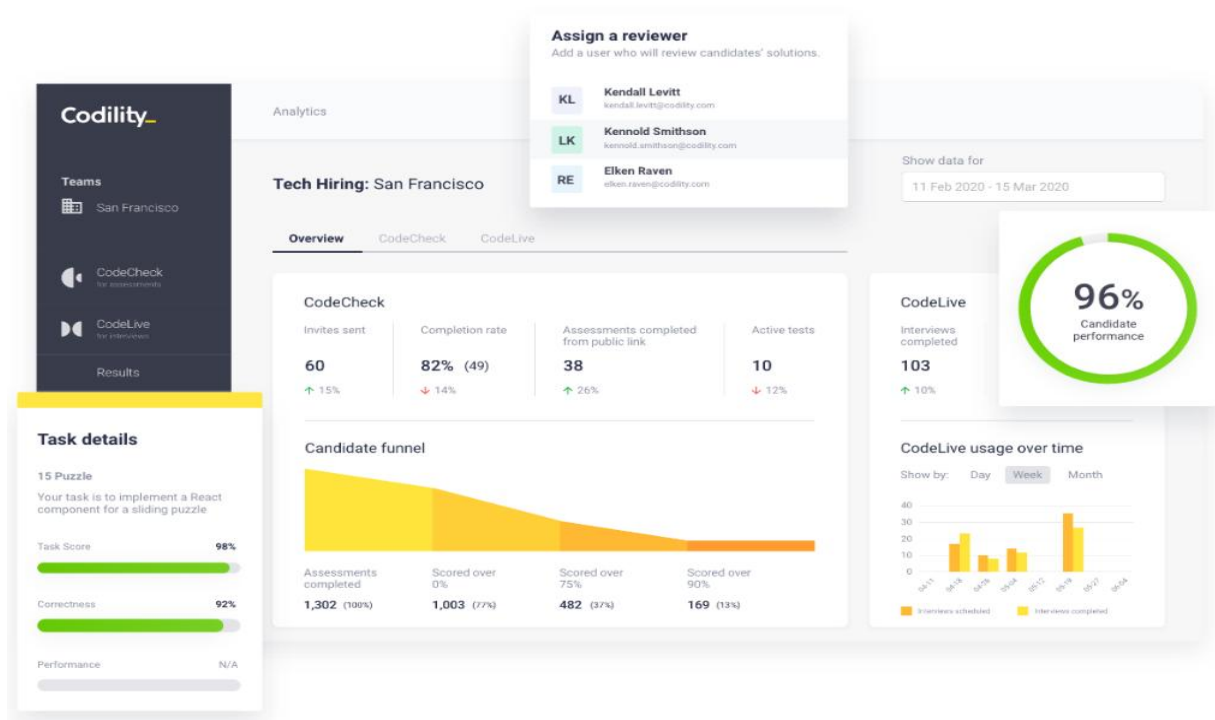


Figure 2.11: Codility’s analytics view for technical hiring, including CodeCheck completion metrics, candidate funnel progression, and performance scoring summaries used to evaluate coding assessments at scale [30].

2.2.1.10 Greenhouse

Greenhouse is a leading Applicant Tracking System (ATS) known for its structured hiring approach and extensive integration ecosystem. Its primary value is in providing a scalable, highly configurable platform that helps organizations manage every stage of recruiting, from job approvals and postings, through interviews and scorecards, to offers all while promoting consistent, data-driven hiring practices. [17] As shown in Figure 2.12.

Strengths

- Highly regarded ATS for mid-size and enterprise companies
- Offers strong customization features:
 - Custom interview pipelines, scorecards, and candidate fields
 - Structured evaluation improves decision quality and consistency
- User-friendly interface with a relatively low learning curve
- Automates workflows like:
 - Interview scheduling
 - Email triggers and reminders
- Supports collaborative hiring:

- Enables team feedback, @mentions, and approval flows
- Provides modular add-ons for onboarding and CRM functions
- Extensive integration ecosystem:
- Strong focus on inclusive and structured hiring:
 - Offers tools and reports for diversity & inclusion tracking

Limitations

- Complex setup and configuration:
 - Initial implementation requires detailed planning (e.g., roles, workflows, permissions)
- May be overkill for small companies needing only basic functionality
- Certain advanced customizations can be difficult or limited without workarounds
- Reporting challenges:
 - Built-in reports lack deep flexibility
 - Custom reports may require technical or BI expertise
- Customer support inconsistency:
 - Smaller accounts may face slower response times
- Premium pricing:
 - Considered one of the more expensive ATS options
 - May not suit budget-sensitive teams or SMBs looking for leaner solutions

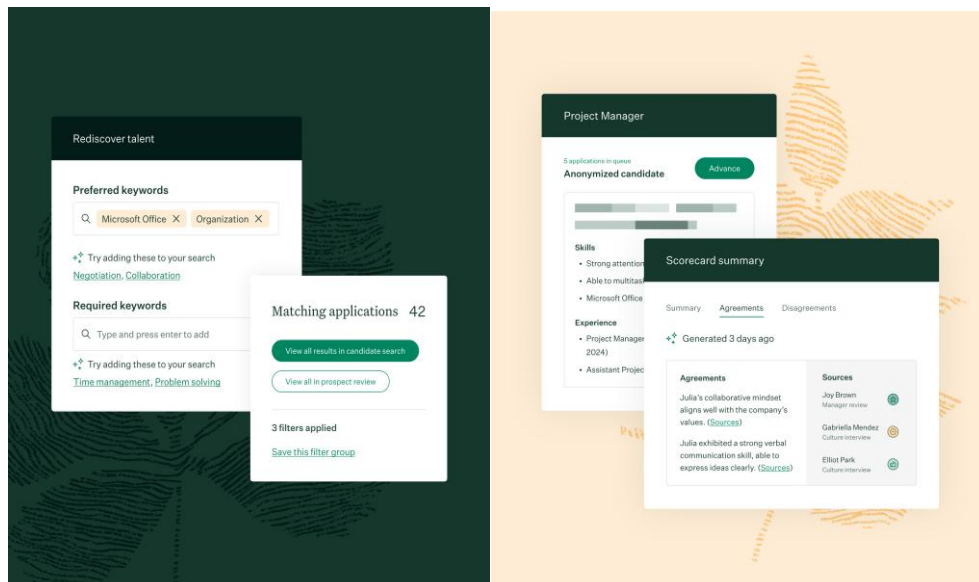


Figure 2.12: Greenhouse's candidate matching (preferred/required keywords) and structured scorecard workflow, including an anonymized candidate view and agreement/disagreement summaries to standardize hiring decisions [17].

2.2.1.11 Ashby

Ashby is a newer all-in-one recruitment platform (ATS) that has gained popularity, especially among tech startups and high-growth companies, for its modern design and integrated analytics. Ashby's value proposition is that it combines the capabilities that usually require several tools as ATS, scheduling automation, email sourcing, and advanced analytics into one cohesive and flexible system. It is also distinctive for offering month-to-month pricing and a standalone analytics product, making it attractive to firms that want flexibility and powerful reporting without long contracts. [31] As shown in Figure 2.13.

Strengths

- Functions as an all-in-one recruiting platform:
 - Combines ATS, sourcing (light CRM), scheduling, and analytics in a single interface
- Streamlines workflows:
 - Recruiters and coordinators can manage everything in one place, reducing tool-switching
- Modern, intuitive user interface:
 - Clean design with high usability
 - Short training time and quick team adoption
- Strong analytics and reporting capabilities:
 - Customizable dashboards for metrics like time-to-hire, funnel conversions, interviewer load, and diversity
 - Analytics module can be purchased standalone
- Ideal for data-driven teams needing flexible, visual performance insights
- Flexible pricing model:
 - Offers monthly subscriptions, rare among ATS platforms
 - Supports teams with fluctuating hiring needs or who prefer avoiding long-term contracts

Limitations

- As a newer entrant, still maturing in feature coverage:
 - Lacks multi-language support and dedicated mobile app
 - May not yet support complex, enterprise-grade workflows
- UI quirks and occasional bugs:
 - Common in emerging platforms, though support is responsive

- Onboarding can be complex:
 - Rich settings and features may require weeks of training to fully master
- Some advanced features are gated behind add-ons:
 - Advanced analytics and expanded sourcing/email tools may require higher-tier plans
- Per-employee costs about 5\$ to 8\$ PEPM can become expensive as hiring scales
 - Not always cheaper than competitors in the long run

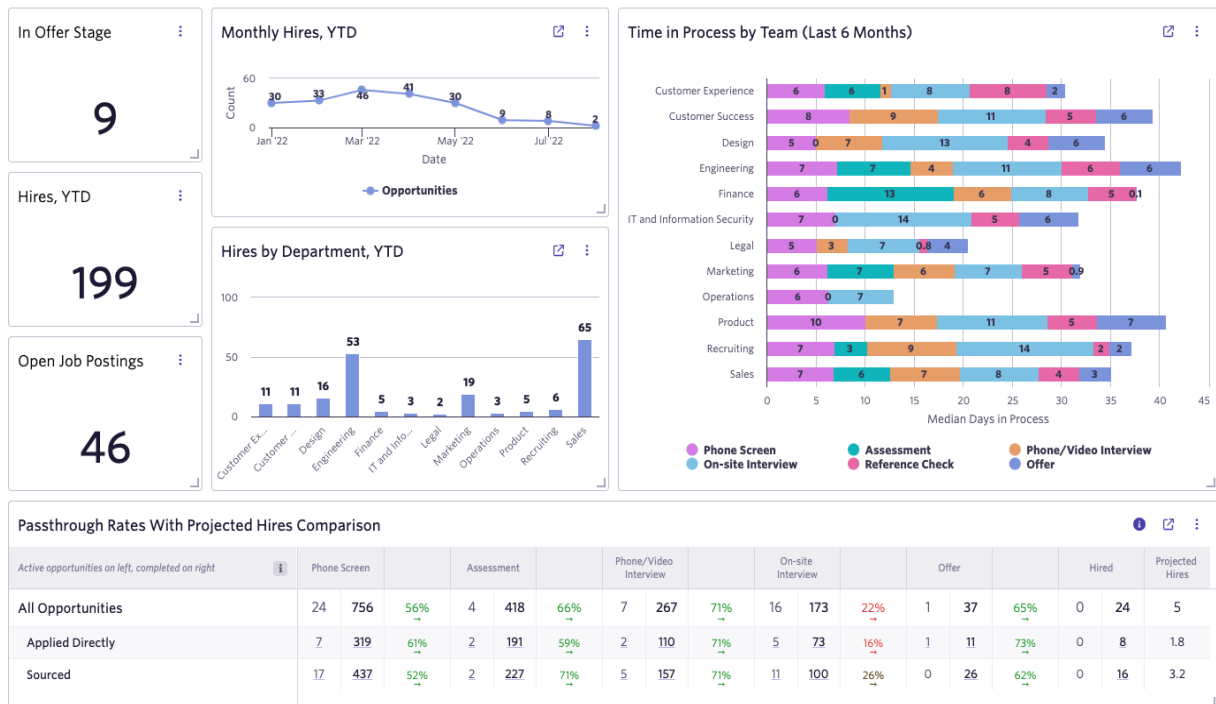


Figure 2.13: Ashby's reporting dashboard with key hiring KPIs (hires, open roles, monthly trends), stage-level passthrough rates, and time-in-process breakdown by team to support data-driven recruiting optimization [31].

2.2.1.12 SmartRecruiters

SmartRecruiters is a cloud-based enterprise Talent Acquisition Suite (ATS+CRM) known for its user-friendly, “social” interface and strong collaborative features. Its motto of “Hiring Success” reflects a focus on not just tracking applicants, but also engaging hiring teams and providing a positive candidate experience. [5] as shown in Figure 2.14.

Strengths

- Modern, sleek UI (social-feed style) that is intuitive for recruiters and hiring managers.
- Strong mobile app, enabling hiring on-the-go and higher adoption (managers can review/advance candidates from phone).

- Large integration marketplace with many connectors to build a tailored tech stack.
- Good balance between enterprise power + usability:
 - Configurable workflows, permissions, and compliance features
 - Still accessible for mid-market teams without heavy daily admin work
- Supports global/multinational hiring:
 - Multi-language postings
 - Local compliance settings and global support for distributed teams
- Strong collaboration features:
 - Internal comments/discussion on candidates
 - Feedback requests and automated reminders that keep stakeholders aligned

Limitations

- Implementation/setup can be heavy:
 - Configuring workflows, roles, and integrations often takes time compared to simpler ATS tools
- Limited onboarding module:
 - Stops mostly at offer + basic new-hire tasks
 - Often requires integration with HRIS/onboarding tools for full onboarding workflows
- No native video interviewing:
 - Relies on integration partners for video interviews (some competitors offer built-in video)
- Support experience may vary:
 - Some smaller customers report less responsive support post go-live compared to enterprise clients
- Mobile app not fully feature-parity with web version:
 - Some admin tasks still require desktop
- Reporting/analytics can be less advanced than specialized tools:
 - Power users may need external BI or advanced reporting solutions for deep analytics

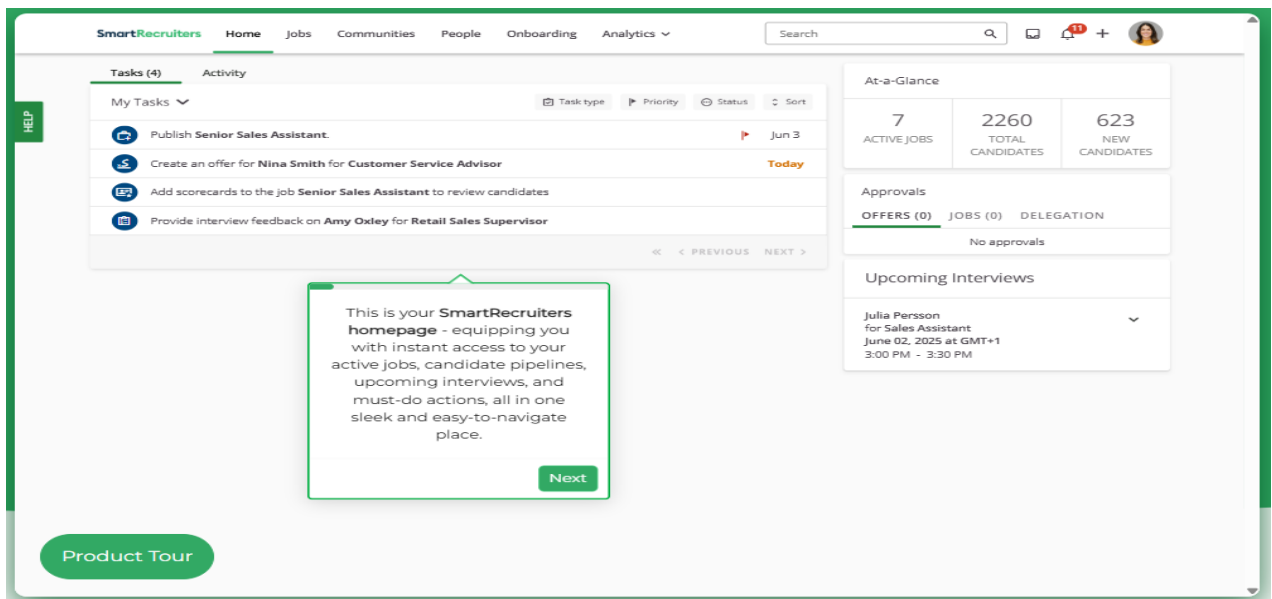


Figure 2.14: SmartRecruiters' main dashboard with task lists, at-a-glance hiring activity (active jobs and candidate counts), approvals, and upcoming interviews which is supporting collaborative and workflow-driven recruitment operations [5].

Table 2.1: Competitive Landscape Comparison of PATHS vs. Leading Recruitment Platforms

Company	Core Category	Core Concept	Sourcing	Interview / Assessment	Human-in-the-Loop	Primary Strength	Primary Limitation	Target Market
PATHS	Agentic Recruiting	Customizable agentic workflows	Hybrid (Outbound + Inbound)	Agentic RAG interviews	High	Deep reasoning + flexibility	New category, requires trust	Start-ups & Mid-Enterprise
Paradox [24]	Hiring Automation	Chatbot-driven hiring	Inbound	Chatbot screening	Low	Speed	Shallow evaluation	High-volume retail/service
Beamery [25]	Talent CRM	Talent lifecycle	Inbound + Database	None	Medium	Long-term talent pools	No evaluation intelligence	Global enterprises
Elevatus [26]	Compliance AI	Compliance-first + video AI	Inbound + Database	Video AI	Medium	Regulatory alignment	Rigid workflows	Governments and large orgs
HireVue [23]	Interview AI	Video and game-based assessments	Inbound	One-way video interviews	Medium	Scalable assessments	Bias	Large enterprises
Eightfold AI [27]	Talent Intelligence	Skills graph	Inbound + Internal mobility	None	High	Skill-based matching	No interviews	Large enterprises
SeekOut [28]	Sourcing Intelligence	AI-powered talent search	Outbound	None	High	Candidate discovery	Stops at sourcing	Tech recruiters
Pymetrics [29]	Assessment Platform	Cognitive and behavioural games	Inbound	Game-based assessments	Low	Bias-audited testing	Early-career	Enterprise
HackerRank [22]	Tech Assessment	Coding validation	Inbound	Coding challenges	Medium	Objective skill tests	Narrow scope	Engineering teams
Codility [30]	Tech Assessment	Coding and algorithm tests	Inbound	Coding challenges	Medium	Engineering signal	No conversation	Engineering teams
Greenhouse [17]	ATS	Structured hiring system	Inbound	Scorecards	Very High	Process consistency	No intelligence	Tech start-ups
Ashby [31]	ATS + Analytics	Analytics-driven recruiting ops	Inbound	Structured interviews	Very High	Reporting	No automation	High-growth start-ups
SmartRecruiters [5]	ATS Platform	Enterprise ATS	Inbound	Partner-based	High	Ecosystem	Not intelligent	Mid-to-large enterprises

The Table 2.1. Is stating benchmarks PATHS against major recruitment tools across core category and concept, sourcing approach, interview/assessment capability, degree of human-in-the-loop, primary strengths and limitations, and the target market segment each solution is designed for.

2.2.2 Literature Review

Mahajan, et al. [50], This paper focuses on scaling remote recruitment for geographically distributed candidate pools by combining Service Cloud infrastructure with AI-driven analytics and a human-AI interaction layer. Its core contribution is treating remote hiring as a cloud workflow problem: centralize candidate data, run AI matching, and keep recruiters in the loop to improve trust, transparency, and governance. Reported outcomes include about 55% time-to-hire reduction (from 45 to 60 - from 20 to 25 days), about 27% improvement in matching accuracy, and about 40% reduction in cost per hire, while explicitly adding bias detection and explain-ability features to reduce ethical risk.

For PATHS, this is highly aligned with the “scale + compliance” objective, but it still assumes that data is already clean and available. PATHS is better at solving the upstream fragmentation and passive-candidate problem: it is designed to unify from sourcing to engagement to evaluation to development, including entity resolution + normalization across LinkedIn/GitHub/PDFs and RAG-enhanced personalized outreach at scale in two areas this paper flags as fragile due to data dependency and real-time sync issues.

Rahman, SM Masudur, et al. [51], This research reframes recruitment as part of ESG execution, proposing a conceptual framework where AI supports both environmental sustainability (e.g., reducing travel/paper and lowering emissions by up to about 30%) and social sustainability (e.g., anonymized profiling to reduce bias, diversity and inclusion). It highlights AI’s operational value in automating repetitive steps like screening so recruiters can focus on higher-value strategy, but it remains largely conceptual and warns that privacy and bias risks persist without audits and governance.

PATHS also treats hiring as more than “fill a vacancy” it’s a system that needs governance and measurable outcomes. Where PATHS is better is that it is not only a sustainability roadmap: it is built as an end-to-end operating system that connects interview evidence to post-hire development plans, directly targeting the hiring–development disconnect and retention problem.

Mariappan, Prabha, et al. [52], This paper targets a narrow but common bottleneck: manual resume screening. It proposes an ML pipeline (tokenization + TF-IDF + Random Forest) trained on a resume dataset (RESUME-20K), reporting about 95% accuracy for categorizing resumes into job roles, and wraps it in a usable Streamlit interface for real-time uploading and feedback. The main limitation is generalizability as TF-IDF works well for that dataset but may not transfer across domains, formats, or markets without retraining.

In PATHS terms, this paper is a component, not a system. PATHS is better at what happens before and after screening: it addresses passive talent discovery, outreach effectiveness (personalization at scale), structured evaluation, and the “single source of truth” problem caused by fragmented profiles, while also turning evaluation results into a development pathway instead of discarding them after hiring.

Pathak, Gangesh, et al. [53], This case study is closest to PATHS architecturally. It argues that recruitment tools are fragmented (chatbots here, schedulers there, scorers elsewhere) and proposes a collaborating multi-agent workflow that covers sourcing, vetting, evaluation, and decision support. In deployment (500+ engineers), it reports about 65% reduction in time-to-hire, about 91% agreement with human technical reviewers, and strong candidate satisfaction (4.6/5), largely due to asynchronous and flexible interviews. It also highlights real risks: weak soft-skill assessment (about 67% accuracy), LLM hallucinations (fabricated feedback), and cheating in async formats.

Koman, Gabriel, et al. [54], This paper is positioned for the public sector: it argues that e-recruitment is the foundational module of e-HRM and a prerequisite for “Smart Government,” linking digital recruitment to outcomes like transparency, efficiency, and cost reduction. Its contribution is mainly theoretical (grounded theory + conceptual model) and it calls for real-world validation in government settings.

For PATHS, this supports your narrative that recruitment modernization is a system-level reform, not just tooling. PATHS is better at operationalizing the model: it specifies how to collapse a fragmented stack into a unified platform and how to enforce structured, auditable evaluation while keeping humans in control, key requirements for high-stakes, compliance-heavy contexts.

Abdelraouf, Mohamed, and Mona Kadry, et al. [55], This study is valuable for your Egypt/MENA framing. It examines how AI modernizes recruitment in Egyptian private banks, emphasizing the tension between “high-tech automation” and the “high-touch” human element in service sectors. It reports speed and efficiency gains via automated screening and communication, and argues that bias can be reduced when AI is supervised correctly, while warning about implementation complexity, misemployment risk, and the need for ethical oversight and training.

This maps directly onto PATHS’s philosophy: augmentation, not replacement. Where PATHS is better is that it bakes human-in-the-loop control into the system design while also solving structural failures that banks face at scale: outreach effectiveness, inconsistent interviews, and the lack of a development link that drives early turnover.

Alkhawaldeh, Muthnna Mohammad Khalaf, et al. [56], This paper argues that AI adoption in Saudi startups is less a tooling issue and more a human capital system issue. It shows that practices like attracting, developing, and empowering talent drive knowledge sharing, which then enables AI adoption. Its limitation is scope (new digital businesses/startups) and its thesis that adoption depends on intermediate organizational capabilities, not simply buying AI.

This is strategically useful for PATHS positioning: it supports the idea that PATHS must be adopted as a workflow, not just software. Where PATHS is better is that it provides the missing operational bridge: it turns hiring signals into structured data and then converts that data into development actions which is helping organizations build the “talent system” that makes AI stick.

Awad, Ashraf, et al. [57], This paper studies AI adoption in a UAE government context using acceptance and “perceived usefulness/ease of use” as key predictors (TAM), supported by SEM analysis. It argues AI can improve efficiency and accuracy, but emphasizes that ethical risks (fairness, transparency, data security) and user experience determine whether adoption succeeds especially in government ministries.

This complements PATHS by highlighting the adoption barrier is trust + usability matter as much as raw capability. PATHS is better because it is explicitly designed to reduce adoption friction by collapsing multiple tools into one unified flow and keeping recruiters in control, while still delivering advanced capabilities (agentic workflows, context-aware interviews, and integrated sourcing/engagement).

Es-said, Boulmane, et al. [58], targets a strategic bottleneck: CV-based hiring misses real skill evidence and often fails to capture motivation/adaptability. It proposes a 7-stage, data-driven hiring framework that uses e-learning analytics/behavioral traces to build richer candidate profiles and improve recommendation/matching quality beyond keywords. The main limitation is data availability + fairness/privacy: not all candidates have comparable access to e-learning platforms, and learning behavior can become a proxy for resources or opportunity unless strong governance exists.

In PATHS terms, this is an evidence source, not a complete hiring system. PATHS enhances the benefit by combining e-learning evidence with multi-source profiles, then applying structured evaluation and converting outcomes into a development pathway (skill gaps to learning plan) that is done after the process of evaluation of the candidate for recommending him the best learning and development path.

Wahyuningrum, Sri Rizqi, et al. [59], targets a common operational bottleneck that recruitment is dynamic (skills drift, changing job needs), but most systems are static. It proposes a self-adaptive recruitment approach using the MAPE-K model (Monitor then Analyze then Plan then Execute over a Knowledge base), arguing AI + Big Data should enable continuous updates for sourcing, matching, and even turnover prediction. The main limitation is that adaptivity depends on high-quality feedback data and strong governance; without it, the system can become a non-transparent loop that amplifies bias or silently changes decisions of the system and the choose of candidates.

In PATHS terms, this is an architectural principle that PATHS can implement concretely using agent workflows and feedback loops (outreach response then messaging update; evaluation outcomes and scoring adjustments). PATHS enhances benefits by enforcing audit logs, explainability, and HITL gates, but is limited when outcome data (retention/performance) is missing or siloed.

Rajapakshe, Wasantha. Et al. [60], Focuses in recruiters struggle with volume, scheduling, screening, and communication. It surveys AI integration across recruitment tasks such as resume screening, chatbots, interview scheduling, and predictive analytics, highlighting productivity gains and shifting HR toward data-driven decision making. The main limitation is the classic triad: bias, privacy, and explain-ability, plus adoption barriers where stakeholders may not trust automated recommendations. In PATHS terms, this is a broad AI-in-hiring mapping; PATHS enhances the benefit by operationalizing governance: anonymization gates, evidence vs inference separation, and human-readable rationales for shortlists.

Jain, Rituraj, et al. [61], States that unstructured interviews are subjective and hard to scale. It proposes deep learning approaches for multimodal sentiment/emotion analysis (facial expressions, speech, text) to automate and standardize candidate evaluation in video interviews. The main limitation is high risk of spurious inference and bias: emotion not means competence, cultural variation is large, privacy concerns are severe, and explain-ability is weak which is making this one of the most sensitive AI hiring directions. In PATHS terms, this is a risky component rather than a safe core. PATHS enhances the benefit by focusing interviews on structured, job-relevant evidence (rubrics, work samples, skills demonstration) and restricting any affective signals behind strict governance and transparency.

Table 2.2: Feature Matrix Comparison of PATHS vs. Related Research Systems.

Ref	HITL	Bias detection / fairness	Anonymization	Explain-ability	Audit logs and auditability	Compliance / governance focus	Integrity controls	Self-adaptive feedback loop
PATHS	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
[50]	Yes	Yes	Not stated	Yes	Not stated	Yes	Not stated	Not stated
[51]	Not stated	Yes	Yes	Not stated	Not stated	Yes	Not stated	Not stated
[52]	Not stated	Not stated	No	No	No	No	No	No
[53]	Not stated	Not stated	Not stated	Not stated	Not stated	Yes	Yes	Not stated
[54]	Not stated	Not stated	Not stated	Not stated	Not stated	Yes	Not stated	Not stated
[55]	Yes	Yes	Not stated	Not stated	Not stated	Yes	Not stated	Not stated
[56]	Not stated	Not stated	No	No	No	Not stated	No	No
[57]	Not stated	Yes	Not stated	Yes	Not stated	Yes	Not stated	Not stated
[58]	Not stated	Yes	Not stated	Not stated	Not stated	Yes	Not stated	Not stated
[59]	Not stated	Yes	Not stated	Not stated	Not stated	Yes	Not stated	Yes
[60]	Not stated	Yes	Not stated	Yes	Not stated	Yes	Not stated	Not stated
[61]	Not stated	Yes	Not stated	No	Not stated	Yes	Not stated	Not stated

The Table 2.2. maps PATHS against the reviewed academic works using a binary feature matrix (Yes/No) across end-to-end workflow coverage, sourcing and outreach capabilities, data unification (entity resolution/normalization), evaluation components (screening, interviews, assessments), and governance requirements (human-in-the-loop, bias/fairness, explain-ability, auditability, and adaptivity). It highlights which papers propose complete systems versus single-module contributions, and where PATHS differentiates by unifying the full recruitment lifecycle and connecting hiring evidence to post-evaluation development pathways.

Chapter 3

Proposed solution analysis and design

3.1 Solution Methodology

The solution is designed as a multi-agent system in which the overall workflow is divided into modules, and each module contains a dedicated agent responsible for a specific task. These agents collaborate to deliver a fair evaluation of candidates against both the technical requirements and the organizational criteria defined by the company. There are six main agents in the system.

The first agent is the Scraping and Searching Agent, which collects candidate information from targeted sources based on what the other agents need in order to complete the required hiring steps. This sourcing stage operates in two modes: Outbound and Inbound. In the Inbound mode, the organization provides candidate data directly, which cuts off the candidate-search stage; in this case, the process starts directly from the outreach stage. In the Outbound mode, the organization provides only the requirements, and we perform the candidate search and sourcing based on these requirements.

After sourcing, another agent performs the Bias-Guardrail step, where candidates are anonymized so the evaluation focuses strictly on their skills, capabilities, and ability to perform the job tasks, regardless of attributes such as age, location, or gender (unless these attributes are explicitly required in the job requirements).

Next, the system takes the filtered list of candidates who meet the requirements and applies a Scoring Agent that evaluates them based on fit to role, experience, and additional relevant features. The system then selects and ranks the Top-K candidates according to the organization's needs.

After scoring, the Outreach Agent proceeds with candidate engagement. At this stage, candidates are de-anonymized so they can be contacted. However, a common issue arises: candidates' contact information may have changed, or the available data may be outdated. To address this, the system includes a Contact Enrichment step using MCP tools as *Hunter.io* to enrich and validate contact details before outreach.

Once a candidate responds positively and accepts to proceed, a Scheduler Agent enables the candidate to select an appropriate time for the next stages, including the assessment and the HR interview, and (for technical roles) the technical interview. For non-technical roles, the workflow proceeds with an interview process appropriate to the role type.

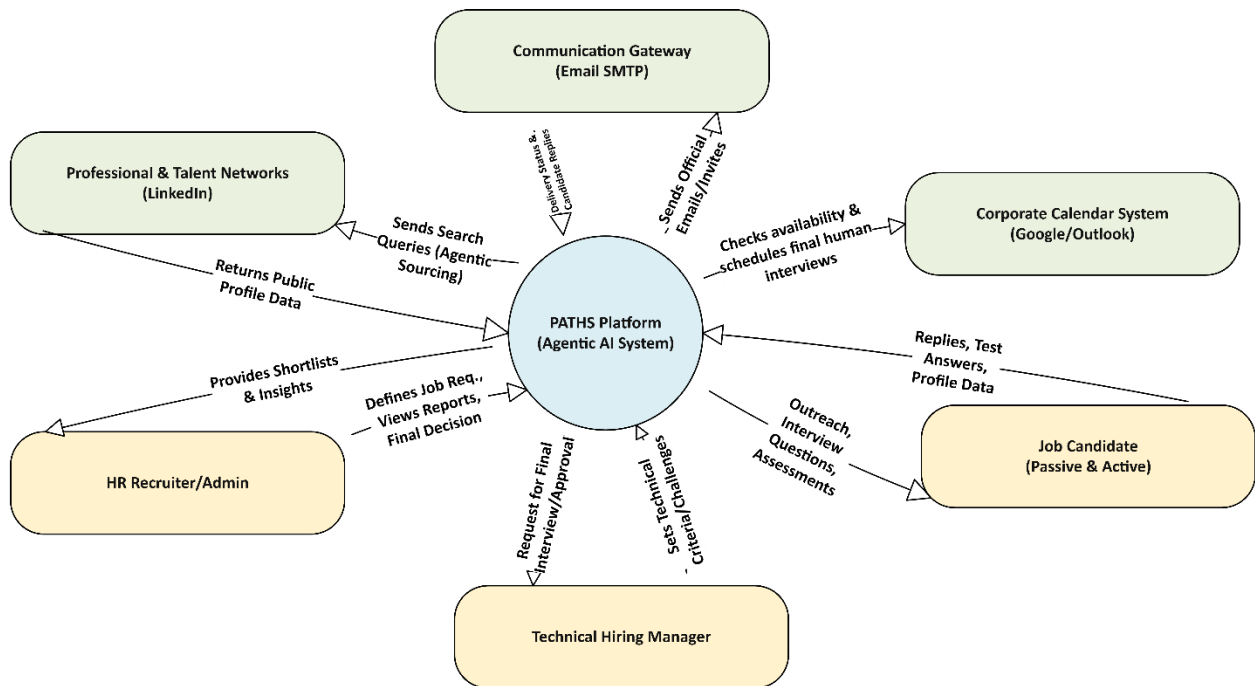


Figure 3.1 System Context Diagram

For technical roles, after scheduling, there are three key agents that operate alongside the interviewer:

1. A RAG Agent for the Organization and HR Interview, which supports verifying whether the candidate satisfies the organization’s criteria.
2. A Technical RAG Questions Agent, which generates and provides relevant questions for both the technical interview and the assessment.
3. A Summary Agent, which compiles and summarizes key outcomes for HR, including the candidate’s strengths, weaknesses, and other important points to support review.

These outputs help the human decision-maker validate whether the candidate’s score and evaluation are reasonable and aligned with the defined criteria. Finally, the Decision Support Agent generates the final scoring and produces a detailed report containing final notes and the candidate’s strengths. If the candidate is accepted, the report supports onboarding and development planning; if the candidate is rejected, the report is shared with the candidate to help them improve and potentially reapply for the same role or position in the future. If none of the Top-K candidates pass the process, the system repeats the cycle starting again from the initial stage.

All of these steps operate under a Human-in-the-Loop (Human HR) supervision approach, where the human HR professional makes the final decision on whether to continue, stop, or adjust the process.

3.2 System analysis

This section presents a structured analysis of the proposed PATHS system to ensure the solution is built on clear, realistic, and measurable requirements. Based on the recruitment challenges identified in the problem statement such as long time-to-hire, weak outreach performance, fragmented candidate data, inconsistent evaluation, and the disconnect between hiring and development PATHS is analyzed as an end-to-end platform rather than a collection of isolated tools.

The purpose of this analysis is to translate the business problem into a complete system view: defining who interacts with the system, what the system must do (functional requirements), how well it must perform (non-functional requirements like privacy, fairness, explain-ability, and reliability), and how information flows between modules across the recruitment lifecycle. The outcome of this section is a clear blueprint that guides the design decisions in later chapters, including architecture, database design, AI components, and user interface workflows.

As shown in Figure 3.2. That's represents our Knowledge Graphs that structures recruitment data as connected entities. The Organization creates Job Requirements, which define the skills and constraints needed for a role. Candidates apply to specific Job Requirements and also maintain direct relationships to Skills through a HAS_SKILL link, enabling skill-based matching and retrieval. Applications act as the operational record tracking a candidate's journey and status. A key addition is the GrowthPlan entity, where outcomes from an application can led to a development pathway which is going to capture gaps and recommendations instead of discarding rejected candidates and supporting long-term talent development and re-engagement.

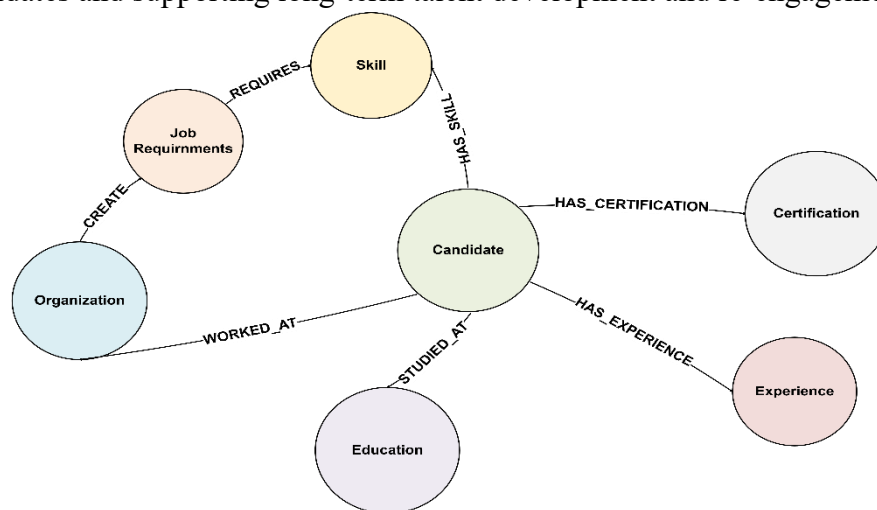


Figure 3.2: Knowledge Graph for Recruitment Data Modeling.

As shown in Figure 3.3. This diagram shows PATHS as a modular agentic system where each agent owns a distinct capability. The Sourcing Agent finds candidates aligned with requirements, the Scoring/Ranking Agent selects top-k candidates while keeping them anonymized, and the Contact Enrichment Agent resolves reliable communication channels. The Outreach Agent then engages candidates and hands accepted/interested candidates to the Scheduling Agent, which coordinates interviews and assessments. Evaluation is split into technical interview, technical assessment, and HR interview agents, each grounded by a dedicated knowledge base to keep evaluation consistent and role-specific. The final Decision Support Agent aggregates all signals into a structured report with scores, strengths/weaknesses, and recommended improvement areas, enabling either hiring or actionable candidate development.

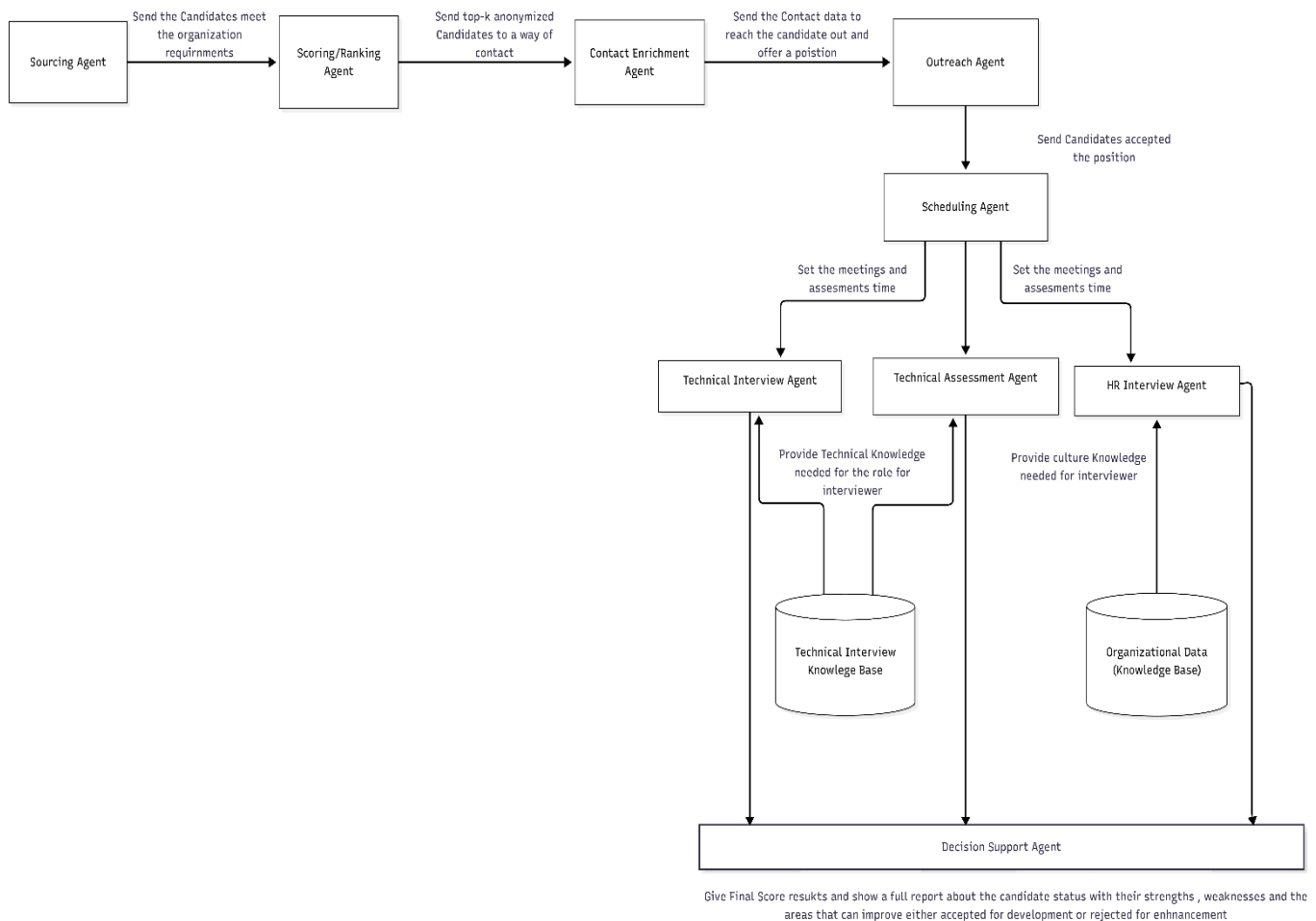


Figure 3.3: Modular Multi-Agent Recruitment Architecture and Agents Communication.

As shown Figure 3.4. This schema models the end-to-end talent pipeline as a traceable graph of decisions and evidence. An organization (users with roles such as recruiter/hiring_manager/admin) creates Job records, each defined by structured attributes (title, salary range, location, status) and detailed criteria (e.g., technical_criteria, required_experience_years). Jobs connect to Skill through JOB_SKILL, which captures both the required proficiency level and an importance weight, so matching can be scored rather than treated as simple keyword overlap.

On the candidate side, CANDIDATE stores identity/contact fields plus recruiting-specific metadata such as candidate_type (active/passive), experience_years, and flexible JSON fields (profile_data, sources, resolved_duplicates) to support enrichment and entity resolution. Candidates relate to skills via CANDIDATE_SKILL, which records proficiency_level, concrete evidence (e.g., portfolio/GitHub/interview proof), and last_verified which enables explainable scoring and periodic re-validation instead of static “self-claimed” skills.

The APPLICATION table is the central transaction layer that ties a candidate to a job and tracks the pipeline status (applied/sourced/shortlisted/assessed/rejected/hired), along with operational fields like apply_date, shortlist_rank, and source_platform. Each application can produce one or more ASSESSMENT records (chat/coding/behavioral), storing score, feedback, raw responses, date, and evaluator (AI/human), which supports human-in-the-loop review and auditability. Finally, GROWTH_PLAN links outcomes back to development by attaching structured plan details, goals, progress status, and review dates to the same application so evaluation insights can directly translate into a personalized improvement pathway rather than being discarded after hiring decisions.



Figure 3.4: Entity-Relationship Diagram (ERD) for PATHS core hiring workflow which linking organizations, jobs, candidates, skills, applications, assessments, and post-selection growth plans through normalized many-to-many mappings and audit-friendly records.

As shown in Figure 3.5. This use-case style diagram clarifies who interacts with which PATHS functions. Organization Recruiters control job creation, hiring criteria definition, shortlisting, and scheduling, reflecting ownership of pipeline management and governance. Organization Interviewers participate in technical assessment, AI-generated interview questions, and structured technical/HR interviews, reflecting evaluation responsibilities. The Organization Hiring Manager performs the final hiring decision, reinforcing accountability at the last step. The Candidate interacts primarily with interview stages and receives outcome notifications. This partitioning supports role-based access control (RBAC) by limiting each user type to the system functions they are responsible for.

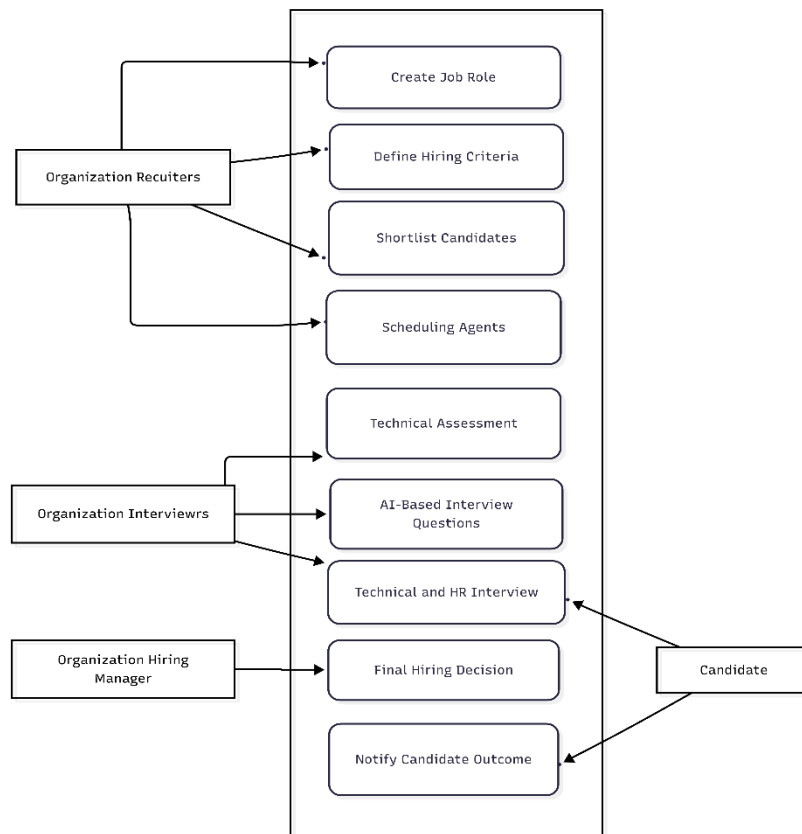


Figure 3.5: PATHS Stakeholder Use-Case and Role Responsibilities (RBAC View).

As shown in Figure 3.6. This diagram focuses on outbound engagement and converting interest into scheduled evaluations. HR first approves the shortlist for outreach, then the Contact Enrichment Agent identifies candidate contact channels and stores channel confidence in ATS/CRM. The Outreach Agent generates personalized messages using RAG by combining company/team context from the Organization Knowledge Base with candidate-specific evidence (projects/skills), then sends tracked multi-channel outreach and logs delivery status. If the candidate responds positively, the system proposes times and formats and the Scheduling Agent collects availability, confirms a slot, and saves meeting details and next steps back into ATS/CRM. If the candidate does not respond or messages bounce, the workflow marks the outcome for retry logic or alternative channel selection.

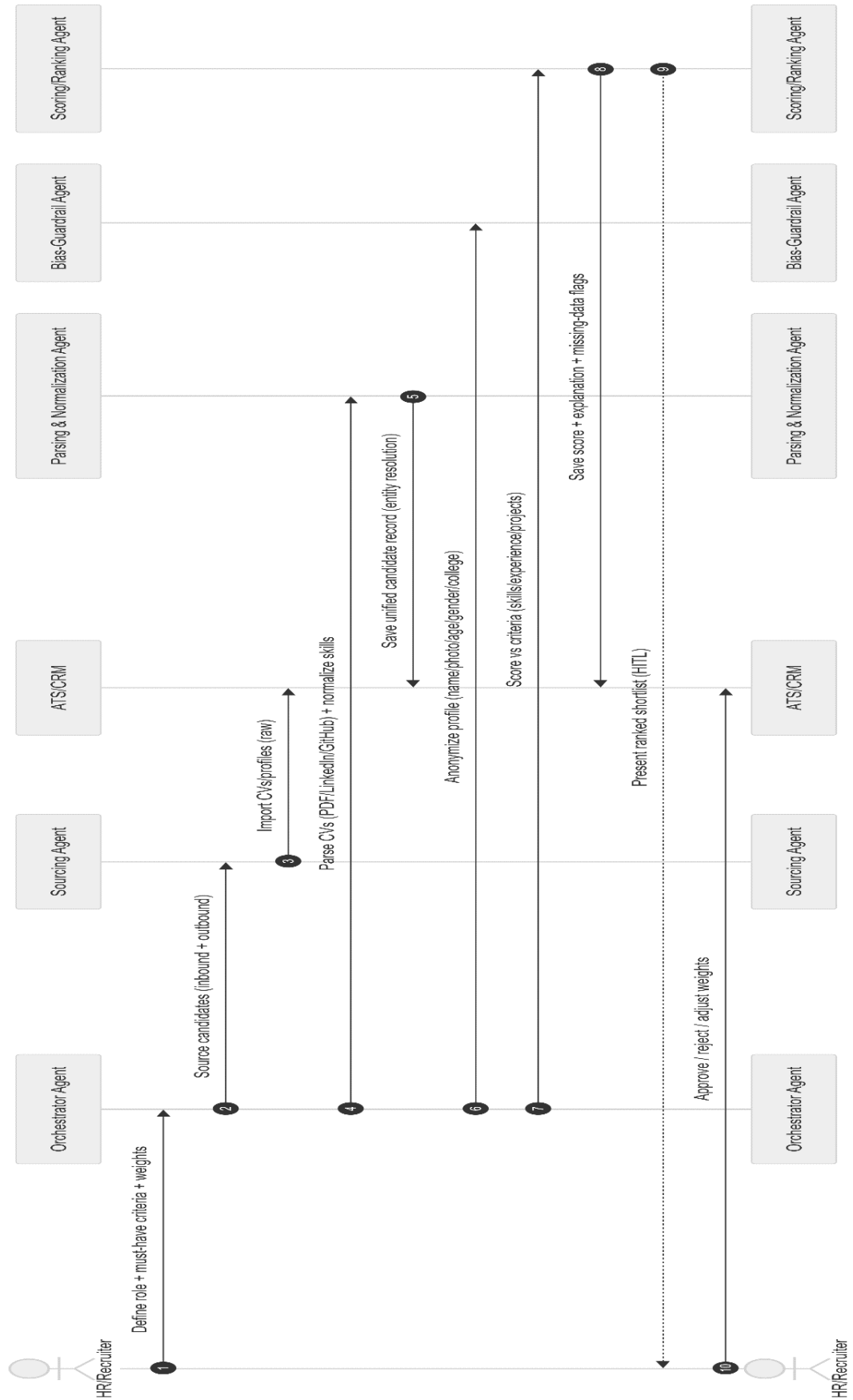


Figure 3.6: Candidate Shortlisting Orchestration with Bias Guardrails and Approval.

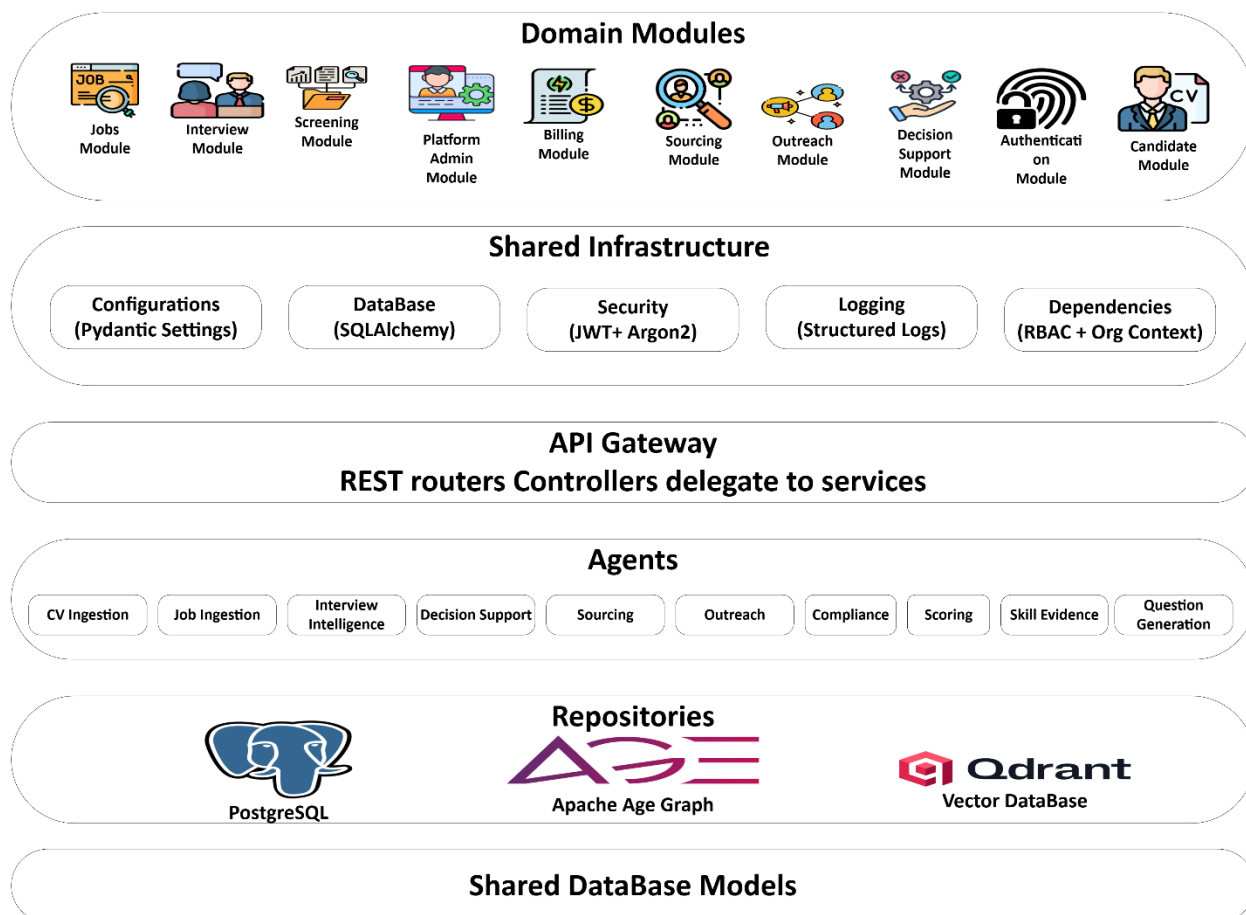


Figure 3.7: Layered modular architecture of PATHS showing the separation between business domain modules, shared infrastructure services, REST API controllers, AI agents, data repositories, and shared database models.

This Figure 3.7. represents the internal design architecture of the PATHS system. Unlike the implementation architecture, which focuses on deployment technologies such as Next.js, FastAPI, PostgreSQL, and Qdrant, this diagram explains how the backend is logically organized from a software design perspective.

The architecture follows a modular monolithic design. This means that PATHS is built as one unified backend application, but internally it is divided into clear modules, layers, and responsibilities. This design is suitable for the project because PATHS covers many recruitment functions, such as jobs, candidates, sourcing, interviews, screening, outreach, and decision support. Without modular separation, all these features would become mixed together and difficult to maintain. The report already describes PATHS as a system that combines sourcing, engagement, evaluation, and development into one end-to-end solution, while keeping recruiters and hiring managers in control of the AI-supported workflow.

At the top of the diagram, the Domain Modules represent the main business areas of the platform. Each module is responsible for a specific part of the recruitment lifecycle. For example, the Jobs Module manages job creation, job descriptions, requirements, and job status. The Candidate Module manages candidate profiles, CV data, applications, and candidate-related records. The Interview Module handles interview scheduling, interview information, and interview outputs. The Screening Module supports candidate screening and early evaluation. The Sourcing Module is responsible for finding or importing candidates, while the Outreach Module manages personalized communication with candidates. The Decision Support Module supports final hiring recommendations, summaries, and decision packages. The Authentication Module handles login, user identity, and secure access. The Platform Admin Module manages administrative operations, and the Billing Module is prepared for subscription or payment-related functionality.

The second layer is the Shared Infrastructure layer. This layer contains reusable services that are needed by almost every domain module. For example, Configurations using Pydantic Settings are used to manage environment variables and application settings in a structured way. Database using SQLAlchemy provides the connection between the application and the relational database. Security using JWT and Argon2 protects the system by authenticating users and hashing passwords securely. Logging using structured logs helps trace system behavior, errors, and important actions. Dependencies using RBAC and organization context allow the backend to check the current user role and organization before allowing access to sensitive actions. This is especially important in PATHS because different users, such as recruiters, candidates, hiring managers, organization admins, and platform admins, should not have the same permissions.

The API Gateway layer represents the REST API boundary of the backend. In this design, the API gateway is not a separate external service; it refers to the FastAPI routers and controllers that receive requests from the frontend. These controllers should not contain heavy business logic. Their role is to receive the request, validate the input, apply authentication and authorization, and then delegate the work to the appropriate service inside the relevant domain module. For example, when a recruiter requests candidate screening, the screening router receives the request, checks permissions, and calls the screening service rather than performing the full scoring process directly inside the route.

The Agents layer represents the intelligent AI-driven part of PATHS. This layer includes agents such as CV Ingestion, Job Ingestion, Interview Intelligence, Decision Support, Sourcing, Outreach, Compliance, Scoring, Skill Evidence, and Question Generation. These agents are responsible for tasks that require reasoning, extraction, ranking, generation, or decision support.

For example, the CV Ingestion Agent extracts useful information from candidate CVs, the Job Ingestion Agent analyzes job requirements, the Scoring Agent evaluates the fit between a candidate and a job, and the Question Generation Agent creates interview questions based on the job and candidate context. This supports the project's goal of using agentic workflows and RAG-based intelligence while still keeping human approval in the loop.

The Repositories layer shows the main data storage technologies used by the system. PostgreSQL is the main relational database and stores structured records such as users, organizations, jobs, candidates, applications, interviews, assessments, scores, and decisions. Apache AGE Graph is used to represent graph relationships, such as links between candidates, skills, jobs, companies, experiences, and evidence. This is useful because recruitment data is naturally connected, and a graph structure helps the system understand relationships more clearly. Qdrant Vector Database stores embeddings for semantic search and similarity matching. This allows PATHS to compare CVs, job descriptions, skills, and knowledge base content based on meaning, not only exact keyword matching.

At the bottom, the Shared Database Models layer represents the common data schema used across the system. These models define the main entities and relationships of PATHS, such as Candidate, Job, Organization, Application, Skill, Interview, Assessment, Score, Decision, and Audit Log. Having shared database models ensures that all modules work with consistent data definitions. For example, the Candidate Module, Screening Module, Interview Module, and Decision Support Module may all need candidate information, but they should all refer to the same candidate model rather than creating different versions of the same data.

A typical flow through this architecture starts when a recruiter performs an action from the frontend, such as creating a job or screening a candidate. The request reaches the REST API controller in the API Gateway layer. The controller checks the user's authentication, role, and organization context using the shared infrastructure layer. Then it forwards the request to the correct domain module. The domain service may call one or more agents, such as the Job Ingestion Agent, Scoring Agent, or Skill Evidence Agent. These agents retrieve and store information through the repositories layer using PostgreSQL, Apache AGE, and Qdrant. Finally, the result is returned back through the API to the frontend.

The main strength of this design is that it keeps PATHS organized and extensible. Each module has a clear responsibility, shared services are reused instead of duplicated, AI agents are separated from normal business logic, and data access is controlled through repositories. This makes the system easier to test, debug, and improve. It also supports future expansion; for example,

a new module for onboarding, learning paths, or analytics can be added without rewriting the whole backend.

In the context of the design and analysis chapter, this diagram is important because it explains how the system is structured before implementation details are introduced. It shows that PATHS is not just a collection of pages or AI prompts, but a well-organized software architecture where recruitment workflows, security, AI reasoning, data storage, and governance are separated into clear layers. This separation supports the main objective of PATHS: building a unified AI-assisted hiring system that reduces fragmentation while maintaining fairness, auditability, and human oversight.

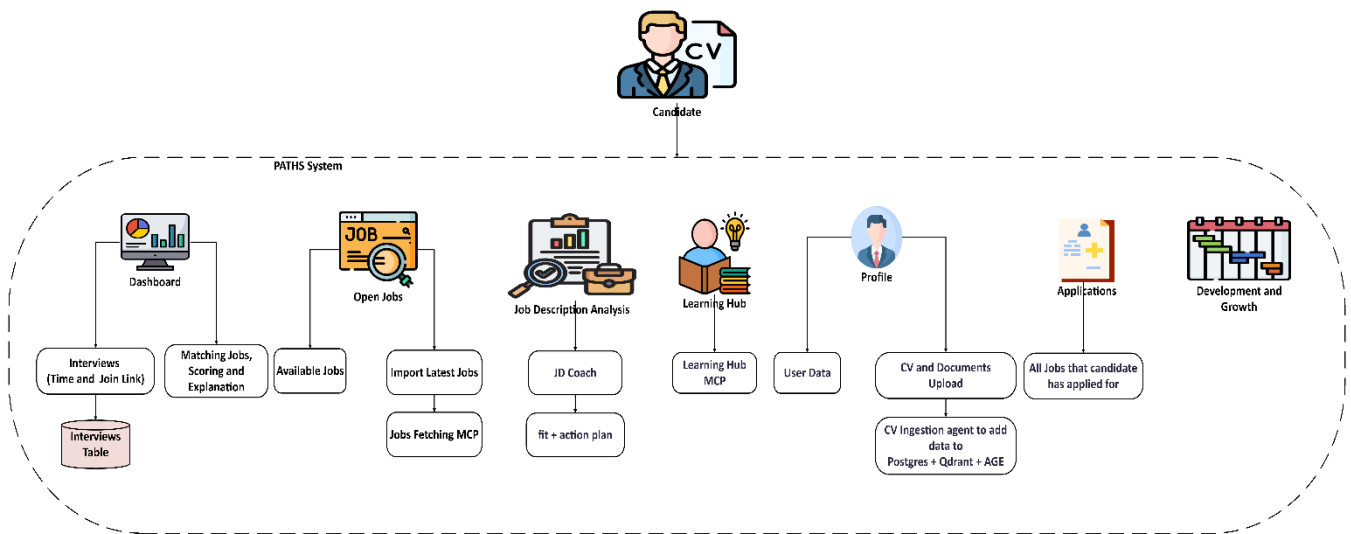


Figure 3.8: HR-side recruitment workflow in PATHS showing how recruiters manage dashboards, jobs, candidate sourcing, anonymized evaluation, interview preparation, assessments, and final hiring decisions with automated feedback and development plans.

The candidate-side design workflow in the system presented in Figure 3.8. It explains how a candidate interacts with the platform after logging in, and how the system supports the candidate across the main stages of the hiring journey: discovering jobs, uploading a CV, receiving matching recommendations, preparing for roles, attending interviews, and tracking applications.

At the top of the diagram, the candidate profile icon represents the candidate as the central user of this workflow. All features shown below are designed around the candidate's needs. Instead of treating the candidate as only a CV record inside the recruiter dashboard, PATHS gives the candidate a dedicated portal where they can manage their data, view opportunities, receive AI-supported recommendations, and follow their recruitment progress.

The first part of the workflow is the Interviews section. This allows the candidate to view upcoming interviews, including the scheduled time and the joining link. From a system design

perspective, this feature is connected to the Interviews Table, where interview records are stored. This ensures that whenever a recruiter or hiring manager schedules an interview, the candidate can see the updated interview information from the same source of truth. This avoids confusion caused by scattered email links or manual coordination.

The second feature is Matching Jobs, Scoring, and Explanation. This part shows the candidate which jobs are most suitable for their profile. The system does not only display job recommendations; it also provides a score and an explanation. This is important because candidates should understand why a job is recommended to them. For example, the explanation may show that the candidate is matched because of relevant skills, previous experience, project evidence, or similarity between the CV and the job description. This makes the recommendation more transparent and useful than a simple job list.

The Available Jobs section gives the candidate access to the open opportunities inside the platform. These are the jobs currently available for application. In the design, this feature represents the normal job browsing experience, where the candidate can explore roles, read job descriptions, and decide whether to apply. It works together with the matching feature, but it is slightly different: available jobs show the full opportunity pool, while matching jobs show the most relevant roles based on the candidate profile.

The Import Latest Jobs feature is connected to a Jobs Fetching MCP component. This means the system can fetch or update job opportunities from external or connected job sources. In the design chapter, this can be described as an integration component that helps keep job listings fresh. Instead of relying only on manually created jobs, PATHS can support importing or synchronizing job opportunities through a dedicated fetching service. This improves the candidate experience because the available jobs list becomes more dynamic and up to date.

The JD Coach is one of the intelligent candidate-support features in the diagram. JD stands for Job Description. The purpose of the JD Coach is to help the candidate understand a job description more clearly and compare it with their own profile. It is connected to the output called fit + action plan. This means the system can explain how suitable the candidate is for a specific job and what they should improve before applying. For example, it may tell the candidate that they match the Python and machine learning requirements, but need to improve cloud deployment or MLOps knowledge. This turns the job description from a static text into a personalized guidance tool.

The Learning Hub MCP represents the candidate development support part of the system. After the platform identifies skill gaps, the candidate can be guided toward learning resources,

courses, or development recommendations. This is important because PATHS is not only designed to help companies hire; it also aims to help candidates grow. In a humanized sense, the system does not simply reject or accept a candidate. It gives them feedback and a practical path to become stronger for future opportunities.

The User Data section represents the candidate's personal and profile information. This may include basic profile details, education, experience, skills, preferences, and other candidate-specific records. This part is important because most AI-supported features depend on the quality of the candidate profile. Better user data leads to better job matching, better JD coaching, better skill-gap analysis, and better interview preparation.

The CV and Documents Upload section allows the candidate to upload their CV and supporting documents. This is one of the most important entry points in the candidate workflow. The uploaded CV is not treated as a passive file only. Instead, it is sent to the CV Ingestion Agent, which extracts meaningful information from the document and adds it to the system's storage layers: PostgreSQL, Qdrant, and Apache AGE.

In this design, PostgreSQL stores the structured candidate data, such as name, skills, education, experience, applications, and interview records. Qdrant stores vector embeddings that support semantic matching between candidate profiles and job descriptions. Apache AGE stores graph-based relationships, such as the connection between candidates, skills, projects, jobs, companies, and evidence. This combination allows PATHS to understand the candidate from multiple perspectives: structured records, semantic similarity, and relationship-based knowledge.

The final part of the diagram is All Jobs that the Candidate Has Applied For. This section gives the candidate a clear history of their applications. The candidate can track which jobs they applied to and potentially see their status in the pipeline. This improves transparency because the candidate does not lose track of their progress after submitting applications. It also supports a more professional candidate experience, where the hiring journey feels organized rather than unclear.

Overall, this diagram shows that the candidate module in PATHS is not just a simple profile page. It is a complete candidate-support environment. The candidate can upload documents, build a stronger profile, discover available jobs, receive matching explanations, understand job descriptions, get an action plan, access learning support, track applications, and join interviews from one place.

From a design and analysis perspective, the strength of this workflow is that it connects candidate data, AI agents, job records, semantic search, graph relationships, and application tracking into one continuous experience. This supports the main goal of PATHS: reducing

fragmentation in recruitment while keeping the process transparent and useful for both recruiters and candidates.

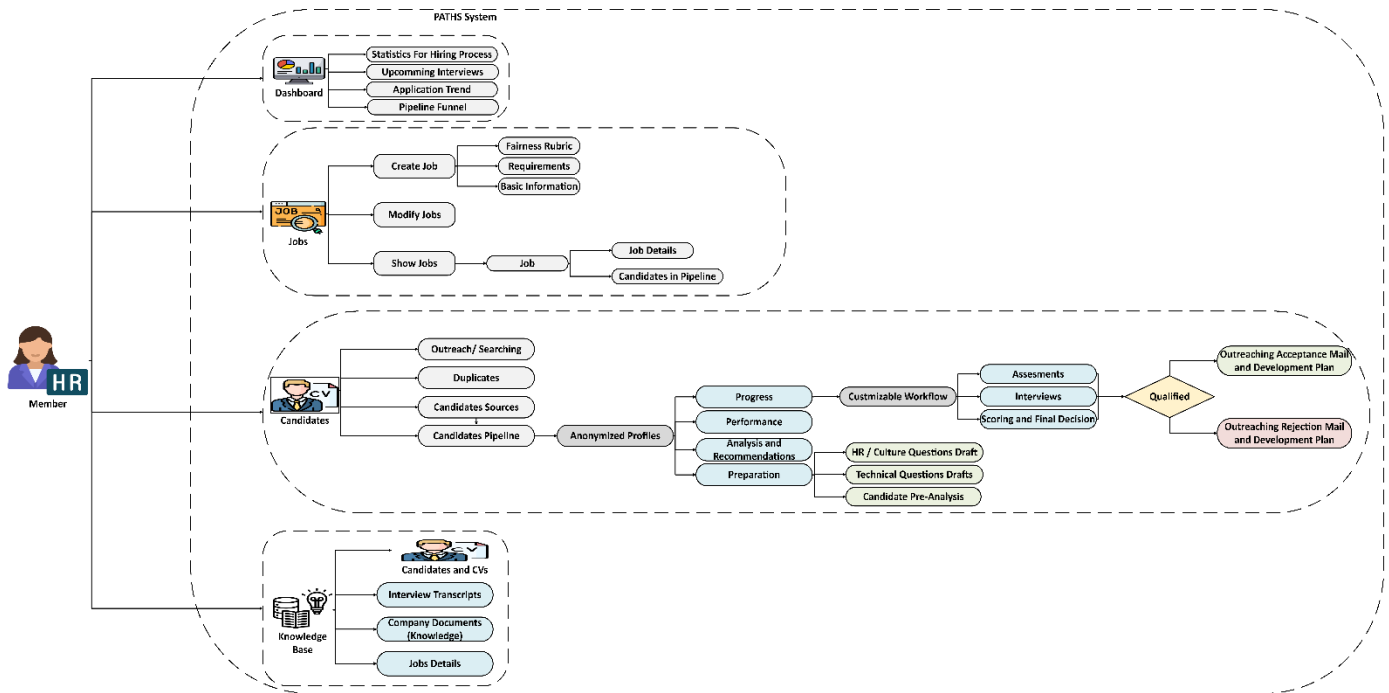


Figure 3.9: HR-side recruitment workflow in PATHS showing how recruiters manage dashboards, jobs, candidate sourcing, anonymized evaluation, interview preparation, assessments, and final hiring decisions with automated feedback and development plans.

The HR and recruiter workflow inside the PATHS system as shown in the Figure 3.9. It shows how the recruiter moves through the main hiring activities, starting from monitoring recruitment statistics, managing jobs, sourcing candidates, preparing interviews, evaluating candidates, and finally making a hiring decision. The purpose of this diagram is to explain the logical flow of the HR side before discussing the technical implementation.

The workflow starts from the HR user, who is the main actor in this part of the system. The HR user interacts with different modules in the platform, but all modules are connected around one goal: helping the recruiter manage the hiring process in a more organized, fair, and intelligent way.

At the top of the diagram, the first area represents the Recruitment Dashboard. This dashboard gives HR a quick overview of the hiring process through several indicators, such as statistics for the hiring process, upcoming interviews, application trends, and the pipeline funnel. This part is important because recruiters need a clear summary of what is happening across the organization. Instead of checking every job and candidate manually, the dashboard gives a high-level view of workload, progress, and hiring performance.

The next part is the Jobs Management flow. The recruiter can create jobs, modify jobs, and show jobs. When creating a job, the system collects important information such as the basic job information, requirements, and the fairness rubric. The fairness rubric is especially important in PATHS because it helps standardize how candidates will be evaluated. Instead of allowing every recruiter or hiring manager to judge candidates differently, the rubric defines the criteria that should be used during screening and decision-making. This supports fairness, consistency, and auditability.

After a job is created, it becomes part of the Job entity in the system. From there, the recruiter can access job details and view candidates in the pipeline. This means each job is not treated as a separate isolated record, but as a living hiring process connected to applicants, sourced candidates, screening outputs, interviews, and final decisions.

The middle section of the diagram focuses on candidate sourcing and pipeline management. The recruiter can perform outreach/searching, handle duplicates, manage candidate sources, and track the candidate pipeline. This is one of the most important parts of PATHS because real recruitment data often comes from many places, such as CV uploads, imported candidates, internal databases, LinkedIn-style sourcing, and previous applications. Without proper source management and duplicate detection, the same candidate may appear multiple times with different information. PATHS solves this by organizing candidate data into a cleaner pipeline.

After candidates are collected, they are converted into anonymized profiles. This is a key design decision in the system. Before evaluation, sensitive identity-related information can be hidden so that recruiters and hiring managers focus more on skills, experience, evidence, and job fit. This helps reduce bias and supports a more structured evaluation process.

From the anonymized profiles, the workflow moves into analysis. The system provides progress, performance analysis and recommendations, and preparation. This means PATHS does not simply store candidate profiles; it actively helps the recruiter understand where each candidate stands. The system can summarize candidate strengths, identify gaps, recommend next steps, and prepare the recruiter for interviews or decisions.

The system also includes a customizable workflow. This means the hiring process does not have to be identical for every organization or every job. Some roles may require a technical assessment, others may require HR screening first, and some may need culture-fit questions or additional manager review. PATHS supports this flexibility by allowing the workflow to adapt to the organization's hiring needs.

Connected to this workflow are several preparation outputs: HR/Culture Questions Draft, Technical Questions Drafts, and Candidate Pre-Analysis. These outputs are generated to help interviewers prepare better before meeting the candidate. For example, the system may generate technical questions based on the job requirements and the candidate's profile, or HR/culture questions based on the organization's values and the role expectations. Candidate pre-analysis gives the interviewer a clear summary before the interview starts.

At the bottom of the diagram, there is a knowledge support area containing interview transcripts, company documents, and job details. These items act as contextual knowledge for the system. Company documents can provide information about company culture, policies, role expectations, and internal requirements. Interview transcripts can be reused for summaries, evaluation evidence, and audit trails. Job details provide the role-specific context needed for scoring, question generation, and recommendations. This makes the AI support more grounded and relevant instead of producing generic outputs.

The next stage is the evaluation stage, which includes assessments, interviews, and scoring and final decision. This part represents the structured evaluation process. Candidates may complete assessments, attend interviews, and receive scores based on predefined criteria. The final score and decision are not meant to replace the recruiter; instead, they support the recruiter with evidence, explanations, and structured recommendations.

After evaluation, the workflow reaches the Qualified decision point. This is represented as a decision diamond because the process branches into two possible outcomes. If the candidate is qualified and selected, the system supports sending an outreach acceptance mail and development plan. If the candidate is not selected, the system supports sending an outreach rejection mail and development plan.

The development plan is an important part of PATHS. Even when a candidate is rejected, the system can still provide useful feedback and recommend areas for improvement. This makes the hiring process more human and valuable for candidates. Instead of ending with a generic rejection message, PATHS can provide a more constructive outcome based on the candidate's actual gaps and strengths.

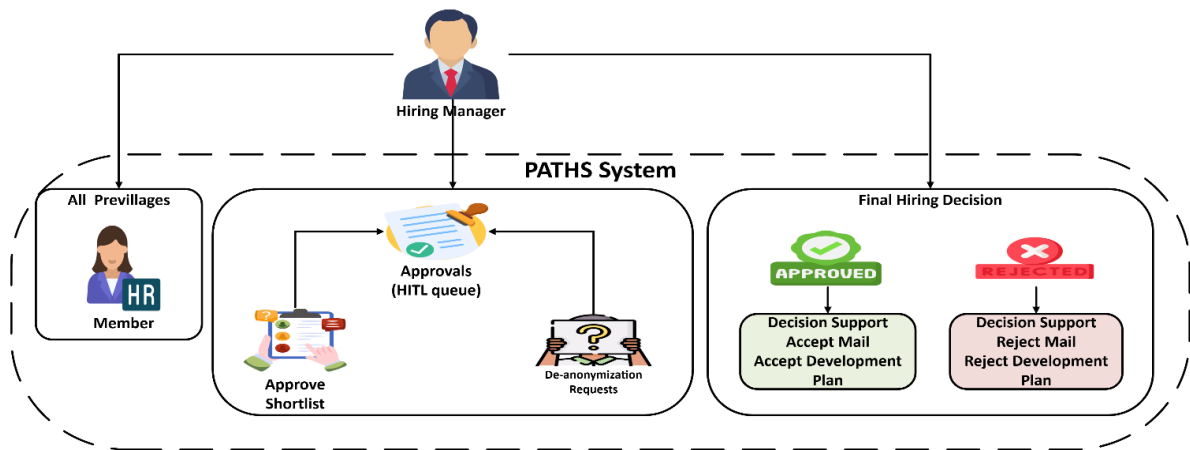


Figure 3.10: Human-in-the-loop approval and final hiring decision workflow, showing how HR actions, approval queues, de-anonymization requests, and decision-support outputs are controlled before candidate decision

Human-in-the-loop decision workflow in the system as shown in Figure 3.10. It explains how the system prevents fully automated hiring decisions by keeping HR and hiring managers involved at the most sensitive points of the recruitment process: shortlist approval, candidate de-anonymization, and the final hiring decision.

At the top of the diagram, the manager icon represents the hiring manager or decision-maker. This actor is responsible for reviewing important recruitment outputs before the system moves forward. The HR user appears on the left side because HR initiates or manages many of the workflow actions, but the final approval process is designed to involve human supervision rather than allowing the AI system to act alone.

The first main area in the figure is the Approvals Queue, also called the HITL queue. HITL stands for Human-in-the-Loop. This queue acts as a checkpoint where sensitive AI-assisted outputs are paused until a responsible human reviews them. In PATHS, this is important because recruitment decisions affect people’s careers and must be handled carefully, transparently, and fairly.

One of the actions inside this approval area is Approve Shortlist. After the system screens and ranks candidates, it may generate a shortlist of the most suitable candidates for a specific job. However, this shortlist is not automatically accepted as the final answer. Instead, HR or the hiring manager reviews it first. This allows the human user to check whether the shortlist makes sense, whether the explanations are reasonable, and whether any candidate should be added, removed, or reviewed again.

The second action is De-anonymization Requests. In PATHS, candidate profiles can be anonymized during evaluation to reduce bias. This means information such as names or identity-related details may be hidden while the candidate is being assessed. If HR needs to reveal the

candidate's identity later, for example before contacting the candidate or moving to a later hiring stage, a de-anonymization request is sent for approval. This prevents unnecessary exposure of candidate identity and ensures that de-anonymization happens only when there is a valid hiring reason.

The right side of the figure shows the Final Hiring Decision stage. At this point, the candidate has already passed through previous stages such as screening, assessment, interview preparation, interviews, and scoring. The final decision has two possible outcomes: Approved or Rejected.

If the candidate is Approved, the system supports the hiring team by generating a Decision Support Accept Mail and an Accept Development Plan. The acceptance email can be personalized based on the role, candidate profile, and evaluation results. The development plan is also important because it helps the organization start the candidate's journey with a clear understanding of strengths, gaps, and recommended growth areas.

If the candidate is Rejected, the system supports the team by generating a Decision Support Reject Mail and a Reject Development Plan. This is a humanized part of the PATHS workflow. Instead of sending a cold rejection message, the system can help provide meaningful feedback and future improvement suggestions. This improves the candidate experience and keeps the relationship professional, even when the candidate is not selected.

The most important idea in this diagram is that PATHS uses AI as a decision-support tool, not as an automatic decision-maker. The AI can recommend, summarize, explain, generate emails, and prepare development plans, but the final decision remains under human control. This is especially important in recruitment because hiring decisions involve fairness, privacy, legal responsibility, and ethical judgment.

From a system design perspective, this workflow also improves auditability. Every approval, shortlist confirmation, de-anonymization request, and final decision can be recorded in the system. This means the organization can later review who approved a step, when it happened, and what information was used. This makes the hiring process more transparent and easier to justify.

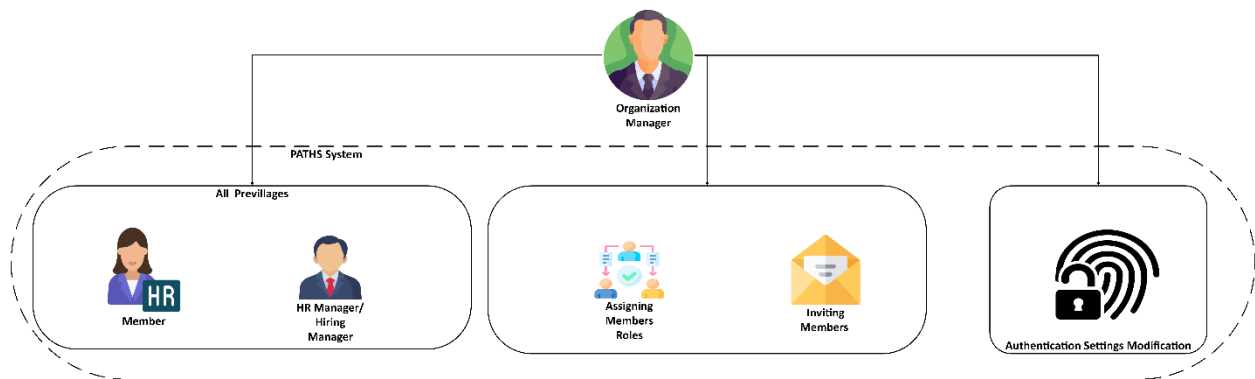


Figure 3.11: Organization administration workflow in PATHS showing how the admin manages HR users, hiring managers, member invitations, role assignment, email communication, and authentication settings.

The Figure 3.11. represents the administration and user-management workflow in the PATHS system. It explains how the organization admin manages the people who use the platform and controls how they access the recruitment system. In PATHS, this part is important because the system is not used by one user only. It is used by several stakeholders, such as HR recruiters, hiring managers, organization administrators, and candidates. Each user must have the correct access level according to their responsibility.

At the top of the diagram, the main user represents the Organization Admin or the user responsible for managing the organization workspace. This admin acts as the controller of the internal hiring team. Their role is not necessarily to perform every recruitment task, but to make sure that the correct people are added to the system, assigned to the correct roles, and given secure access to the appropriate modules.

On the left side, the diagram shows the HR user. This represents recruiters or HR team members who use PATHS to manage jobs, candidates, screening, interviews, outreach, and pipeline activities. The admin can create or manage HR accounts so that recruiters can access the hiring tools they need. However, HR users should not automatically have full system control. Their permissions should be limited to recruitment operations, depending on the organization’s policy.

The second user icon represents the Hiring Manager. Hiring managers are usually responsible for reviewing shortlisted candidates, approving important decisions, participating in interviews, and giving the final judgment on whether a candidate fits the role or team. In PATHS, the hiring manager may have access to candidate evaluations, interview summaries, decision-support reports, and approval requests, but not necessarily to all administration settings.

The middle icon represents member management and role assignment. This is where the admin can organize users inside the organization. The admin may add new users, assign them roles such as HR, Hiring Manager, or Admin, and control what each user can do inside the platform. This supports Role-Based Access Control (RBAC), which means that the system gives each user

permissions based on their role. For example, an HR user may create jobs and manage candidates, while a hiring manager may approve shortlists and make final hiring decisions.

The envelope icon represents email-based invitations and notifications. When the admin adds a new HR user or hiring manager, the system can send an invitation email. This email may include an invitation link, login instructions, or temporary access details. This makes onboarding new organization members easier and more professional. It also reduces manual communication because the platform itself handles the invitation process.

On the right side, the authentication icon represents Authentication Settings Modification. This part refers to the security settings that control how users log in and how access is protected. These settings may include password rules, authentication configuration, account activation, access control, and possibly future support for stronger login mechanisms. In the design chapter, this shows that user management is not only about adding people; it is also about protecting the system from unauthorized access.

The main idea of this workflow is that PATHS separates recruitment actions from administrative control. HR users and hiring managers use the platform to perform hiring tasks, but the admin manages the organizational structure behind them. This separation is important because recruitment data is sensitive. Candidate CVs, evaluations, interview notes, scores, and decisions should only be visible to the right users.

From a system design perspective, this workflow supports three important principles: security, accountability, and controlled collaboration. Security is achieved by authentication settings and role-based permissions. Accountability is achieved because every user has a clear role inside the system. Controlled collaboration is achieved because HR users and hiring managers can work together on the same hiring process without having identical access rights.

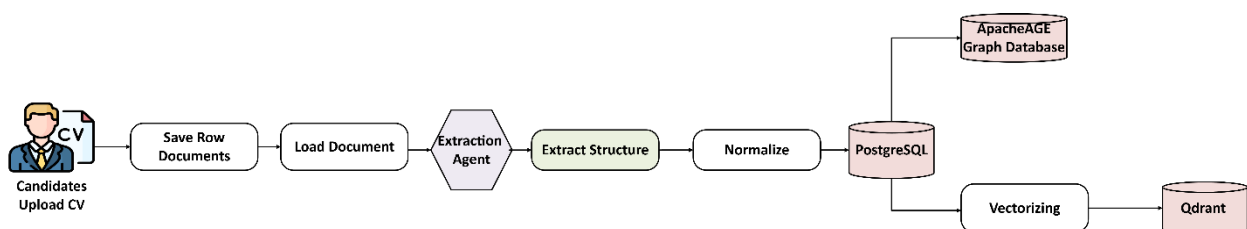


Figure 3.12: CV and document ingestion pipeline in PATHS showing how uploaded candidate documents are stored, loaded, extracted, normalized, and indexed across PostgreSQL, Apache AGE graph database, and Qdrant vector database.

This figure represents the CV and document ingestion workflow in PATHS. It explains how the system handles a candidate's uploaded CV or supporting documents and transforms them from unstructured files into structured, searchable, and usable recruitment data.

The workflow starts with the candidate CV/document upload. At this stage, the candidate provides a CV, resume, portfolio file, certificate, or any other supporting document. These files usually contain important information such as skills, education, work experience, projects, certifications, tools, and achievements. However, the information is usually unstructured because every CV has a different format. Therefore, the system cannot directly use the uploaded file for matching or scoring until it is processed.

The first step is Save Raw Documents. In this stage, the system stores the original uploaded file before making any changes to it. This is important because the raw document acts as the original source of truth. If the extraction process needs to be repeated later, or if the recruiter wants to review the original CV, the system can always return to the uploaded version. This also supports traceability because the structured profile can be linked back to the original document.

After saving the file, the system moves to Load Document. This step reads the uploaded file and prepares it for processing. Depending on the document type, the loader may extract text from formats such as PDF, DOCX, or plain text. The purpose of this stage is to convert the file into machine-readable content that can be passed to the extraction agent.

The next stage is the Extraction Agent. This agent is responsible for understanding the content of the document and identifying the important candidate information inside it. For example, it may detect the candidate's skills, previous job titles, education history, years of experience, projects, certifications, languages, tools, and contact-related fields. This agent is important because CVs are not always written in the same structure. One candidate may list skills in a table, another may describe them inside project descriptions, and another may place them in a summary paragraph. The extraction agent helps convert these different formats into a consistent structure.

After that, the workflow moves to Extract Structure. This means the extracted information is organized into defined fields. Instead of keeping the CV as one long text, the system separates it into meaningful sections such as skills, education, experience, projects, certifications, and summary. This structured representation makes the candidate profile easier to store, search, compare, and evaluate.

The next step is Normalize. Normalization is needed because candidates may describe the same concept in different ways. For example, one CV may mention "Machine Learning," another

may say “ML,” and another may mention “predictive modeling.” The normalization process attempts to standardize these terms so the system can compare candidates fairly and consistently. It can also clean duplicate skills, standardize dates, unify job titles, and convert extracted information into the format expected by the database.

After normalization, the processed data is stored in PostgreSQL. PostgreSQL is used for structured and transactional data. It stores the candidate profile, extracted skills, education records, work experience, document metadata, and other reliable system records. This allows the candidate module, recruiter module, screening module, and decision-support module to access the same clean candidate information.

The diagram also shows Apache AGE Graph Database. This graph layer is used to represent relationships between entities. For example, a candidate can be connected to skills, skills can be connected to jobs, projects can provide evidence for skills, and companies can be connected to previous experiences. This is useful because recruitment is not only about isolated fields; it is about relationships. The graph database helps PATHS understand how candidate evidence, skills, roles, and job requirements are connected.

The final AI-related step is Vectorizing. In this stage, the system converts the extracted text or structured candidate information into numerical embeddings. These embeddings represent the meaning of the candidate’s CV and profile in a way that can be compared semantically with job descriptions, requirements, and knowledge base documents.

The embeddings are then stored in Qdrant, which is the vector database. Qdrant allows PATHS to perform semantic search and similarity matching. This means the system can find candidates who are conceptually relevant to a job even if the exact keywords are different. For example, a job may require “backend API development,” while a candidate writes “built REST services using FastAPI.” Traditional keyword search may miss this connection, but vector similarity can capture the relationship more effectively.

This pipeline turns a simple uploaded CV into three useful forms of data: structured records in PostgreSQL, relationship-based knowledge in Apache AGE, and semantic embeddings in Qdrant. This makes the candidate profile usable for job matching, skill evidence analysis, scoring, interview preparation, and decision support.

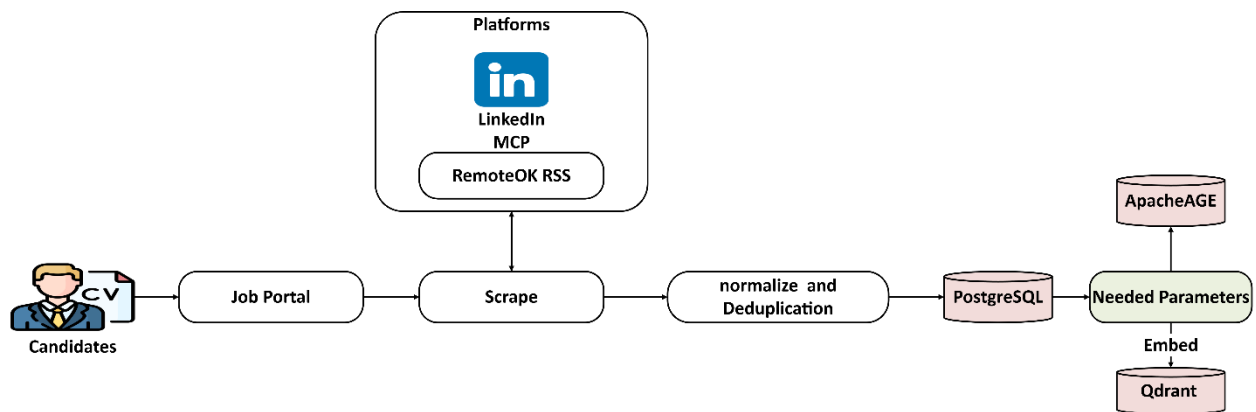


Figure 3.13: External job sourcing and ingestion workflow in PATHS showing how jobs are collected from job portals, LinkedIn-style sources, and RemoteOK RSS, then scraped, normalized, deduplicated, stored, and indexed in PostgreSQL, Apache AGE, and Qdrant.

The Figure 3.13. represents the external job sourcing and ingestion workflow in PATHS. It explains how the system collects job opportunities from external sources, processes them, removes duplicates, and prepares them for candidate matching and recommendation.

The workflow starts from the job portal / external source side. In PATHS, job data can come from different places, such as job portals, LinkedIn-style sources, and RSS-based sources like RemoteOK RSS. The purpose of this step is to keep the platform updated with recent job opportunities instead of relying only on manually created jobs inside the system.

After identifying the job source, the next step is Scrape. This step means that the system retrieves job information from the selected source. The collected data may include job title, company name, location, job type, required skills, experience level, salary range, job description, application link, and posting date. In a real implementation, this should be done through legal and reliable methods such as official APIs, RSS feeds, approved integrations, or scraping that respects the source's terms of service and privacy requirements.

Once the jobs are collected, they pass through the normalization and deduplication stage. This is one of the most important parts of the pipeline because external job data is usually messy. Different websites may write the same skill or role in different formats. For example, one source may write "ML Engineer," another may write "Machine Learning Engineer," and another may write "AI/ML Engineer." Normalization helps convert these variations into a consistent format that the system can understand.

Deduplication is also needed because the same job may appear on more than one platform. For example, a company may post the same role on LinkedIn, RemoteOK, and another job board. Without deduplication, the candidate may see repeated jobs, and the matching system may produce

inaccurate results. Therefore, PATHS compares job title, company, location, description similarity, and application links to detect and merge duplicate jobs.

After cleaning the data, the structured job information is stored in PostgreSQL. PostgreSQL acts as the main database for reliable job records. It stores fields such as job title, company, source, location, requirements, description, posting date, and status. This makes the job available to the candidate portal, recruiter dashboard, matching module, and analytics module.

The diagram also shows Needed Parameters. This means that after the job is stored, the system extracts the most important parameters required for matching and recommendation. These parameters may include required skills, preferred skills, years of experience, seniority level, job category, employment type, location, remote/hybrid/on-site status, and keywords from the job description. These parameters are later used by the recommendation and scoring components.

The system then uses embedding to convert the job description and extracted requirements into vector representations. These embeddings capture the meaning of the job beyond exact keywords. For example, if a job requires “building REST APIs using FastAPI,” the system can understand that this is related to backend engineering, Python, API development, and web services.

The embeddings are stored in Qdrant, the vector database. Qdrant allows PATHS to perform semantic matching between candidate profiles and job descriptions. This means candidates can be matched with jobs based on meaning and relevance, not only direct keyword overlap. This improves the quality of recommendations, especially when candidates and job descriptions use different wording for the same skills.

The workflow also connects to Apache AGE, the graph database layer. Apache AGE can represent relationships between jobs, skills, companies, industries, candidates, and required experiences. For example, a job can be connected to several skills, each skill can be connected to related skills, and candidates can be connected to the same skill nodes through their CVs or projects. This graph structure helps PATHS reason about job requirements in a more connected way. This workflow transforms external job postings into clean, structured, searchable, and matchable job records. It supports the candidate side by keeping the available jobs page updated, and it supports the AI matching system by preparing jobs in a format that can be compared against candidate profiles.

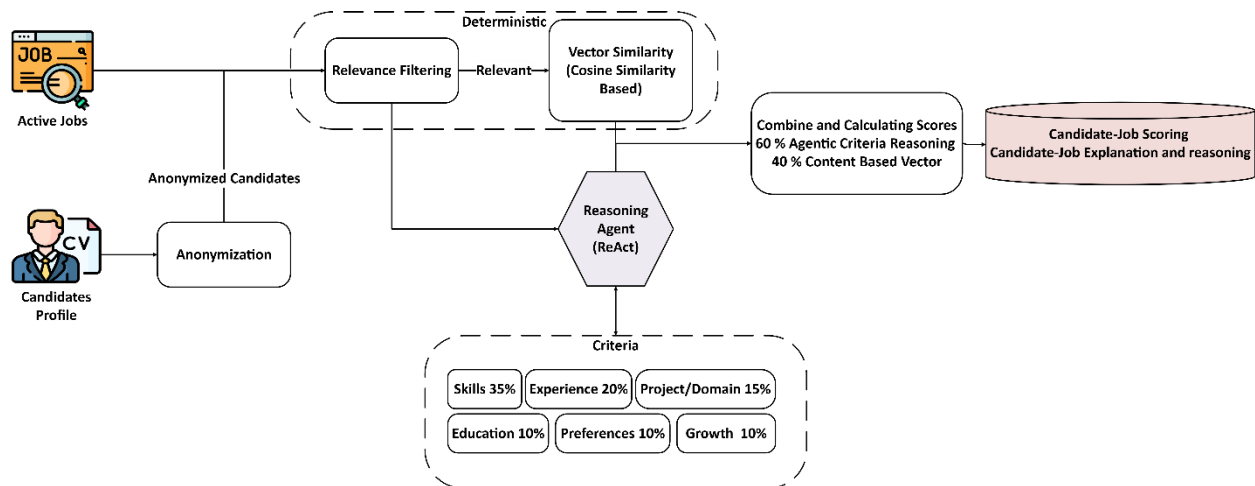


Figure 3.14: Candidate-job scoring pipeline in PATHS showing anonymization, relevance filtering, vector similarity, ReAct reasoning, weighted evaluation criteria, and the final explainable candidate-job score.

The Figure 3.14. shows the candidate-job scoring workflow in PATHS. It explains how the system evaluates whether a candidate is suitable for a specific job while keeping the process fair, explainable, and based on both semantic similarity and structured reasoning.

The workflow starts with two main inputs: the job and the candidate profile. The job contains the requirements, responsibilities, required skills, preferred qualifications, experience level, and role context. The candidate profile contains information extracted from the CV, profile data, projects, education, skills, and other supporting documents.

Before the candidate is evaluated, the system applies anonymization. This step removes or hides identity-related information from the candidate profile, such as name, gender indicators, photo, contact details, and other personal identifiers. The purpose is to reduce bias during the early evaluation stage. In other words, the system focuses on what the candidate can do, not who the candidate is.

After anonymization, the anonymized candidate profile is passed into the scoring pipeline. The first technical step is relevance filtering. This acts as a gate before deeper scoring happens. The goal is to quickly check whether the candidate is generally relevant to the job or not. For example, if the job is for an AI Engineer and the candidate profile has no connection to AI, programming, data, machine learning, or related fields, the system can mark the profile as not relevant and avoid unnecessary scoring.

If the candidate passes the relevance filter, the workflow continues to vector similarity. This part is based on cosine similarity, which compares the semantic meaning of the candidate profile with the job description. The system converts both the job and candidate information into embeddings, then measures how close they are in meaning. This is useful because candidates and job descriptions may use different words to describe the same concept. For example, a job may

say “LLM application development,” while the candidate may write “built RAG systems using LangChain and vector databases.” Vector similarity helps detect that these are strongly related.

In parallel, the workflow uses a Reasoning Agent based on ReAct. ReAct means the agent reasons step by step and can use available tools or retrieved information before producing a judgment. In this diagram, the reasoning agent does not simply generate a random opinion. It evaluates the candidate using clear weighted criteria.

The scoring criteria are divided into six parts:

1. Skills-35%

This is the most important criterion. It measures how well the candidate’s technical and professional skills match the job requirements. For example, if the role requires Python, FastAPI, LangChain, PostgreSQL, and vector databases, the system checks whether these skills exist in the candidate profile and whether there is evidence supporting them.

2. Experience-20%

This criterion evaluates the candidate’s previous work experience, internships, projects, or practical exposure. It checks whether the candidate has worked on similar tasks, similar domains, or similar responsibilities before.

3. Project/Domain-15%

This part focuses on the relevance of the candidate’s projects and domain background. For technical roles, projects are very important because they provide evidence of practical ability. A candidate with strong projects in the same domain may receive a higher score even if their years of experience are limited.

4. Education-10%

This criterion evaluates the candidate’s academic background and whether it supports the role. It is not the only factor, but it provides additional context, especially for early-career candidates.

5. Preferences-10%

This part considers alignment between the job and the candidate’s preferences, such as location, remote/hybrid/on-site work, job type, seniority level, and career interests. A technically strong candidate may still be a weak match if the role does not fit their preferences.

6. Growth-10%

This criterion estimates the candidate’s growth potential. It considers whether the candidate shows learning ability, adaptability, project progression, or evidence that they can close

small gaps with guidance. This is important because PATHS is not designed only to find the “perfect” candidate; it also supports identifying candidates who can grow into the role.

After calculating the reasoning-based criteria, the system combines them with the vector similarity score. The final score is calculated using a blended approach: 60% agentic criteria reasoning and 40% content-based vector similarity. This combination is important because each method has a different strength. The reasoning agent gives structured, explainable judgment across skills, experience, projects, education, preferences, and growth. The vector similarity adds semantic matching and helps detect meaning-based similarity between the job and candidate.

The final output is stored as Candidate-Job Scoring, along with the candidate–job explanation and reasoning. This means the system does not only produce a number. It also explains why the candidate received that score. For example, it may say that the candidate scored highly because they have strong Python and ML experience, relevant projects, and good domain alignment, but lost points due to missing cloud deployment experience.

This explanation is very important for transparency. Recruiters and hiring managers should not see only “82% match.” They should understand what caused the score, what the candidate’s strengths are, and where the gaps exist. This makes the scoring system more useful, more trustworthy, and easier to review.

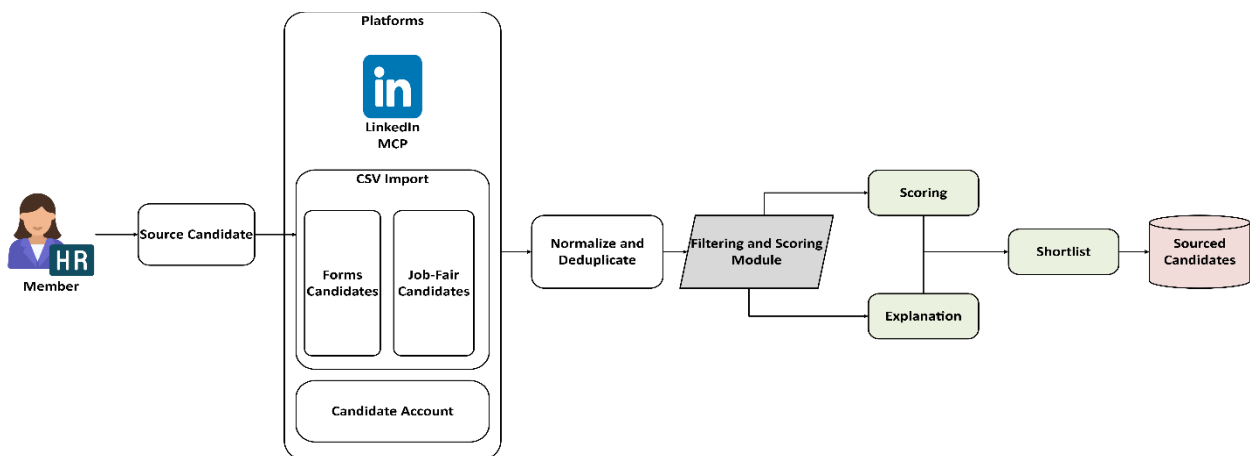


Figure 3.15: Candidate sourcing and shortlisting workflow in PATHS showing how HR imports candidates from multiple sources, creates candidate accounts, normalizes and deduplicates records, then applies filtering, scoring, and explanation before adding candidates to the sourced-candidates list.

The Figure 3.15. shows the candidate sourcing workflow in PATHS. It explains how HR users can bring candidates into the system from different sources, clean and organize their data, evaluate their fit for a job, and finally produce a shortlist of sourced candidates.

The workflow starts with the HR user, who initiates the sourcing process by selecting the Source Candidate function. This step represents the recruiter's need to search for potential candidates beyond only the people who directly applied to a job. In real recruitment, good candidates may come from different places, such as job fairs, CSV files, LinkedIn-style sourcing, manual forms, or previous candidate records. Therefore, PATHS allows the recruiter to collect candidates from multiple entry points instead of depending on one fixed source.

One of the main input methods shown in the diagram is CSV Import. This allows HR to upload a structured file containing candidate information. The CSV may include fields such as candidate name, email, phone number, skills, education, experience, source, and the job they are being considered for. This is useful when recruiters already have candidate lists from job fairs, events, spreadsheets, or external sourcing campaigns.

Inside the CSV import area, the diagram shows two types of candidate sources: Form Candidates and Job-Fair Candidates. Form candidates are candidates who submit their data through an online form. Job-fair candidates are candidates collected during physical or virtual recruitment events. This distinction is important because the same platform can support both digital sourcing and real-world recruitment activities.

The diagram also includes LinkedIn-style sourcing through the LinkedIn icon. This represents external professional profile sourcing, where recruiters may discover potential candidates from professional networks or open-to-work style sources. In the design, this source is treated as another input channel that can feed candidate data into the platform. The goal is to support outbound recruitment, not only inbound applications.

After candidates are imported or collected, the system creates or updates a Candidate Account. This means that a candidate is not stored only as a temporary row in a file. Instead, PATHS converts the candidate into a system-level record that can later be used for matching, screening, outreach, interviews, and decision support. If the candidate already exists, the system should update the existing record rather than create a duplicate account.

The next stage is Normalize and Deduplicate. This is a critical step because candidate data from different sources is often messy and repeated. For example, the same candidate may appear once in a CSV file, once from a job fair form, and once from an external profile. The system must detect possible duplicates using fields such as email, phone number, name similarity, LinkedIn/profile URL, and CV information. Normalization also standardizes the data format. For example, "Machine Learning," "ML," and "machine-learning" should be treated consistently. This improves the quality of later matching and scoring.

After the candidate records are cleaned, they are passed to the Filtering and Scoring Module. This module checks whether each candidate is relevant to the selected job or sourcing target. It may filter out candidates who are clearly unrelated, incomplete, or below the minimum requirements. Candidates who pass the relevance check are then scored based on their fit with the job. The scoring output is shown in two parts: Scoring and Explanation. The scoring part gives a numerical or ranked evaluation of how suitable the candidate is. The explanation part is equally important because recruiters should understand why the system recommends a candidate. For example, the system may explain that a candidate is recommended because they match the required Python and machine learning skills, have relevant project experience, and meet the job seniority level. It may also mention missing skills or weaker areas.

After scoring and explanation, the system produces a Shortlist. This shortlist contains the strongest candidates based on the filtering and scoring process. However, the shortlist should support recruiter review rather than act as an automatic final decision. HR can inspect the explanations, compare candidates, and decide who should move forward in the pipeline.

The final output is stored as Sourced Candidates. These are candidates who were discovered or imported through sourcing activities and are now available inside PATHS for further recruitment actions. They can later be contacted through outreach, moved to the candidate pipeline, matched to jobs, scheduled for interviews, or reviewed by hiring managers.

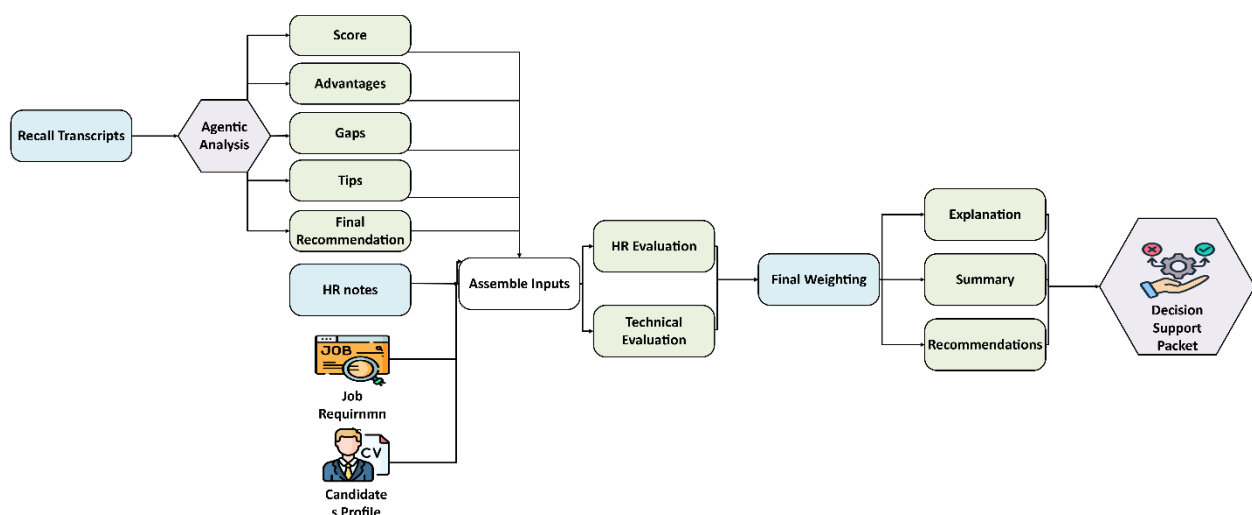


Figure 3.16: Interview analysis and decision-support workflow in PATHS showing how interview transcripts, HR notes, job requirements, candidate data, and agentic analysis are assembled into HR and technical evaluations, weighted, and converted into a final decision-support packet.

The flow of the interview analysis and decision-support as shown in the Figure 3.16. It explains how the system transforms interview data and candidate information into a structured evaluation package that can help HR and hiring managers make a fair and informed decision.

The workflow starts with Recall Transcripts, which represent the interview transcript generated from the interview session or note-taking component. Instead of depending only on memory or manual notes, PATHS can use the transcript as an evidence source. This is important because interviews often contain valuable details about the candidate's technical knowledge, communication style, problem-solving approach, confidence, and role understanding.

The transcript is passed to an Agentic Analysis component. This agent analyzes the interview content and extracts useful evaluation outputs. It does not only summarize the conversation; it breaks the interview into meaningful decision elements.

The agentic analysis produces several outputs:

1. Score

This represents the candidate's performance score based on the interview. The score can reflect how well the candidate answered questions, explained concepts, solved problems, or demonstrated role-related understanding.

2. Advantages

This section highlights the candidate's strengths. For example, the candidate may show strong technical understanding, clear communication, relevant project experience, or good alignment with the role.

3. Gaps

This section identifies weaknesses or missing areas. For example, the candidate may lack experience in a required tool, show limited depth in a certain technical topic, or need more practice in practical problem-solving.

4. Tips

This part provides helpful guidance, either for the recruiter, hiring manager, or candidate. It may suggest what to ask in the next stage, what to verify, or what the candidate should improve.

5. Final-Recommendation

This is the agent's overall recommendation based on the interview analysis. It can suggest whether the candidate should move forward, needs further review, or is not currently suitable for the role.

After the analysis stage, the system combines this information with other important inputs. These include HR Notes, Job Details, and Candidate Profile/CV Data. This is important because a good hiring decision should not depend on the interview transcript alone. The system needs to understand the job requirements, the candidate's background, and the human recruiter's observations.

The next step is Assemble Inputs. This stage collects all available evidence into one evaluation context. It brings together the transcript analysis, HR notes, job requirements, candidate profile, CV evidence, assessment results, and any previous screening outputs. This makes the final evaluation more complete and prevents decisions from being based on isolated information. After assembling the inputs, the workflow separates the evaluation into two main tracks:

6. HR-Evaluation

This focuses on the non-technical and role-fit aspects of the candidate. It may include communication, professionalism, motivation, culture fit, availability, expectations, attitude, and general suitability for the organization.

7. Technical-Evaluation

This focuses on the candidate's technical ability. It may include programming knowledge, AI/ML skills, backend/frontend knowledge, problem-solving, project depth, domain expertise, and ability to meet the technical requirements of the job.

The results from HR evaluation and technical evaluation are then passed into Final Weighting. This step combines the different evaluation parts according to the importance of each criterion. For example, for a technical AI Engineer role, the technical evaluation may have a higher weight, while for an HR or customer-facing role, communication and culture fit may have stronger influence. This makes the system flexible and adaptable to different job types.

After final weighting, the system generates three main decision-support outputs:

8. Explanation

This explains why the system reached a certain evaluation result. It shows the reasoning behind the score and recommendation, making the decision more transparent for HR and hiring managers.

9. Summary

This gives a short and clear overview of the candidate's performance, strengths, weaknesses, and overall suitability.

10. Recommendations

This provides suggested next steps, such as moving the candidate forward, requesting another interview, assigning an additional assessment, accepting the candidate, rejecting the candidate, or preparing a development plan.

All outputs are combined into the Decision Support Packet. This packet is the final structured report given to HR or the hiring manager. It is not meant to automatically hire or reject the candidate. Instead, it supports the human decision-maker with organized evidence, clear reasoning, and practical recommendations. The main value of this workflow is that it makes interview evaluation more consistent and evidence-based. Traditional interviews can be subjective because different interviewers may focus on different things or forget details after the meeting. PATHS reduces this problem by using transcripts, structured analysis, HR notes, job requirements, and weighted evaluation to create a more balanced view of the candidate.

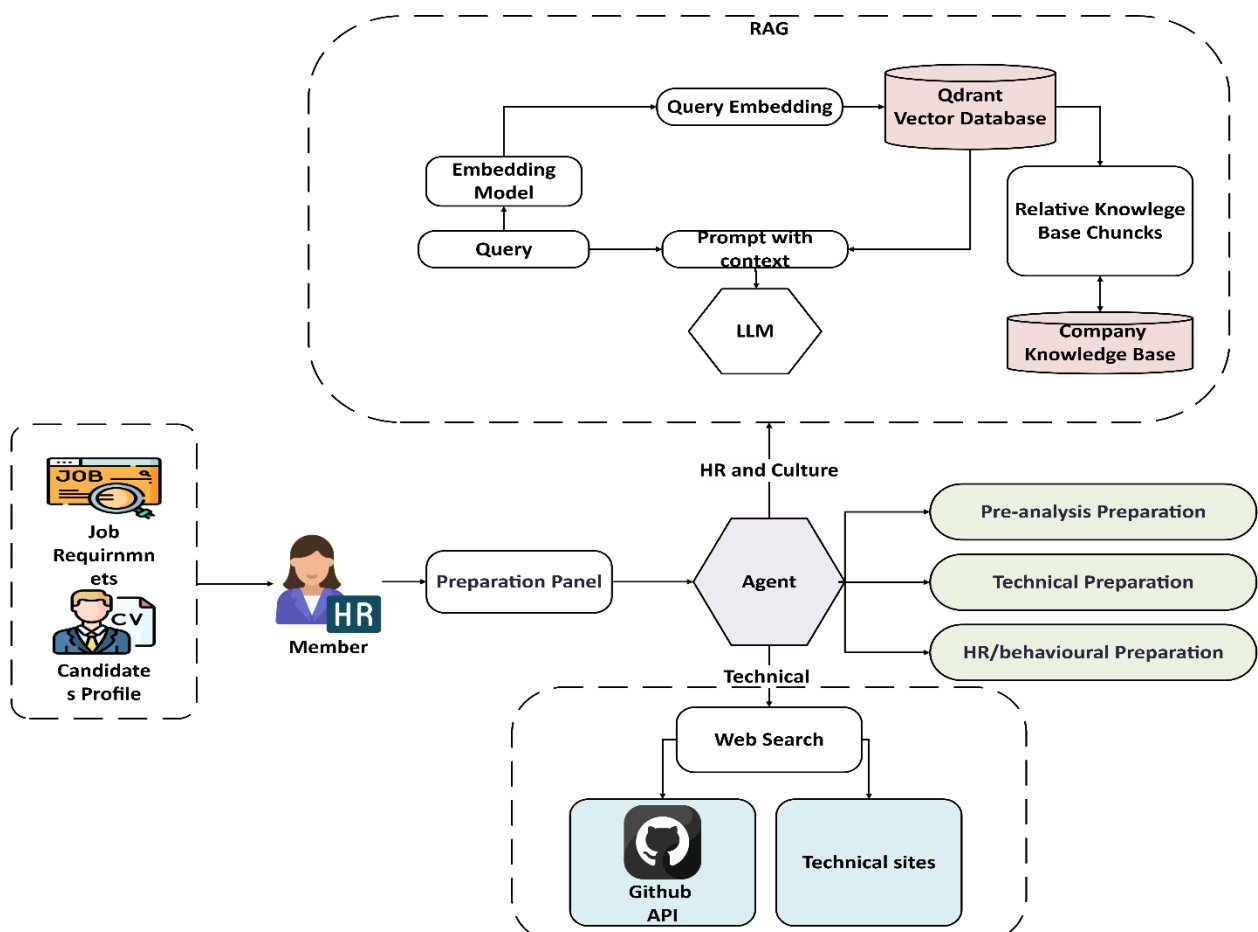


Figure 3.17: RAG-based interview preparation workflow in PATHS showing how job data, candidate data, company knowledge, semantic retrieval, web search, GitHub evidence, and agentic reasoning are combined to generate pre-analysis, technical preparation, and HR/behavioral preparation outputs.

The Figure 3.17. represents the interview preparation and RAG-based intelligence workflow in PATHS. It explains how the system prepares HR users and interviewers before meeting a candidate by combining the job requirements, candidate profile, company knowledge base, external technical evidence, and AI reasoning. The workflow begins with three main inputs: the job, the candidate profile, and the HR user. The job provides the role requirements, responsibilities, needed skills, seniority level, and evaluation criteria. The candidate profile

provides the candidate's CV data, skills, projects, experience, education, and uploaded documents. The HR user interacts with the system through the Preparation Panel, which acts as the interface where interview preparation outputs are requested and displayed.

The upper part of the diagram represents the Retrieval-Augmented Generation (RAG) workflow. In this part, the system takes a query related to the job or candidate. For example, the query may ask: "Prepare interview questions for this AI Engineer candidate based on the job description and company requirements." This query is passed to an embedding model, which converts the query into a numerical representation called a query embedding.

The query embedding is then sent to Qdrant Vector Database. Qdrant searches for the most relevant stored knowledge chunks based on semantic similarity. These chunks may come from the Company Knowledge Base, such as company policies, role expectations, technical standards, culture documents, hiring rubrics, or internal evaluation guidelines. The result is a set of relevant knowledge base chunks that provide context for the AI model.

After the relevant chunks are retrieved, the system builds a prompt with context. This means the LLM does not answer from the query alone. Instead, it receives the user request together with the retrieved company-specific information, job details, and candidate context. This makes the output more accurate and more aligned with the organization, rather than producing generic interview questions.

The LLM then generates a response based on the retrieved context. In this workflow, the LLM is not used as a simple chatbot. It is used as a controlled reasoning component that receives grounded information and produces structured outputs for interview preparation.

The middle part of the diagram shows the Agent layer. This agent organizes the preparation process into two main directions: HR and Culture preparation and Technical preparation.

The HR and Culture path focuses on behavioral, communication, motivation, teamwork, culture fit, and role-alignment questions. For example, the agent may generate questions about how the candidate handles pressure, works with teams, communicates with stakeholders, or aligns with company values. This helps HR conduct a more structured and fair interview instead of asking random or inconsistent questions.

The Technical path focuses on the candidate's technical skills and evidence. Here, the agent can use Web Search, GitHub API, and technical sites. The purpose is to verify and enrich the technical preparation. For example, if a candidate claims experience with FastAPI, LangChain, PostgreSQL, or machine learning deployment, the system can prepare technical questions that are connected to those skills. If GitHub data is available, the agent can also review repositories, project

descriptions, technologies used, and code-related evidence to generate more relevant technical questions. The output of this workflow is shown on the right side. PATHS produces three main preparation outputs:

1. Pre-analysis Preparation

This gives HR and interviewers a quick overview of the candidate before the interview. It may include the candidate's strengths, weak areas, relevant projects, possible concerns, and suggested focus points. This helps the interviewer enter the interview already prepared.

2. Technical Preparation

This includes technical questions, skill-verification points, project-based questions, and possible follow-up questions. For example, instead of asking a generic question like "Do you know machine learning?", the system can ask about a specific project, tool, or model mentioned in the candidate profile.

3. HR/Behavioral Preparation

This includes HR and culture-fit questions. These questions help evaluate communication, motivation, teamwork, adaptability, and professional behaviour. This is important because technical ability alone is not enough for hiring decisions.

The main value of this workflow is that it makes interviews more personalized, structured, and evidence-based. In traditional recruitment, interviewers may prepare questions manually or repeat the same questions for every candidate. In PATHS, the preparation is adapted to the specific job, the specific candidate, and the company's own knowledge base.

This also improves fairness. Since the system prepares questions based on job requirements and evaluation context, the interview becomes more consistent and less dependent on the interviewer's personal memory or subjective judgment. At the same time, the HR user remains in control and can review, edit, or reject the generated preparation before using it.

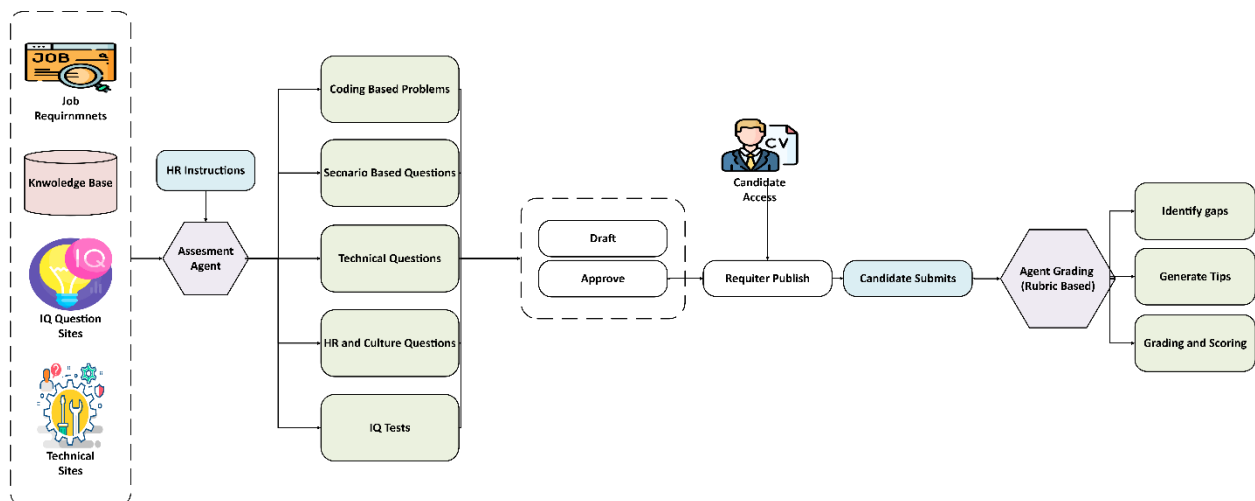


Figure 3.18: Assessment generation and grading workflow in PATHS showing how the assessment agent uses job requirements, HR instructions, and the knowledge base to create different assessment types, publish them after recruiter approval, receive candidate submissions, and generate rubric-based grading, gaps, and improvement tips.

The shown Figure 3.18. represents the assessment workflow in the PATHS system. It explains how technical and non-technical assessments are generated, reviewed, published, submitted by candidates, and finally graded using a structured rubric-based process.

The workflow starts from the job context. The assessment is not created randomly; it is connected to the job requirements, the required skills, the role level, and the type of candidate being evaluated. For example, an AI Engineer role may require Python, machine learning, model evaluation, and deployment questions, while a Frontend Engineer role may focus more on React, UI logic, state management, and API integration.

On the left side, the workflow uses three main sources of input: the Knowledge Base, HR Instructions, and the organization's assessment rules or question style. The Knowledge Base may include company documents, technical standards, role expectations, previous assessment templates, or hiring rubrics. HR Instructions allow the recruiter or HR user to guide the assessment generation process. For example, HR may specify the difficulty level, number of questions, time limit, allowed tools, prohibited tools, or whether the test should focus on coding, scenarios, technical theory, or behavioral evaluation.

These inputs are passed to the Assessment Agent. This agent is responsible for generating the assessment content according to the job and HR instructions. Instead of manually writing all questions, the recruiter can use the agent to draft a structured assessment that matches the role requirements. The Assessment Agent can generate different types of assessment items. The diagram shows several categories:

1. Coding-Based Problems

These are practical programming tasks used to evaluate implementation ability. They are useful for technical roles such as AI Engineer, Backend Engineer, Frontend Engineer, Data Scientist, or Software Tester.

2. Scenario-Based Questions

These questions test how the candidate thinks in real-world situations. For example, the candidate may be asked how they would handle a production bug, design a model evaluation pipeline, improve API performance, or debug a failed deployment.

3. Technical Questions

These questions evaluate theoretical and applied knowledge. They may cover frameworks, algorithms, tools, databases, machine learning concepts, software engineering principles, or role-specific technologies.

4. HR and Culture Questions

These questions focus on communication, teamwork, motivation, ownership, problem-solving behaviour, and cultural alignment. They help HR evaluate whether the candidate fits the organization's working style.

5. IQ Tests

These may be used when the organization wants to evaluate logical reasoning, pattern recognition, analytical thinking, or general problem-solving ability.

After the assessment is generated, it does not go directly to the candidate. It first enters the Draft stage. This means the assessment is prepared as an editable version. The recruiter or HR user can review it, modify questions, remove unsuitable items, adjust difficulty, and ensure that the assessment is aligned with the role.

The next step is Approve. This is a human review checkpoint. It ensures that the assessment is not published automatically without recruiter supervision. This is important because AI-generated questions may sometimes need correction, simplification, or alignment with the company's actual hiring standards.

After approval, the recruiter publishes the assessment through the Recruiter Publish step. Once published, the assessment becomes available to the candidate. The candidate then completes the assessment and submits their answers through the Candidate Submits step.

After submission, the answers are passed to the Agent Grading component. This grading process is rubric-based, meaning the candidate is evaluated according to predefined criteria rather

than a vague or subjective judgment. A rubric may include correctness, completeness, technical depth, clarity, problem-solving approach, code quality, communication, and alignment with the expected answer. The grading agent produces three main outputs:

1. Identify Gaps

The system identifies what the candidate is missing. For example, the candidate may understand the theory but lack practical implementation, or they may write working code but fail to handle edge cases.

2. Generate Tips

The system provides improvement tips based on the candidate's gaps. This makes the assessment useful not only for selection, but also for learning and development.

3. Grading and Scoring

The system calculates a structured score for the assessment. This score can later be used in the candidate evaluation pipeline, combined with interview performance, CV analysis, skill evidence, and final decision support.

The main value of this workflow is that it makes assessments more consistent, role-specific, and explainable. Instead of giving every candidate a generic test, PATHS can generate assessments based on the actual job requirements and the organization's evaluation criteria. At the same time, the recruiter remains in control through the draft and approval stages.

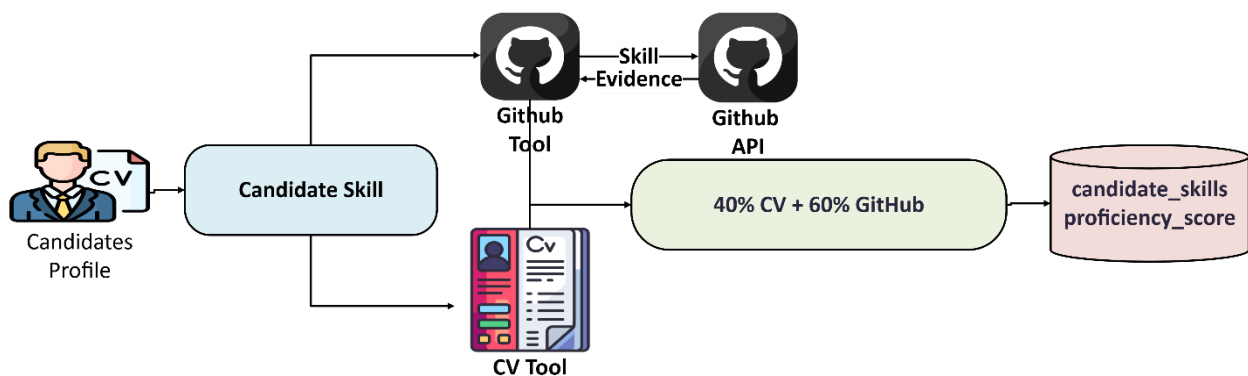


Figure 3.19: Candidate skill evidence workflow in PATHS showing how candidate skills are verified using CV evidence and GitHub evidence, then converted into a weighted proficiency score stored in the candidate skills table.

This figure represents the skill evidence and proficiency scoring workflow in PATHS. It explains how the system evaluates whether a candidate's listed skills are only mentioned in the CV or actually supported by practical evidence, especially from GitHub projects.

The workflow starts with the candidate profile. The candidate may have several skills extracted from their CV, uploaded documents, profile form, or previous applications. These skills

are represented in the diagram as Candidate Skill. For example, a candidate may list skills such as Python, FastAPI, Machine Learning, React, PostgreSQL, or Docker.

However, PATHS does not treat every skill as equally reliable just because it appears in the CV. A CV is useful, but it is mostly self-reported. A candidate can mention a skill without showing strong proof of using it. That is why the system introduces a Skill Evidence layer.

The skill evidence layer checks the candidate's skill from two main sources: CV evidence and GitHub evidence.

The CV evidence comes from the candidate's uploaded CV or documents. The system checks where and how the skill appears. For example, if "Python" appears only once in a skills list, this is weaker evidence than if Python appears inside multiple project descriptions, internship experiences, and technical achievements. The CV gives an initial signal about the candidate's claimed skills.

The GitHub evidence provides a stronger practical signal. If the candidate connects a GitHub profile, the system can inspect repositories, project descriptions, programming languages, README files, topics, and visible code-related activity. This helps the system check whether the candidate has actually used the skill in real projects. For example, if the candidate claims FastAPI and has a GitHub repository containing FastAPI backend code, this gives stronger evidence than CV text alone.

The diagram shows the final weighting as 40% CV + 60% GitHub. This means the system gives more weight to GitHub evidence because it is closer to practical proof. The CV still matters because it provides context, experience, education, and role descriptions, but GitHub is weighted higher because it can show real implementation evidence.

The final proficiency score can be represented as:

Equation 1. Skill Proficiency Score

$$40\% \text{ CV Evidence Score} + 60\% \text{ GitHub Evidence Score}$$

After calculating the weighted score, the result is stored in the `candidate_skills_proficiency_score` field. This score represents how strong the candidate appears to be in a specific skill based on available evidence. For example, a candidate may have Python with a proficiency score of 88, FastAPI with 76, and Docker with 54, depending on how much evidence exists in both the CV and GitHub.

This workflow is important because it helps PATHS move from simple keyword matching to evidence-based skill evaluation. Traditional recruitment systems may only check whether a skill

exists in the CV. PATHS goes further by asking: Is there evidence that the candidate has actually used this skill? This makes matching, scoring, and shortlisting more reliable.

A typical scenario can be described as follows: the candidate uploads a CV and connects a GitHub profile. PATHS extracts skills from the CV, then checks GitHub repositories for supporting evidence. For each skill, the system calculates a CV-based evidence score and a GitHub-based evidence score. These two scores are combined using the 40/60 weighting, and the final value is saved as the candidate’s proficiency score for that skill.

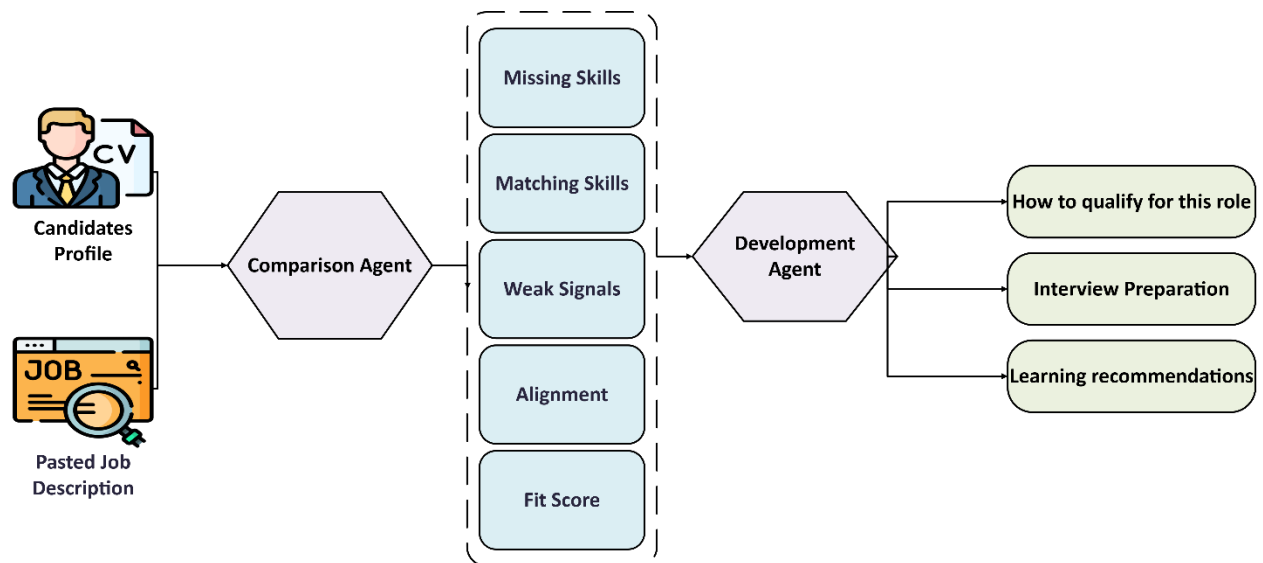


Figure 3.20: Candidate-job comparison and development recommendation workflow in PATHS showing how the system compares a candidate profile with a posted job description, identifies missing and matching skills, evaluates weak signals and alignment, calculates a fit

The Figure 3.20. represents the candidate-to-job comparison workflow in PATHS. It explains how the system helps a candidate understand how well they fit a specific posted job and what they need to improve in order to become more qualified for that role.

The workflow starts with two main inputs: the candidate profile/CV and the posted job description. The candidate profile contains the candidate’s skills, education, experience, projects, uploaded CV data, and possibly GitHub or portfolio evidence. The job description contains the role requirements, required skills, responsibilities, preferred qualifications, seniority level, and expected experience.

These two inputs are passed to the Comparison Agent. This agent is responsible for analyzing the gap between the candidate and the job. Instead of only saying “matched” or “not matched,” the agent breaks the comparison into several meaningful outputs that are useful for both candidate support and recruitment decision-making.

The first output is Missing Skills. This section identifies the important skills required by the job but not clearly found in the candidate profile. For example, if the job requires Docker, AWS, or model deployment and the candidate profile does not show evidence of these skills, the system marks them as missing. This helps the candidate understand exactly what they need to learn or strengthen.

The second output is Matching Skills. This section highlights the skills that already match between the candidate and the job. For example, if the job requires Python, machine learning, SQL, or FastAPI, and these are found in the candidate profile, the system identifies them as matching skills. This gives the candidate confidence and helps recruiters quickly see the areas where the candidate is already strong.

The third output is Weak Signals. This means the system found partial or uncertain evidence for some requirements, but the evidence is not strong enough. For example, the candidate may mention “cloud” once in the CV without showing a project or practical experience, or they may list “React” as a skill without clear proof of usage. Weak signals are useful because they show areas that may need verification during interviews or assessments.

The fourth output is Alignment. This measures the overall relationship between the candidate’s background and the job direction. Alignment is broader than skills. It may include whether the candidate’s projects, experience, career interests, education, and preferred job type fit the role. For example, a candidate may not have every required skill, but their project background and learning direction may still align well with the job.

The fifth output is the Fit Score. This score summarizes the comparison in a simple form. It gives the candidate and recruiter a quick indication of how suitable the candidate is for the role. However, the fit score should not be understood as a final hiring decision. It is a decision-support indicator that must be interpreted together with the explanation, missing skills, matching skills, weak signals, and human review.

After the comparison outputs are generated, they are passed to the Development Agent. This agent transforms the analysis into practical guidance. This is an important part of PATHS because the system is not designed only to evaluate candidates; it also supports candidate development.

The Development Agent produces three main outputs.

The first output is How to qualify for this role. This section gives the candidate a clear action plan for becoming more suitable for the job. For example, it may recommend learning a

missing tool, building a small project, improving a portfolio, gaining experience in a specific framework, or preparing evidence for a weak skill.

The second output is Interview Preparation. This provides personalized preparation advice based on the job requirements and the candidate's current profile. For example, if the candidate has weak evidence in system design or deployment, the system may suggest preparing for questions in these areas. This makes interview preparation more focused instead of generic.

The third output is Learning Recommendations. This section suggests learning paths, topics, or resources that can help the candidate close the identified gaps. For example, the system may recommend learning Docker basics, practicing SQL queries, studying MLOps workflows, or building a project related to the target role.

The main value of this workflow is that it makes the job-matching process more useful and human-centered. Traditional systems often show only a match percentage or reject a candidate silently. PATHS gives the candidate a clearer explanation: what matches, what is missing, what is weak, how aligned they are, and what they can do next.

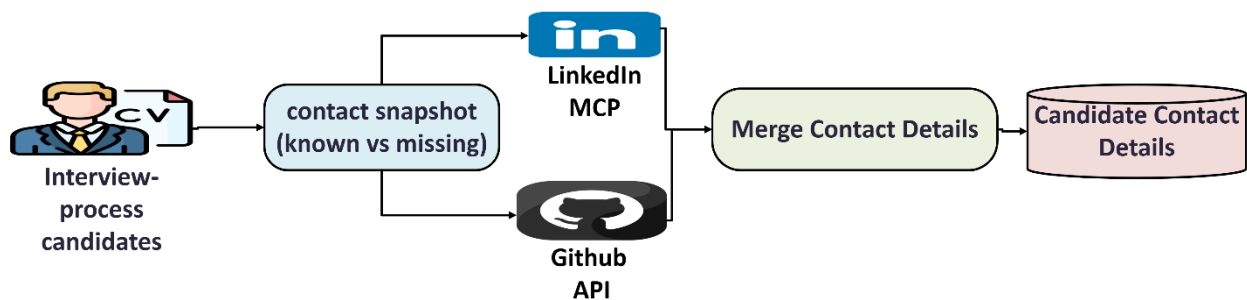


Figure 3.21: Candidate contact enrichment workflow in PATHS showing how interview-process candidates are reviewed for missing contact information, enriched from professional sources such as LinkedIn and GitHub, merged into a unified contact profile, and stored as candidate contact details.

The Figure 3.21. represents the candidate contact enrichment workflow in PATHS. It explains how the system handles candidates who are already inside the interview or recruitment process but have incomplete contact information. In recruitment systems, candidate data is often fragmented. A candidate may have a CV uploaded in the system, a LinkedIn profile, a GitHub account, and some partial contact details, but not all of this information is available in one clean record. This workflow is designed to solve that problem.

The workflow starts with interview-process candidates. These are candidates who have already entered the recruitment pipeline and may be shortlisted, assessed, or scheduled for interviews. At this stage, having accurate contact information is important because HR may need

to send interview links, assessment invitations, acceptance emails, rejection emails, or follow-up messages.

The next step is the contact snapshot, described as known vs. missing. This means the system first checks what contact information is already available for the candidate and what is still missing. For example, the system may already have the candidate's email but not their phone number, LinkedIn URL, GitHub profile, portfolio link, or alternative contact method. This snapshot helps the system avoid unnecessary searching and focuses only on the missing or incomplete fields.

After identifying the missing contact details, the workflow uses external professional sources such as LinkedIn and GitHub. LinkedIn can provide professional profile information, career history, public profile links, and sometimes contact-related signals. GitHub can help verify the candidate's technical identity, repositories, username, portfolio evidence, and public profile information. These sources are not treated as separate candidate records; instead, they are used to enrich the existing candidate profile.

The next stage is Merge Contact Details. This is an important step because the system may collect candidate information from multiple places, and the data may not always be identical. For example, a candidate may use one email in their CV, another email in GitHub, and a different profile name on LinkedIn. The merge process compares the collected information and combines it into one unified contact record. It should also avoid overwriting reliable information with uncertain data unless confidence is high or HR confirms it.

The final output is Candidate Contact Details. This represents the cleaned and merged contact information stored in the system. These details can then be used by the outreach module, interview scheduling module, assessment invitation module, and decision-support communication module. Instead of HR manually searching for candidate contacts across different platforms, PATHS organizes the data into one reliable place.

From a design perspective, this workflow supports the idea of a single source of truth for candidate communication. It reduces manual work for recruiters, minimizes duplicate or inconsistent contact records, and improves the reliability of communication with candidates.

This workflow also supports better candidate experience. When contact details are complete and correct, candidates are less likely to miss interview invitations, assessment links, or important updates. It also helps HR avoid sending messages to outdated or incorrect contact channels.

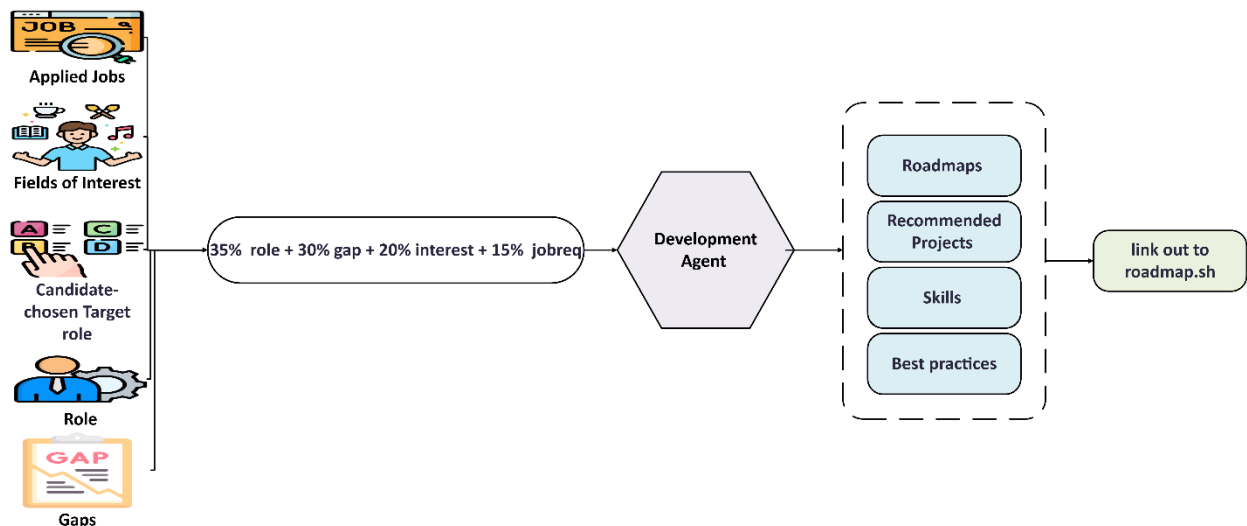


Figure 3.22: Personalized development recommendation workflow in PATHS showing how the candidate’s target role, skill gaps, interests, and job requirements are weighted and processed by the Development Agent to generate roadmaps, recommended projects, required skills, best practices, and external learning links.

This figure represents the personalized development workflow in PATHS. It explains how the system supports candidates after comparing their profile with a target job or career role. The main purpose of this workflow is not only to tell the candidate whether they are qualified or not, but to guide them toward becoming more qualified through a clear and practical development plan.

The workflow starts from the left side with several inputs. These inputs represent the information needed to understand the candidate’s current situation and future goal. The first input is the job or target role, which defines what the candidate wants to qualify for. For example, the target role may be AI Engineer, Backend Engineer, Data Scientist, Software Tester, or Biomedical Engineer.

The second input represents the candidate’s current profile and background. This includes the candidate’s CV, skills, education, experience, projects, and previous application data. This information helps the system understand what the candidate already has and what they still need to improve.

The third input is the candidate’s chosen target role. This is important because development recommendations should not be generic. A candidate preparing for an AI Engineer role needs a different roadmap than a candidate preparing for a Frontend Engineer or QA Engineer role. By allowing the candidate to choose a target role, PATHS can personalize the output according to the candidate’s own career direction.

The fourth input represents the gap analysis. This is the difference between what the candidate currently has and what the target role requires. For example, the candidate may already

know Python and machine learning, but may be missing Docker, cloud deployment, or MLOps. These missing areas become the basis for the development plan. The figure shows a weighted formula:

Equation 2 weighted formula for Development Plan creation

$$35\% \text{ role} + 30\% \text{ gap} + 20\% \text{ interest} + 15\% \text{ job requirements}$$

This means the Development Agent does not generate recommendations randomly. It uses a weighted logic to decide what matters most. The role has the highest weight because the roadmap must be aligned with the candidate's target career path. The gap has the second-highest weight because the system should focus on what the candidate is missing. The interest factor keeps the plan personalized and motivating, because candidates are more likely to complete a plan that matches their interests. Finally, job requirements ensure that the recommendations remain practical and connected to real market needs.

After these inputs are collected and weighted, they are passed to the Development Agent. This agent is responsible for converting the analysis into a useful learning and growth plan. It acts as the bridge between evaluation and improvement. Instead of stopping at “you are missing these skills,” it answers the more important question: “What should you do next?”

The Development Agent produces four main outputs.

The first output is Roadmaps. These are structured learning paths that guide the candidate step by step. For example, for an AI Engineer role, the roadmap may include Python, mathematics, machine learning, deep learning, model deployment, APIs, databases, and MLOps. This gives the candidate a clear direction instead of leaving them confused about where to start.

The second output is Recommended Projects. Projects are important because they turn learning into evidence. For example, the system may recommend building a resume screening model, a FastAPI-based ML deployment project, a RAG chatbot, or a dashboard for model evaluation. These projects help the candidate prove their skills and later strengthen their CV, GitHub, and interview answers.

The third output is Skills. This section lists the skills the candidate should focus on. These may include missing technical skills, weak skills, or supporting soft skills. For example, the system may recommend improving SQL, Docker, React, data pre-processing, communication, or system design depending on the selected role.

The fourth output is Best Practices. This gives the candidate practical advice about how to improve professionally. For example, it may suggest documenting projects properly, writing clean

GitHub README files, practicing interview explanations, following coding standards, building portfolio projects, or preparing role-specific case studies.

Also, an external link to roadmap.sh. This means the system can connect the candidate to external structured roadmaps when useful. Instead of generating everything from scratch, PATHS can combine its personalized recommendations with trusted public learning roadmaps. This makes the development plan more practical and easier for the candidate to follow.

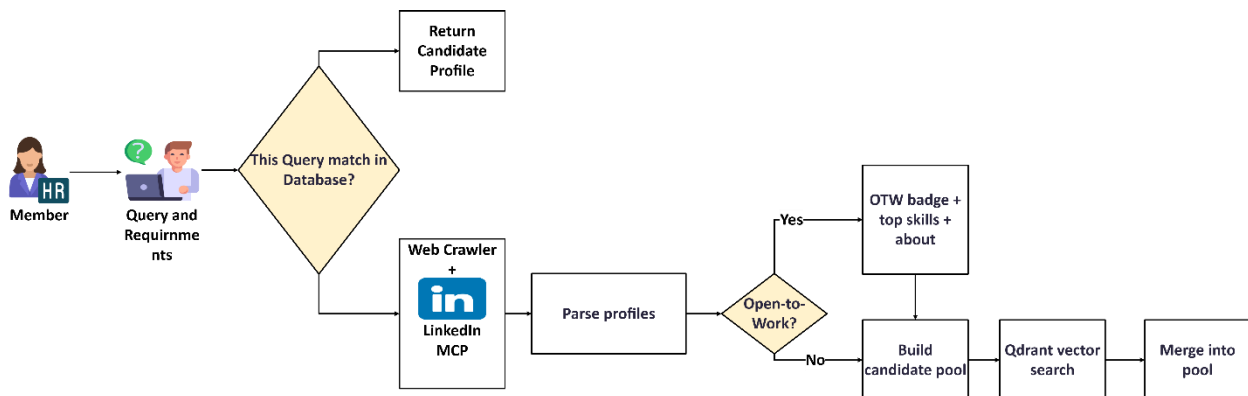


Figure 3.23: LinkedIn-based outbound sourcing workflow in PATHS showing how HR search queries are first checked against the internal candidate database, then expanded through LinkedIn MCP crawling, Open-to-Work filtering, profile enrichment, semantic search, deduplication, and imported candidate profile creation.

The Figure 3.23. represents the external candidate sourcing workflow in PATHS, specifically the flow used when HR wants to search for candidates who may not already exist inside the system database. The purpose of this workflow is to support outbound recruitment, where recruiters search for suitable candidates instead of depending only on applicants who apply directly.

The workflow starts with the HR user, who enters a candidate search query. For example, HR may search for “Junior AI Engineer with Python and FastAPI,” “Backend Developer in Egypt,” or “Data Scientist open to work.” This query represents the recruiter’s hiring need.

The first decision point is “Does this query match the database?”. Before searching externally, PATHS checks whether there are already suitable candidates stored inside the internal database. This is important because the system should reuse existing candidate records when possible instead of repeatedly collecting the same data. If matching candidates already exist, the system directly returns the candidate profile to HR.

If the internal database does not contain enough relevant candidates, the workflow moves to the external sourcing stage. Here, PATHS uses a web crawler / LinkedIn MCP component. MCP stands for Model Context Protocol, and in this context it represents a controlled connector that allows the system to search or retrieve candidate-related information from LinkedIn-style

professional sources. This step helps the system discover potential candidates outside the existing database.

After collecting profiles, the system performs profile parsing. The parsed profiles are stored as *ExternalCandidate* records and grouped under an *ExternalCandidateBatch*. This is useful because each sourcing run can return many profiles, and the system needs to track which candidates came from which search query, when they were collected, and what source produced them. The next decision point is Open-to-Work? This step checks whether the discovered candidate appears to be open to new job opportunities. If the candidate is marked as open to work, the system performs additional profile enrichment. For each candidate, the system retrieves more detailed information such as the person profile, top skills, about section, and other available public professional information. The diagram also mentions concurrency, which means the system can process multiple candidate profiles in parallel to reduce waiting time.

After enrichment, the system builds a candidate pool with the source labeled as LinkedIn. This pool represents the group of externally sourced candidates that may be relevant to the recruiter's query.

The next step is semantic search using Qdrant vector search. Instead of depending only on keyword matching, PATHS converts the recruiter query and candidate information into embeddings, then compares them semantically. This allows the system to find candidates who are meaningfully relevant even if they do not use the exact same words as the job description. For example, a candidate may write "built REST APIs using FastAPI," while the recruiter searches for "Python backend developer." Semantic search helps connect these related meanings.

The semantic search returns database candidate rows. These rows represent candidate records that are either already stored or newly prepared to be stored in the system. After that, the workflow moves to merge into pool. This step combines candidates from the internal database and external LinkedIn source into one clean candidate pool. The merge step also includes deduplication. This is very important because the same candidate may already exist in PATHS from a CV upload, a previous application, a CSV import, or another external sourcing run. Deduplication prevents repeated candidate cards and keeps the database clean. It can compare identifiers such as name, profile link, email, phone number, GitHub URL, and similarity between profile content. The final output is a candidate card shown as Imported, with an option to View Profile. This means the sourced candidate is now available inside the platform and can be reviewed by HR. The recruiter can open the profile, inspect the candidate's skills and background, and decide whether to add the candidate to the hiring process.

This workflow helps PATHS solve one of the major recruitment problems: recruiters often miss strong passive candidates because those candidates have not applied directly. By combining internal database search, LinkedIn-style sourcing, Open-to-Work detection, profile enrichment, semantic search, and deduplication, PATHS creates a more complete sourcing pipeline.

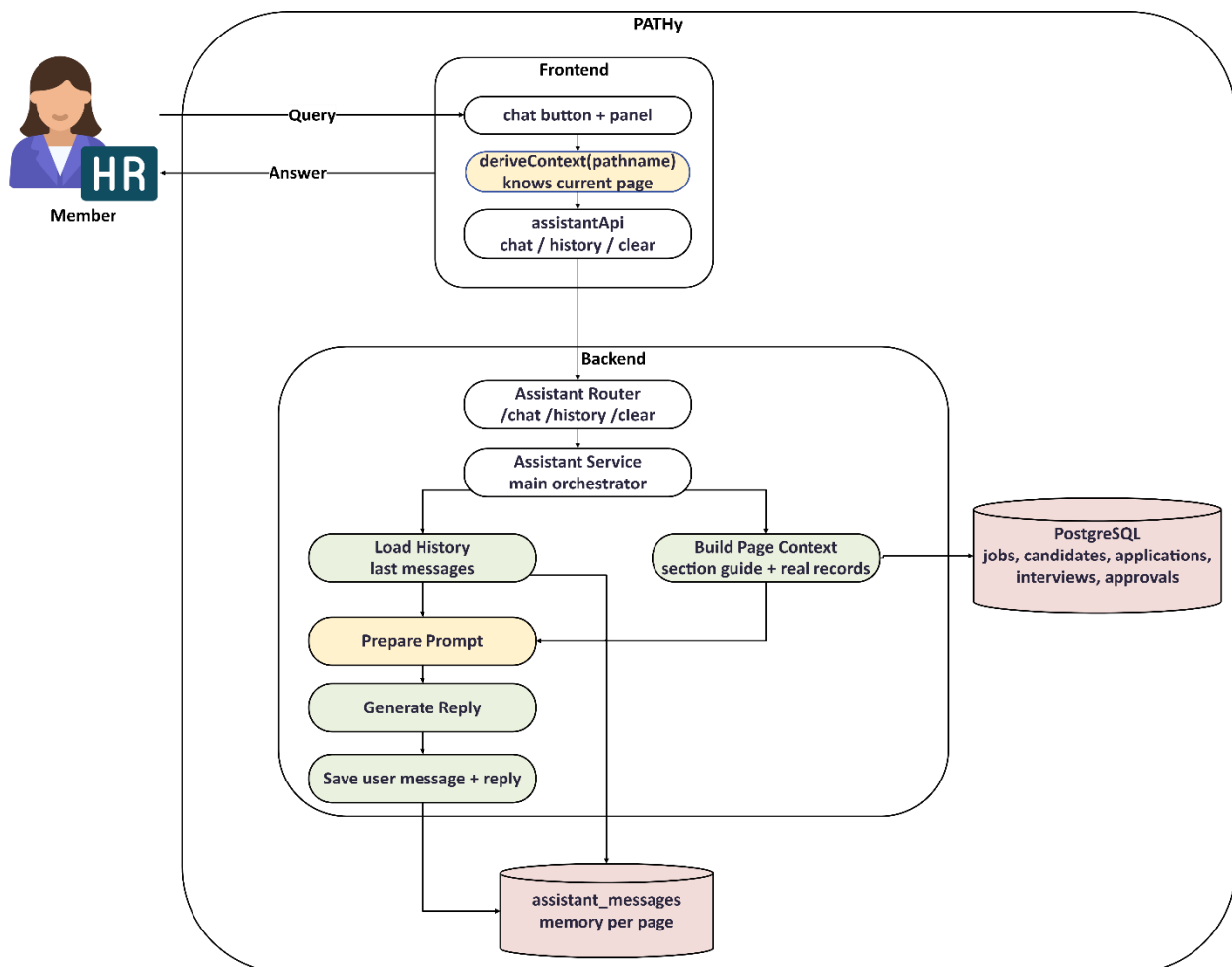


Figure 3.24: Context-aware HR assistant workflow in PATHS showing how the floating chat interface detects the current page, sends the user query to the assistant API, retrieves page-specific data and conversation history, generates a grounded reply, and stores memory separately for each page context.

The Figure 3.24. represents the context-aware AI assistant workflow inside PATHS. It explains how the HR user can communicate with a floating assistant anywhere inside the platform, and how the assistant responds based on the page the user is currently viewing.

The workflow starts with the HR user, who asks a question through the chat interface. The purpose of this assistant is not to behave like a general chatbot only. Instead, it is designed to understand the current recruitment context. For example, if the HR user is on the jobs page, the assistant should answer about jobs. If the user is on a candidate profile, it should answer using that candidate's information. If the user is on interviews, approvals, or applications, the assistant should respond according to that section.

On the frontend side, the assistant is shown as a chat button and panel. This means the user can open a small floating chat window without leaving the current page. This design improves usability because HR users can ask questions while continuing their normal workflow.

The next frontend component is `deriveContext(pathname)`. This is an important part of the design. It reads the current page route or pathname and uses it to understand where the user is inside the platform. For example, the pathname may indicate that the user is viewing the dashboard, a specific job, a candidate profile, an interview page, or an approval page. Based on this, the system derives the current context.

After the context is detected, the frontend communicates with the backend through the `assistantApi`, which supports actions such as chat, history, and clear. The chat action sends a new user query. The history action retrieves previous messages for the current page context. The clear action removes the saved conversation for that specific context when needed.

The request then reaches the backend through the Assistant Router, which exposes endpoints such as `/chat`, `/history`, and `/clear`. The router acts as the API boundary. Its role is to receive the request, validate it, check authentication, and forward the request to the assistant service. It does not contain the main reasoning logic itself.

The core logic is handled by the Assistant Service, which acts as the main orchestrator. This service coordinates all the steps needed to answer the HR user properly. It decides what history to load, what page context to build, how to prepare the prompt, how to generate the reply, and how to save the conversation afterward.

The first internal step is Load History — last messages. The assistant retrieves recent previous messages from the `assistant_messages` table. This allows the assistant to remember the recent conversation within the same page. For example, if HR previously asked about a candidate's score and then asks "why is it low?", the assistant can understand the follow-up because it has the recent chat history.

The second step is Build Page Context — section guide + real records. This is what makes the assistant context-aware. The backend does not only rely on the user's question. It also retrieves relevant real data from PostgreSQL, such as jobs, candidates, applications, interviews, and approvals. For example, if the HR user is viewing a job page, the assistant may retrieve that job's title, requirements, candidates in pipeline, and related screening information. If the user is viewing a candidate page, it may retrieve candidate profile data, application history, interview status, and scoring outputs.

The diagram shows that PostgreSQL stores core recruitment records such as jobs, candidates, applications, interviews, and approvals. This database acts as the reliable source of truth for the assistant. The assistant's answer is therefore grounded in the actual platform data, not only generated from general knowledge.

After history and page context are collected, the system moves to Prepare Prompt. This step combines the user's question, recent conversation history, page context, and relevant database records into one structured prompt. The goal is to give the AI model enough information to answer accurately and specifically. This avoids generic replies and helps the assistant provide useful operational support.

The next step is Generate Reply. At this point, the AI model produces an answer for the HR user. The reply may explain a page, summarize a candidate, clarify a score, suggest next actions, describe why a candidate is in a certain pipeline stage, or guide HR through a system feature. Because the reply is based on page context and real records, it becomes more useful than a normal chatbot response.

After generating the answer, the workflow moves to Save user message + reply. Both the HR user's query and the assistant's response are stored in the `assistant_messages` table. This gives the assistant memory over time.

A key design detail in this figure is memory per page. The assistant does not use one global memory for everything. Instead, messages are stored according to the current page or context. This means the conversation about the dashboard stays separate from the conversation about a specific candidate or job. This is important because HR may ask similar questions in different places, but the correct answer depends on the current page.

For example, the question "Why is this candidate recommended?" should have a different answer depending on which candidate profile is open. Similarly, "What should I do next?" should produce different guidance on the approvals page, interview page, or jobs page. Page-specific memory prevents confusion and keeps the assistant relevant.

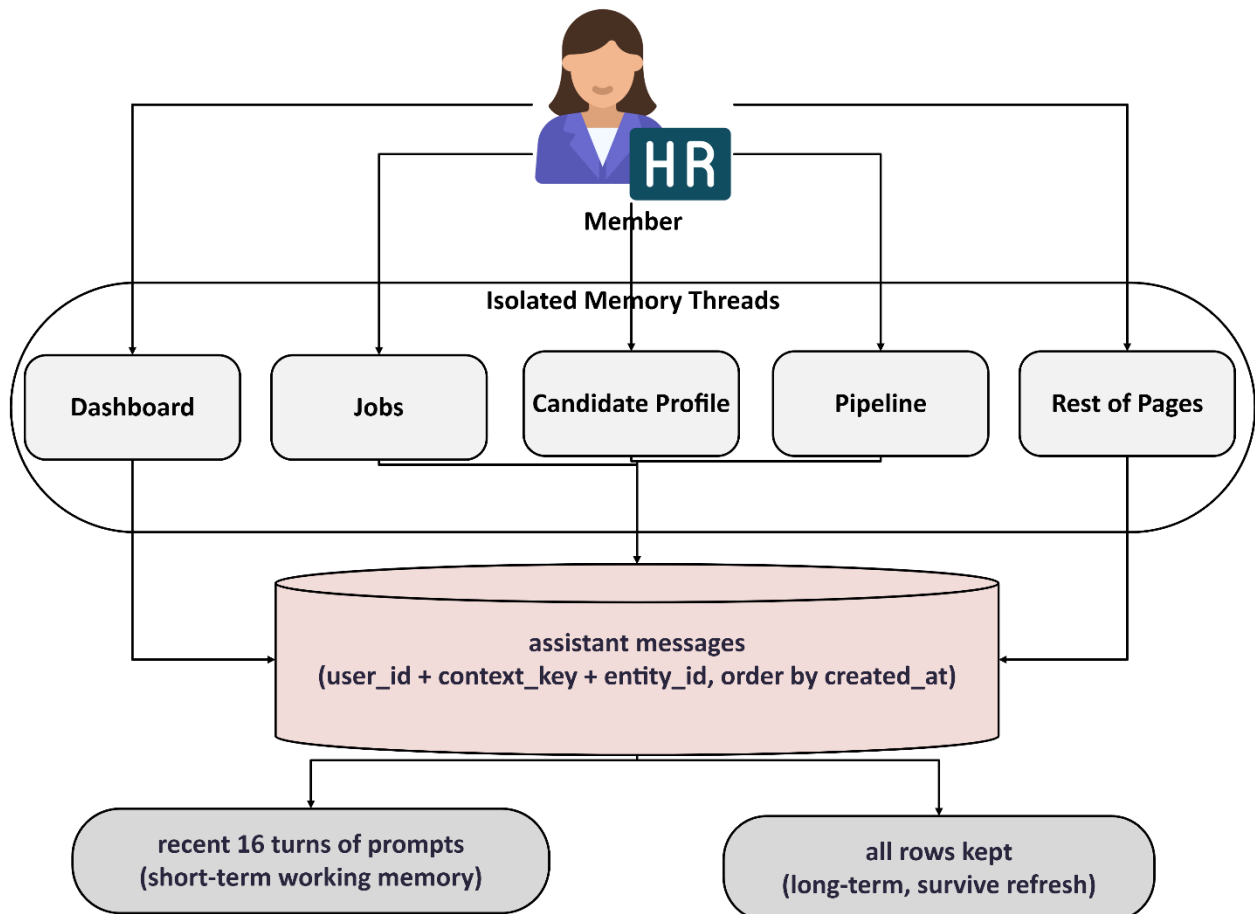


Figure 3.25: Isolated memory-thread design for the PATHS context-aware assistant, showing how each page context keeps a separate conversation history while storing all assistant messages in one structured database table.

The Figure 3.25. represents the memory design of the context-aware assistant in PATHS. It explains how the assistant keeps conversations organized across different pages of the recruiter platform, so that the HR user receives answers that are relevant to the current page without mixing unrelated conversations together.

At the top of the diagram, the HR user is the main actor. The HR user may open the assistant from different places inside the platform, such as the dashboard, jobs page, candidate profile, pipeline page, or any other page. Each of these pages has a different meaning and contains different data, so the assistant must understand the context before answering.

The middle layer shows Isolated Memory Threads. This means that the assistant does not use one shared conversation history for the entire system. Instead, it separates memory by page or context. For example, the assistant conversation on the Dashboard is stored separately from the conversation on Jobs, and both are stored separately from the conversation on a specific Candidate Profile or Pipeline page.

This design is important because the same question can mean different things depending on the page. For example, if HR asks, “What should I do next?” on the dashboard, the answer may be about reviewing overall hiring statistics or upcoming interviews. But if the same question is asked on a candidate profile, the answer should be about that candidate’s score, interview status, or next hiring action. Isolated memory prevents the assistant from confusing these contexts.

The diagram shows several memory threads:

1. Dashboard

This thread stores assistant conversations related to high-level hiring statistics, upcoming interviews, application trends, pipeline funnel insights, and general recruitment performance.

2. Jobs

This thread stores conversations related to job creation, job requirements, job details, candidates connected to a specific job, and job-based recommendations.

3. Candidate

Profile

This thread stores conversations related to a specific candidate. This may include questions about the candidate’s CV, skills, score, interview results, assessment performance, or suitability for a job.

4. Pipeline

This thread stores conversations related to candidate movement across hiring stages, such as sourced, shortlisted, assessed, interviewed, approved, or rejected.

5. Rest of Pages

This represents all other pages that may also need assistant support, such as approvals, interviews, reports, settings, or organization administration.

- All of these isolated threads are stored in one database table called *assistant_messages*. The table uses three important identifiers: *user_id*, *context_key*, and *entity_id*.
- The *user_id* identifies the HR user who owns the conversation. This ensures that each user has their own assistant memory.
- The *context_key* identifies the page or section where the conversation happened. For example, the context key may represent dashboard, jobs, candidate profile, pipeline, interviews, or approvals.
- The *entity_id* identifies a specific object inside that page when needed. For example, if the HR user is viewing a specific candidate profile, the entity ID can be the candidate ID. If the

user is viewing a specific job, the entity ID can be the job ID. This allows the assistant to keep a separate memory not only per page type, but also per specific candidate or job.

- The messages are ordered by *created_at*, which means the assistant can retrieve the conversation in the correct chronological order. This is important for understanding follow-up questions. For example, if HR asks, “Why is this candidate recommended?” and then asks, “What are the weak points?”, the assistant can understand that “weak points” refers to the same candidate and same conversation thread. At the bottom of the diagram, there are two types of memory behaviour.

The first is recent 16 turns of prompts, which represents the assistant’s short-term working memory. In practice, the assistant does not need to load the full conversation every time. It only loads the most recent messages, such as the last 16 turns, to keep the prompt focused and efficient. This helps the assistant answer follow-up questions without sending too much unnecessary history to the AI model.

The second is all rows kept, which represents long-term memory that survives refresh. Even if the user refreshes the browser, closes the page, or returns later, the previous conversation records remain stored in the database. This makes the assistant more reliable because the user does not lose context after navigation or refresh. The main value of this design is that it gives the assistant contextual memory without confusion. A recruiter may have many active workflows at the same time: one conversation about a job, another about Candidate A, another about Candidate B, and another about the pipeline. If all messages were stored in one global thread, the assistant could mix information between them. By isolating memory threads, PATHS keeps each conversation clean and relevant.

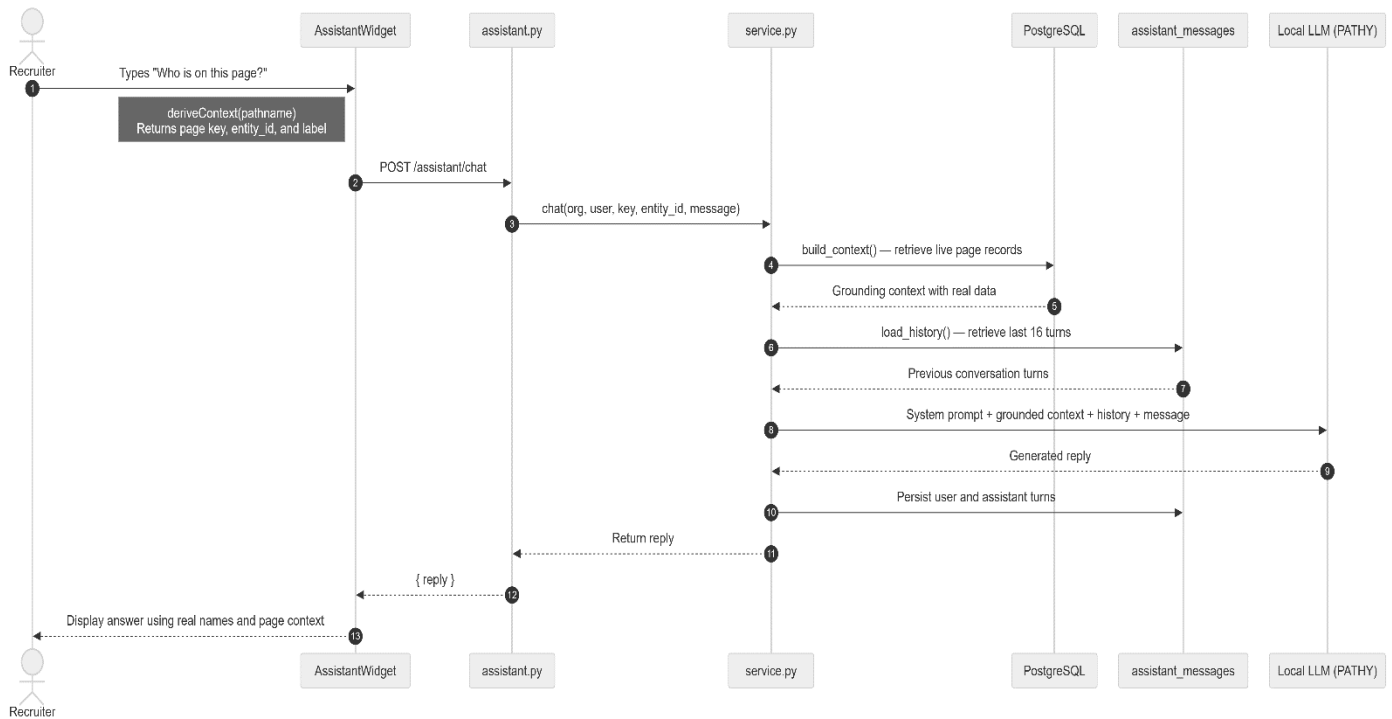


Figure 3.26: Context-Aware PATHY Assistant Interaction Sequence for Retrieving Page-Specific Data, Maintaining Conversation History, and Generating Grounded Responses.

The Figure 3.26. presents the complete interaction sequence followed when a recruiter communicates with the PATHY assistant from within the PATHS web application. The sequence begins when the recruiter enters a natural-language question through the *AssistantWidget*, such as "Who is on this page?". Rather than treating the question as an isolated request, the widget first derives the current page context from the application pathname. This context identifies the active page type, the relevant entity identifier, and a descriptive label. Consequently, the assistant can distinguish whether the recruiter is viewing a job, candidate profile, interview, organization, or another recruitment workspace before responding.

The *AssistantWidget* then sends the recruiter's message together with the derived page context to the backend through the POST /assistant/chat endpoint. The request is initially handled by *assistant.py* and forwarded to *service.py*, where the main assistant orchestration logic is executed. The service layer calls *build_context()* to retrieve the live records associated with the current page from PostgreSQL. This grounding step is essential because it enables PATHY to use current system data, including the actual names, statuses, and relationships stored in PATHS, instead of relying solely on general language-model knowledge or generic assumptions.

To maintain continuity throughout the conversation, *service.py* also retrieves the most recent interaction history from the *assistant_messages* store. In the illustrated implementation, the

last sixteen conversation turns are loaded and supplied as part of the prompt context. This short-term memory allows PATHY to understand follow-up questions, preserve references made earlier in the discussion, and avoid unnecessary repetition. For example, after the recruiter asks about a candidate, the assistant can correctly interpret a later question such as "What is their current status?" within the same conversation and page context.

The service combines four inputs before invoking the local PATHY language model: the system prompt, the grounded page records, the retrieved conversation history, and the recruiter's new message. The local LLM generates a response using this combined context and returns it to the service layer. Both the recruiter's message and the generated response are subsequently persisted in *assistant_messages*. Persisting these turns supports future conversational continuity, auditability, and troubleshooting, as the system retains a record of the information supplied to and generated by the assistant.

The reply is returned through *assistant.py* to the *AssistantWidget* and displayed to the recruiter in the interface. Overall, this design integrates page-level context awareness, live database grounding, conversational memory, and local language-model generation in one controlled workflow. It enables PATHY to provide relevant and personalized assistance while remaining connected to real recruitment records and preserving an auditable interaction history. This approach improves usability for recruiters because answers are tailored to the specific task and entity currently being reviewed rather than being generated as generic chatbot responses.

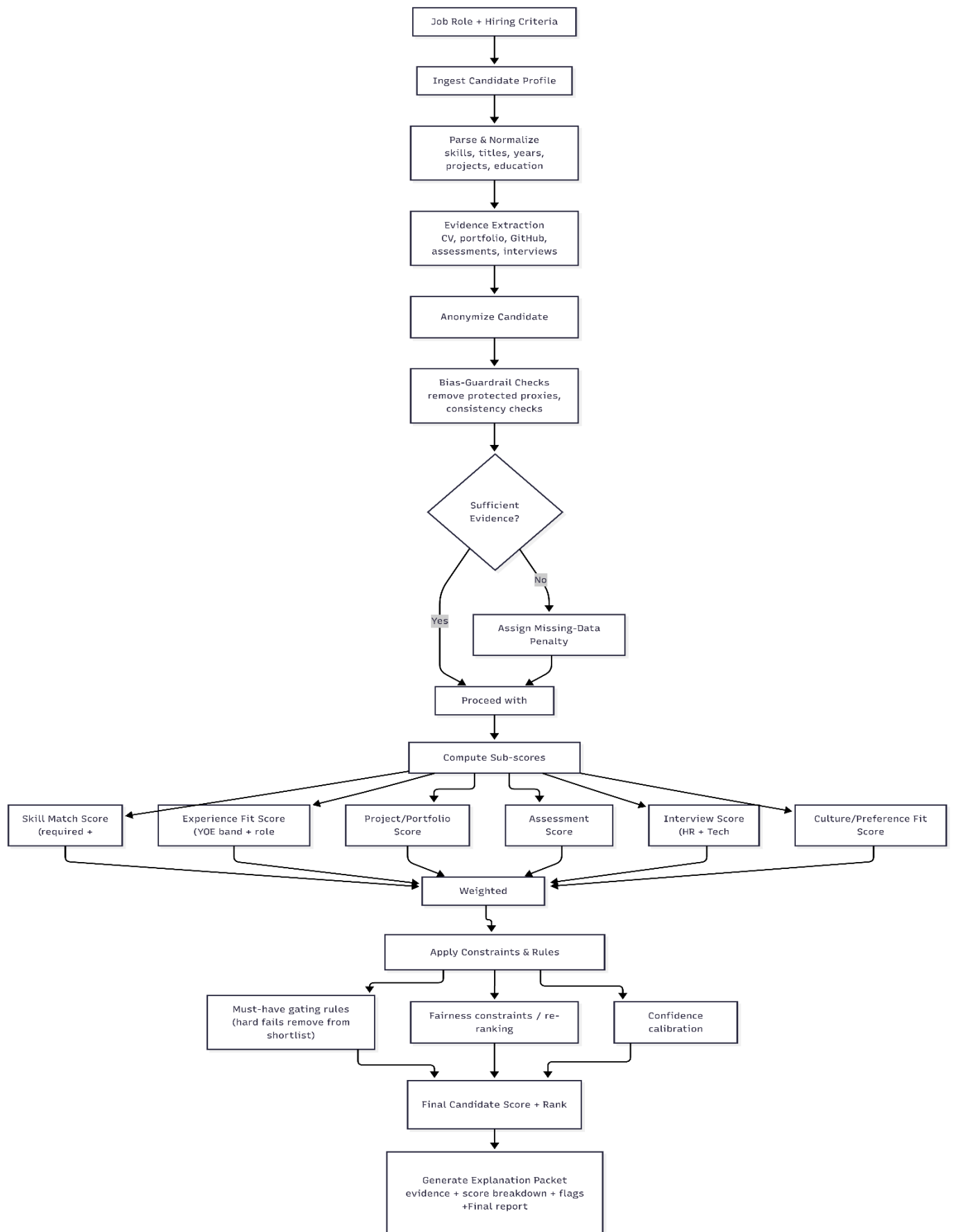


Figure 3.27: Candidate Scoring Pipeline with Evidence Gating, Bias Guardrails, and Explainable Ranking.

As shown in Figure 3.7, PATHS follows a *scoring-first, evidence-driven* pipeline designed to produce a candidate rank that is traceable, auditable, and defensible not just a “black-box” score.

After the recruiter defines the Job Role + Hiring Criteria, the system ingests the candidate profile (CV + any linked sources) and runs a parsing & normalization layer that converts unstructured text into structured fields such as (*standardized job titles, skill entities, years-of-experience bands, project signals, education level*). This stage also performs entity resolution to reduce duplicates and unify identity across sources (e.g., CV vs GitHub vs portfolio). Next, PATHS runs evidence extraction to collect verifiable signals from multiple channels CV claims, portfolio artifacts, GitHub repositories/commits, assessment outputs, and interview transcripts then links each extracted claim to an evidence item (source, timestamp, and confidence).

Before any scoring happens, candidates are anonymized (e.g., removing direct identifiers and sensitive proxies), and then a dedicated bias-guardrail layer checks for protected-proxy leakage and inconsistent or suspicious data (for example, inflated experience, duplicated projects, or mismatched timelines). At this point, the pipeline applies the Sufficient Evidence Gate: if evidence coverage is weak (missing portfolio, missing assessment, incomplete work history, etc.), the system does not “hallucinate confidence”; instead, it applies a missing-data penalty and lowers the score confidence, ensuring candidates are not unfairly ranked too high (or too low) based on thin signals.

Once the candidate passes the gate (or receives the penalty), PATHS computes multiple **sub-scores** in parallel:

- Skill Match Score: overlap between required skills and verified skills (with priority for must-have skills).
- Experience Fit Score: alignment of years-of-experience band + role similarity (title/level/domain).
- Project/Portfolio Score: strength of evidence from real work (complexity, relevance, recency).
- Assessment Score: structured evaluation from technical tasks/tests (when applicable).
- Interview Score: combined HR + technical interview rubric outputs (structured criteria, not “gut feeling”).
- Culture/Preference Fit Score: alignment with constraints (work mode, location, salary range, etc.).

These sub-scores are then aggregated using a weighted model (weights reflect the role type; e.g., technical roles may weight assessment/project signals higher). After weighting, PATHS enforces constraints & rules in three layers:

1. Must-have gating rules (hard fails remove the candidate from shortlist if essential criteria are not met).
2. Fairness constraints / re-ranking to reduce systematic skew while preserving merit-based ordering.
3. Confidence calibration so the final rank reflects both performance and certainty (a strong score with weak evidence is flagged and treated cautiously).

As a final step, the system outputs Final Candidate Score + Rank, and generates an Explanation Packet that includes:

- (a) the evidence items used
- (b) a breakdown of each sub-score and its weight contribution
- (c) missing-data penalties (if applied)
- (d) bias-guardrail flags
- (e) an audit-ready summary explaining *why* the candidate is ranked at that position.

This design directly addresses the project's core risks as biased evaluation, fragmented candidate data, and inconsistent decisions by ensuring every score is backed by evidence and logged for review.

As shown in Figure 3.8. This sequence diagram details execution order, storage points, and human approvals across assessment and interview stages. For technical roles, PATHS enforces an assessment step as a pre-interview filter, assigning a rubric, delivering the assessment, storing answers, and generating pass/fail explanations. A human-in-the-loop checkpoint allows HR/hiring managers to approve continuation or stop early, preventing fully automated rejection without oversight. HR and technical interviews are run as structured sessions using RAG-grounded rubrics and question banks pulled from the organizational knowledge base, and transcripts plus scores are persisted into the ATS/CRM. The Decision Support Agent aggregates CV + assessment + interviews into a decision packet with evidence, after which the final human decision is stored. A GrowthPlan Agent then converts observed gaps into milestones and attaches the plan to the candidate record for development or future re-matching.

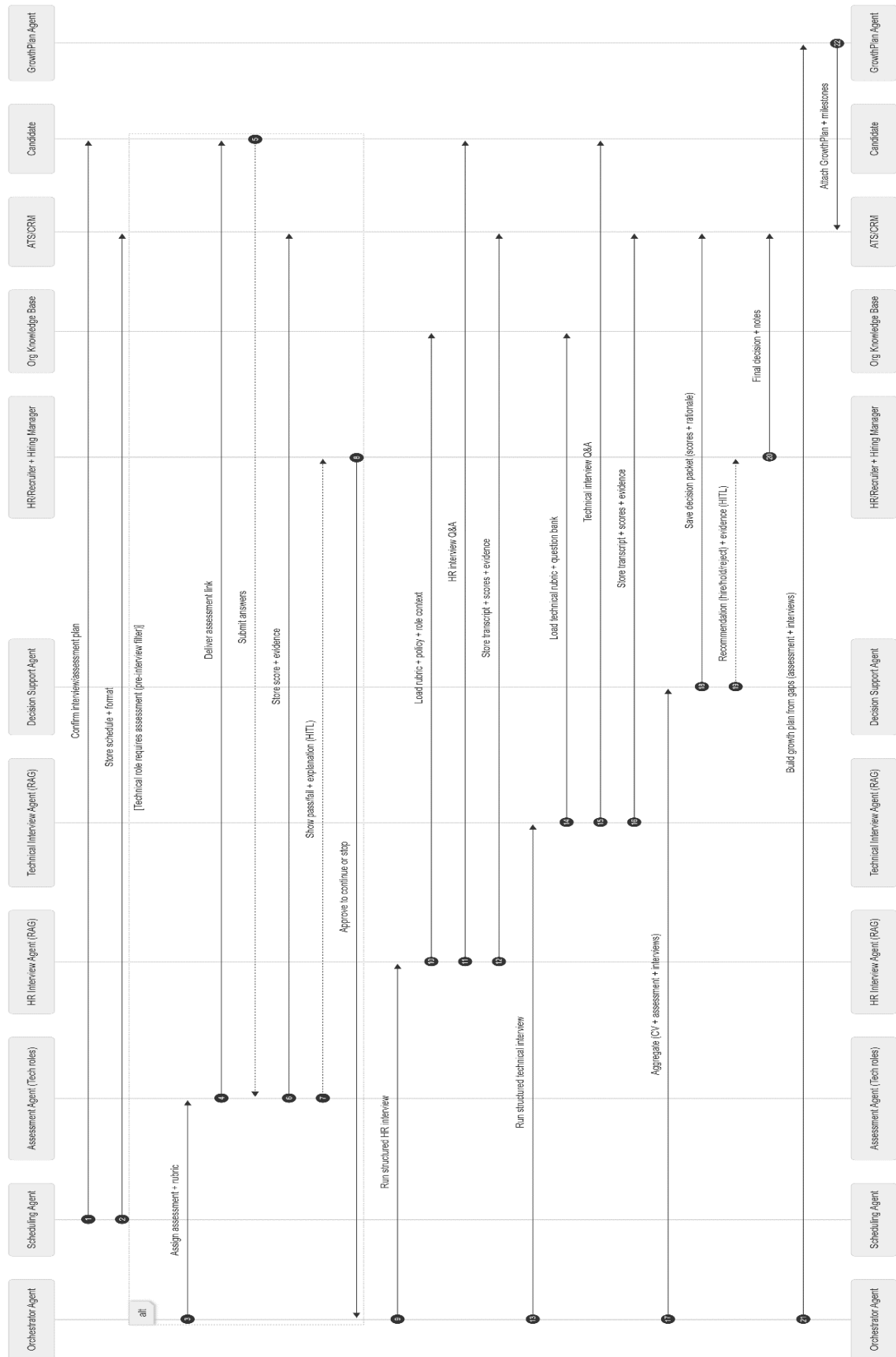


Figure 3.28: End-to-End Evaluation Orchestration with Assessment Gating, RAG Interviews, and HTL Decision Support.

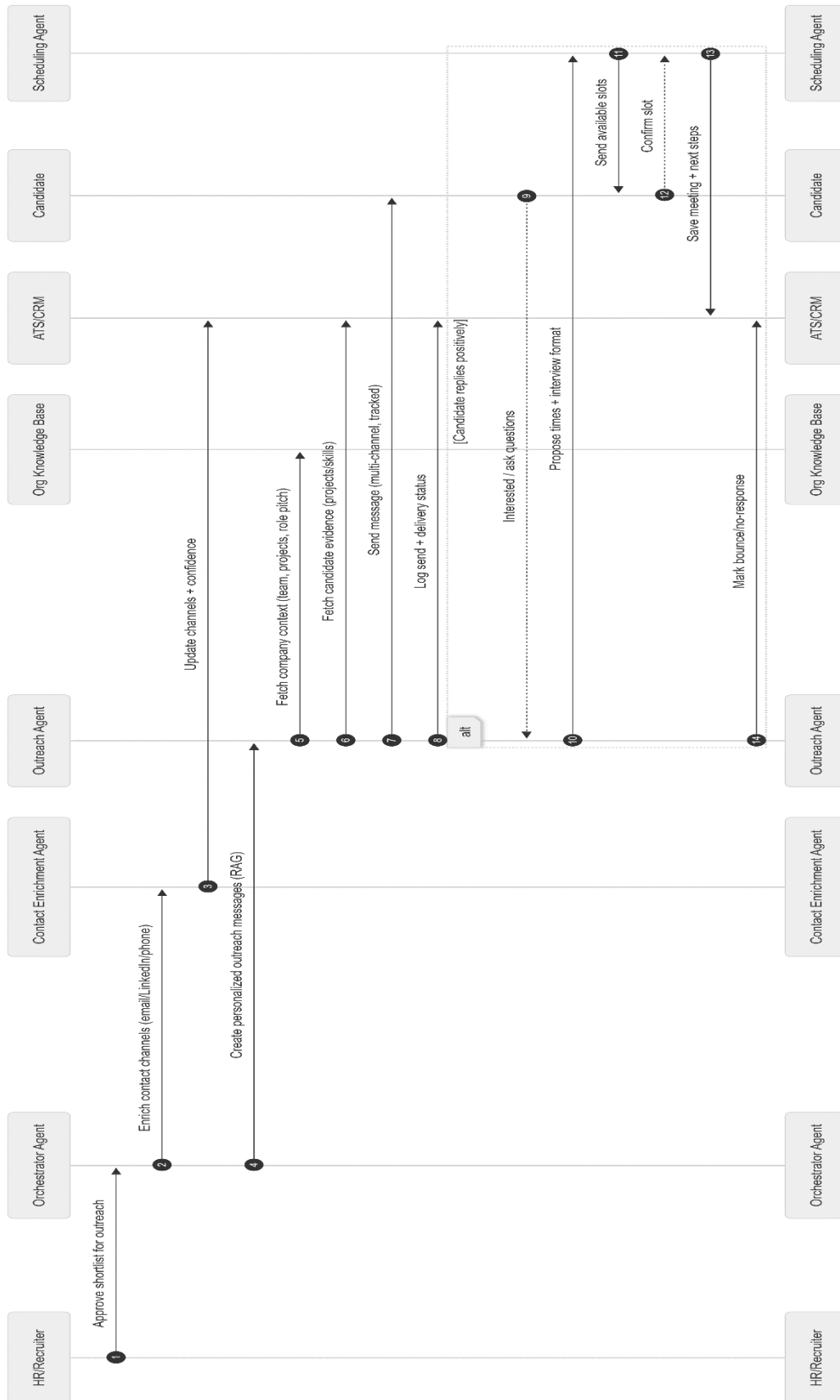


Figure 3.29: Outreach-to-Scheduling Orchestration with RAG Personalization and Multi-Channel Tracking.

As shown in Figure 3.9. This diagram focuses on outbound engagement and converting interest into scheduled evaluations. HR first approves the shortlist for outreach, then the Contact Enrichment Agent identifies candidate contact channels and stores channel confidence in ATS/CRM. The Outreach Agent generates personalized messages using RAG by combining company/team context from the Org Knowledge Base with candidate-specific evidence (projects/skills), then sends tracked multi-channel outreach and logs delivery status. If the candidate responds positively, the system proposes times and formats and the Scheduling Agent collects availability, confirms a slot, and saves meeting details and next steps back into ATS/CRM. If the candidate does not respond or messages bounce, the workflow marks the outcome for retry logic or alternative channel selection.

3.2.1 Functional/ Non-functional Requirements

The inputs include:

- The organization's job requirements (technical skills, responsibilities, seniority, and organizational criteria).
- The sourcing mode selection (Inbound: organization provides candidate data, or Outbound: PATHS searches for candidates).
- Candidate data (CV/resume, portfolio links, LinkedIn/GitHub profiles, and any ATS/imported candidate records).
- Scraped/search results and extracted candidate features (skills, experience, projects, keywords).
- Bias-guardrail configuration (attributes to anonymize or retain only if required by the job).
- Outreach details (available contact information and preferred communication channels).
- Contact enrichment results from MCP tools (e.g., Hunter.io) when contact data is missing or outdated.
- Candidate responses to outreach (accept/decline/no response) and scheduling preferences.
- Organization knowledge base content used by RAG (policies, role description, evaluation rubric, HR guidelines).
- Interview and assessment outcomes (scores, interviewer notes, and feedback).

The outputs include:

- A filtered candidate list that meets the minimum job requirements.
- An anonymized candidate shortlist with Top-K ranking and scoring rationale.
- Enriched and validated contact information (when enrichment is triggered).
- Outreach messages and outreach status tracking (sent/delivered/bounced/replied).
- A scheduled interview and assessment plan (confirmed dates/times and stage flow).
- RAG-generated HR and technical interview/assessment questions aligned to requirements.
- A summarized interview report highlighting key points, strengths, and weaknesses.
- A final decision-support report with final score, justification, and recommendations:
 - If accepted: final notes that support onboarding and development.
 - If rejected: actionable feedback for improvement and potential reapplication.
- A pipeline status outcome (candidate accepted, rejected, or cycle restarted if no Top-K passes).

3.2.2 Functional/ Non-functional Requirements

3.2.2.1 Functional Requirements

3.2.2.1.1 *Organization setup and job intake*

The system shall support organization setup and job intake by capturing the complete job profile, including the role title, responsibilities, must-have and nice-to-have skills, seniority level, location/remote constraints, and all organizational criteria. The system shall also allow configuring the workflow mode as either Inbound (where the organization provides a candidate list and the system starts directly from outreach) or Outbound (where the organization provides only requirements and PATHS performs candidate sourcing). In addition, PATHS shall enable defining an evaluation rubric that includes scoring dimensions, weights, thresholds, and Top-K selection rules, while providing Human-in-the-Loop checkpoints where HR can approve, override, or stop/continue the pipeline.

3.2.2.1.2 Candidate sourcing (Outbound) and data ingestion (Inbound)

The system shall support candidate sourcing and data ingestion depending on the selected mode. In Outbound mode, the system shall scrape/search across defined sources to discover candidates who match the specified requirements. In Inbound mode, the system shall import candidate data provided by the organization through supported formats such as CSV, ATS exports, manual lists, or APIs. The system shall also support building multi-source candidate profiles and maintaining continuous updates when new candidate information becomes available, such as updated projects or job changes.

3.2.2.1.3 Candidate data normalization and entity resolution

The system shall perform candidate data normalization and entity resolution to ensure data consistency and reduce duplication. This includes merging duplicate identities across platforms into a unified candidate record, transforming unstructured candidate data (skills, experience, projects) into structured fields usable by downstream agents, and maintaining provenance by storing which source contributed each field and when it was collected.

3.2.2.1.4 Bias-guardrail anonymization (fair evaluation)

To ensure fairness, The system shall implement a bias-guardrail anonymization stage. The system shall anonymize identity attributes that are not required by the job (such as name, gender, age, and location), and ensure that evaluation agents only access job-relevant signals (skills, experience, portfolio, and assessment data). The system shall also support controlled de-anonymization only after outreach approval, ensuring that identity details are not exposed prematurely.

3.2.2.1.5 Filtering, scoring, and ranking (Top-K)

The system shall support filtering, scoring, and ranking by removing candidates who do not meet minimum requirements and computing a fit score for the remaining candidates based on skills match, seniority alignment, relevant experience, portfolio signals, and other required features. The system shall then rank candidates and produce a Top-K shortlist, including a rationale for each candidate's score and position. HR shall be able to adjust criteria and weights and re-run scoring when needed.

3.2.2.1.6 Outreach and personalization

The system shall support outreach and personalization by generating role-appropriate outreach messages (email/LinkedIn and other channels as applicable) aligned with both the job context and

the candidate's background. The system shall track delivery status and engagement metrics such as sends, bounces, opens, replies, and response rate, and it shall handle outreach failure scenarios including no reply, email bounce, or spam indicators.

3.2.2.1.7 Contact enrichment

The system shall include contact enrichment through MCP integration to address outdated or missing contact information. When required, the system shall trigger enrichment workflows using MCP tools (such as Hunter.io), validate and enrich contact details (including email format and domain health where possible), and store both the confidence level and the source of the enriched information to preserve traceability.

3.2.2.1.8 Scheduling and interview orchestration

The system shall support scheduling and interview orchestration by allowing candidates to select available time slots for assessments, HR interviews, technical interviews (for technical roles), and non-technical interviews (for non-technical roles). The system shall automatically handle time-zone conversions, confirmations, reminders, and also support rescheduling and cancellations to reduce coordination overhead.

3.2.2.1.9 Interview support agents (RAG-based)

The system shall provide interview support agents using RAG, including an HR/Organization RAG Agent that retrieves company policies, role expectations, and evaluation criteria to guide structured HR evaluation, and a Technical RAG Questions Agent that generates role-specific interview and assessment questions aligned with the stated requirements. The system shall ensure that generated questions map directly to the scoring rubric dimensions to maintain consistency between evaluation stages.

3.2.2.1.10 Summary and decision support

The system shall generate summaries and decision support outputs by producing structured interview notes that highlight key points, strengths, and weaknesses, and by computing a final score and producing a final report with justification and recommendations. The report shall include development feedback for rejected candidates and must be exportable in formats suitable for HR workflows, including PDF, document formats, or ATS notes.

3.2.2.1.11 Feedback loop and cycle repeat

The system shall support a feedback loop and cycle repetition. If none of the Top-K candidates pass, the system shall restart the workflow from the beginning (or from sourcing in Outbound mode). The system shall also capture rejection reasons and feed them back into criteria refinement to improve subsequent cycles. In addition, supporting a post-hire development handoff by converting assessment and interview gaps into recommended learning or development plans for accepted candidates, and optionally for rejected candidates as part of constructive feedback.

3.2.2.2 Non-Functional Requirements

3.2.2.2.1 Privacy, Data Protection, and Role-Based Access Control

The system must ensure strong privacy and data protection by encrypting sensitive data both at rest and in transit. The system should follow a data-minimization approach, collecting only what is necessary to execute the recruitment workflow, while enforcing clear data retention and deletion policies. In addition, PATHS must implement role-based access control so that permissions differ across HR staff, interviewers, and administrators, and it must securely handle scraped or enriched personal data through consent-aware and legitimate-use controls, consistent with the Business Problem Statement and the intended governance model.

3.2.2.2.2 Fairness, Bias Mitigation, and Auditability

The system must ensure fairness, bias control, and auditability by enforcing anonymization as a mandatory gate before any scoring or interview evaluation occurs. The system should maintain audit logs covering all major decision-related actions, including score updates, de-anonymization events, and the model/agent outputs used in decision-making. Where legally and ethically permitted, PATHS should provide bias monitoring indicators such as selection-rate comparisons across groups and support compliance-ready reporting for any automated decision-support processes.

3.2.2.2.3 Explain-ability, Transparency, and Confidence Communication

The system must provide explain-ability and transparency by ensuring that every shortlist or ranking outcome includes a human-readable justification describing why a candidate was scored and ranked in that position. The system must clearly separate retrieved evidence (facts extracted or retrieved from sources) from agent inference or reasoning, and it should provide confidence indicators and flags when information is missing, uncertain, or incomplete to avoid misleading conclusions.

3.2.2.2.4 Reliability, Availability, and Failure Recovery

The system must achieve high reliability and availability, particularly for time-sensitive services such as outreach and scheduling. The system should fail gracefully; for example, if a contact enrichment tool fails, PATHS should either fall back to alternative channels or flag the candidate for human HR intervention. In addition, transient failures in APIs, scraping components, or communication services should be handled through retry and recovery mechanisms to maintain pipeline continuity.

3.2.2.2.5 Performance, Scalability, and Concurrency Control

The system must meet performance and scalability requirements by supporting high-volume candidate pools while maintaining acceptable latency across key operations, including search/scraping, Top-K scoring, RAG retrieval, and report generation. The system should enable parallel execution of agents where possible while ensuring that workflow states remain consistent and that concurrency does not lead to duplicate actions or conflicting updates.

3.2.2.2.6 Integration Security and Abuse Prevention

The system must ensure robust security across all integrations and system components. This includes secure storage and rotation of API keys for MCP tools (such as Hunter.io), email providers, and ATS integrations. The system must protect against prompt injection and malicious payloads embedded in resumes, links, or external sources, and it must implement rate limiting and abuse prevention mechanisms, such as outreach throttling and scraping limits, to prevent misuse and service disruption.

3.2.2.2.7 Data Quality, Integrity, and Traceability

The system must maintain high data quality and integrity by setting clear requirements for deduplication accuracy to avoid merging different individuals or splitting one candidate into multiple records. The system should validate structured fields such as emails and phone numbers, enforce consistency in core profile attributes, and maintain versioning for candidate profiles and evaluation results so that decisions remain traceable and reproducible over time.

3.2.2.2.8 Maintainability, Modularity, and Extensibility

The system must support maintainability and extensibility through a modular agent architecture that allows agents to be added, replaced, or upgraded without breaking the overall recruitment workflow. The system should support configuration-driven workflows tailored to different organizations and job roles, and it must maintain clear agent interfaces and input/output schemas to simplify debugging, upgrades, and future expansion.

3.2.2.2.9 Observability, Monitoring, and Operational Alerting

The system must provide strong observability and monitoring through centralized logging of every agent action and each pipeline stage. The system should offer dashboards for key performance indicators such as time-to-hire, response rates, bounce rates, and pass-through rates across stages, and it must generate alerts for abnormal patterns such as sudden bounce spikes, retrieval failures, or scoring drift that may indicate operational or model issues.

3.2.2.2.10 Interoperability and External-System Integration

The system must ensure interoperability and integration by supporting integration with ATS and HR systems for importing candidates, exporting reports, and synchronizing pipeline statuses. It should support common formats such as CSV and JSON, enable resume/PDF parsing and API-based connectors where needed, and maintain consistent identifiers across tools and sources to reduce fragmentation and improve lifecycle tracking.

3.2.2.2.11 Usability and Human-in-the-Loop Control

The system must ensure usability for both HR teams and interviewers by providing a simple and clear interface for setting criteria and weights, reviewing Top-K candidates and rationales, approving de-anonymization, and managing scheduling and summary outputs. Human override must remain easy and always available to preserve the Human-in-the-Loop approach defined in the Business Problem Statement and to ensure that final decision authority remains with human HR professionals.

Chapter 4

Implementation

The chapter is presenting the practical implementation of PATHS, including the software stack, execution environment, deployment topology, security boundary, AI orchestration layer, and the data-flow architecture that connects the web client, FastAPI backend, PostgreSQL/Apache AGE graph layer, Qdrant vector store, and model-inference services. The chapter is written as an implementation record rather than a purely conceptual design: it explains what was built, why each component was selected, and how the components collaborate to support the full recruitment workflow.

4.1 Implementation Overview

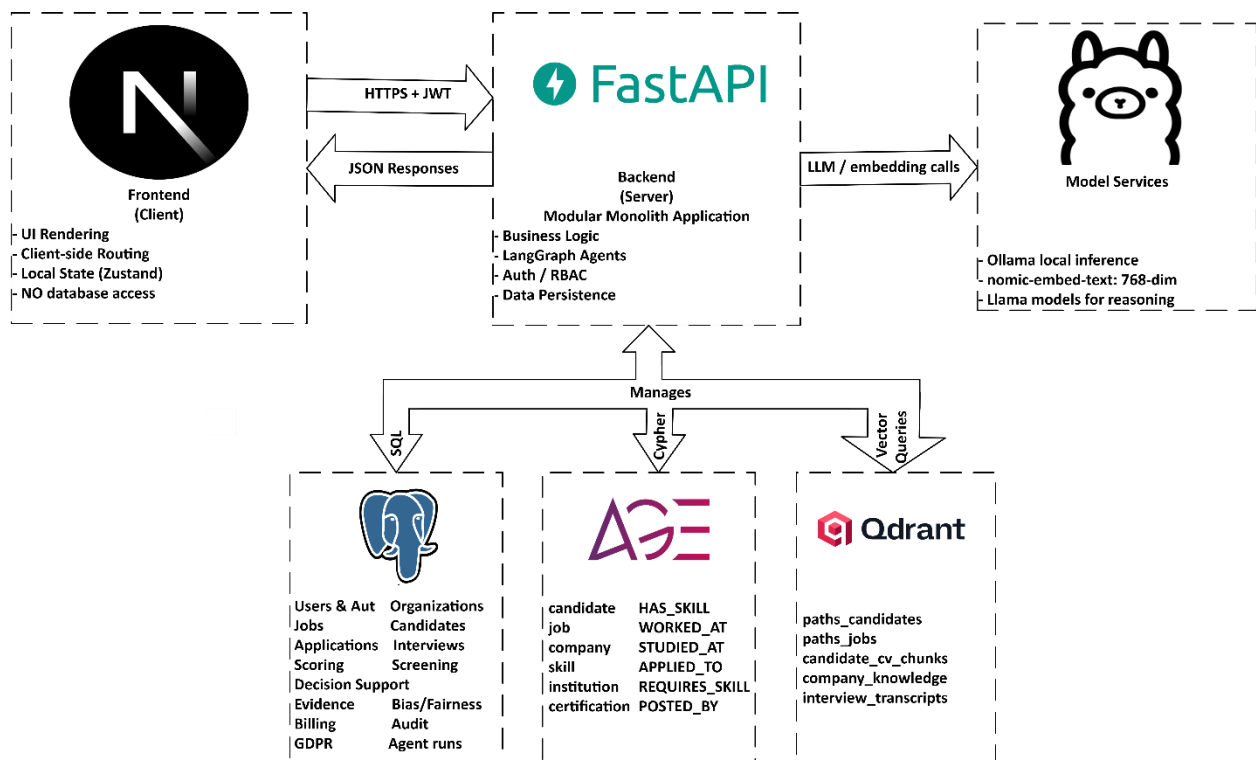


Figure 4.1: high-level implementation architecture showing the separation between the Next.js frontend, FastAPI backend, model services, and the polyglot persistence layer consisting of PostgreSQL, Apache AGE, and Qdrant [73], [78], [96], [97].

The architecture shown in Figure 4.1 follows a **RESTful client-server architecture**, where the frontend and backend are separated into two independent layers that communicate through standardized HTTP requests and JSON responses. In this design, the frontend does not contain business logic related to scoring, database access, AI model execution, or authorization decisions. Instead, it acts as the presentation layer that displays the interface, collects user inputs, and sends requests to the backend. The backend then becomes the trusted application layer responsible for processing requests, enforcing security rules, executing business logic, communicating with databases, and interacting with model services [73].

In RESTful architecture, the system is organized around resources. A resource represents an important object in the system, such as a user, organization, job, candidate, application, interview, score, report, or audit record. Each resource is accessed through a clear API endpoint, and standard HTTP methods define the action being performed. For example, GET is used to retrieve data such as candidate lists or job details, POST is used to create new records such as jobs or applications, PUT or PATCH is used to update existing records such as candidate pipeline status, and DELETE is used to remove or deactivate records when allowed. This makes the communication between the frontend and backend predictable, maintainable, and easy to extend.

In PATHS, this RESTful design is applied through the communication between the Next.js frontend and the FastAPI backend. When a recruiter opens the jobs page, the frontend sends an authenticated request to the backend to retrieve the available jobs. When a candidate uploads a CV, the frontend sends the file and related metadata to the backend. When a hiring manager reviews a decision-support report, the frontend requests the report from the backend, and the backend retrieves the required data from PostgreSQL, Apache AGE, and Qdrant before returning a structured JSON response. Therefore, the frontend is not responsible for knowing where the data comes from; it only communicates with the backend API [62], [73], [78], [96], [97].

A major advantage of using RESTful architecture in PATHS is stateless communication. Each request sent from the frontend to the backend contains the necessary authentication information, usually through a JWT token in the Authorization header. The backend verifies this token for every protected request and checks whether the user has the required role and permissions. This is especially important in PATHS because different users have different access rights. For example, a candidate should only access their own profile and applications, a recruiter can manage jobs and candidates within their organization, a hiring manager can review final decisions, and the platform administrator can manage system-wide settings and monitoring pages.

This architecture also improves security because the frontend never connects directly to PostgreSQL, Apache AGE, Qdrant and Ollama. All database credentials, model service endpoints, API keys, and internal infrastructure details remain hidden inside the backend environment. The backend acts as the only gateway to the data and model layers. This prevents users from bypassing authorization rules and ensures that sensitive operations such as scoring, de-anonymization, interview evaluation, and decision support are validated and logged before execution [60], [78], [96], [97], [98].

The RESTful approach also supports maintainability and scalability. Since the frontend and backend are separated, each layer can be developed, tested, deployed, and updated independently. The frontend team can improve dashboards, forms, and user interaction without modifying database logic, while the backend can improve scoring algorithms, agent workflows, data models, and integrations without changing the visual interface. This separation is suitable for PATHS because the project includes multiple complex modules, such as candidate sourcing, CV ingestion, AI screening, interview scheduling, RAG-based question generation, decision support, and analytics.

RESTful architecture provides a clean integration point for future extensions. If PATHS later needs to support a mobile application, external ATS integration, organization-specific portals, or third-party HR tools, these systems can reuse the same backend APIs. This makes the architecture flexible and future-ready. That RESTful architecture allows PATHS to maintain a clear separation between the user interface, application logic, data persistence, and AI services, while ensuring secure, structured, and traceable communication across the entire recruitment workflow.

4.2 Tools and Libraries

The implementation is divided into a frontend client stack, backend application stack, data and model-inference services, and host-level hardware requirements. The separation below documents the concrete technologies used by each layer and clarifies the responsibility of each component in the final system.

4.2.1 Front-end (Web Application)

The frontend is implemented as a typed React application using Next.js. Its responsibility is to provide interactive dashboards for candidates, recruiters, organization members, hiring managers, and platform administrators. It manages UI state, validates form inputs, and calls backend APIs, but it does not connect directly to any database or model service.

From a technical implementation perspective, the frontend does not connect directly to any database or model service because it is designed as a secure client-side presentation layer rather than a data-processing or persistence layer. The Next.js application is responsible for rendering the user interface, managing client-side routing, storing temporary UI state, and sending authenticated HTTP requests to the backend. It communicates only with the FastAPI server through HTTPS-based REST APIs using JWT authentication. This separation ensures that sensitive database credentials, model service endpoints, API keys, and internal infrastructure details are never exposed to the browser.

This design is important for security, maintainability, and governance. If the frontend connected directly to databases or model services, database credentials and AI service keys could be exposed, users might bypass backend authorization rules, and critical actions such as scoring, de-anonymization, candidate evaluation, and audit logging could become inconsistent or unsafe. By forcing all requests through the backend, PATHS ensures that every operation is validated, logged, and controlled according to the system’s business rules and human-in-the-loop governance model.

Therefore, the client remains thin and secure, while the backend acts as the only trusted gateway to the platform’s databases, knowledge graph, vector store, and AI services. This implementation choice supports a clean three-tier architecture: the frontend handles interaction and visualization, the backend handles business logic and AI orchestration, and the data/model layer handles persistence, retrieval, and inference.

Table 4.1: Frontend technology stack and dependency versions used in the web application.

Concern	Tool/Library	Version
Framework	Next.js (App Router, Turbopack)	16.2.4
UI runtime	React / React-DOM	19.2.4
Language	TypeScript	5.9.3
Client state	Zustand	5.0.12
Server state	TanStack React Query	5.100.5
Forms and validation	React Hook Form + Zod	7.74.0 / 4.3.6
Styling and design system	Tailwind CSS + shadcn/ui + Base UI	4.2.4 / 4.5.0 / 1.4.1
Icons and motion	lucide-react + framer-motion	1.11.0 / 12.38.0
Data visualization	Recharts	3.8.1
User-experience helpers	sonner, cmdk, @dnd-kit	2.0.7 / 1.1.1 / 6.3.1
Frontend observability	@sentry/nextjs	8.55.2
Tooling	pnpm + ESLint	10.33 / 9

As shown in the Table 4.1. the Concern column represents the technical responsibility or architectural area covered by each dependency in the frontend layer. Instead of listing tools randomly, the table groups them according to their role in building, rendering, validating, styling, monitoring, and maintaining the PATHS web application. This makes the implementation clearer because each technology is connected to a specific frontend concern, such as routing, state management, forms, visualization, or developer tooling.

The frontend framework used in PATHS is Next.js 16.2.4, including the App Router and Turbopack. This concern refers to the main application framework responsible for routing, page rendering, server components, and the overall web application structure. Next.js was selected because it provides a modern production-ready React framework with strong support for modular routing, optimized builds, and scalable frontend architecture. Version 16.2.4 was used because it

is the installed project version and is compatible with the selected React 19 runtime, providing the required stability and performance for the PATHS dashboard and user portals.

The UI runtime concern refers to the core component rendering layer of the application. We use React and React-DOM 19.2.4 to build reusable interface components such as dashboards, forms, tables, modals, sidebars, and workflow screens. React was chosen because its component-based model supports maintainable and reusable UI development, which is essential for a system that includes multiple user roles such as candidates, recruiters, hiring managers, organization administrators, and platform administrators. The selected version aligns with the Next.js version used in the project and supports the latest React features required by the application.

The language concern is handled using TypeScript 5.9.3. TypeScript adds static typing to the frontend codebase, helping reduce runtime errors and improving maintainability. In PATHS, this is particularly important because the frontend communicates with many backend API endpoints related to jobs, candidates, scoring, interviews, organizations, analytics, and administrative functions. TypeScript allows API responses, form values, and component properties to be represented with clear types, making the system easier to develop, debug, and extend. Version 5.9.3 was selected because it is the project's locked version and provides modern language support while remaining compatible with the rest of the frontend stack.

The client state concern is managed using Zustand 5.0.12. Client state refers to temporary frontend state that does not necessarily need to be fetched from the server, such as authentication session indicators, interface flags, selected tabs, filters, sidebar state, and local workflow preferences. Zustand was selected because it is lightweight, simple to integrate, and avoids the complexity of heavier global state frameworks. This makes it suitable for PATHS, where most persistent business data is handled by the backend while the frontend only needs efficient local state management. Version 5.0.12 was used because it is stable within the current project dependency set.

The server state concern is handled using TanStack React Query 5.100.5. Server state refers to data fetched from the backend, such as job lists, candidate records, screening results, interview schedules, organization members, analytics metrics, and admin reports. React Query was chosen because it provides built-in caching, background refetching, loading states, error handling, and invalidation after mutations. This is important in PATHS because many pages depend on live backend data and must remain synchronized after actions such as creating jobs, updating candidate stages, running scoring, or scheduling interviews. Version 5.100.5 was selected to match the installed project dependency and to benefit from the mature v5 API.

The forms and validation concern is implemented using React Hook Form 7.74.0 and Zod 4.3.6. This concern covers all user input flows, including login, registration, job creation, candidate profile completion, organization setup, assessment configuration, and administrative forms. React Hook Form was selected because it provides efficient form handling with minimal re-rendering, while Zod provides schema-based validation that ensures form data follows the expected structure before being submitted to the backend. This combination improves reliability and user experience by catching invalid input early. The selected versions were chosen because they are compatible with the current React and TypeScript stack and are locked in the project configuration [63], [67], [68].

The styling and design system concern is covered by Tailwind CSS 4.2.4, shadcn/ui 4.5.0, and Base UI 1.4.1. This concern defines the visual consistency and usability of the PATHS interface. Tailwind CSS provides utility-first styling for fast and consistent layout development, while shadcn/ui and Base UI provide reusable interface primitives and components. These tools were selected because the system contains many repeated UI patterns, including cards, tables, dialogs, buttons, inputs, navigation panels, and dashboard sections. Using a shared design system reduces inconsistency and accelerates development. The selected versions were used because they match the installed frontend dependency versions and provide compatibility with the project's modern React and Next.js setup.

The icons and motion concern is handled using lucide-react 1.11.0 and framer-motion 12.38.0. Icons are used throughout the system to improve readability and make actions easier to recognize, such as job status, candidate stages, analytics indicators, settings, alerts, and navigation items. Framer Motion supports controlled animations that improve user experience without distracting from the workflow. These libraries were selected because they are lightweight, React-friendly, and suitable for a professional dashboard interface. The chosen versions are the project's installed versions and are compatible with React 19.

The data visualization concern is implemented using Recharts 3.8.1. This concern covers charts and visual analytics used in recruiter dashboards, platform analytics, hiring funnels, sourcing-channel breakdowns, response-rate monitoring, and administrative reporting. Recharts was selected because it integrates naturally with React components and provides clear, customizable charting capabilities suitable for business dashboards. In PATHS, this helps transform recruitment activity into measurable indicators such as active jobs, candidate volume, pipeline progress, interview status, and hiring outcomes. Version 3.8.1 was used because it is the installed version and supports the visualization requirements of the application.

The user-experience helpers concern includes sonner 2.0.7, cmdk 1.1.1, and @dnd-kit 6.3.1. This concern focuses on improving interaction quality inside the platform. Sonner is used for toast notifications that inform users about successful actions, errors, warnings, and system updates. cmdk supports command-palette interactions, allowing users to quickly search or trigger actions inside the dashboard. @dnd-kit provides drag-and-drop behaviour, which is useful for interactive workflow interfaces such as candidate pipeline movement, stage management, or configurable boards. These tools were chosen because they improve usability without adding unnecessary complexity, and their selected versions reflect the locked project dependencies.

The frontend observability concern is handled by @sentry/nextjs 8.55.2. Observability refers to the ability to monitor errors, diagnose failures, and understand production issues. In PATHS, this is important because the frontend includes several critical workflows, such as authentication, job creation, screening result display, scheduling, and admin operations. Sentry helps capture client-side errors and provides diagnostic information that supports faster debugging and system maintenance. Version 8.55.2 was selected because it is compatible with the Next.js setup used in the project and provides stable monitoring support for production deployment.

The tooling concern is covered by pnpm 10.33 and ESLint 9. Tooling refers to the development utilities used to manage packages, enforce code quality, and maintain consistency across the team. pnpm was selected because it is fast, disk-efficient, and reliable for managing JavaScript dependencies in a modern frontend project. ESLint was selected to enforce consistent coding standards and detect common errors before runtime. These tools are essential for maintaining a clean and scalable codebase as the frontend grows. The selected versions correspond to the development environment used in the project and ensure consistent installation and linting behaviour across machines.

4.2.2 Backend Server Stack

The backend is implemented in Python using FastAPI and Uvicorn. It exposes the REST API, verifies JWTs, enforces role-based access, validates incoming payloads, runs business logic, coordinates agent workflows, communicates with all persistence services, and writes audit records.

Table 4.2: Backend technology stack used in the web application.

Concern	Tool/Library
Web framework	FastAPI + Uvicorn (ASGI)
Language/runtime	Python 3.12
ORM and migrations	SQLAlchemy + Alembic
Database driver	psycopg (binary)
Configuration and schemas	Pydantic + pydantic-settings
AI orchestration	LangGraph + LangChain

Local model client	langchain-ollama + langchain-text-splitters
Vector store client	qdrant-client
Authentication/security	PyJWT, passlib, bcrypt, argon2-cffi, cryptography
Document ingestion	pypdf, python-docx, BeautifulSoup4
Document generation	reportlab
Scheduling/scraping	APScheduler + optional Playwright
Resilience	tenacity, bleach, httpx, requests
Observability	prometheus-fastapi-instrumentator, sentry-sdk, OpenTelemetry
Testing	pytest + pytest-asyncio

As shown in Table 4.2. which is the backend responsibility covered by each technology in the PATHS implementation. The backend is not described as a random collection of libraries; instead, each library is mapped to a specific implementation concern such as API delivery, database access, configuration management, AI orchestration, authentication, document processing, monitoring, or testing. This makes the backend architecture easier to understand because every selected tool is connected to a clear technical function within the system.

The web framework concern is implemented using FastAPI with Uvicorn as the ASGI server. FastAPI is responsible for exposing the backend API endpoints used by the frontend, validating incoming requests, applying dependency injection, and generating OpenAPI documentation automatically. Uvicorn runs the FastAPI application using the asynchronous ASGI standard, which is suitable for modern web services that handle multiple concurrent requests. This combination was chosen because PATHS requires a fast, modular, and well-documented API layer that can support many services, including authentication, jobs, candidates, scoring, interviews, organizations, analytics, and platform administration.

The language and runtime concern is handled using Python 3.12. Python was selected as the main backend language because it has strong support for artificial intelligence, machine learning, natural language processing, data processing, and API development. Since PATHS depends heavily on AI agents, retrieval-augmented generation, document parsing, embeddings, and model integrations, Python provides a practical ecosystem for implementing both business logic and AI workflows in one backend environment. Python 3.12 was used because it is a modern stable runtime with performance improvements and compatibility with the selected libraries.

The ORM and migrations concern is implemented using SQLAlchemy and Alembic. SQLAlchemy provides a structured way to interact with the relational database using Python models instead of writing all database operations as raw SQL. This improves maintainability and makes database interactions easier to test and extend. Alembic is used for database migrations, allowing the database schema to evolve safely over time as new tables, columns, indexes, and relationships are added. This is important in PATHS because the system contains many relational

entities such as users, organizations, jobs, candidates, applications, interviews, scores, audit logs, and decision-support records.

The database driver concern is handled using `psycopg` in its binary distribution. This library provides the actual connection layer between the Python backend and PostgreSQL. While SQLAlchemy manages the ORM layer, `psycopg` is responsible for executing database communication efficiently and reliably. It was selected because PostgreSQL is the main system-of-record database in PATHS, and the backend needs stable connectivity for transactional operations such as user authentication, job creation, candidate updates, application tracking, scoring persistence, and audit logging.

The configuration and schemas concern is implemented using `Pydantic` and `pydantic-settings`. `Pydantic` is used to define request and response schemas, ensuring that data exchanged between the frontend and backend follows the expected structure and types. It also helps validate API inputs before they reach the business logic layer. `pydantic-settings` is used to manage environment-based configuration such as database URLs, API keys, JWT secrets, CORS settings, model service endpoints, and deployment-specific values. These tools were chosen because PATHS requires strict validation, typed configuration, and secure separation between code and environment-specific secrets.

The AI orchestration concern is handled using `LangGraph` and `LangChain`. `LangChain` provides integrations and utilities for building LLM-based applications, including retrieval, prompt construction, tools, and model connections. `LangGraph` extends this by supporting stateful, multi-step workflows, which is important for agentic systems. In PATHS, these tools are used to coordinate workflows such as candidate scoring, screening, sourcing, decision support, outreach personalization, and interview preparation. They were selected because PATHS is not based on a single model call; it requires controlled multi-step reasoning pipelines with memory, state, tool usage, and human-in-the-loop checkpoints.

The local model client concern is implemented using `langchain-ollama` and `langchain-text-splitters`. `langchain-ollama` allows the backend to communicate with local Ollama models for embedding generation and local LLM inference. This supports privacy-preserving and cost-controlled model execution when cloud APIs are not required or unavailable. `langchain-text-splitters` is used to divide long documents, such as CVs, company policies, job descriptions, and interview transcripts, into smaller chunks suitable for embedding and retrieval. These tools were chosen because PATHS depends on retrieval-augmented generation and semantic search, both of which require clean text chunking and reliable embedding generation.

The vector store client concern is handled using qdrant-client. Qdrant stores vector embeddings that represent candidates, jobs, CV chunks, company knowledge documents, and interview transcripts. The qdrant-client library allows the backend to create collections, insert embeddings, search by semantic similarity, and retrieve relevant evidence for RAG workflows. It was selected because PATHS requires semantic matching rather than only keyword-based search. This enables the system to compare candidates and jobs based on meaning, retrieve relevant evidence from documents, and support explainable AI-assisted recommendations.

The authentication and security concern are implemented using PyJWT, passlib, bcrypt, argon2-cffi, and cryptography. PyJWT is used to issue and verify JSON Web Tokens for stateless authentication between the frontend and backend. passlib, bcrypt, and argon2-cffi support secure password hashing, ensuring that user passwords are not stored in plain text. cryptography provides lower-level encryption utilities when secure handling of sensitive data is required. These tools were selected because PATHS handles sensitive recruitment data, including candidate profiles, organization records, assessments, and hiring decisions. Therefore, secure authentication, password protection, and role-based access control are essential implementation requirements.

The document ingestion concern is handled using pypdf, python-docx, and BeautifulSoup4. Which is needed to process unstructured documents such as candidate CVs, resumes, company knowledge documents, job descriptions, and scraped pages. pypdf is used for extracting text from PDF files, python-docx is used for reading Word documents, and BeautifulSoup4 is used for parsing HTML content from web pages or scraped sources. These libraries were chosen because recruitment data often arrives in different file formats, and the system must convert these sources into structured and searchable text before applying embedding, scoring, or RAG-based analysis.

The document generation concern is implemented using reportlab. This library is used to generate PDF reports, such as candidate decision reports, interview summaries, scoring explanations, and development feedback documents. Report generation is important in PATHS because the system must produce outputs that can be reviewed by recruiters, hiring managers, candidates, and administrators. reportlab was selected because it allows the backend to generate structured PDF documents programmatically, supporting auditability and professional reporting.

The scheduling and scraping concern are handled using APScheduler and optional Playwright. APScheduler is used to run periodic or time-based tasks, such as scheduled data ingestion, background checks, job-posting updates, or automated maintenance workflows. Playwright can be used when browser automation is required for scraping or interacting with web pages that cannot be accessed through simple HTTP requests. These tools were chosen because PATHS includes

workflows that may need to run automatically over time, especially in sourcing, external job ingestion, and automated recruitment pipeline updates.

The resilience concern is handled using `tenacity`, `bleach`, `httpx`, and `requests`. `tenacity` provides retry mechanisms for operations that may fail temporarily, such as external API calls, model-service communication, or network-dependent tasks. `bleach` is used for sanitizing user-generated or external HTML content, reducing the risk of unsafe input. `httpx` and `requests` are used for outbound HTTP communication with external services and APIs. These tools were chosen because PATHS integrates with several external and internal services, and the backend must handle failures, sanitize inputs, and communicate reliably with third-party systems.

The observability concern is implemented using `prometheus-fastapi-instrumentator`, `sentry-sdk`, and `OpenTelemetry`. Observability allows developers and administrators to understand what is happening inside the backend during development and production. Prometheus instrumentation exposes metrics such as request counts, latency, and error rates. Sentry captures exceptions and runtime errors. OpenTelemetry supports distributed tracing and deeper visibility into request flows. These tools were selected because PATHS contains multiple backend services and AI workflows, so monitoring is necessary to detect failures, performance bottlenecks, model-service errors, and unstable pipeline behaviour.

The testing concern is handled using `pytest` and `pytest-asyncio`. `pytest` is used to write and execute unit and integration tests for backend logic, API endpoints, services, and utility functions. `pytest-asyncio` extends testing support to asynchronous components, which are common in FastAPI applications and AI-service calls. These tools were chosen because PATHS requires reliable backend behavior across many workflows, including authentication, candidate management, scoring, document ingestion, scheduling, and decision support. Testing helps verify correctness before deployment and reduces the risk of breaking critical recruitment workflows.

The backend stack was selected to support a secure, modular, AI-enabled, and maintainable implementation. FastAPI provides the API foundation, SQLAlchemy and Alembic manage structured persistence, Pydantic enforces validation, LangGraph and LangChain orchestrate agentic workflows, Qdrant supports semantic retrieval, and the security, monitoring, resilience, and testing libraries ensure that the system can operate reliably in a real recruitment environment. The selected tools align with the core architecture of PATHS, where the backend acts as the trusted control layer between the frontend, the databases, the vector store, and the AI model services [73], [75], [76], [97], [101], [102], [103].

4.2.3 Data and Model-Inference Services

PATHS uses three main data services and two model-inference paths. PostgreSQL acts as the system of record, Apache AGE stores relationship-oriented recruitment knowledge inside the PostgreSQL engine, Qdrant stores embeddings for semantic retrieval, Ollama enables local inference. The embedding dimension is standardized at 768 dimensions using the nomic-embed-text model. Standardizing the vector size across all collections avoids schema mismatch problems and simplifies retrieval pipelines across candidate profiles, job descriptions, CV chunks, company knowledge, and interview transcripts.

Table 4.3: Data and inference services used in the system.

Service	Image / engine	Default Internal Port
PostgreSQL + Apache AGE	apache/age:latest	5432
Qdrant	qdrant/qdrant:latest	6333
Ollama	ollama/ollama:latest	11434

The Table 4.3. represents the infrastructure component responsible for storing, retrieving, or generating data inside the PATHS platform. Unlike frontend or backend libraries, these services run as separate engines or containers and are accessed by the FastAPI backend through internal network connections. Each service has a dedicated responsibility in the architecture: PostgreSQL with Apache AGE stores structured and graph-based data, Qdrant stores vector embeddings, Ollama performs local model inference.

The Image / engine column identifies the container image or execution environment used to run each service. In the development environment, Docker images are used to make the infrastructure reproducible and easy to launch on any machine. For example, `apache/age:latest` runs PostgreSQL with the Apache AGE extension included, `qdrant/qdrant:latest` runs the Qdrant vector database, and `ollama/ollama:latest` runs local model inference.

The Port column indicates the network port through which each service receives requests. PostgreSQL with Apache AGE uses port 5432, Qdrant uses port 6333, and Ollama uses port 11434. These ports are useful in local development because the backend can connect to services running on the same machine or inside the Docker Compose network.

PostgreSQL with Apache AGE is used as the main structured data and graph storage engine in PATHS. PostgreSQL stores transactional records such as users, organizations, jobs, candidates, applications, interviews, scores, billing records, audit logs, and decision-support outputs. Apache AGE extends PostgreSQL with graph database capabilities, allowing PATHS to model recruitment relationships as connected entities such as candidates, jobs, companies, skills, institutions, certifications, and applications. This choice is useful because it keeps relational and graph data

close to each other inside one database engine, reducing architectural complexity while still supporting graph-style queries for relationship-based recruitment analysis.

Qdrant is used as the vector database layer. Its responsibility is to store embeddings generated from candidate profiles, CV chunks, job descriptions, company knowledge documents, and interview transcripts. These embeddings allow PATHS to perform semantic search and similarity-based retrieval. This is important because recruitment matching cannot depend only on exact keywords; two candidates may describe the same skill using different terms, and two job descriptions may require similar capabilities without using identical wording. Qdrant allows the system to compare meaning rather than only text overlap, which supports RAG, candidate-job matching, evidence retrieval, and explainable decision support.

Ollama is used as the local model-inference service. It allows the backend to run local models such as `nomic-embed-text` for embedding generation and `llama3.1:8b` for local LLM execution. This is useful during development and demonstrations because the system can perform AI-related tasks without depending entirely on paid cloud services. It also improves privacy control because sensitive recruitment data can be processed locally when required. In PATHS, Ollama supports embedding generation, local reasoning fallback, and offline AI functionality.

In the development environment, these services are typically launched using Docker Compose. The containers run on the same machine, and their ports may be exposed to localhost so the developer can inspect, test, and debug them easily. For example, the backend can connect to PostgreSQL, Qdrant, and Ollama through Docker service names or local ports. This setup is suitable for implementation, testing, and graduation-project demonstrations because it is simple, reproducible, and does not require a complex cloud infrastructure.

In deployment, the architecture changes mainly in terms of exposure, security, persistence, and scalability. The backend may still run as a container, but databases and model services should not be publicly exposed to the internet. PostgreSQL with Apache AGE should be placed behind a private network, protected with strong credentials, persistent storage, automated backups, and restricted access. Qdrant should also run with persistent storage and private access because it contains semantic representations of sensitive candidate and organization data. Ollama may run on a separate machine or GPU-enabled instance if local inference is required in production, because LLM inference is resource-intensive and may not be suitable for small CPU-only servers.

Another important deployment difference is that some services may be replaced by managed cloud alternatives. PostgreSQL can be deployed as a managed database if the hosting provider supports the required extensions; however, Apache AGE may require a self-managed PostgreSQL instance if the managed provider does not support that extension. Qdrant can be deployed either

as a self-hosted container or through a managed Qdrant Cloud service. Ollama can be self-hosted on a GPU server, replaced with another model-serving provider [78], [85], [96], [97], [98].

Therefore, the main difference between development and deployment is that development prioritizes simplicity and local accessibility, while deployment prioritizes security, reliability, persistence, and controlled access. In development, ports may be exposed for convenience; in production, database and model-service ports should be internal only. In development, one machine can run the full stack; in production, services may be separated across private networks, managed databases, GPU instances, or cloud APIs. This separation ensures that PATHS remains secure and scalable while preserving the same logical architecture: the frontend communicates only with the backend, and the backend is the trusted gateway to the relational database, knowledge graph, vector database, and model services.

4.2.4 Host Requirements

PATHS is a software-only system. Therefore, the relevant hardware consists of the host machines that run the application and its supporting services. The development environment can operate on a single workstation, while deployment benefits from additional memory and storage throughput because local inference, vector storage, and graph queries can be resource-intensive.

Table 4.4: Development Baseline, Current and Recommended Cloud Deployment and Requirements.

Resource	Development Baseline	Current Cloud Server	Recommended Cloud Deployment
Operating system	Windows 11 Pro with Linux containers	Ubuntu 24.04 LTS	Ubuntu 24.04 LTS
Cloud provider	Localhost / developer machine	AWS EC2, eu-north-1	AWS EC2 or equivalent VPS/cloud VM
Instance type	Local machine	m7i-flex.large	4 vCPU cloud VM
CPU	8-core local CPU	2 vCPU	4 vCPU
RAM	16 GB	8 GB	8 GB
Swap space	Not required locally if RAM is sufficient	8 GB swap file	8 GB swap file
Storage	SSD with approximately 200 GB	30 GB gp3 disk	50 GB SSD/gp3 disk
Network access	Localhost	Public IP + domain	Public IP + domain
Public ports	Local development ports only	80, 443, 22	80, 443, 22
Internal service ports	3000 and 8001 locally	3000 and 8001 behind proxy	3000 and 8001 behind proxy
Container orchestration	Docker Desktop + Docker Compose	Docker Engine + Docker Compose plugin	Docker Engine + Docker Compose plugin

Reverse proxy	Not required locally	Caddy 2	Caddy 2
Database storage	Local Docker volumes	Docker volume on gp3 disk	Backed-up Docker volume on 50 GB disk
Vector storage	Local Docker volume	Qdrant Docker volume	Backed-up Qdrant volume
Local model storage	Local Ollama cache	Ollama Docker volume	Ollama Docker volume or hosted embeddings
Security model	Local-only access	HTTPS via Caddy	HTTPS via Caddy with restricted SSH

As shown in the Table 4.4. showing the resource, the development baseline and the current cloud server. development baseline represents the environment used during implementation and testing before production deployment. Since the PATHS application is composed of several backend, frontend, database, vector search, and AI-related services, local development was performed using Docker-based Linux containers on a Windows 11 Pro machine. This made it possible to run a Linux-compatible production-like environment while still using a standard developer workstation. The local machine used an 8-core CPU, 16 GB of RAM and 8 GB of GPU, which was sufficient for writing code, running containers, tuning agents, testing APIs, building the frontend, and validating the interaction between services.

The production version deployed on AWS cloud server. This decision was necessary because the system had to be accessible through a public domain, support HTTPS traffic, and remain available for testing and demonstration from different devices. The current cloud server is hosted on AWS EC2 in the eu-north-1 region using an m7i-flex.large instance. This instance provides 2 vCPUs and 8 GB of RAM, which proved sufficient for the current running production workload. The operating system is Ubuntu 24.04 LTS, which was selected because it is stable, widely supported, and well-suited for Docker-based production deployments.

The CPU requirement differs between development and production because each environment has a different purpose. In development, the 8-core local CPU helps with running the editor, Docker Desktop, browser testing, backend services, frontend builds, and debugging tools at the same time. In production, the current 2 vCPU server is enough for the measured workload because the application is not continuously CPU-heavy during idle operation. However, a 4 vCPU server is recommended for a cleaner deployment because it provides more headroom during Docker image builds, backend processing, database activity, concurrent user requests, and local embedding operations.

Memory is one of the most important resources in the system deployment because several services run together in the same Docker Compose stack. The system includes PostgreSQL with

Apache AGE, Qdrant, Ollama, the FastAPI backend, the Next.js frontend, and the Caddy reverse proxy. Although the measured runtime memory consumption is not high during idle operation, some operations are memory-sensitive, especially the frontend build process and local AI-related services. For this reason, the current cloud server uses 8 GB of RAM together with an 8 GB swap file. The swap file acts as a safety layer that reduces the risk of out-of-memory failures during temporary spikes [78], [85], [96], [97], [98].

Storage is the main limitation in the current production deployment. The current server uses a 30 GB gp3 disk, but this is considered tight because Docker images, build cache, Ollama model files, database pages, graph data, vector collections, and persistent volumes all consume disk space over time. In particular, Ollama is one of the heaviest local components because it stores model-related files. Therefore, the recommended production storage size is 50 GB. This gives the system enough space for application images, database growth, vector embeddings, logs, cache, backups, and future updates without immediately creating disk pressure.

A GPU is optional for the correctness, while Ollama is mainly used locally for inference and embedding generation. This means the system can operate correctly on a CPU-only server. However, a GPU may become useful in future versions if the system is expanded to run larger local language models, heavier embedding workloads, or more advanced AI inference directly on the server.

The network configuration is different between local development and production. During development, services can safely run on localhost because they are only accessed by the developer. In production, the system requires a public IP address and a domain name so users can access the platform through the browser. The public ports are limited to port 443 for HTTPS, port 80 for HTTP redirection and certificate validation, and port 22 for SSH administration. Internal application ports such as 3000 for the frontend and 8001 for the backend are kept behind the reverse proxy and should not be exposed directly to the public internet.

Docker Compose was used as the main orchestration mechanism because it allows all system services to be defined, started, stopped, and managed together. This is suitable for the system because the system depends on multiple services that must communicate reliably: the frontend, backend, relational and graph database, vector database, local embedding service, and reverse proxy. Using containers also improves reproducibility because the same service definitions can be reused across development, testing, and production with limited configuration changes.

Caddy was used as the production reverse proxy because it simplifies secure deployment. Instead of exposing the backend and frontend directly, Caddy receives public traffic and routes it internally to the correct service. It also manages HTTPS certificates automatically using Let's

Encrypt. This makes the deployment more secure and professional, especially because users access the platform through a public domain rather than raw IP addresses or direct development ports.

The persistent storage layer is critical because the system stores important recruitment data, including user accounts, organizations, jobs, candidates, skills, applications, graph relationships, vector embeddings, and AI-generated outputs. PostgreSQL with Apache AGE stores relational and graph data, while Qdrant stores vector embeddings used for semantic search and retrieval-augmented generation. These volumes must be treated as production assets, not temporary container files. Therefore, regular backups are required to protect the system from data loss and to support safe redeployment or migration to a larger server in the future.

4.3 Environment and Platform

4.3.1 Container-First Runtime Topology

PATHS is designed as a container-first platform. The infrastructure stack is described through Docker Compose so that PostgreSQL/AGE, Qdrant, Ollama, the backend API, and the web client can be started in a predictable topology. This reduces environmental differences between development, testing, and demonstration machines. Inside the Compose network, services communicate through container hostnames such as postgres, qdrant, and ollama. Outside the network, the browser communicates with the published backend endpoint. This distinction keeps internal database and model endpoints separate from the public client surface [78], [85], [97], [98].

Table 4.5: Container Port Mapping.

Container	Published Port	Internal Port
paths_postgres PostgreSQL + Apache AGE	5432	5432
paths_qdrant Qdrant	6333	6333
paths_ollama Ollama	11434	11434
paths_backend FastAPI	8001	8000
paths_web Next.js	3000	3000

The Table 4.5. shows the containers port mapping. *paths_postgres* container represents the main transactional and graph database layer of the platform. It runs PostgreSQL together with Apache AGE, allowing the system to store both conventional relational records and graph-based recruitment relationships in the same database environment. This is important because the system does not only store simple tables such as users, organizations, jobs, candidates, and applications; it also needs to model relationships between candidates, skills, jobs, organizations, evidence, and growth plans. When the database container starts for the first time, the initialization script

automatically creates the `paths_graph` graph, ensuring that the graph environment is ready without requiring manual database setup after deployment.

The `paths_qdrant` container is responsible for vector storage and semantic retrieval. In PATHS, vector search is needed because candidate profiles, job descriptions, CV content, organizational knowledge, and interview-related information are not always matched effectively through exact keywords only. Qdrant stores embeddings in a persistent vector collection so that the system can retrieve semantically related information during matching, ranking, and Retrieval-Augmented Generation workflows. Its persistent volume, `paths_qdrant_data`, ensures that vector data is not lost when the container is restarted or updated.

The `paths_ollama` container provides the local AI model service used mainly for embedding-related operations. Instead of treating AI processing as a temporary runtime feature, Ollama is deployed as a dedicated service with persistent storage. Its model files and inference cache are stored through the `paths_ollama_data` volume, which avoids downloading or rebuilding model data after every deployment. This improves stability and reduces deployment time. Although the system can use hosted LLM inference for larger reasoning tasks, keeping Ollama locally gives PATHS more control over embedding generation and semantic indexing.

The `paths_backend` container runs the FastAPI application, which acts as the core service layer of the PATHS system. It exposes the application programming interfaces used by the frontend and coordinates communication with the database, vector store, embedding service, and AI workflow components. The backend is published on port 8001 externally while the application itself listens internally on port 8000. This separation makes it easier to map the service into the Docker environment while keeping the internal application configuration stable. During startup, the backend waits until PostgreSQL passes its health check, which prevents the application from starting before the database is ready to accept connections.

The `paths_web` container runs the Next.js frontend application, which provides the user interface for recruiters, hiring managers, candidates, and administrators. It is published on port 3000 and also listens internally on port 3000. The frontend is configured using `NEXT_PUBLIC_API_URL`, which tells the browser where to send API requests. This configuration is essential because the frontend and backend are separate services, and the browser must know the correct backend endpoint in both development and production environments.

This container structure separates the responsibilities of the platform into clear operational units. The database handles relational and graph persistence, Qdrant supports semantic search, Ollama supports local embedding operations, FastAPI manages backend logic and AI workflow orchestration, and Next.js provides the user-facing interface. This separation improves

maintainability because each service can be updated, restarted, monitored, and scaled independently without affecting the entire system. It also makes the deployment easier to reproduce because all services are defined and managed through Docker Compose rather than being installed manually on the server.

4.3.2 Configuration and Environment Variables

Configuration is externalized through environment variables and documented in the project environment template. The most important variables include DATABASE_URL, QDRANT_URL, OLLAMA_BASE_URL, AGE_GRAPH_NAME, CORS_ORIGINS and SECRET_KEY. The same application image can therefore run-in different environments by changing configuration rather than editing source code.

Table 4.6: Key configuration variables

Variable	Purpose
DATABASE_URL	Connection string for PostgreSQL/AGE.
QDRANT_URL	HTTP endpoint for Qdrant vector storage.
OLLAMA_BASE_URL	Local inference endpoint for embeddings and local LLM calls.
AGE_GRAPH_NAME	Name of the recruitment knowledge graph, defaulting to paths_graph.
CORS_ORIGINS	Allowed web origins for browser requests.
SECRET_KEY	JWT signing secret and security-sensitive application secret.

4.3.3 System Inputs and Outputs

Table 4.7: Principal input categories

Input category	Implementation handling
Candidate resumes	PDF or DOCX resumes parsed and chunked for profile extraction and embedding.
Candidate profile data	Skills, education, experience, certifications, preferences, and account info. entered by onboarding.
Job descriptions	Organization-created job posts and externally ingested job postings.
Organization knowledge documents	Company policies, culture documents, rubrics, onboarding materials, and evaluation guidelines.
Interview transcripts	Textual interview records embedded for screening, summarization, and decision support.
Recruiter/admin actions	Pipeline changes, scoring requests, member invitations, feature flags, and settings.
Natural-language queries	Assistant questions answered through RAG over graph and vector data.

The system accepts both structured and unstructured inputs. Structured inputs include job requirements, candidate profile fields, recruiter actions, and workflow decisions. Unstructured inputs include resumes, organization documents, and interview transcripts. The backend converts these inputs into normalized records, graph relationships, vector embeddings, and auditable decision-support outputs. As shown in the Table 4.7.

Table 4.8: Principal output categories.

Output category	Purpose
Candidate-job match scores	Ranked fit indicators supported by evidence and confidence signals.
Ranked shortlists	Top-K candidate lists for recruiter review and human approval.

Screening summaries	AI-assisted summaries of screening, assessment, and interview outcomes.
Fairness and bias checks	Anonymization status, guardrail flags, and reviewable risk indicators.
Evidence trails	Source-linked support for candidate skills, experience, projects, and interview claims.
Decision-support recommendations	Structured decision packets for hiring manager review.
Exported reports	PDF or document-ready outputs for decision records and HR workflows.

As shown in the Table 4.8. the system produces several structured outputs that support the recruitment workflow from candidate evaluation to final decision-making. These outputs include candidate-job match scores, ranked shortlists, screening and interview summaries, fairness and bias-check indicators, evidence trails, decision-support recommendations, and exportable reports. Each output is generated to help recruiters and hiring managers understand not only which candidates are suitable, but also why they were recommended. By presenting scores together with supporting evidence, confidence signals, and human-review checkpoints, the system ensures that AI-generated results remain transparent, traceable, and practically useful within the hiring process.

4.4 Backend Implementation and Module Organization

Although the backend is deployed as one service, it is organized as a set of modules that reflect the main recruitment domains. This organization supports maintainability, testing, and future extraction of services if the platform scales beyond the modular-monolith stage. The routing layer is kept thin. API routers validate requests and delegate business operations to services. Services implement domain rules and coordinate repository operations, agent calls, and audit entries. This separation helps avoid mixing HTTP details with recruitment logic. The repository/data-access layer isolates database-specific operations. SQLAlchemy is used for relational tables, AGE/Cypher operations are used for graph updates and relationship queries, and the Qdrant client is used for vector upserts and semantic retrieval. This structure prevents domain services from depending on raw database connection details. Long-running operations such as document ingestion, external job import, embedding generation, and outreach workflows are designed to be handled asynchronously or through scheduled jobs where appropriate. This prevents the user interface from blocking while computational or network-heavy tasks complete [96], [97], [101].

Table 4.9: Backend modules and their implementation responsibilities.

Module	Responsibility
Authentication and users	Registration, login, password hashing, JWT issuance, session validation, and role metadata.
Organizations and members	Organization profiles, member invitations, roles, membership status, and administrative settings.
Jobs and requirements	Job creation, status management, criteria capture, sourcing mode, and role-specific evaluation configuration.
Candidates and profiles	Candidate records, CV ingestion, skills, education, experience, enrichment sources, and duplicate resolution.

Applications and pipeline	Candidate-job relationships, application status, shortlist rank, source platform, and pipeline stage transitions.
Scoring and screening	Fit-score calculation, evidence retrieval, matching confidence, bias guardrails, and explanation packet generation.
Interviews and assessments	Scheduling, interview type, transcript storage, question generation, assessment outputs, and summary records.
Decision support	Final decision packet, hire/reject support, development-plan handoff, and report export.
Agents and runs	Stateful agent execution, tool calls, retrieved context, model responses, and run-level observability.
Platform administration	Platform-wide overview, analytics, access requests, organizations, users, agent monitor, system health, feature flags, audit logs, settings, and graph/vector database inspection.

The Table 4.9. shows the backend implementation of the system which was organized into several functional modules to keep the system maintainable, scalable, and easy to extend. Each module is responsible for a specific part of the recruitment workflow, while all modules work together to deliver the complete hiring pipeline. This modular structure also reduces complexity because authentication, job management, candidate processing, scoring, interviews, decision support, and administration are not mixed into one large service. Instead, each part of the platform has a clear responsibility and communicates with the other modules through well-defined backend APIs.

The authentication and user's module forms the security foundation of the system. It manages user registration, login, password hashing, JWT token generation, session validation, and role metadata. This module ensures that each user is correctly identified before accessing the platform and that permissions can be enforced according to the user's role, such as recruiter, hiring manager, interviewer, candidate, or platform administrator. Since PATHS handles sensitive recruitment data, secure authentication is essential to prevent unauthorized access and protect candidate and organization information.

The organizations and members module manages the organizational structure inside the platform. It stores organization profiles, member invitations, user roles, membership status, and administrative settings. This allows each organization to manage its own hiring team while keeping its data separated from other organizations. For example, an organization can invite recruiters, hiring managers, or interviewers and assign each member a specific role. This module is important because PATHS is not designed for a single user only; it supports collaborative hiring where different people participate in different stages of the recruitment process.

The jobs and requirements module is responsible for defining the hiring need. It handles job creation, job status management, sourcing mode selection, criteria capture, and role-specific

evaluation configuration. This module stores the details that later guide sourcing, screening, scoring, and interview generation. For example, when a recruiter creates a job, the system captures the required skills, responsibilities, seniority level, work mode, location, and evaluation criteria. These requirements become the reference point used by the AI components to compare candidates against the role in a structured and consistent way.

The candidates and profiles module stores and manages candidate-related information. It includes candidate records, CV ingestion, extracted skills, education, experience, enrichment sources, and duplicate resolution. Since candidate information can come from multiple sources such as CVs, manually entered profiles, imported files, GitHub, LinkedIn, or external enrichment tools, this module is responsible for building a unified candidate profile. It helps reduce data fragmentation by connecting related information into one record and identifying possible duplicates. This improves the reliability of the scoring process because the system evaluates a cleaner and more complete candidate profile.

The applications and pipeline module connects candidates to specific job opportunities. It stores the relationship between a candidate and a job, including the application status, shortlist rank, source platform, and current pipeline stage. This module allows the system to track the candidate journey from sourcing or application through screening, shortlisting, assessment, interview, decision, and final outcome. By maintaining clear pipeline states, PATHS can show recruiters where each candidate is in the process and prevent confusion or duplicated actions across the hiring team.

The scoring and screening module is one of the core intelligence modules in the system. It calculates candidate-job fit scores by comparing candidate evidence against job requirements. It also retrieves supporting evidence, estimates matching confidence, applies bias guardrails, and generates explanation packets. The purpose of this module is not only to rank candidates but also to explain why each candidate received a specific score. This makes the screening process more transparent and supports the human-in-the-loop approach, where recruiters can review the AI recommendation before approving or rejecting candidates.

The interviews and assessments module manages the evaluation stages after screening. It supports interview scheduling, interview type selection, transcript storage, question generation, assessment outputs, and summary records. For technical roles, this module can support technical assessments and role-specific interview questions. For HR or organizational interviews, it can help structure the evaluation around company criteria and soft-skill requirements. Storing transcripts, notes, scores, and summaries also makes the process auditable and helps hiring managers review candidate performance in a consistent way.

The decision support module converts the collected recruitment evidence into a final decision-ready output. It prepares the final decision packet, supports hire or reject recommendations, connects the outcome to a development-plan handoff, and enables report export. This module is important because the final hiring decision should not depend on isolated scores only. Instead, it should combine profile evidence, screening results, assessment outcomes, interview summaries, and recruiter feedback into one structured report. If the candidate is accepted, the report can support onboarding and development planning. If the candidate is rejected, the system can still generate constructive feedback and improvement recommendations.

The agents and runs module manage the execution of AI agents inside the system. It records stateful agent runs, tool calls, retrieved context, model responses, and run-level observability data. This is important because PATHS uses agentic workflows rather than simple static automation. Each agent may retrieve information, call tools, generate outputs, or pass results to another module. By storing agent runs and execution details, the system becomes more observable and easier to debug. This also supports transparency because the team can review what the agent used, what it generated, and how it contributed to the workflow.

The platform administration module provides the PATHS administrator with system-wide visibility and control. It includes the platform overview, analytics, access requests, organizations, users, agent monitor, system health, feature flags, audit logs, settings, and graph/vector database inspection. This module is different from the organization-level administration because it is designed for the PATHS platform administrator who supervises the entire system. Through this module, the administrator can monitor operational health, inspect agent activity, manage organizations and users, review access requests, check audit logs, and inspect knowledge graph and vector database content. This supports governance, debugging, maintenance, and continuous improvement of the platform.

These modules show the end-to-end nature of the system. The implementation does not treat recruitment as a single task, but as a connected workflow that begins with secure user access and organization setup, continues through job definition, candidate profile management, scoring, screening, interviews, and decision support, and ends with reporting, monitoring, and administration. This modular backend design makes the platform easier to maintain, improves separation of responsibilities, and provides a strong foundation for extending the system with additional AI agents, integrations, analytics, and governance features in the future.

4.5 Agentic AI Implementation

PATHS uses agentic workflows to coordinate recruitment tasks that can't be handled effectively through a single static query or a simple rule-based process. Activities such as candidate sourcing, candidate-job scoring, screening support, interview preparation, outreach personalization, and decision support require multiple connected steps. Each task depends on collecting context, retrieving relevant evidence, applying business and fairness rules, generating human-readable outputs, and saving the final results for review and auditing.

To support this design, we use LangGraph to represent recruitment workflows as stateful graphs of operations. Each workflow is implemented as a compiled LangGraph StateGraph, where a typed state object is passed between nodes throughout the execution. This state gradually accumulates the job context, candidate information, retrieved evidence, intermediate decisions, generated outputs, and any errors that occur during the process.

Each step inside the graph is implemented as a separate node, usually as a Python function with one clear responsibility. For example, one node may discover candidates, another may extract evidence, another may calculate candidate scores, and another may apply bias guardrails. The execution order is controlled through graph edges, while conditional edges allow the workflow to respond dynamically to different outcomes. For instance, in the CV ingestion pipeline, if parsing fails, the workflow does not continue to downstream storage or scoring. Instead, it routes the execution to a *handle_failure* node, ensuring that corrupted or incomplete data is not inserted into the system.

The graphs are invoked asynchronously using *ainvoke*, which allows long-running recruitment tasks to execute without blocking the API layer. Every workflow execution is recorded as an agent run in PostgreSQL, including its status, current node, related entity, timestamps, and any errors encountered. This makes agent behaviour observable, traceable, and auditable, which is especially important in recruitment because decisions must be explainable and reviewable by human stakeholders.

For model serving, PATHS routes all chat and reasoning LLM calls through a single provider facade located in *app/services/llm_provider.py*. This abstraction allows the system to switch between cloud-based and local model execution without changing the agents themselves. In the offline configuration, PATHS uses a local Ollama instance running *llama3.1:8b* and *llama3.1:70b* for reasoning and natural-language generation. Embeddings are generated locally using Ollama's *omic-embed-text* model, which produces 768-dimensional vectors for semantic search, RAG

retrieval, and candidate-job similarity operations. This separation ensures that embedding generation remains local and consistent, regardless of which chat or reasoning provider is used.

Accordingly, the AI layer in PATHS is divided into two main groups. The first group consists of LangGraph agents, which orchestrate multi-step recruitment pipelines such as sourcing, ingestion, screening, scoring, outreach, interview preparation, and decision support. The second group consists of focused reasoning agents, which perform specific LLM-based reasoning tasks and can be invoked either directly by the API layer or from inside LangGraph nodes. Together, these agents form the operational intelligence layer of PATHS. The system implements seven LangGraph agents in addition to a set of focused reasoning agents, as summarized in Table 4.11. Each agent reads from and writes to the data stores most appropriate for its responsibility, such as PostgreSQL for transactional records, the vector database for semantic retrieval, and the knowledge graph for relationship-based reasoning. However, the agents do not make final hiring decisions independently. Each workflow terminates at a point where a human recruiter, interviewer, or hiring manager remains responsible for approval, rejection, or further review. This design preserves human authority while allowing the system to automate repetitive coordination, evidence retrieval, personalization, and decision-support tasks [76], [78], [97].

Table 4.10: Agentic and Reasoning Agents, Data Stores, and Human-in-the-Loop Checkpoints.

Agent	Stores used	Human checkpoint
CV Ingestion	• PostgreSQL • AGE • Qdrant (write)	Candidate confirms the parsed profile
Job Ingestion	• PostgreSQL • AGE • Qdrant (write)	Recruiter approves the job posting
Sourcing	• Qdrant (search) • PostgreSQL • AGE	Recruiter reviews the sourced list
Screening	• PostgreSQL • Qdrant • AGE	Shortlist approval
Interview Intelligence	• PostgreSQL • Qdrant ('interview__transcripts' + 'company__knowledge' RAG)	Interviewer reviews scorecards
Decision Support	• PostgreSQL • Qdrant • AGE	Final hiring decision
Outreach	• PostgreSQL • Gmail/Calendar	Recruiter approves the message
Scoring Agent	• PostgreSQL • Qdrant	Recruiter reviews the score
Candidate-Job Reasoning Agent	• Qdrant • AGE • PostgreSQL	Recruiter reviews the rationale
Decision-Support Reasoning	• PostgreSQL • Qdrant	Final hiring decision
Assessment Agent	• PostgreSQL • Qdrant ('technical_questions' MCP)	Recruiter approves the assessment
Assistant (PATHy)	• AGE • Qdrant • PostgreSQL	Advisory only (no autonomous action)

4.6 Knowledge Graph Synchronization

The knowledge graph is synchronized from key relational records. When a candidate profile is created or updated, the system can create or update candidate nodes and edges to skills, institutions, certifications, companies, and applications. When a job is created, the system can create job nodes and REQUIRES_SKILL edges. This graph representation supports relationship-aware retrieval for scoring and assistant queries. The graph does not replace PostgreSQL as the system of record. Instead, it provides an additional relationship-oriented view of recruitment data. PostgreSQL remains authoritative for transactional records, while Apache AGE supports graph traversal and explainable relationship paths [78], [96].

4.7 Vector Database Collections and Retrieval Use Cases

Table 4.11: Qdrant collections and their retrieval use cases

Collection	Stored content	Main use case
paths_candidates	Candidate profile summaries and structured profile text.	Candidate discovery, similarity ranking, and profile search.
paths_jobs	Job descriptions, responsibilities, and requirements.	Job-candidate matching and role similarity retrieval.
candidate_cv_chunks	Chunked CV/resume text.	Fine-grained evidence retrieval for skills, projects, and experience.
company_knowledge	Organization policies, culture, evaluation rubrics, and onboarding material.	RAG-grounded interview questions and organization-specific assistant responses.
interview_transcripts	Interview text and summarized responses.	Screening summaries, decision support, and growth-plan generation.

Each document or record added to a vector collection includes metadata such as source type, organization identifier, job identifier, candidate identifier, timestamp, and access constraints where applicable. This metadata is critical because semantic retrieval must not leak data across organizations or expose content to unauthorized users.

4.8 Recommendation System Implementation

The recommender answers two important questions. First, how close is the candidate's profile to the job description and required skills? Second, can the system explain why this candidate is or is not suitable for the role? To answer these questions, PATHS combines three complementary signals:

1. Content-based Similarity: Candidate and job embeddings are compared in Qdrant using cosine similarity.

2. LLM-based Reasoning: Anonymized candidate-job pair is evaluated by a scoring agent through the configured LLM provider.
3. Graph-based Relationship: Apache AGE stores explicit relationships such as candidate skills, job-required skills, and candidate-job match edges.

The final score is calculated on a 0-100 scale. The graph is mainly used for filtering, relationship-based explain-ability, and storing match links, while the two numeric scoring signals are the LLM score and the vector similarity score. This design allows PATHS to avoid depending on only one technique. Vector search gives fast semantic retrieval, the LLM gives deeper contextual reasoning, and the graph provides structured relationship evidence that can be reviewed and queried.

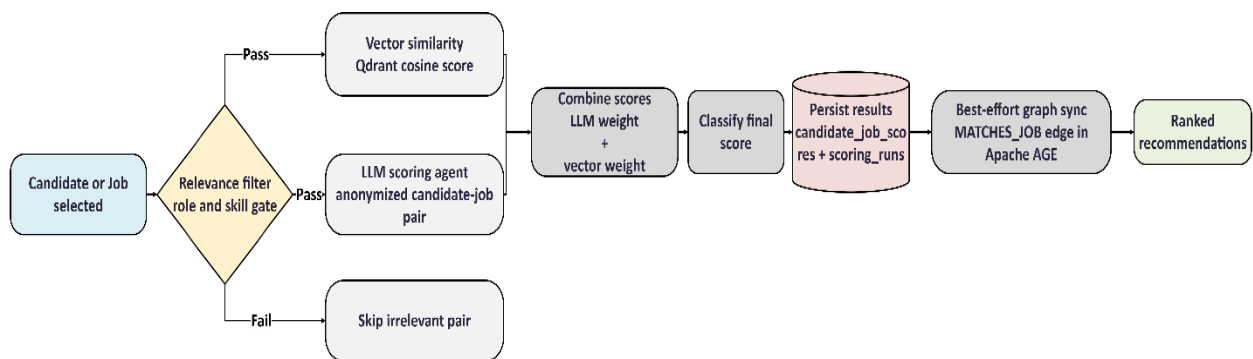


Figure 4.2: Hybrid Candidate-Job Recommendation Pipeline.

The Figure 4.2. shows that the recommender follows a structured pipeline. The process starts by loading the candidate and job information, then checks whether the pair is relevant enough to be scored. After that, the system calculates a vector similarity score and asks the LLM scoring agent to evaluate the anonymized profile. These scores are combined into a final score, classified into a match band, stored in PostgreSQL, and optionally synchronized into Apache AGE as a MATCHES_JOB relationship.

The figure highlights that the recommender is not only a scoring function, but a full operational pipeline that reads from multiple stores, applies relevance checks, handles missing data, and stores auditable results. The main advantage of this design is that expensive reasoning is only applied after cheaper filtering and retrieval steps. This improves efficiency because the LLM is not called for every possible candidate-job pair. Instead, PATHS first removes clearly irrelevant pairs using role-family and skill-based gates, then applies deeper reasoning only where it is useful.

4.8.1 Score Combination Formula

The final recommendation score is generated by combining the LLM reasoning score and the vector similarity score. In the current implementation, the LLM score has a higher weight because it can evaluate context, relevance, seniority, evidence, and role fit more deeply than vector similarity alone. The vector score remains important because it gives a stable semantic similarity signal based on embeddings.

Equation 3. Candidate Recommendation Logic

$$final_score = (agent_Score \times W_a) + (vector_Similarity \times W_v)$$

The implementation includes two safeguards. First, the weights are renormalized so that they always sum to 1.0, even if the environment variables are misconfigured. Second, the final score is clamped between 0 and 100 and rounded to three decimal places. This prevents invalid scores such as negative values or scores above 100.

4.8.2 Embeddings and Content-Based Similarity

The content-based part of the recommender depends on embeddings generated using Ollama's `nomic-embed-text` model. This model produces 768-dimensional vectors that represent the semantic meaning of candidate profiles and job descriptions. These vectors are stored in Qdrant and compared using cosine similarity. In the implementation, candidate vectors are stored in the `paths_candidates` collection and job vectors are stored in the `paths_jobs` collection. The recommender does not generate these vectors directly during scoring. Instead, embeddings are produced earlier by the ingestion workflows, then reused by the recommender when scoring is requested.

The vector similarity service follows several implementation rules:

- each candidate vector point uses the `candidate_id` as its *Qdrant* point identifier;
- each job vector point uses the `job_id` as its *Qdrant* point identifier;
- missing vectors do not crash the scoring run;
- if a candidate or job vector is missing, the similarity score is treated as 0.0 and the run is marked with a vector-missing status;
- cosine similarity is normalized into a 0-100 score before being combined with the LLM score.

This approach makes the system robust. A missing embedding should reduce confidence or score quality, but it should not stop the full recommendation pipeline. This is important because

real candidate data is often incomplete, especially when candidates are imported from CVs, CSV files, or external sources.

4.8.3 Graph-Based Matching Signals

Apache AGE is used to represent explicit relationships between candidates, skills, jobs, and matches. While Qdrant captures semantic similarity, the graph captures structured relationships that are easier to explain. For example, candidate skills are represented through `HAS_SKILL` edges, while job requirements are represented through `REQUIRES_SKILL` edges. This allows PATHS to explain a recommendation in a more transparent way. Instead of only saying that two vectors are similar, the system can show that a candidate has Python, FastAPI, and PostgreSQL skills, while the job explicitly requires these skills.

When a candidate is scored against a job, the recommender also attempts to create or update a `MATCHES_JOB` edge in Apache AGE. This graph edge allows the match itself to become part of the knowledge graph. As a result, administrators and future agents can query candidate-job relationships directly, inspect matching patterns, and support explainable decision-making. The graph signal is especially useful for three purposes:

1. Relevance filtering, by checking whether a candidate and job share important skill relationships.
2. Explain-ability, by showing which candidate skills match which job requirements.
3. Auditability, by keeping candidate-job match relationships query-able over time.

The same scoring core is reused across multiple parts of the PATHS platform. This avoids duplicating matching logic and ensures that candidate-side, recruiter-side, and organization-side recommendations follow the same scoring principles.

4.8.4 Robustness and Fairness Controls

The recommender is designed to handle imperfect real-world data. Candidate profiles may be incomplete, vectors may be missing, external services may fail, and some candidate-job pairs may not contain enough evidence for confident scoring. For this reason, PATHS applies graceful degradation rather than stopping the entire workflow.

The main robustness controls include:

- Missing vector handling: if a vector is missing, the system assigns a similarity value of 0.0 instead of crashing.
- Fallback retrieval: if Qdrant is unavailable in semantic search, the system can fall back to SQL-based keyword or skill-overlap search.

- Per-pair isolation: each candidate-job scoring pair is handled independently, so one failed pair does not invalidate the whole batch.
- Error persistence: failures are stored in `scoring_errors` for later debugging.
- Best-effort graph sync: if writing the *MATCHES_JOB* edge to Apache AGE fails, the scoring result can still remain valid in PostgreSQL.

Fairness is also considered in the recommendation process. Before the LLM scoring agent evaluates a candidate, the candidate profile is anonymized using the scoring prompt builder. This removes or hides identifying information that should not influence the match score. The goal is to make the scoring agent focus on job-relevant evidence such as skills, experience, projects, and assessments rather than personal identity attributes.

The recommender also supports explain-ability by combining three types of evidence. Qdrant provides semantic similarity, the LLM provides a written rationale, and Apache AGE provides explicit relationships such as *HAS_SKILL*, *REQUIRES_SKILL*, and *MATCHES_JOB*. Together, these signals make the recommendation easier to justify than a black-box score.

4.9 Model Context Protocol (MCP)

The Model Context Protocol (MCP) is an open-source standard for connecting AI applications to external systems. Using MCP, AI Agents can connect to data sources (e.g. local files, databases), tools (e.g. search engines, calculators) and workflows (e.g. specialized prompts) which enables them to access key information and perform tasks.

It is implemented as a standardized integration layer that enables the platform to communicate with external capability providers through a uniform tool-based interface. Instead of tightly coupling the backend to a specific third-party service, API design, or scraping mechanism, MCP allows PATHS to treat external services as independent tool servers that can be invoked through a consistent protocol. This design is particularly important for recruitment workflows, where candidate information, enrichment services, communication tools, and scheduling systems may exist outside the core platform.

Within PATHS, MCP is mainly used to support external candidate sourcing and profile enrichment. The most important implementation is the LinkedIn MCP sourcing provider, which enables the system to retrieve candidate profiles and Open-to-Work signals through an MCP-compatible server. In addition, the same tool-oriented approach is reused in the skill-evidence subsystem, where candidate evidence can be collected from CVs, GitHub, and LinkedIn through a unified interface. Optional MCP hooks are also provided for Google Calendar and Gmail

integrations, although these bridges are disabled by default and are only activated when the deployment environment requires them.

The purpose of introducing MCP in PATHS is not only technical integration, but also architectural flexibility. By separating the PATHS application from the external data provider, the system can replace or extend external sources without changing the main API layer or the recruiter workflow. This makes the platform more maintainable, vendor-neutral, and easier to adapt to different deployment environments.

Table 4.12: MCP usage across system.

Usage Module	Nature
LinkedIn candidate sourcing	Full MCP client using Streamable-HTTP and JSON-RPC 2.0 against a LinkedIn MCP server
Skill-evidence tools	MCP-style tool abstraction for collecting candidate evidence from multiple sources
Calendar and Gmail bridges	Optional feature-flagged hooks for Google MCP-compatible services

As shown in Table 4.12. The core MCP implementation in PATHS is the *LinkedInMcpSourcingProvider*. This provider acts as a real MCP client that communicates with a LinkedIn MCP server using the Streamable-HTTP transport. The MCP server is responsible for maintaining the external session, handling authentication, and enforcing access control rules. PATHS does not directly manage the LinkedIn session and does not embed LinkedIn-specific scraping logic inside the main backend. Instead, the backend sends protocol-level requests to the MCP server and then maps the returned candidate information into the internal PATHS data model.

This separation creates a clean compliance and responsibility boundary. The external MCP server owns the connection to the external source, while PATHS only consumes structured results through a controlled interface. This is important because recruitment systems deal with sensitive candidate information, and therefore external data access must be handled through authorized, traceable, and configurable mechanisms.

The transport configuration works as the LinkedIn MCP provider communicates with the server using JSON-RPC 2.0 messages over HTTP. The MCP endpoint is configured through the *LINKEDIN_MCP_URL* environment variable, which allows the same codebase to run in different environments without modification. For local development, this endpoint may point to a local MCP server, while in production it can point to a secured internal service.

The client accepts both standard JSON responses and Server-Sent Events (SSE) responses by using the following accept types:

application/json, text/event-stream

This is necessary because the MCP Streamable-HTTP transport allows the server to return either an immediate JSON response or a streamed SSE response. The provider therefore includes

decoding logic for both response types. The client also identifies itself using a fixed protocol version and client name, allowing the server to recognize the PATHS sourcing component during the initialization phase.

As shown in the Figure 4.3. before invoking any external tool, the MCP client follows a structured communication sequence. First, the client sends an initialize request to establish the MCP session. The server responds with a session identifier through the `Mcp-Session-Id` header. This session identifier is then included in subsequent calls to maintain continuity between the client and server.

After initialization, the client sends the notifications/initialized message to inform the server that the client is ready to begin tool execution. Once the session is active, the provider sends a `tools/call` request to invoke the required MCP tool. At the end of the operation, PATHS attempts to close the MCP session using an HTTP DELETE request. This step is handled as a best-effort clean-up mechanism to reduce the risk of leaving browser or tool sessions open on the MCP server.

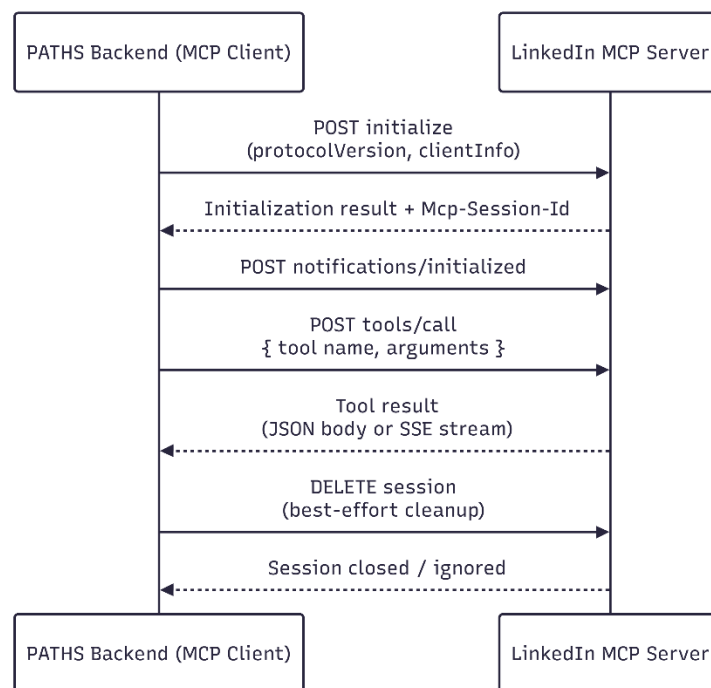


Figure 4.3: MCP Streamable-HTTP handshake and tool call.

4.9.1 LinkedIn MCP Sourcing Provider

The LinkedIn MCP provider uses two main tools exposed by the MCP server. The first tool, `search_people`, is used to search for candidate profiles based on recruiter-defined keywords and,

optionally, location. The second tool, `get_person_profile`, is used to retrieve additional details about a specific candidate profile, such as public skills and Open-to-Work evidence.

Table 4.13: LinkedIn MCP tools used.

Tool	Purpose
<code>search_people</code>	Searches for candidate profiles matching the recruiter query
<code>get_person_profile</code>	Enriches a selected candidate profile with additional details

The implementation deliberately sends only the arguments supported by the external MCP tool. This avoids runtime failures caused by unsupported keyword arguments. The number of returned candidates is also capped according to the request made by the PATHS sourcing service, ensuring that the recruiter interface receives a controlled and predictable candidate list.

The enrichment step is especially important because search results alone may contain limited profile information. By calling `get_person_profile`, PATHS can verify whether an Open-to-Work signal is available and can retrieve stronger evidence such as top skills. This improves the quality of the candidate ranking process because the system is not relying only on short search snippets.

Because the MCP server may return either a JSON body or an SSE stream, the provider first checks the response content type. If the response is a standard JSON response, the body is parsed directly. If the response is returned as text/event-stream, the provider reads the stream and extracts the first valid data: payload that contains either a JSON-RPC result or error.

After the response is decoded, the provider processes the MCP content envelope. Tool outputs may be returned inside `content[]`.text fields, and these text values may themselves contain JSON-encoded profile information. The implementation therefore flattens this structure and decodes the profile content into a usable internal format. If structured content is available, it is used as a fallback.

The provider also filters the returned information to avoid treating irrelevant page content as candidate records. For example, generic page text, cookie banners, login-wall messages, or interstitial content should not be interpreted as real candidate data. Candidate entries are therefore built only when meaningful person-profile references and profile-related fields are available.

After parsing the MCP response, each candidate is converted into the provider-independent *ExternalCandidatePayload* model. This internal payload includes the candidate name, headline, current role or company, location, profile URL, skills, and Open-to-Work signal when available. Field values are also normalized and truncated where necessary to remain compatible with database column limits.

A key implementation principle is that Open-to-Work information must never be fabricated. The signal is only stored when it is explicitly provided by the external source or can be supported by candidate-visible profile evidence. This protects the reliability of the sourcing process and prevents the system from making unsupported assumptions about a candidate’s job-search status.

```
class CandidateSourcingProvider(abc.ABC):
    @abc.abstractmethod
    async def fetch_open_to_work_candidates(self, input) -> list[ExternalCandidatePayload]:
        ...

    async def fetch_profile_details(self, *, username) -> dict:
        ...
```

The recruiter-facing sourcing page does not communicate directly with the MCP server. Instead, it sends a request to the PATHS backend, and the backend selects the appropriate sourcing provider through a common abstraction. This abstraction is represented by the *CandidateSourcingProvider* interface, which defines the operations required for fetching Open-to-Work candidates and retrieving profile details.

The provider factory function, *get_sourcing_provider(name)*, resolves the concrete provider based on the explicit provider requested by the caller, the default configured provider, or the LinkedIn MCP provider as the fallback default. This allows PATHS to support more than one sourcing strategy without changing the API layer.

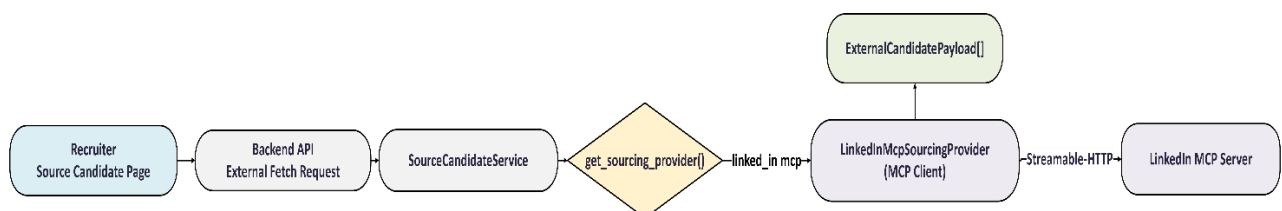


Figure 4.4: Source-candidate provider abstraction.

4.9.2 Skill-Evidence MCP-Style Tools

The skill-evidence subsystem follows the same MCP-oriented design philosophy. Instead of hardcoding each evidence source into the scoring logic, PATHS models each source as a tool that returns a standardized *EvidenceResult*. This allows the evidence collection process to treat CVs, GitHub profiles, and LinkedIn profiles consistently, even though each source has a different access mechanism and reliability level.

Table 4.14: Skill-evidence tools.

Tool	Evidence Source	Implementation Role
------	-----------------	---------------------

CVEvidenceTool	Candidate CV, experience records, and certifications	Uses first-party candidate data and formats it as structured evidence
GithubEvidenceTool	Public GitHub repositories, languages, and README content	Collects project-based evidence and supports stronger behavioural verification
LinkedinEvidenceTool	LinkedIn public profile	Can use a configured MCP server or fall back to a best-effort public fetch

As shown in the Table 4.14. Each tool returns not only evidence snippets, but also an explicit availability status. Examples include *available*, *not_configured*, *url_missing*, *blocked*, *error*, and *no_match*. This distinction is important because it separates a true lack of evidence from a technical inability to collect evidence. For example, a candidate should not be penalized in the same way when GitHub is missing, when GitHub is blocked, and when GitHub was checked but no relevant evidence was found.

The `collect_skill_evidence` orchestrator calls the available evidence tools, evaluates the returned snippets, and aggregates the results into a skill-evidence score. The weighting strategy is re-based when a source is unavailable. In the current implementation, CV and GitHub are treated as the main weighted sources, and a missing source is excluded from the denominator rather than automatically counted as zero. This produces fairer scoring because the system avoids punishing candidates for sources that could not be accessed.

When an LLM is available, the evidence snippets can be evaluated using the configured local Ollama model. If the LLM is unavailable, the system falls back to a deterministic snippet-count strategy. This fallback maintains continuity of the scoring process and prevents the evidence subsystem from becoming dependent on a single model provider.

The most important MCP-ready property in this subsystem is that the LinkedIn evidence tool can be redirected to a real MCP server through configuration using `LINKEDIN_MCP_SERVER_URL`. No code changes are required. This makes the tool interface a stable integration seam and supports future deployment flexibility.

4.9.3 Google Calendar and Gmail MCP Bridges

PATHS also includes optional MCP hooks for Google Calendar and Gmail services in the outreach subsystem. These hooks are implemented as feature-flagged helpers and are disabled by default. This is because the primary outreach workflow can operate using SMTP and booking links, while the primary Google integration path can still be implemented through direct OAuth where required.

The MCP bridge therefore acts as an optional deployment alternative. When `MCP_ENABLED` and the relevant per-service flag are enabled, the helper functions expose MCP hints such as `mcp:google-calendar` or similar service references. This allows an operator running a Google

Calendar or Gmail MCP server to connect those services into the scheduling and messaging workflow without changing the main application logic.

```
def calendar_mcp_link_hint() -> str | None:
    if settings.mcp_enabled and settings.mcp_google_calendar_enabled:
        return "mcp:google-calendar"
    return None
```

This feature-flagged design keeps the default deployment simple while still supporting more advanced MCP-based environments. It also prevents optional integrations from introducing unnecessary complexity into the core hiring workflow.

4.10 Security, Privacy, RBAC, and Auditability

Security is implemented as a cross-cutting concern rather than an isolated feature. All protected requests require JWT verification, and role-based access control determines whether the user can access candidate profiles, scoring evidence, organization data, platform administration tools, or graph/vector inspection screens.

Candidate privacy is especially important because PATHS stores resumes, contact information, interview results, and AI-generated evaluations. The system therefore follows a data-minimization principle: it collects and processes only the fields required for recruitment workflows. Sensitive values are stored and transmitted through secured channels, and secrets such as JWT keys, model provider keys, and integration tokens are injected through environment variables rather than hard-coded.

The anonymization layer reduces bias risk by hiding identity attributes before evaluation. The implementation separates the evaluation stage from the outreach stage: candidates can be scored and shortlisted using job-relevant signals, while identity and contact information are exposed only when a human-approved workflow requires communication. De-anonymization events should be logged so that the organization can review who revealed a candidate and why.

Auditability is implemented by recording important state transitions and decision-related operations. Examples include score generation, shortlist approval, de-anonymization, interview summary creation, final hiring decision, report export, and platform-admin actions. These logs are necessary for accountability, debugging, compliance review, and explaining decisions to stakeholders. Also, the Table 4.15. shows the implemented security and governance, and their purposes.

Table 4.15: Security and governance controls implemented by design.

Control	Purpose
JWT authentication	Verifies user identity before protected API access.
Role-based access control	Restricts actions by user role: candidate, recruiter, hiring manager, organization admin, or platform admin.
Anonymization gate	Prevents identity-related fields from influencing early evaluation.
Controlled de-anonymization	Exposes identity/contact data only after approved workflow steps.
Audit logging	Records decision-related operations and platform-administration activity.
Input sanitization	Reduces risk from malicious resumes, scraped pages, or prompt-injection text.
Secret externalization	Keeps credentials and API keys out of source code.
Organization-level data boundaries	Prevents cross-organization retrieval from graph/vector collections.

4.11 System Testing

System testing was implemented to validate PATHS as a complete operational platform rather than as a collection of isolated modules. Because the system depends on several connected components, including the Next.js frontend, FastAPI backend, PostgreSQL with Apache AGE, Qdrant, and Ollama, the testing strategy had to verify both internal service correctness and real user-facing behaviour. For this reason, PATHS was evaluated using a layered testing approach that combines backend unit and integration tests with browser-based end-to-end tests executed against the running development stack.

4.11.1 Testing Strategy

PATHS follows a testing pyramid in which lower layers verify focused backend behaviour, while upper layers validate complete system journeys. At the base of the pyramid, unit and service tests examine individual business rules such as authentication, scoring, matching, role-based access control, candidate anonymization, deduplication, and data normalization. The integration layer then verifies that larger backend workflows, including CV ingestion, job ingestion, and unified pipeline execution, operate correctly across multiple services. Finally, the end-to-end layer uses Playwright to simulate real browser behavior across the full stack, ensuring that user actions move correctly from the interface to the backend APIs and into the underlying data stores.

This layered structure is important because no single test type can prove the complete reliability of an AI-driven hiring platform. Backend tests are fast and precise, making them suitable for validating business logic in isolation. However, they cannot fully confirm that a real user can log in, navigate to the correct portal, or be prevented from accessing another role's area. End-to-end

tests fill this gap by validating the system from the user's perspective while still exercising the real frontend, API layer, database, graph layer, and vector database.

The E2E suite was intentionally designed around the four actor types supported by the system: anonymous visitor, Candidate, Organization member, and Platform Admin. This makes the testing strategy directly aligned with the access-control model of PATHS and ensures that the system is not only available, but also correctly separated by responsibility and authorization level. As shown in the Figure 4.5.

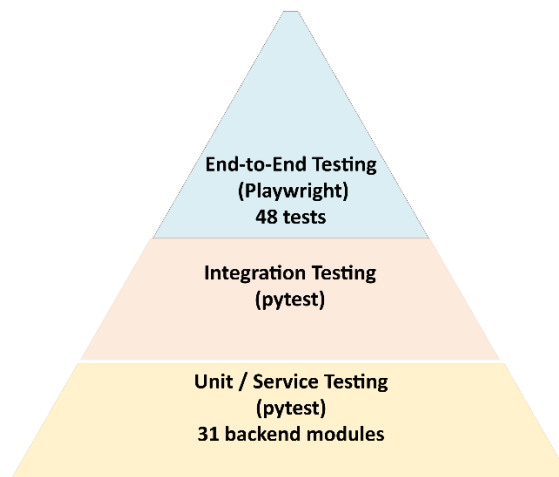


Figure 4.5. Layered testing pyramid combining backend unit/integration tests with full-stack Playwright end-to-end tests across four user personas.

The testing implementation used tools that match the technical structure of PATHS. Backend validation was performed with pytest (Used to test backend services, repositories, APIs, and deterministic business logic such as authentication, RBAC, scoring, matching, and data validation.) and pytest-asyncio (Used to test asynchronous backend workflows, especially service calls and API operations that depend on async FastAPI/database execution.), while end-to-end validation was performed with Playwright Test (Used to run real browser-based end-to-end tests that simulate how users interact with PATHS through the frontend.) running in Chromium (Used as the browser engine for executing Playwright tests in a consistent and automated testing environment.). The runtime under test was the complete PATHS stack rather than a mocked environment.

Table 4.16. Testing tools and runtime components used to validate the system.

Layer	Framework / Tool	Version / Source
Backend unit and integration testing	pytest and pytest-asyncio	Project requirements file
End-to-end browser testing	Playwright Test (@playwright/test)	1.49+ using Chromium
Runtime under test	Next.js 16, FastAPI, PostgreSQL with Apache AGE, Qdrant, and Ollama	Live development stack

The Table 4.16. shows the mechanisms of observability and reliability. Observability is required because PATHS contains several asynchronous and AI-driven operations. The backend integrates metrics, structured logging, tracing, and error reporting so that failures can be diagnosed across API calls, database queries, agent runs, embedding generation, and external integrations.

Reliability is addressed through health checks, retries, graceful fallback, and stateful workflow logging. For example, if a contact-enrichment call fails, the system should not silently proceed with an invalid outreach action; it should either retry, select an alternative channel, or flag the case for human review.

Table 4.17. Observability and reliability mechanisms.

Mechanism	Implementation role
API health and metrics	Prometheus FastAPI instrumentation and service health endpoints.
Error reporting	Sentry SDK for backend and frontend runtime errors.
Tracing	OpenTelemetry for tracking requests through service and agent layers.
Agent run logs	Stored prompts, retrieved context identifiers, outputs, and execution status where safe.
Database health	PostgreSQL readiness checks and migration status.
Vector/graph diagnostics	Collection availability, vector-size consistency, graph initialization status, and query errors.

4.12 Deployment and Operational Considerations

PATHS is deployed as a containerized web application running on a single cloud server. The production version is live at <https://pathstalent.com>, and the full stack is orchestrated using Docker Compose. This deployment approach was selected because it keeps the system reproducible, easy to redeploy, and independent from manually installed server dependencies. Instead of installing PostgreSQL, Qdrant, Node.js, Python, or the reverse proxy directly on the machine, each service runs inside its own Docker container with a clearly defined responsibility.

The production stack consists of six main containers. PostgreSQL with Apache AGE is used as the relational and graph database layer, Qdrant is used as the vector database for embeddings and semantic search, Ollama is used locally for the nomic-embed-text embedding model, the FastAPI backend serves the API and LangGraph agent workflows, the Next.js frontend provides the user interface, and Caddy acts as the public reverse proxy. Caddy also manages automatic HTTPS certificates through Let's Encrypt, meaning that users access the platform securely through the domain without manually configuring SSL certificates.

In the current production configuration, reasoning and chat-based LLM calls are routed through ollama using *meta-llama/llama-3.3-70b-instruct*. This handles reasoning and embedding generation remains local through Ollama. This separation is important operationally because embeddings must remain consistent for retrieval and similarity search, while reasoning models can be changed or upgraded through the provider layer without redesigning the system.

The production server runs on AWS EC2 in the eu-north-1 region using an m7i-flex.large instance with 2 vCPUs and 8 GB RAM. The operating system is Ubuntu 24.04 LTS, with Docker Engine and Docker Compose installed as the only required server-level software. The system currently uses a 30 GB gp3 disk and an 8 GB swap file. Runtime resource usage is relatively low, with the full stack consuming around 450 MB of container memory when idle and approximately 1.3 GB of total host memory including the operating system. However, the frontend build process can require several gigabytes of RAM, so swap is important to avoid out-of-memory failures during deployment.

The recommended deployment target for a clean production redeployment is 2–4 vCPUs, 8 GB RAM, 50 GB disk, Ubuntu LTS, Docker and Docker Compose, ports 22, 80, and 443, and a domain name pointing to the server’s public IP address. A 30 GB disk can run the system, but it is considered tight because Docker images, Ollama models, build cache, database volumes, and vector data grow over time. The current server disk usage is already high, so increasing the disk size to 50 GB or periodically pruning the Docker build cache is recommended.

From a networking perspective, only ports 80 and 443 should be publicly available for user access. Port 80 is used for HTTP redirection and Let’s Encrypt certificate challenges, while port 443 is the main HTTPS entry point. Port 22 is required for server administration through SSH, but it should be restricted to trusted IP addresses whenever possible. Internal service ports such as PostgreSQL, Qdrant, Ollama, the direct backend port, and the direct frontend port should not be exposed publicly once Caddy is handling traffic through the domain.

Persistence is one of the most important operational concerns in this deployment. The critical production data is stored in Docker volumes, especially the PostgreSQL/AGE volume and the Qdrant vector-store volume. These volumes contain candidate records, organizations, jobs, agent runs, graph relationships, embeddings, and retrieval data. Therefore, they must be treated as production assets and backed up regularly. If these volumes are deleted or replaced without a backup, the application containers can be rebuilt, but the business data may be lost.

Secrets management is also essential. Sensitive values such as database credentials, JWT secrets, provider credentials, and future email/calendar integration keys should not be stored directly in source code. They should be injected through environment variables or a deployment secret mechanism. This keeps the system safer and makes it easier to rotate credentials without changing application code.

Because PATHS processes candidate and organization data, production deployment must follow responsible security practices. The domain must always use HTTPS, CORS should be restricted to trusted frontend origins, authentication tokens and cookies should be configured securely, and

internal data services should remain private. Logs should be retained for debugging and auditability, but they should not expose sensitive candidate data unnecessarily. Rate limiting, backup policies, access control, and monitoring are also part of responsible operation because PATHS is not only a normal web application; it is a hiring decision-support system.

Operational monitoring should cover both infrastructure and agent behavior. At the infrastructure level, the team should monitor disk usage, memory usage, container health, restart loops, and HTTP errors. At the application level, PATHS should monitor API failures, failed agent runs, Qdrant retrieval errors, database errors, and unusual delays in workflows such as CV ingestion, scoring, outreach, and decision support. This allows the team to detect problems early and maintain the reliability expected from a production recruitment platform.

Table 4.18. Deployment Responsibilities and Operational Requirements.

Area	Operational Requirement
Public domain	Serve the production system through https://pathstalent.com using a domain connected to the server public IP.
Reverse proxy and HTTPS	Use Caddy as the public entry point, reverse proxy, HTTP-to-HTTPS redirect layer, and automatic Let's Encrypt certificate manager.
Frontend hosting	Run the Next.js 16 frontend inside the web Docker container and expose it through Caddy instead of direct public access.
Backend hosting	Run the FastAPI backend inside the backend Docker container with environment variables, health checks, and access through the reverse proxy.
Database service	Run PostgreSQL 16 with Apache AGE inside the postgres container to support relational records and graph relationships.
Vector database	Run Qdrant inside the qdrant container for embedding storage, semantic retrieval, and similarity search.
Embedding service	Run Ollama locally only for embeddings using <code>nomic-embed-text</code> ; keep the Ollama service private and not publicly exposed.
LLM reasoning	Route reasoning and chat-based LLM calls through Ollama using <code>meta-llama/llama-3.3-70b</code> , reducing local compute requirements.
Server requirements	Use a Docker-capable Linux server; recommended production target is 2–4 vCPUs, 8 GB RAM, 50 GB disk, Ubuntu LTS, and 8 GB swap.
Network exposure	Publicly expose only ports 80 and 443; restrict SSH on port 22; keep database, vector store, Ollama, backend, and frontend internal where possible.
Persistence	Store PostgreSQL/AGE data and Qdrant vectors in persistent Docker volumes, especially <code>pgdata</code> and <code>qdrant_data</code> .
Backups	Back up database and vector-store volumes regularly because they contain the operational and AI retrieval state of the system.
Secrets management	Store JWT secrets, database credentials, OpenRouter keys, and provider keys in environment variables or a secure secret manager, not in source code.
Disk management	Monitor disk usage, prune Docker build cache when needed, and prefer a 50 GB disk because Ollama images, build cache, and volumes can quickly fill smaller disks.
Monitoring	Track container health, logs, API errors, agent run failures, database errors, retrieval failures, memory usage, and disk usage.
Security controls	Enforce HTTPS, strict CORS origins, private internal services, secure authentication settings, rate limiting, and controlled log retention.

As shown in Table 4.18, the production deployment of PATHS depends on several responsibilities working together rather than on a single hosting decision. The frontend, backend, database, vector store, embedding service, reverse proxy, secrets, backups, and monitoring each play a specific role in keeping the system secure and usable. This is particularly important because PATHS handles candidate records, organization data, agent outputs, and decision-support information, so operational reliability and data protection are part of the system’s core implementation rather than optional deployment improvements.

As shown in the Figure 4.1. This is the landing page. It communicates the product’s value in one sentence to speed and fairness so readers instantly understand the goal before seeing any internal screens. The call-to-actions (“Get Started” / “Request Demo”) also hint at two adoption paths: self-serve onboarding for smaller teams versus demo-driven onboarding for organizations. From a narrative perspective, this figure is ideal as the first image because it frames everything that follows: PATHS is not just automation, it’s automation with responsible decision-making.

4.13 Implementation Challenges

The implementation of PATHS was not limited to building separate frontend, backend, and AI components. The main challenge was integrating all of these parts into one reliable hiring platform that could support real recruitment workflows. PATHS combines a modern Next.js web interface, a FastAPI backend, agentic AI workflows, PostgreSQL, Apache AGE, Qdrant, hosted LLM services, local LLM fallback, and external integrations such as Google Calendar and Gmail. As a result, several implementation challenges appeared during development, especially around latency, reliability, data consistency, security, fairness, deployment, and third-party integrations.

This section presents the most important challenges encountered during implementation. For each challenge, the practical impact is described, followed by the mitigation strategy adopted in the system. The purpose of this section is to show that PATHS was not only designed conceptually, but also engineered with realistic operational problems in mind.

Table 4.19. Summary of Implementation Challenges and Mitigation Strategies.

Challenge	Area	Mitigation Strategy
LLM latency	AI / LLM	Asynchronous execution, right-sized models, local Ollama fallback, and bounded scoring workload
Matching threshold tuning	AI / LLM	Weighted score calibration, vector normalization, relevance filtering, and score bands
CV data quality	Data	Structured extraction, normalization, failure routing, and human confirmation
Duplicate candidates	Data	Deduplication workflow, source hashing, and unified candidate synchronization
Cross-store data drift	Data	PostgreSQL as the source of truth and synchronized graph/vector ingestion

Apache AGE and Cypher quirks	Data	agtype parsing, safe label handling, and parameterized query values
Authentication and role permissions	Security	JWT authentication, RBAC, backend guards, and expired-token handling
Tenant isolation in vector search	Security	Metadata-scoped retrieval using organization and role filters
Bias and fairness	Security / Ethics	Candidate anonymization, bias guardrails, and controlled de-anonymization
Deployment issues	Deployment	Single Docker Compose stack, explicit port mapping, AGE initialization, and environment-based configuration
Google Calendar and Gmail OAuth	Integration	Encrypted token storage, webhook/polling fallback, and send rate limiting
Next.js 16 changes	Frontend	Version pinning, framework-specific implementation patterns, and type checking
External job scraping	Integration	Scheduled scraping, rate-limit awareness, optional Playwright setup, and stub mode

4.13.1 LLM Latency

One of the first practical challenges was the latency introduced by Large Language Model calls. Several PATHS features depend on LLMs, including candidate scoring, decision support, screening explanations, outreach generation, and the context-aware assistant. These operations are naturally slower than traditional database queries because they require either a network request to a hosted provider or local model inference. The issue becomes more visible when an agentic workflow performs multiple LLM calls in sequence.

This latency directly affects the user experience. For example, a recruiter running candidate screening expects the interface to remain responsive while the backend evaluates candidate-job fit. If every candidate required a blocking LLM call, the system would quickly become slow, especially when handling multiple candidates or jobs.

To address this, LLM execution was implemented asynchronously wherever possible. Agent runs are tracked so long-running operations remain observable instead of appearing as frozen requests. In addition, PATHS uses different model sizes depending on the task. Lightweight models are preferred for high-volume tasks such as initial scoring, while larger models are reserved for more complex decision-support tasks where deeper reasoning is needed. The system also supports local Ollama execution for development. The workload is bounded by limiting the number of jobs scored per candidate and by computing embeddings once during ingestion rather than recomputing them during every match.

4.13.2 Matching Threshold Tuning

Candidate-job matching in PATHS combines two different types of signals: LLM-based judgment and vector similarity. While this improves the quality of matching, it also creates a calibration challenge. Raw cosine similarity values usually fall within a narrow range, while LLM

scores can vary depending on the prompt, model behaviour, and available evidence. If these two signals are combined without normalization, the final score may become misleading.

To solve this, vector similarity is first normalized into a 0–100 range. The final score is then calculated as a weighted blend between the agent score and the vector score. In the implemented configuration, the agent score receives a higher weight because it considers structured reasoning and job context, while the vector score acts as a semantic similarity support signal. The combined score is clamped between 0 and 100 to prevent invalid outputs.

PATHS also applies a relevance filter before scoring. For example, candidates from completely unrelated role families should not be ranked simply because they share a few generic keywords. This filter reduces noise, improves ranking quality, and decreases unnecessary LLM calls. Finally, scores are mapped into interpretable bands such as Excellent, Strong, Good, Possible, and Weak. These bands make the score easier for recruiters to understand and provide a practical basis for shortlist decisions.

4.13.3 Data Quality from CVs

CV ingestion was one of the most sensitive implementation areas because candidate CVs arrive in many formats and structures. Some CVs are PDF files, others are DOCX files, and their layouts differ significantly. Important information may be missing, written in free text, or expressed using inconsistent terminology. For example, one candidate may write “JavaScript,” another may write “JS,” and another may list a framework without explicitly naming the language.

Poor extraction quality can affect the entire system. If the system extracts incorrect skills or misses important experience, the candidate profile, graph relationships, vector embeddings, and matching results may all become unreliable.

PATHS addresses this through a dedicated CV ingestion pipeline. The pipeline first parses the document using suitable parsing libraries, then applies structured extraction to convert unstructured text into fields such as skills, education, experience, projects, and certifications. After that, entity normalization is applied to reduce inconsistencies in names, skills, and job titles.

A key implementation decision was to avoid writing corrupted or incomplete data into the database. If parsing or extraction fails, the workflow is routed to a failure handler instead of saving partial data across PostgreSQL, Apache AGE, and Qdrant. In addition, candidate confirmation is included as a human-in-the-loop step, allowing extracted information to be reviewed before it is used for scoring and matching.

4.13.4 Duplicate Candidates

Duplicate candidate records can appear in several ways. A candidate may upload a CV more than once, be imported from a CSV file, appear in a sourced profile, or exist in both internal and external datasets. Duplicates are especially problematic because they split the candidate's evidence across multiple records and can cause the same person to appear multiple times in shortlists.

To reduce this issue, the sourcing and ingestion workflow includes a deduplication step. Source hashing is used to detect repeated imports, while entity normalization helps identify equivalent skills, names, and profile attributes. Skill lists are also cleaned case-insensitively so repeated variations do not inflate a candidate's profile. Finally, the system includes a unified synchronization step to keep the candidates relational, graph, and vector representations aligned.

This implementation choice is important because recruitment systems depend heavily on trust. Recruiters must feel that the candidate pool represents real people accurately, not repeated or inconsistent records.

4.13.5 Cross-Store Data Drift and ID Mismatch

PATHS uses three data stores: PostgreSQL for transactional records, Apache AGE for graph relationships, and Qdrant for vector search. While this architecture provides strong capabilities, it also introduces a synchronization challenge. During early development and demo seeding, some mock records remained in the graph or vector database even after they no longer existed in PostgreSQL. This created a mismatch between the relational, graph, and vector ID spaces. The impact was clear in admin and matching views. The system could display candidates or jobs that were no longer part of the live operational database, which could reduce trust in the platform.

The adopted mitigation was to treat PostgreSQL as the system of record. Administrative and matching views filter entities against PostgreSQL so only real operational records are shown by default. At the same time, the platform-admin graph and vector database inspector allow developers or administrators to inspect orphaned records when debugging is needed. New CV and job ingestion workflows write to PostgreSQL, Apache AGE, and Qdrant in a synchronized process, reducing the likelihood of future drift.

4.13.6 Apache AGE and Cypher Query Handling

Apache AGE introduced specific implementation challenges because it returns graph values using the agtype format. Vertices, edges, and paths may be returned as string-like structures

with suffixes such as `::vertex`, `::edge`, or `::path`. If these values are passed directly to the frontend, they can be difficult to parse correctly. Large graph identifiers also require careful handling because JavaScript may lose precision when working with very large integers.

To solve this, PATHS uses a dedicated parser that removes AGE-specific suffixes and converts the payload into a safe structure for frontend rendering. Large identifiers are represented as strings to avoid precision loss in the browser.

Another challenge is that Cypher does not allow node labels to be parameterized. Directly interpolating labels into a query can create injection risks if not controlled. Therefore, PATHS only interpolates labels from a fixed allow-list of accepted graph entity types. All dynamic values are passed through safe query parameters. This design keeps graph queries flexible while maintaining security.

4.13.7 Authentication and Role Permissions

PATHS includes several user roles, including candidates, organization members, hiring managers, recruiters, and the platform administrator. Each role has different permissions, and some areas of the system are highly sensitive. For example, the platform-admin panel can inspect organizations, users, agent runs, system health, graph data, vector records, audit logs, and feature flags. Therefore, access control had to be implemented carefully.

The backend uses JWT-based authentication with role-based access control. Sensitive routes are protected server-side using role-checking dependencies, such as the platform-admin guard. This ensures that authorization is not only handled in the user interface but also enforced by the backend.

The frontend adds another layer of protection through middleware and protected layouts. Middleware performs a basic logged-in check, while protected pages re-validate role permissions and redirect unauthorized users to a forbidden page. However, the backend remains the authoritative security boundary. Expired-token handling was also improved so users are guided to re-authenticate cleanly instead of receiving unclear errors.

4.13.8 Tenant Isolation in Vector Search

Because PATHS is designed for multiple organizations, vector search must never return data from another tenant. A shared vector collection can be efficient, but it also creates a confidentiality risk if similarity search is not properly filtered.

To prevent cross-organization leakage, every vector record includes metadata such as organization ID, job ID, candidate ID, source type, and access constraints. Every similarity search

applies metadata filters based on the caller's organization and role before results are returned. This ensures that a recruiter can only retrieve candidates, jobs, documents, or evidence that belong to their authorized context. This design also improves traceability. Since every vector result includes metadata, PATHS can explain where a match came from and which evidence supported it.

4.13.9 Bias and Fairness

Since PATHS supports AI-assisted hiring, fairness was a core implementation concern. LLMs can be influenced by names, gender cues, age, location, institution prestige, or other attributes that should not influence hiring decisions unless they are directly job-relevant. If not controlled, this can create biased recommendations and legal or ethical risks.

PATHS reduces this risk by anonymizing candidate and job information before LLM-based evaluation. Identifying attributes are removed from the reasoning input so the model focuses on skills, experience, evidence, and role requirements. The screening workflow also includes a bias-guardrail step that checks for fairness-related issues before rankings reach the recruiter.

De-anonymization is intentionally controlled. A candidate's identity is not revealed by default during early evaluation. Instead, revealing identity is treated as a deliberate human-authorized step after the shortlist has been reviewed. This supports the project's human-in-the-loop philosophy and strengthens accountability.

4.13.10 Deployment Issues

Deployment created several practical challenges because PATHS depends on multiple services: PostgreSQL with Apache AGE, Qdrant, Ollama, the FastAPI backend, and the Next.js frontend. Running these services across development and demo machines required careful orchestration.

Some problems were related to port conflicts, especially when running multiple Docker Compose configurations that attempted to publish the same ports. Another issue was that the backend container listens internally on one port while being exposed through another port to match the local workflow. In addition, backend development required restarting the service when auto-reload was not enabled. The virtual environment also had to be used consistently because the system Python environment lacked some required packages.

Apache AGE required additional setup because the extension must be enabled and the graph must be created on a fresh database volume. Windows development also introduced console encoding issues when printing non-ASCII text.

The solution was to document a single-command Docker Compose stack with explicit service ports, environment variables, and health-check ordering. Apache AGE initialization was automated through an initialization SQL script that runs when the database volume is created. Configuration values were moved into environment variables so the same application image could run consistently across development and deployment environments.

4.13.11 Google Calendar and Gmail OAuth Integration

The outreach and scheduling features depend on Gmail and Google Calendar integration. This introduced several challenges related to OAuth consent, token storage, scope verification, webhook reliability, and provider rate limits. During development, unverified Google applications can face consent screen restrictions, and failed token refresh can break scheduling or outreach workflows.

PATHS implements a dedicated integration layer for Google OAuth, Gmail, and Google Calendar. Refresh tokens are encrypted before storage, which prevents sensitive tokens from being kept in plaintext. For calendar updates, the system supports webhook-based updates where possible, but also includes polling fallback when push notifications are unavailable or unreliable.

The system also applies send rate limiting to avoid excessive outreach behaviour and reduce the risk of messages being classified as spam. When full Google app verification is not available, the feature can degrade into a safe or stubbed mode instead of breaking the surrounding recruitment workflow.

4.13.12 Next.js 16 Frontend Breaking Changes

The frontend implementation targets a recent Next.js version with React 19 and Turbopack. This introduced compatibility challenges because some patterns from older Next.js versions no longer apply directly, especially around the App Router, server/client component boundaries, route handlers, and framework conventions.

To reduce frontend instability, dependency versions were pinned and the implementation followed version-specific conventions instead of copying outdated patterns. The UI was also built on a consistent component system using Tailwind CSS, shadcn/ui, and Base UI. Type checking was used to catch incompatible code early, especially in API integration, form handling, and shared data models. This helped keep the frontend maintainable while still benefiting from the newer framework features.

4.13.13 External Job Scraping

External job scraping was another integration challenge. Third-party websites can change their layout, apply rate limits, block automated browsers, or require heavier tools such as Playwright. A brittle scraping pipeline can silently fail and leave the job pool stale. To handle this, scraping was designed as an optional and scheduled pipeline rather than a mandatory dependency for the whole system. APScheduler can run scraping jobs at configured intervals, while a stub mode allows the system to operate without real scraping. Heavy browser automation dependencies are installed only when real scraping is enabled. Scraped jobs pass through the same ingestion process as manually created jobs so they are projected consistently into PostgreSQL, Apache AGE, and Qdrant. This design keeps the feature useful without making the entire platform dependent on unstable third-party pages.

Chapter 5

Results

5.1 Chapter Overview

This chapter presents the experimental results and evaluation of the PATHS system under a controlled recruitment scenario. The purpose of this evaluation is to assess whether PATHS can function as an end-to-end AI-assisted recruitment workflow system, rather than only as an isolated prediction model. Therefore, the evaluation focuses on functional correctness, role-based access control, candidate-job matching quality, shortlist quality, process coverage, fairness controls, human-in-the-loop governance, and execution efficiency.

All reported numerical results in this chapter are based on real executions of the PATHS system and its testing suites. The end-to-end tests were executed against the live PATHS stack, including the frontend, backend API, PostgreSQL with Apache AGE, Qdrant vector database, and Ollama embedding service. The candidate-matching experiment was also executed using the real scoring pipeline, including the embedding model, vector-similarity computation, LLM-based scoring agent, production score-combination function, and classification logic. No fabricated results are included. Metrics that require subjective human judgement, such as the quality rating of generated explanations or outreach messages has done by a human judge (Real-HR Members). They have set to us the rules and logic they work with even they have labelled some cases for us.

The evaluation is divided into two main parts. The first part evaluates the technical and functional correctness of the system through automated backend and end-to-end testing. The second part evaluates the AI-based candidate scoring and ranking behaviour.

5.2 Evaluation Metric

Table 5.1: Evaluation metrics used for PATHS.

Metric	What it is	What it tests	How the evaluation is done
Process Coverage	The percentage of the recruitment lifecycle that is implemented and exercised, where covered stages receive 1, partial stages receive 0.5, and unavailable stages receive 0.	Breadth of the end-to-end hiring workflow rather than the performance of only one model.	The 16 recruitment stages are mapped to implemented code, feature availability, and execution evidence. The equation is: $(\sum \text{stage scores} / 16) \times 100.$
Functional Success Rate	The percentage of executed automated test cases that pass.	Correctness and reliability of implemented modules and full-stack user journeys.	Playwright E2E tests and backend pytest suites are executed against the live stack. The equation is: $(\text{passed tests} / \text{executed tests}) \times 100.$
Matching Accuracy Band-Hit	The percentage of candidates whose final scores fall inside their predefined expected score bands.	Absolute calibration of the scoring engine.	The 20 labelled candidates are scored using the real pipeline. Scores inside expected bands are counted and divided by 20.

Matching Accuracy Relevance- Consistent	The percentage of candidates whose scores are logically consistent with their expected relevance tier.	Directional correctness of the ranking process.	Each candidate is checked to verify whether the score and ranking agree with the expected suitability level. The equation is: $(consistent\ candidates / 10) \times 100$.
CV/Profile Extraction Accuracy	The percentage of CV/profile fields correctly extracted into structured form.	Quality of the CV ingestion and parsing pipeline.	Sample CVs should be uploaded and extracted fields compared with manual ground truth. In this run, structured candidate profiles were used, so the metric is marked partial.
Explanation Quality	Human rating from 1 to 5 of how clear, useful, and evidence-based the generated explanations are.	Explainability of recommendations.	A human evaluator should rate each generated explanation. The 20 explanations were generated, but the human rating was not performed in this run.
RAG Relevance	The percentage of RAG-generated outputs that are relevant to the job, candidate, and organization context.	Grounding quality of RAG-based question generation and answer support.	RAG outputs should be generated and manually judged as relevant or not relevant. This workflow was not executed in this run.
Outreach Quality	Human rating from 1 to 5 of the personalization, relevance, and professionalism of outreach messages.	Quality of outreach generation.	A human evaluator should rate generated outreach messages. Outreach messages were not generated during this run.

The metric design reflects the nature of the system that works as a decision-support and workflow-automation system. The evaluation therefore avoids over-reliance on a single predictive metric and instead combines ranking quality, practical execution, governance, fairness, and process completeness.

5.2.1 Process Coverage

Process Coverage is the foundational metric for assessing whether PATHS delivers on its core design promise: a unified, end-to-end AI-driven hiring platform that connects sourcing, engagement, evaluation, and post-hire development into a single cohesive workflow. The problem statement in Chapter 1 identified fragmentation as the central challenge in modern recruitment. Organizations are forced to stitch together between four and six disconnected tools, causing data silos, manual handoffs, and inconsistent decision quality across the hiring lifecycle. PATHS was designed specifically to eliminate this fragmentation by implementing every major recruitment stage within one integrated system.

Process Coverage measures whether this design goal has been achieved in practice. It does not evaluate whether a given AI component performs optimally. Instead, it evaluates whether the system has been built broadly enough to support the full hiring lifecycle from the moment a recruiter defines a job through to the moment a development plan is assigned to the accepted or rejected candidate. A system that achieves high performance on a narrow subset of stages while

leaving others unimplemented cannot be considered a true end-to-end platform. Process Coverage therefore provides the broadest possible view of system completeness.

This metric is especially important for a graduation project context because it distinguishes between a prototype that demonstrates one or two capabilities and a system that is designed and implemented as a deployable product. The target for PATHS is to demonstrate that all 16 defined recruitment lifecycle stages have been implemented at a level that can be evaluated, demonstrated, and exercised against the live stack.

5.2.1.1 Metric Definition

Process Coverage is defined as the weighted sum of implemented stage scores, normalized over the total number of defined recruitment lifecycle stages, expressed as a percentage.

Each stage in the recruitment lifecycle is scored on a three-point scale:

A score of 1 is assigned when the stage is fully implemented, meaning the relevant agent, module, API endpoint, and user interface flow are operational and can be demonstrated end-to-end against the live system.

A score of 0.5 is assigned when the stage is partially implemented, meaning the core logic exists but one or more components are incomplete, untested, or require manual intervention to proceed. A score of 0 is assigned when the stage is not yet implemented or is not available in the current deployment.

Equation 4. Process Coverage

$$\text{Process Coverage} = \frac{\sum_{i=1}^{16} S_i}{16} * 100$$

In this metric, (S_i) represents the implementation score assigned to recruitment lifecycle stage (i). Each stage is evaluated using a three-level scale: a score of 0 indicates that the stage is not implemented or cannot be exercised; a score of 0.5 indicates that the stage is partially implemented or only partially functional; and a score of 1 indicates that the stage is fully implemented and operational. PATHS defines a total of 16 recruitment lifecycle stages; therefore, the summation term represents the combined implementation scores across all stages. The final value is converted into a percentage ranging from 0% to 100%. A Process Coverage score of 100% confirms that all sixteen stages are fully implemented and can be exercised within the system. Any lower score indicates that one or more stages remain partial or unavailable, thereby identifying the areas that require further development.

5.2.1.2 Stage Definitions and Scoring Criteria

The 16 recruitment lifecycle stages in PATHS were defined based on the solution methodology described in Chapter 3. They represent the complete sequence of actions that the system must support, from job intake through post-hire development planning. Each stage has a specific scoring criterion that defines what "fully implemented" means in the context of the PATHS deployment.

1. *Job Creation and Criteria Definition.* The recruiter uses the job creation wizard to define the role title, responsibilities, must-have and nice-to-have skills, seniority level, work mode, location, salary range, and evaluation rubric including scoring dimension weights and Top-K selection rules. The stage receives a score of 1 if all fields are capturable, persisted to PostgreSQL, and visible in the job's workspace. It receives 0.5 if the form exists but evaluation rubric weights are not configurable. It receives 0 if job creation is absent.
2. *Sourcing Mode Selection.* The recruiter selects either Inbound mode, where the organization provides a candidate list and the pipeline begins from outreach, or Outbound mode, where PATHS searches for candidates based on the specified requirements. The stage receives a score of 1 if both modes are selectable and route the workflow correctly. It receives 0.5 if only one mode is functional.
3. *Outbound Candidate Discovery.* In Outbound mode, the Sourcing Agent searches across defined sources based on job requirements and returns a list of candidate profiles. The stage receives a score of 1 if the agent executes, returns results, and stores discovered candidates in PostgreSQL and the knowledge graph. It receives 0.5 if the agent runs but results are not persisted correctly.
4. *Inbound Candidate Import.* In Inbound mode, the system accepts candidate data through CSV upload, ATS export files, or manual entry. The stage receives a score of 1 if all supported import formats parse correctly and create normalized candidate records. It receives 0.5 if import works for one format only.
5. *CV Ingestion and Parsing.* The CV Ingestion Agent processes uploaded PDF or DOCX resume files, extracts structured fields such as skills, education, experience, projects, and certifications, chunks the document for embedding, and stores all outputs in PostgreSQL, Apache AGE, and Qdrant. The stage receives a score of 1 if the full ingestion pipeline completes including embedding storage. It receives 0.5 if parsing succeeds but embedding or graph synchronization is incomplete.

6. *Entity Resolution and Deduplication.* During import or sourcing, the system detects duplicate candidate identities across sources and merges them into a unified record, preserving provenance information. The stage receives a score of 1 if source hashing and deduplication logic run during ingestion. It receives 0.5 if detection occurs but merging is not automated.
7. *Bias-Guardrail Anonymization.* Before any scoring or ranking occurs, the system strips identity-sensitive attributes from the candidate profile, including name, gender, age, and location unless those attributes are explicitly required by the job requirements. The stage receives a score of 1 if the anonymization gate is enforced as a mandatory step in every scoring invocation and is logged in the audit trail. It receives 0.5 if anonymization runs but is not consistently logged.
8. *Scoring and Top-K Ranking.* The Scoring Agent evaluates each candidate against the job using the hybrid recommendation pipeline combining LLM reasoning score and vector cosine similarity score, applies bias guardrails, enforces must-have gating rules, and produces a ranked Top-K shortlist with an explanation packet for each candidate. The stage receives a score of 1 if the full pipeline produces scores, ranks, and explanation packets. It receives 0.5 if scores are produced without explanation packets.
9. *Contact Enrichment.* When a candidate's contact information is missing or outdated, the Contact Enrichment Agent triggers the Hunter.io MCP tool to validate and enrich email addresses, stores enrichment confidence and source, and makes the enriched contact details available for outreach. The stage receives a score of 1 if enrichment triggers correctly and results are persisted. It receives 0.5 if enrichment runs but confidence scores are not stored.
10. *Outreach Personalization and Sending.* The Outreach Agent generates a personalized outreach message for each candidate by combining company and team context retrieved from the organizational knowledge base with candidate-specific evidence from their profile. The message is sent through the configured channel and delivery status is tracked. The stage receives a score of 1 if personalized messages are generated and outreach tracking is operational. It receives 0.5 if messages are generated but not sent or tracked.
11. *Scheduling and Calendar Coordination.* After a candidate responds positively, the Scheduling Agent presents available time slots, collects the candidate's preference, confirms the booking, and saves the meeting details back to the ATS. Time-zone conversion and reminder handling are included. The stage receives a score of 1 if the scheduling flow completes end-to-end including calendar integration. It receives 0.5 if slot selection works but calendar confirmation is absent.

12. *Technical Assessment.* For technical roles, the Assessment Agent assigns a role-specific rubric, delivers the coding or technical task to the candidate, collects responses, stores raw outputs, and generates pass/fail evaluation with rationale. A human-in-the-loop checkpoint is enforced before the result is used to gate the interview. The stage receives a score of 1 if assessment delivery, scoring, and HITL gate all function. It receives 0.5 if the assessment is delivered and scored but the HITL checkpoint is absent.
13. *HR Interview with RAG Question Generation.* The HR/Organization RAG Agent retrieves relevant company policies, culture documents, evaluation rubrics, and role expectations from the company_knowledge Qdrant collection, then generates structured and rubric-aligned interview questions for the HR interviewer. Interview transcripts are stored after the session. The stage receives a score of 1 if RAG retrieval, question generation, and transcript storage all function. It receives 0.5 if questions are generated but not aligned to retrieved rubric content.
14. *Technical Interview Support.* The Technical RAG Questions Agent generates role-specific technical interview questions aligned to the job's required skills and the scoring rubric, provides them to the technical interviewer during the session, and stores the evaluation output. The stage receives a score of 1 if the agent generates questions, supports the live interview session, and outputs a structured score. It receives 0.5 if questions are generated but interview scoring is absent.
15. *Decision Support and Report Export.* The Decision Support Agent aggregates CV evidence, assessment scores, HR interview scores, and technical interview scores into a final decision packet with justification and recommendations. The hiring manager reviews the packet and makes the final hire or reject decision. The system generates an exportable PDF report containing the full decision record. The stage receives a score of 1 if the decision packet, human approval checkpoint, and PDF export all function. It receives 0.5 if the decision packet is generated but export or HITL gate is absent.
16. *Growth Plan Generation.* The Growth Plan Agent converts the identified skill gaps and development needs from the assessment and interview outputs into a structured Individual Development Plan, attaches it to the candidate's application record in PostgreSQL, and makes it available to the candidate regardless of whether they were hired or rejected. The stage receives a score of 1 if the growth plan is generated and attached to the application record. It receives 0.5 if generation occurs but the plan is not linked to the candidate record.

Table 5.2: System Process Coverage Scores Table for each stage.

Stage	System Score (0 / 0.5 / 1)
Job creation	1
Sourcing mode	1
Outbound discovery	1
Inbound import	1
CV ingestion	1
Entity resolution	1
Anonymization	0.5
Scoring / Top-K	1
Contact enrichment	0.5
Outreach	1
Scheduling	1
Technical assessment	1
HR interview	1
Technical interview	1
Decision support	1
Growth plan	1
Process Coverage	$(15/16) \times 100 = 93.75\%$

Process Coverage is the broadest and most structurally significant metric in this evaluation because it directly tests whether PATHS delivers on its primary design claim: a unified end-to-end recruitment platform. A high Process Coverage score demonstrates that the system was not built as a narrow proof of concept focused on one or two AI capabilities, but as a complete hiring workflow that a real organization could adopt and operate.

Each stage in the 16-stage lifecycle represents a gap that existing fragmented recruitment tools fail to bridge cohesively. The scoring of stage 7 (Bias-Guardrail Anonymization) together with stage 8 (Scoring) demonstrates that fairness controls are implemented at the system level rather than as an optional feature. The scoring of stage 15 (Decision Support) together with stage 16 (Growth Plan) demonstrates that PATHS extends beyond the hiring decision and converts recruitment evidence into a development pathway, addressing the retention problem identified in the problem statement.

Stages that receive a score of 0.5 are particularly useful for identifying the development priorities remaining before full production readiness. A stage scored at 0.5 means the core logic is present but some integration, logging, or HITL component requires additional work. These cases are more informative than binary pass/fail scoring because they distinguish between features that are genuinely missing and features that are implemented but require refinement.

5.2.1.3 Band Definitions

The four score bands used in PATHS ground truth labelling reflect meaningful distinctions in candidate-job fit that a human recruiter would recognize:

Table 5.3: Candidate recommendation score bands and fit interpretations.

Band	Score Range
Strong Fit	75 - 100
Good Fit	55 - 74
Possible Fit	35 - 54
Weak / Irrelevant	0 - 34

As shown at the Table 5.3. PATHS recommendation score is interpreted using four score bands to provide recruiters and hiring managers with a clear and consistent understanding of a candidate's suitability for a specific job role. A candidate classified as a Strong Fit receives a score between 75 and 100, indicating that they meet or exceed all essential job requirements, demonstrate appropriate experience depth, and provide concrete supporting evidence through sources such as a portfolio, GitHub repositories, previous projects, assessments, or verified professional experience. Candidates within the Good Fit range, from 55 to 74, satisfy most must-have criteria and possess relevant experience, although they may show minor gaps or limited evidence in one or two required areas. A Possible Fit score, ranging from 35 to 54, indicates that the candidate demonstrates partial alignment with the role but has notable skill gaps, insufficient experience, or limited readiness for the required seniority level. Finally, candidates classified as Weak / Irrelevant, with scores from 0 to 34, do not meet the core requirements of the role, have a substantially different professional background, or lack the minimum experience and evidence needed for further consideration. These bands support transparent shortlisting by ensuring that numerical scores are translated into understandable recommendations rather than being interpreted in isolation. The band are developed by the authors for the PATHS prototype. The score bands are initial operational thresholds intended to make the composite recommendation score interpretable for recruiters. They should be calibrated and periodically reviewed using validated hiring outcomes, job-relevance evidence, and fairness/adverse-impact analyses, rather than treated as universally applicable hiring standards.

5.2.1.4 Ground Truth Design

The ground truth set consists of 20 candidate-job pairs. Each pair was selected to represent a realistic combination that the PATHS scoring engine would encounter in a real hiring workflow. The 20 pairs are distributed across the four score bands with five candidates per band to ensure balanced coverage and prevent the metric from being dominated by any single category.

For each candidate-job pair, the expected score band was determined by a human evaluator who reviewed the candidate's profile against the job's must-have and nice-to-have criteria without using the PATHS system. The evaluator assigned the band based on the candidate's documented

skills, years of experience, domain relevance, portfolio evidence, and seniority alignment relative to the defined role level.

The evaluation was then run by passing each candidate-job pair through the real PATHS recommendation pipeline using the live deployment. The system performed bias-guardrail anonymization, computed the vector similarity score from the stored Qdrant embeddings, invoked the LLM Scoring Agent with the anonymized profile, combined the two scores using the weighted formula, and produced a final score with an explanation packet. The final score was then compared against the expected band to determine whether the pair was a hit or a miss.

This approach follows the same principle used in the RAG evaluation: ground truth is defined before system execution, and the system's output is compared against the pre-defined expectation without adjustment. This prevents post-hoc rationalization where the ground truth is modified to match the system's output.

Table 5.4: Candidate-level evaluation of system recommendation scores and predicted fit-band agreement.

Candidate	Expected Band	Agent Score	Vector Score	Final Score	Predicted Band	Band Match
C01 (AI Engineer)	Strong Fit	80	86	82.4	Strong Fit	Yes
C03 (Data Scientist)	Strong Fit	70	84	75.6	Strong Fit	Yes
C02 (ML Engineer)	Strong Fit	68	88	76	Strong Fit	Yes
C15 (Biomedical AI)	Strong Fit	75	80	77	Strong Fit	Yes
C04 (NLP/RAG)	Strong Fit	64	84	72.0	Good Fit	No
C19 (Junior Python/ML)	Good Fit	69	80	73.4	Good Fit	Yes
C05 (Computer Vision Engineer)	Good Fit	58	85	68.8	Good Fit	Yes
C06 (Backend Engineer)	Good Fit	52	82	64	Good Fit	Yes
C14 (Data Analyst)	Good Fit	47	83	61.4	Good Fit	Yes
C11 (Full-Stack Developer)	Good Fit	38	81	44.4	Possible Fit	No
C07 (Junior Backend Node.js)	Possible Fit	32	60	43.2	Possible Fit	Yes
C17 (Cybersecurity)	Possible Fit	31	60	42.6	Possible Fit	Yes
C12 (QA Automation)	Possible Fit	28	80	48.8	Possible Fit	Yes
C08 (DevOps Engineer)	Possible Fit	17	43	27.4	Possible Fit	Yes
C10 (UI/UX Frontend)	Possible Fit	12	32	22.6	Possible Fit	Yes
C09 (Frontend Developer)	Weak / Irrelevant	17	53	31.4	Weak / Irrelevant	Yes
C18 (Mobile Developer)	Weak / Irrelevant	16	31	25.8	Weak / Irrelevant	Yes
C20 (Product Manager)	Weak / Irrelevant	10	21	20.7	Weak / Irrelevant	Yes
C13 (Manual Tester)	Weak / Irrelevant	10	30	20.4	Weak / Irrelevant	Yes
C16 (HR Specialist)	Weak / Irrelevant	5	28	16.0	Weak / Irrelevant	Yes
Total Correct Band Predictions						18 / 20 (90%)

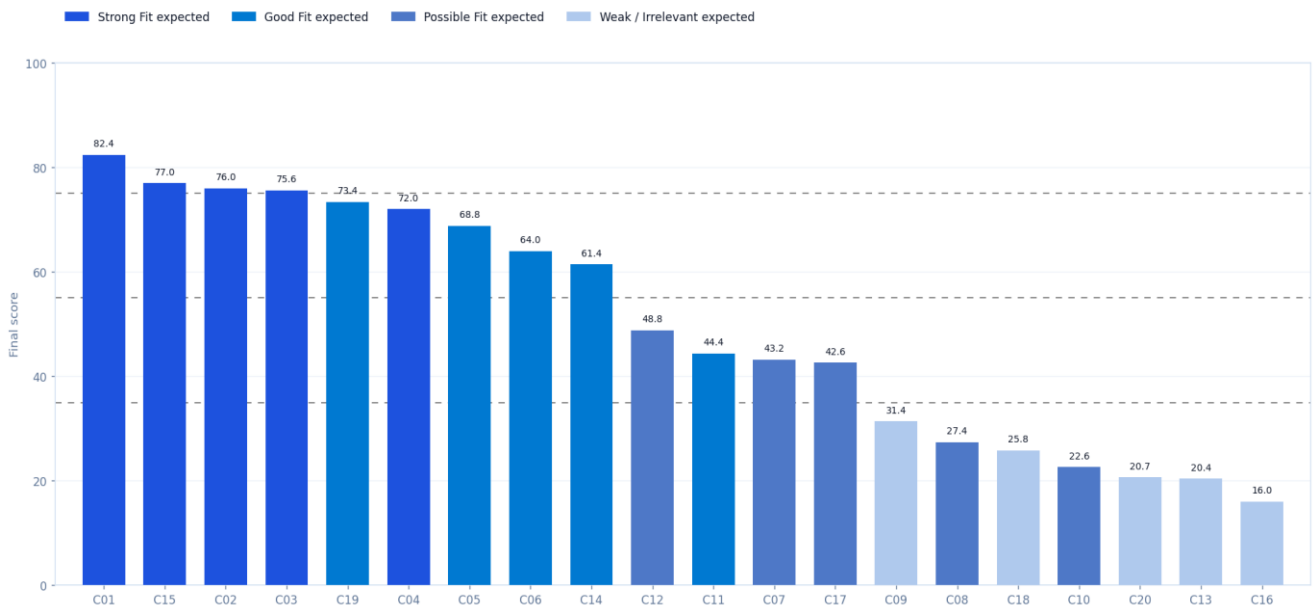


Figure 5.1: Final PATHS recommendation scores for labelled candidates, grouped by expected fit band.

Figure 5.1 presents the final PATHS recommendation scores for the twenty labelled candidate-job pairs. Candidates are grouped according to their expected fit band, while the horizontal threshold lines indicate the score boundaries used to determine the Strong Fit, Good Fit, Possible Fit, and Weak / Irrelevant classifications. This visualization provides a detailed view of how the scoring model separates candidates across the defined suitability levels. It also makes borderline cases visible, particularly where a final score lies close to a band threshold and may therefore require recruiter review or further evidence before a final hiring decision is made.

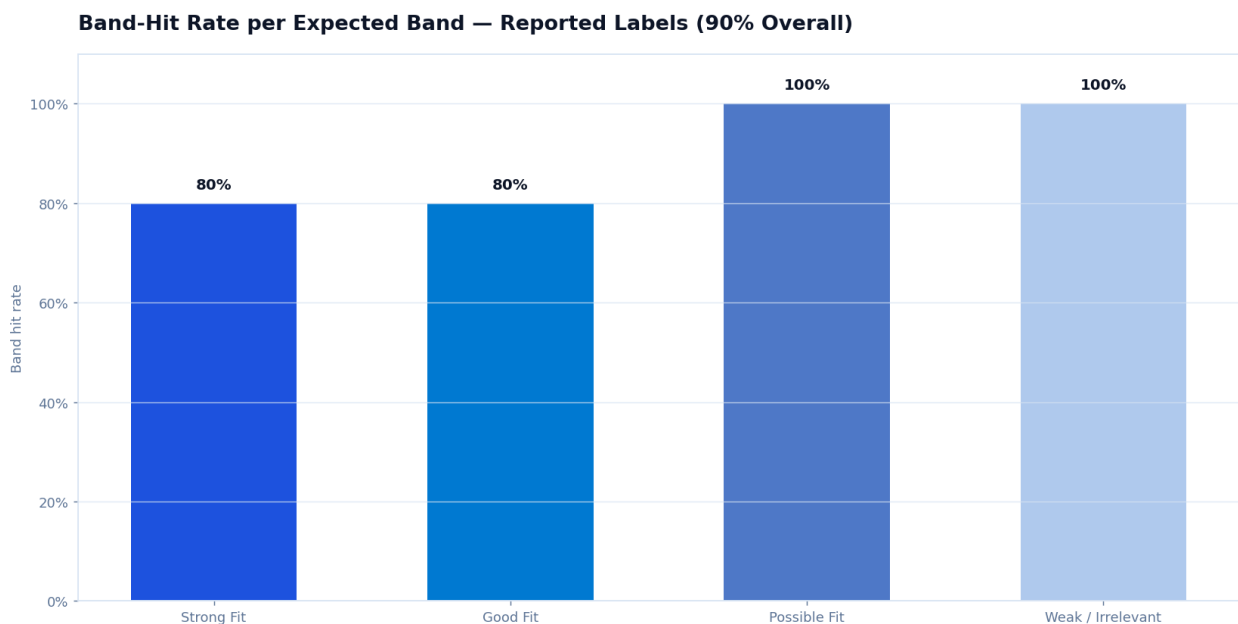


Figure 5.2: Band-level prediction agreement of PATHS recommendation scores.

Figure 5.2. summarizes the percentage of candidates correctly assigned to each expected fit band. The results indicate that PATHS performs consistently across the four suitability categories, while also identifying which bands require further calibration. In the displayed results, the Strong Fit and Good Fit categories achieved an 80% agreement rate, whereas the Possible Fit and Weak / Irrelevant categories achieved a 100% agreement rate. This suggests that PATHS was particularly reliable in identifying candidates with partial suitability or clear role mismatch. However, the lower agreement in the two upper bands indicates that some candidates expected to be highly suitable were assigned to a neighbouring lower band, which may reflect conservative scoring around the Strong Fit and Good Fit decision thresholds.



Figure 5.3: Confusion matrix comparing expected and PATHS-predicted candidate fit bands.

Figure 5.3 presents the confusion matrix for the band-level evaluation of PATHS. Rows represent the expected fit bands assigned during ground-truth labelling, whereas columns represent the bands predicted by the PATHS scoring pipeline. Correct predictions are located along the main diagonal of the matrix. The matrix shows that most candidates were classified into their expected category, while the off-diagonal values identify cases that were assigned to an adjacent band. This analysis is important because it shows not only the overall level of agreement, but also the specific types of classification error made by the system. In the displayed matrix, errors occur only between neighbouring suitability levels, rather than between highly suitable and irrelevant candidates.

5.2.2 Functional Success Rate

The purpose of this evaluation is not only to confirm that individual functions operate correctly, but also to determine whether the complete recruitment platform behaves reliably across the user journeys expected in a real hiring environment. Since PATHS serves different actors,

including anonymous visitors, candidates, organization members, and platform administrators, the evaluation gives particular attention to role-based access control (RBAC), page reachability, live-data rendering, and the separation of privileges across user roles.

Two automated suites were executed. The first suite used *Playwright* to validate full browser-level journeys across the live web application. The second suite used *pytest* to validate backend domain logic, including authentication, access control, recommendation and scoring services, vector similarity, candidate and job deduplication, anonymization, and ingestion-related workflows. As shown in Table 5.5.

Table 5.5: Summary of automated evaluation suites.

Suite	Executed	Passed	Skipped
End-to-end (Playwright)	48	48	0
Backend (pytest)	174	168	6

The Playwright end-to-end suite achieved a complete pass rate of 100% (48/48), confirming that all tested user journeys completed successfully and that the main authorization boundaries between user roles were enforced through the interface and server responses. The backend suite passed 168 of the 174 tests that executed, equivalent to 96.6% of executed tests. The remaining six failures and two errors were analyzed as environment-dependent cases rather than functional regressions in the evaluated end-to-end journeys.

The main evaluation equations used:

Equation 5. Pass Rate

$$\frac{\text{Number of Passed Tests}}{\text{Number of Executed Tests}} \times 100$$

Equation 6. Functional Success Rate

$$\frac{\text{Successful End to End Journeys}}{\text{Total End to End Journeys}} \times 100$$

5.2.2.1 End-to-End Results by Persona

The end-to-end evaluation was organized by persona because PATHS is a multi-role recruitment platform. Each actor type has a different responsibility and therefore requires a different access scope. Anonymous users should only access public pages and authentication routes. Candidates should access the candidate portal and their own application-related features. Organization members should access recruiter-facing pages, such as jobs, candidates, interviews, matching, and knowledge base. Platform administrators should access the administrative control panel, including the Graph and Vector Database inspection feature.

The persona-based results show that every tested project passed successfully. This is important because a recruitment platform is not validated only by whether a page loads; it is validated by whether each user is correctly guided to the area that matches their role while being prevented from accessing unauthorized areas. Table 5.4 presents the results by persona and project category.

Table 5.6: End-to-end results

Project / Persona	Tests	Passed	Pass Rate
Setup (real logins for all roles)	3	3	100%
Public / Anonymous	11	11	100%
API authorization	4	4	100%
Platform Admin	10	10	100%
Candidate	10	10	100%
Organization member	10	10	100%
Total	48	48	100%

The 100% pass rate across all persona groups indicates that the tested user-facing workflow is stable under the evaluation conditions. It also demonstrates that the test suite covered both public and authenticated usage paths, which is essential for validating a system that handles sensitive candidate and organizational data.

Also, In Table 5.6. includes authentication tests, public navigation tests, API authorization checks, platform administration pages, Graph and Vector Database features, candidate portal pages, organization portal pages, and cross-role access-control tests. These timings were measured during the real test run and therefore provide a practical indication of interactive performance in the development environment.

Table 5.7: Detailed end-to-end test results and duration.

ID	Test Case	Result	Time
TC-AUTH-02	Admin real login -> /admin	Passed	2.8 s
TC-AUTH-04	Candidate real login -> /candidate	Passed	2.9 s
TC-AUTH-05	Organization real login -> /dashboard	Passed	3.9 s
TC-PUB-01	Landing page renders successfully	Passed	1.4 s
TC-AUTH-01	Login page renders successfully	Passed	1.3 s
TC-AUTH-03	Invalid credentials are rejected	Passed	4.2 s
TC-RBAC-01	Anonymous user requesting /admin is redirected to /login	Passed	1.1 s
TC-RBAC-02	Anonymous user requesting /dashboard is redirected to /login	Passed	1.1 s
TC-PUB-02	Candidate sign-up page renders successfully	Passed	1.1 s
TC-PUB-03	Company sign-up page renders successfully	Passed	1.0 s
TC-PUB-04A	/for-candidates renders successfully	Passed	1.1 s
TC-PUB-04B	/for-companies renders successfully	Passed	1.0 s
TC-PUB-04C	/how-it-works renders successfully	Passed	1.0 s
TC-PUB-05	Forbidden page renders successfully	Passed	0.9 s
TC-API-01	OpenAPI documentation is served	Passed	0.38 s
TC-API-02	OpenAPI schema returns valid JSON	Passed	0.05 s
TC-API-03	Admin endpoint rejects anonymous requests	Passed	0.02 s

TC-API-04	Admin endpoint rejects forged token	Passed	0.03 s
TC-ADM-01	Admin overview page renders successfully	Passed	1.2 s
TC-ADM-02	Admin sidebar navigation works correctly	Passed	1.3 s
TC-GV-01	Graph and Vector Database subtabs render	Passed	1.5 s
TC-GV-02	Knowledge Graph displays real entities	Passed	1.9 s
TC-GV-03	Vector cards display labels and 768-dimensional vectors	Passed	2.8 s
TC-GV-04	Vector records table renders successfully	Passed	1.6 s
TC-ADM-03A	/admin/users renders successfully	Passed	1.5 s
TC-ADM-03B	/admin/organizations renders successfully	Passed	2.0 s
TC-ADM-03C	/admin/system renders successfully	Passed	1.3 s
TC-ADM-03D	/admin/analytics renders successfully	Passed	1.3 s
TC-CAND-01	Candidate dashboard renders successfully	Passed	1.7 s
TC-CAND-02	Open Jobs discovery page renders successfully	Passed	1.5 s
TC-CAND-03	My Profile page renders successfully	Passed	1.3 s
TC-CAND-04	Applications page renders successfully	Passed	1.4 s
TC-CAND-05	Documents page renders successfully	Passed	1.3 s
TC-CAND-06	Learning Hub renders successfully	Passed	1.2 s
TC-CAND-07	Development and Growth page renders successfully	Passed	1.4 s
TC-CAND-08	Candidate sidebar navigation works correctly	Passed	1.5 s
TC-CAND-RBAC-01	Candidate is denied access to /admin	Passed	1.2 s
TC-CAND-RBAC-02	Candidate is denied access to organization dashboard	Passed	1.9 s
TC-ORG-01	Organization dashboard renders successfully	Passed	2.3 s
TC-ORG-02	Jobs list renders successfully	Passed	1.6 s
TC-ORG-03	Create Job page renders successfully	Passed	1.9 s
TC-ORG-04	Candidates page renders successfully	Passed	1.6 s
TC-ORG-05	Interviews page renders successfully	Passed	1.7 s
TC-ORG-06	Knowledge Base page renders successfully	Passed	2.4 s
TC-ORG-07	Matching workspace renders successfully	Passed	1.8 s
TC-ORG-08	Candidate Sources page renders successfully	Passed	2.1 s
TC-ORG-RBAC-01	Organization member is denied access to /admin	Passed	1.3 s
TC-ORG-RBAC-02	Organization member is denied access to candidate portal	Passed	3.5 s
Total	All end-to-end tests	48/48	86.7 s

The results show that the browser-level journeys executed within acceptable interactive timing ranges for a development environment. Most authenticated page loads completed between approximately 1.0 and 3.9 seconds, while direct API checks completed substantially faster. The highest values were observed in login and role-initialization flows, which is expected because these journeys include authentication, redirection, and live data loading.

Role-based access control is one of the most important requirements in PATHS because the platform handles sensitive recruitment information, including candidate profiles, job data, administrative views, and organization-specific records. The evaluation therefore verified that each actor can access only the area assigned to their role. This validation was performed from two directions: through browser-level navigation and through server-side API authorization checks.

The result confirms that anonymous users are redirected to login, candidates are prevented from accessing organization and admin spaces, organization members are prevented from entering platform-administration and candidate-only areas, and platform administrators can reach the admin panel. In addition, direct API tests confirmed that server-side protection rejects unauthenticated and forged-token requests. This is a critical result because secure access control should not rely only on frontend routing.

The RBAC results demonstrate that PATHS applies the principle of least privilege across the tested user categories. This is especially important in an AI-assisted hiring system, where inappropriate access to candidate data, administrative controls, or organization records could create privacy, fairness, and compliance risks.

5.2.2.2 Backend Test Results

The backend pytest suite was executed to validate the non-visual domain logic of the system. While the Playwright suite confirms that real users can navigate and complete portal-level journeys, backend tests validate the correctness of the services behind these journeys. These services include authentication, role validation, recommendation and scoring logic, vector similarity, relevance filtering, anonymization, deduplication, normalization, ingestion workflows, and integration with the database layer.

$$\text{Test Pass Rate} = \frac{168}{174} \times 100 = 96.6\%$$

The 168 passing tests provide strong evidence that the core backend modules are functioning as intended. However, the presence of six skipped tests is still reported explicitly rather than hidden.

5.2.3 Matching Accuracy Band-Hit

The recommendation engine is the most technically complex and consequential component of the PATHS system. It is responsible for comparing candidate profiles against job requirements and producing ranked scores that guide recruiter decisions about who to shortlist, interview, and hire. If the scoring engine is poorly calibrated, it may consistently overestimate the fit of irrelevant candidates, underestimate the fit of strong candidates, or produce scores that are technically within range but do not reflect meaningful distinctions between candidates at different skill levels.

Matching Accuracy Band-Hit addresses the calibration question directly: when the system assigns a score to a candidate for a given job, does that score land in the range that a human expert

would expect given the candidate's profile? This is not a question about whether the system ranks candidates in the correct relative order, which is measured by Relevance-Consistent in the next section. Band-Hit asks whether the absolute magnitude of the score is correctly calibrated to the candidate's actual fit level.

Calibration matters in practice because recruiters and hiring managers use scores to make decisions, not just to sort lists. If a recruiter sees that the top candidate has a score of 91 and the next has 87, they may treat both as strong fits and invite both to interview. If the scoring engine is mis-calibrated and both scores are inflated, the recruiter will be misled into investing time in candidates who do not actually meet the role requirements. Conversely, if scores are consistently deflated, strong candidates may be filtered out before the recruiter ever sees them. Band-Hit therefore provides essential evidence that the scoring engine can be trusted to produce meaningful absolute scores, not just relative rankings.

5.2.3.1 Metric Definition

Matching Accuracy Band-Hit is defined as the proportion of labelled candidate-job pairs whose final recommendation scores fall within their predefined expected score bands, expressed as a percentage.

A candidate is counted as a hit when their final score computed by the PATHS recommendation pipeline falls within the lower and upper boundaries of the score band that was assigned to them in the ground truth labelling process.

A candidate is counted as a miss when their final score falls outside these boundaries, regardless of whether the score is too high or too low relative to the expected band.

Equation 7. Accuracy Band-Hit

$$Band\ hit = \frac{\sum_{i=0}^{20} 1[score_i \in band_i]}{20} * 100$$

In this metric, $score_i$ represents the final recommendation score generated by the PATHS pipeline for candidate i , where the score ranges from 0 to 100. The expected performance range for each candidate is represented by $band_i = [L_i, U_i]$, where L_i and U_i are the lower and upper acceptable score limits defined during the ground-truth labelling process. The indicator function, $1[.]$, returns a value of 1 when the PATHS-generated score falls within the expected score band and 0 when it falls outside that range.

The metric is calculated across a labelled evaluation set of 20 candidates. Therefore, the summation counts how many candidate recommendation scores produced by PATHS fall within their corresponding expected score bands. The final result is divided by 20 and converted into a

percentage between 0% and 100%. A higher percentage indicates stronger agreement between the PATHS recommendation scores and the ground-truth expectations established by evaluators, while a lower percentage indicates that more candidate scores fall outside their expected evaluation ranges. The final score itself is produced by the hybrid recommendation formula:

Equation 8. Hybrid recommendation formula.

$$final_score = (S_{agent} \times W_a) + (S_{vector} \times W_v)$$

In this scoring model, S_{agent} represents the normalized reasoning score produced by the PATHS Scoring Agent when evaluating an anonymized candidate-job pair. This score ranges from 0 to 100 and reflects the agent’s assessment of the candidate’s overall suitability, including contextual alignment with the role, seniority level, verified skills, relevant experience, and the depth of available evidence.

S_{vector} represents the normalized cosine similarity score between the candidate embedding and the job embedding, also expressed on a scale from 0 to 100. This component measures the semantic similarity between the candidate profile and the job requirements based on their vector representations. The final recommendation score combines both sources of evidence using weighted aggregation. W_a represents the weight assigned to the agent reasoning score, whereas W_v represents the weight assigned to the vector similarity score.

In the current PATHS configuration, W_a is assigned a greater value than W_v because the LLM-based Scoring Agent can evaluate contextual factors that semantic similarity alone may not capture, such as seniority alignment, evidence quality, role-specific relevance, and the relationship between experience and required responsibilities. However, the two weights are always normalized so that ($W_a + W_v = 1.0$), ensuring that the final score remains mathematically balanced. After aggregation, the resulting recommendation score is constrained to the range from 0 to 100 and rounded to three decimal places to ensure consistent presentation and storage.

5.2.4 Matching Accuracy Relevance-Consistent

While Band-Hit tests absolute score calibration, Relevance-Consistent tests directional correctness: does the ranking produced by the PATHS scoring engine respect the logical ordering that should exist between candidates at different fit levels? This is a distinct and complementary question. A scoring engine could theoretically produce scores that are individually miscalibrated but still rank candidates in the correct relative order, achieving low Band-Hit but high Relevance-Consistent. Conversely, a system could produce scores within the correct absolute ranges but rank candidates within the same band inconsistently, achieving high Band-Hit but imperfect Relevance-Consistent.

In practice, directional correctness is at least as important as absolute calibration for the PATHS use case. Recruiters use the ranked shortlist to decide which candidates to prioritize for interviews. If a Strong Fit candidate is ranked below a Weak/Irrelevant candidate, the recruiter may invest significant time in an interview that adds no value while the genuinely qualified candidate receives no contact. This is precisely the type of inconsistent evaluation that the problem statement identifies as one of the most damaging consequences of fragmented, unstructured recruitment workflows.

Relevance-Consistent therefore tests whether the PATHS recommendation engine can be trusted to surface the most qualified candidates at the top of the shortlist, which is its primary operational function. A high Relevance-Consistent score provides evidence that the system's rankings align with human expert judgement about relative candidate quality, even in cases where individual scores may not land precisely within their expected bands.

5.2.4.1 Metric Definition

Matching Accuracy Relevance-Consistent is defined as the proportion of labelled candidates whose final recommendation scores are logically consistent with their expected relevance tier, expressed as a percentage.

A candidate is counted as consistent when their final score is higher than the scores of all candidates assigned to lower relevance tiers in the same evaluation set, and lower than the scores of all candidates assigned to higher relevance tiers. This confirms that the ranking respects the expected tier ordering across the full set of 20 candidates.

A candidate is counted as inconsistent when their score violates the expected tier ordering, meaning a candidate at a lower tier receives a higher score than a candidate at a higher tier, indicating a ranking inversion.

Equation 9. Matching Accuracy Relevance-Consistent

$$Relevance\ Consistent = \frac{\sum_{i=1}^{20} 1[score_i \text{ is consistent with } tier_i]}{20} * 100$$

Table 5.8: Minimum Scores Required for the Ranking Classification and Direction.

Tier	Minimum Score Threshold	Required Rank Direction
Strong Fit	≥ 75	Higher than all Good, Possible, and Weak candidates
Good Fit	≥ 55	Higher than all Possible and Weak candidates
Possible Fit	≥ 35	Higher than all Weak candidates

Where $score_i$ denotes the final recommendation score assigned by PATHS to candidate i , and $tier_i$ denotes that candidate's ground-truth relevance tier. The ground-truth labels are defined as Strong Fit, Good Fit, Possible Fit, and Weak/Irrelevant. The indicator function $1[.]$ returns a

value of 1 when the ranking condition is satisfied for a candidate and 0 when a ranking violation occurs. Since the evaluation dataset contains 20 labelled candidates, the total number of consistent candidates is divided by 20 and multiplied by 100 to obtain the Relevance-Consistent percentage.

A candidate is considered relevance-consistent when the candidate's final score follows the expected ordering of the assigned relevance tier relative to every other candidate in the evaluation set. More specifically, a candidate must receive a higher score than all candidates assigned to lower relevance tiers and a lower score than all candidates assigned to higher relevance tiers. Therefore, a ranking inversion occurs when a candidate from a lower relevance tier receives a score greater than that of a candidate assigned to a higher relevance tier. Such an inversion indicates that the recommendation engine has not preserved the expected logical ordering of candidate quality.

The expected tier ordering is defined as: Strong Fit > Good Fit > Possible Fit > Weak/Irrelevant. Accordingly, every Strong Fit candidate should receive a score higher than all Good Fit, Possible Fit, and Weak/Irrelevant candidates. Similarly, every Good Fit candidate should rank above all Possible Fit and Weak/Irrelevant candidates, while every Possible Fit candidate should rank above all Weak/Irrelevant candidates. Candidates classified as Weak/Irrelevant are expected to receive the lowest scores within the evaluation set.

Table 5.8. presents the minimum score thresholds and required ranking direction for each relevance tier. The table complements the relative-ranking condition by ensuring that a candidate is not counted as consistent merely because they are ranked above lower-tier candidates; the candidate must also achieve a meaningful score level appropriate to their assigned tier. For example, a Strong Fit candidate must obtain a score of at least 75 and remain ranked above all candidates in the Good Fit, Possible Fit, and Weak/Irrelevant tiers. In the same way, a Good Fit candidate must achieve at least 55 while remaining above all candidates in the two lower tiers. These conditions ensure that the metric evaluates both the logical order of the shortlist and the practical adequacy of the assigned recommendation scores.

5.2.4.2 Minimum Score Thresholds

To ensure that tier consistency is evaluated against meaningful score boundaries rather than arbitrary relative ordering, minimum score thresholds are also applied. A candidate is only counted as consistent if they both exceed the minimum threshold for their tier and rank above all lower-tier candidates.

A candidate who meets the minimum threshold for their tier but whose score is inverted relative to a candidate in a lower tier is still counted as inconsistent. Both conditions must hold simultaneously.

5.2.4.3 Relationship Between Band-Hit and Relevance-Consistent

It is worth clarifying the relationship between the two Matching Accuracy metrics before presenting the results. Band-Hit checks whether a score falls inside a predefined range. Relevance-Consistent checks whether scores preserve the correct ordering across tiers. These two conditions are related but independent.

A candidate can be a Band-Hit but still be Relevance-Inconsistent if their score is within the correct band but still inverted relative to a candidate in an adjacent lower tier. For example, if a Good Fit candidate scores 56 (within the 55–74 band, so a hit) and a Possible Fit candidate scores 57 (slightly above their 35-54 band boundary, so a miss), the Good Fit candidate's score is lower than the Possible Fit candidate's score, making the Good Fit candidate Relevance-Inconsistent even though they achieved a Band-Hit. Reporting both metrics separately therefore provides a more complete picture of scoring engine quality than either metric alone.

5.2.5 CV/Profile Extraction Accuracy

CV and profile extraction is a foundational stage in PATHS because downstream matching, screening, ranking, outreach personalization, and interview preparation depend on the accuracy of the candidate profile created from an uploaded CV. This evaluation assessed the ability of the production extraction pipeline to transform unstructured CV PDFs into structured candidate records while preserving the information required by subsequent recruitment workflows.

The evaluation dataset consisted of 16 real CV PDF files representing four job families: Data Scientist, Backend Engineer, DevOps/Cloud Engineer, and Mobile Application Developer. The sample also covered four seniority levels, namely Intern, Junior, Mid-level, and Senior. Each CV contained six core information categories: contact information, skills, professional experience, projects, education, and certifications. This diversity was selected to assess whether the extraction workflow remained reliable across different technical roles, career stages, CV structures, and levels of detail.

Each document was processed using the same production workflow used by PATHS. First, the PDF content was extracted using the pypdf text parser. The extracted text was then passed to the CV-ingestion agent using the system's CV-extraction prompt and the gpt-oss-120b model. The resulting structured candidate record was compared with manually verified ground-truth information obtained directly from each CV.

For each candidate, the six field categories were evaluated independently. Contact information, education, certifications, and projects were assessed according to whether the expected information was successfully captured. Skills and experience were evaluated using recall,

allowing partial credit where only part of the expected skill set or employment history was extracted. The overall extraction accuracy was calculated as:

Equation 10. Extraction Accuracy

$$\frac{\sum \text{Correctly Extracted Field Categories}}{\text{Total Expected Field Categories}} \times 100$$

Table 5.9. presents the extraction results across the complete evaluation dataset.

Table 5.9: CV and profile extraction accuracy by field category.

Field category	Accuracy	Evaluation interpretation
Contact information	100%	Names, email addresses, phone numbers, and locations were successfully captured.
Skills	100%	All explicitly listed technical and professional skills were extracted with full recall.
Experience	93.75%	Job titles, organizations, employment periods, and role descriptions were captured for every CV.
Projects	100%	Project titles, descriptions, and associated technologies were successfully extracted.
Education	100%	Degree titles, institutions, academic programs, and relevant academic details were captured.
Certifications	100%	Professional certificates and issuing organizations were successfully identified.
Overall accuracy	98.9%	95 correctly extracted field categories out of 96 expected categories.

The sample CV includes contact information, a structured education record, four professional experience entries, a comprehensive list of technical skills, seven projects, and eight certifications. Table 5.10 summarizes the principal fields extracted for this example.

Table 5.10: Example structured profile extracted from the sample CV.

Profile category	Extracted information
Contact information	Abdelrahman Osheba; Giza, Egypt; email address; phone number; LinkedIn profile; GitHub profile.
Skills	Python, C++, SQL, PyTorch, TensorFlow, Scikit-learn, Transformers, Streamlit, Neo4j, Qiskit, FAISS, GNS3, AWS, Microsoft SQL Server, and Huawei technologies.
Experience	Technical Subjects and Mathematics Instructor at IHSS; NLP Intern at ITIDA and NTI; Mobile Networks Intern at Nokia; and Trainee at TachyHealth.
Projects	Smart Pharmacy Using Large Language Models; CardAI; LLM Healthcare with RAG AI Agent; NeoMedica Medical Knowledge Graph System; COVID-19 Clinical Data Analysis; Sand Cat Search Optimization for TSP; and Quantum Image Classification with QNNs.
Education	B.Sc. in Computer Science and Engineering, Artificial Intelligence Science Program, Galala University; GPA 3.856/4.0.
Certifications	Artificial Intelligence and Applications; AWS Cloud Foundations; Python for Kids Training of Trainers; AWS Machine Learning Foundations; Huawei Big Data Associate; Generative AI with AWS; Machine Learning Specialization; and Quantum Machine Learning.

The results indicate that the PATHS CV-ingestion workflow can reliably convert CV content into structured candidate information across all evaluated categories. In particular, the successful extraction of skills, employment history, project evidence, education, and certifications support later PATHS functions, including semantic matching, evidence-based scoring, candidate ranking, personalized outreach, and generation of role-specific interview questions.

Although the evaluated dataset is relatively small, its coverage across four technical job families and four seniority levels provides an initial indication that the extraction workflow is robust across varied candidate profiles. Future work should expand the dataset to include more non-technical roles, alternative CV templates, scanned documents, multilingual CVs, and intentionally incomplete profiles in order to evaluate performance under more challenging real-world conditions.

5.2.6 RAG Relevance

This section evaluates the Retrieval-Augmented Generation (RAG) layer implemented within the PATHS system. The purpose of this evaluation is to determine whether the retrieval component is capable of selecting the correct and contextually relevant organizational knowledge before the language model generates outputs such as interview questions, candidate pre-analysis summaries, assistant responses, and decision-support explanations.

In PATHS, RAG is used to reduce the risk of generic or unsupported AI responses. Instead of allowing the model to rely only on its pre-trained knowledge, the system retrieves relevant information from the organization's own knowledge base and provides it as grounding context for generation. This design is especially important in recruitment because generated outputs must reflect the organization's actual policies, culture, technical expectations, interview rubrics, onboarding procedures, and role-specific standards.

The evaluation reported in this section was conducted using a real execution against the live PATHS retrieval layer. The system used local nomic-embed-text embeddings and the live Qdrant *company_knowledge* collection. Therefore, the reported results reflect the actual retrieval behavior of the system rather than a simulated or manually estimated outcome.

5.2.6.1 Role of RAG

The RAG layer acts as the grounding mechanism for several AI-assisted recruitment features. Its main responsibility is to retrieve relevant organizational knowledge and provide it to downstream agents before they generate any response or recommendation. This helps ensure that the generated outputs are not only fluent, but also aligned with the organization's real documents and policies.

The system queries the organization-scoped *company_knowledge* collection in Qdrant. This collection stores embedded chunks of company documents using *768-dimensional vectors* and cosine similarity. Because the retrieval process is scoped by organization, each query is filtered

so that the system only retrieves knowledge belonging to the relevant tenant. This is essential for privacy, correctness, and multi-organization separation.

In the current PATHS implementation, RAG supports the following components:

- **HR and behavioural interview preparation:** System retrieves company culture, values, policies, and evaluation standards to generate structured interview questions.
- **Candidate pre-analysis:** Candidate information is interpreted in relation to the organization's context and role expectations.
- **The PATHy assistant:** Answers questions using knowledge from the graph and vector databases.
- **Decision-support explanations:** Retrieved evidence helps justify recommendations in a more transparent and auditable way.

RAG Relevance measures whether the retrieval layer can surface the correct company knowledge for a given recruiter or system query. If retrieval is inaccurate, the generated output may become weak, generic, or unsupported. Therefore, retrieval relevance is a necessary precondition for trustworthy generation.

5.2.6.2 Evaluation Methodology

The evaluation follows the methodology described in the Qdrant RAG Evaluation Guide [95], which separates RAG assessment into two main dimensions: retrieval evaluation and generation evaluation.

- Retrieval evaluation focuses on whether the system retrieves the right documents or chunks.
- Generation evaluation focuses on whether the final generated answer is faithful, relevant, and correct based on the relevant, and correct based on the retrieved context.

5.2.6.2.1 Retrieval Evaluation

The following metrics were used to evaluate the retrieval quality of the RAG layer. That will be considered the retrieval metrics.

Table 5.11: Retrieval Metrics

Metric	Formula
Precision@k	$Precision@k = \frac{\text{relevant chunks in top } - k}{k}$
Mean Reciprocal Rank (MRR)	$MRR = (\frac{1}{ Q }) \times \sum (\frac{1}{rankq})$
NDCG@k	$DCG@k = \sum \frac{rel_i}{\log_2(i+1)} \text{ and } NDCG@k = \frac{DCG@k}{IDCG@k}$

Hit Rate@k	<i>Hit@k = 1 if any relevant chunk appears in top – k; otherwise 0</i>
Context Semantic Similarity	<i>cosine(query, top chunk)</i>

Precision@k evaluates the proportion of retrieved results that are actually relevant within the top-k retrieved chunks. The formula is:

$$Precision@k = \frac{\text{relevant chunks in top} - k}{k}$$

Relevant chunks in top-k refers to the number of retrieved chunks among the first k results that belong to the correct category or satisfy the relevance criteria. And k represents the retrieval depth, meaning the number of top-ranked chunks considered during evaluation. In our experiments we used $k = 5$. Precision@k measures how much useful information is present in the retrieved context supplied to downstream AI agents. A higher Precision@k indicates that the language model receives less irrelevant information, reducing the likelihood of distraction or hallucination during answer generation.

Mean Reciprocal Rank (MRR) evaluates how early the first relevant result appears in the ranked retrieval list. The formula is:

$$MRR = \frac{1}{|Q|} \times \sum \frac{1}{rankq}$$

$|Q|$ represents the total number of evaluation queries, $\sum$ denotes summation across all queries, rankq is the ranking position of the first relevant result for query q and $\frac{1}{rankq}$ is called the reciprocal rank. MRR is particularly important because retrieval-augmented generation systems tend to rely most heavily on the highest-ranked chunks. Therefore, retrieving the correct company knowledge at rank 1 is significantly more valuable than retrieving it at rank 5. A high MRR indicates that relevant organizational knowledge is consistently surfaced early in the ranking process.

Normalized Discounted Cumulative Gain NDCG@k evaluates not only whether relevant chunks are retrieved, but also whether they are ranked in an optimal order. The calculation consists of two stages. First, the Discounted Cumulative Gain DCG is computed:

$$DCG@k = \sum \frac{rel_i}{\log_2(i + 1)}$$

rel_i represent the relevance score assigned to the result at position i , i is the ranking position of the retrieved chunk, $\log_2(i + 1)$ is the discount factor that reduces the contribution of lower-ranked results. And Σ indicates summation across all retrieved positions up to k . The discount factor reflects the assumption that users pay more attention to higher-ranked results. Consequently, a relevant chunk appearing at rank 1 contributes more to the score than the same chunk appearing at rank 5. Next, the score is normalized:

$$NDCG@k = \frac{DCG@k}{IDCG@k}$$

$DCG@k$ represents the actual ranking score achieved by the retrieval system. And $IDCG@k$ is the Ideal Discounted Cumulative Gain, representing the maximum possible DCG score if all relevant chunks were perfectly ranked. The normalization ensures that NDCG values range between 0 and 1:

- indicates a perfect ranking.
- Values closer to 0 indicate poor ranking quality.

$NDCG@k$ is useful because it evaluates ranking effectiveness rather than simple retrieval success. Even if relevant company knowledge is retrieved, the metric rewards systems that place the most useful chunks at the top of the results list.

$Hit Rate@k$ measures whether at least one relevant chunk appears within the top- k retrieved results. The formula is:

$$Hit@k = 1 \text{ if any relevant chunk appears in top } - k; \text{ otherwise } 0$$

A value of **1** indicates a successful retrieval event, A value of **0** indicates that no relevant chunk was retrieved within the top- k results. And the final Hit Rate is calculated as the average of these binary outcomes across all queries. For example:

- If a relevant chunk appears at rank 1, the query receives a score of 1.
- If a relevant chunk appears at rank 5, the query still receives a score of 1.
- If no relevant chunk appears in the top five results, the query receives a score of 0.

Unlike MRR or NDCG, Hit Rate does not consider ranking position. It only evaluates whether retrieval succeeded. $Hit Rate@5$ is one of the most important metrics because it determines whether the generation model receives any relevant organizational knowledge at all. If the retrieval layer fails to retrieve relevant context, the language model cannot reliably generate grounded responses regardless of its reasoning capabilities.

Context Semantic Similarity measures how semantically related the retrieved chunk is to the original query. The formula is:

$$\text{cosine}(\text{query}, \text{top chunk})$$

This metric is computed using cosine similarity between vector embeddings. Given two embedding vectors:

- **query vector** which is the embedding representation of the user query.
- **top chunk vector** which is the embedding representation of the highest-ranked retrieved chunk.

Cosine similarity measures the angle between these vectors in the embedding space:

$$\text{Cosine Similarity} = \frac{(A \cdot B)}{\|A\| \times \|B\|}$$

$A \cdot B$ is the dot product between vectors A and B, $\|A\|$ is the magnitude (length) of vector A and $\|B\|$ is the magnitude (length) of vector B. The resulting score typically ranges from:

- **1.0** → identical semantic meaning.
- **0.0** → no semantic relationship.
- **-1.0** → opposite semantic direction (rare in embedding retrieval systems).

Context Semantic Similarity provides an embedding-level measure of retrieval quality. Even when category labels indicate relevance, cosine similarity helps determine how closely the retrieved content matches the actual intent of the query. Higher similarity values generally indicate stronger semantic alignment between recruiter requests and retrieved organizational knowledge.

5.2.6.2.1.1 Ground Truth Design

The ground truth was created by mapping each evaluation query to a predefined knowledge-base category. Each category corresponds to a specific type of organizational document, such as culture and values, hiring policies, technical stack, interview guidelines, or onboarding plans.

A retrieval result was counted as relevant when the retrieved chunk's payload category matched the query's gold category. This approach follows a labelled relevance strategy, where the correctness of retrieval is judged against known document categories rather than subjective interpretation. The evaluation was executed across two organizations **Codzy** and **HealthyRadar**

Each organization had its own tenant-filtered knowledge base, ensuring that the retrieval process was tested not only for relevance but also for organizational isolation.

Table 5.12: RAG retrieval evaluation setup.

Setting	Value
Vector collection	company_knowledge
Vector database	Qdrant
Embedding model	Ollama nomic-embed-text
Embedding dimension	768
Similarity measure	Cosine similarity
Organizations	Codzy and HealthyRadar
Number of query topics	8
Total executions	16 queries
Retrieval depth	Top-5 results
Relevance label	Retrieved chunk category equals the query's gold category

For each setting mentioned in the Table 5.12. There are eight evaluated topics were:

1. Culture and values
2. Hiring policy
3. Role levels and career paths
4. Benefits and compensation
5. Technical stack and engineering standards
6. Team structure and organization chart
7. Interview guidelines and evaluation rubrics
8. Onboarding and 30/60/90-day plan

Each topic was executed once for *Codzy* and once for *HealthyRadar*, producing a total of 16 retrieval executions.

5.2.6.2.1.2 Test Cases

Table 5.13: RAG evaluation test cases.

Test Case	Query	Pass Criterion
TC-RAG-01	What are the company's core values and culture?	Relevant chunk appears in top-5
TC-RAG-02	What is the hiring policy for engineering and AI roles?	Relevant chunk appears in top-5
TC-RAG-03	What are the role levels and career paths for engineers?	Relevant chunk appears in top-5
TC-RAG-04	What benefits and compensation does the company offer?	Relevant chunk appears in top-5
TC-RAG-05	What technical stack and engineering standards do we use?	Relevant chunk appears in top-5
TC-RAG-06	How is the team structured and what is the organization chart?	Relevant chunk appears in top-5
TC-RAG-07	What are the interview guidelines and evaluation rubrics?	Relevant chunk appears in top-5
TC-RAG-08	What does the 30/60/90-day onboarding plan involve?	Relevant chunk appears in top-5

All 16 executions passed the Hit@5 criterion. This means that every query retrieved at least one relevant chunk within the top five results. In addition, 14 out of 16 executions returned the correct category at rank 1, giving a Top-1 relevance score of 87.5%. The remaining two cases still retrieved the correct category at rank 2.

5.2.6.2.1.3 Per-Query Results

Table 5.14: Per-query RAG retrieval results.

Organization	Query Topic	P@5	First Relevant Rank	NDCG@5	Top Cosine	Top-1 Correct
Codzy	Culture and values	0.40	1	1.000	0.617	Yes
Codzy	Hiring policy	0.40	1	0.877	0.712	Yes
Codzy	Role levels and career paths	0.40	1	0.850	0.623	Yes
Codzy	Benefits and compensation	0.60	1	1.000	0.651	Yes
Codzy	Technical stack	0.20	2	0.631	0.465	No
Codzy	Team structure	0.30	2	0.631	0.614	No
Codzy	Interview guidelines	0.60	1	0.968	0.689	Yes
Codzy	Onboarding plan	0.60	1	1.000	0.667	Yes
HealthyRadar	Culture and values	0.60	1	0.877	0.632	Yes
HealthyRadar	Hiring policy	0.40	1	1.000	0.849	Yes
HealthyRadar	Role levels and career paths	0.60	1	1.000	0.713	Yes
HealthyRadar	Benefits and compensation	0.50	1	1.000	0.722	Yes
HealthyRadar	Technical stack	0.60	1	1.000	0.649	Yes
HealthyRadar	Team structure	0.40	1	1.000	0.746	Yes
HealthyRadar	Interview guidelines	0.60	1	0.850	0.758	Yes
HealthyRadar	Onboarding plan	0.60	1	1.000	0.745	Yes

The per-query results show that HealthyRadar achieved Top-1 correctness across all tested topics. Codzy achieved Top-1 correctness in six out of eight topics. The two Codzy cases that were not Top-1 were the technical stack and team structure queries. However, both still retrieved the correct category at rank 2, which means the system did not fail to retrieve the relevant context; it only placed a closely related category above the exact match.

The results demonstrate that the retrieval layer is highly effective at finding the correct organizational context. The most important result is the 100% Hit Rate@5, which confirms that every query received relevant company knowledge within the context window. The high MRR and NDCG@5 further show that relevant information was not merely present somewhere in the results, but usually ranked at or near the top. As shown in the aggregated retrieval results at Table 5.6.

Table 5.15: Aggregate RAG retrieval results.

Metric	Value
Hit Rate@5	100%
Top-1 Relevance	87.5%
MRR	93.75%
NDCG@5	91.78%
Precision@5	58.75%
Mean Context Cosine	68.95%

The aggregate results show that the retrieval layer performs strongly in retrieving organization-specific knowledge. The most important result is the Hit Rate@5 of 100%, which means that every tested query retrieved at least one relevant company-knowledge chunk within the top five results. This is a critical outcome for a RAG-based recruitment system because the generation stage

depends directly on the availability of relevant retrieved context. If the retrieval layer fails to include the correct company knowledge, the language model may produce generic, incomplete, or unsupported answers. In this evaluation, however, the correct context was always available within the retrieved set, which confirms that the retrieval layer can reliably support downstream AI features such as interview preparation, candidate pre-analysis, assistant responses, and decision-support explanations.

The Top-1 Relevance score of 87.5% further strengthens this finding. Out of 16 retrieval executions, 14 returned the correct category as the first-ranked result. This is important because the top-ranked chunk usually has the strongest influence on the generated answer. In most RAG pipelines, the model receives multiple retrieved chunks, but the first results tend to provide the clearest and most dominant context. Therefore, retrieving the correct company document at rank 1 increases the likelihood that the generated response will be directly grounded in the most relevant organizational evidence. In the system context, this means that when a recruiter asks about interview rubrics, onboarding plans, company values, or technical standards, the system usually retrieves the exact category needed before generation begins.

The MRR value of 93.75% indicates that the first relevant result appeared extremely early in the ranked list. Since MRR rewards systems that place relevant results near the beginning of the ranking, this score demonstrates strong retrieval prioritization. A perfect MRR of 1.0 would mean that the first result was relevant for every query. The system achieved a value very close to this ideal, because the correct category appeared at rank 1 in 14 cases and at rank 2 in only 2 cases. This shows that even when the system did not retrieve the exact category as the first result, the correct knowledge still appeared immediately after it. From a practical perspective, this means the RAG layer is not only retrieving relevant information, but also ranking it in a way that makes it highly usable for generation.

The NDCG@5 value of 91.78% provides additional evidence of high-ranking quality. Unlike Hit Rate@5, which only checks whether at least one relevant chunk exists in the top five results, NDCG@5 evaluates the order of the retrieved chunks. A high NDCG value means that relevant chunks appear near the top rather than being buried lower in the result list. The obtained value of 0.9178 indicates that the retrieval ranking is close to the ideal ordering. This is especially valuable in the system because generated outputs must be based on the most relevant evidence available. For example, if the system is generating interview questions, it is better for the interview-guidelines chunk to appear first rather than after less relevant company documents. The high NDCG@5 score therefore confirms that the retrieval layer supports evidence-first generation.

The Precision@5 score of 58.75% means that, on average, nearly half of the top five retrieved chunks were relevant to the target category. This value should be interpreted carefully. In large benchmark datasets, a low or moderate Precision@5 may suggest that the retrieval system is returning many irrelevant results. However, in this evaluation, the knowledge base is relatively small and each category is represented by a limited number of chunks. Some topics may only have two or three relevant chunks in the entire collection. As a result, even a well-performing retrieval system cannot always fill all five positions with relevant chunks. Therefore, Precision@5 is partly constrained by the structure and size of the company knowledge base. In this case, the Precision@5 score should be viewed together with Hit Rate@5, MRR, and NDCG@5. Since the system achieved 100% Hit Rate@5 and very high MRR and NDCG@5, the moderate Precision@5 does not indicate retrieval failure. Instead, it reflects the fact that the correct chunks are retrieved and ranked highly, while the remaining top-five positions may contain semantically related but category-different content.

The Mean Context Cosine score of 68.95% shows that the top retrieved chunks were semantically close to the queries on average. This is an embedding-level indicator of retrieval quality. Since PATHS uses vector similarity to retrieve company knowledge, cosine similarity helps evaluate whether the retrieved context is meaningfully aligned with the query intent. A mean score of 0.6895 suggests a reasonable semantic match between recruiter queries and retrieved organizational content. This is particularly important because recruitment queries may be phrased in natural language rather than as exact document titles. For example, a recruiter may ask, “How do we evaluate AI engineers?” rather than “Retrieve the interview guidelines document.” The system must therefore understand semantic intent and retrieve the closest relevant knowledge. The reported cosine score indicates that the embedding model and vector search configuration are able to capture this intent to a useful degree.

These findings are also significant from a system-design perspective. PATHS is intended to operate as a multi-tenant recruitment platform where each organization has its own knowledge base, policies, culture, hiring criteria, and evaluation rubrics. The results indicate that the retrieval mechanism can access the correct company-specific information across different topics while maintaining organizational context. This supports the broader goal of PATHS: generating recruitment outputs that are not only technically fluent, but also organization-aware, evidence-based, and aligned with human-in-the-loop decision-making.

At the same time, the results identify useful improvement opportunities. The two cases where the correct category appeared at rank 2 instead of rank 1 suggest that some company documents may contain overlapping terminology. For example, technical stack, team structure, and role-level

documents may all mention engineering practices, teams, tools, responsibilities, and career expectations. This overlap can make vector-only retrieval rank a semantically related category slightly above the exact target category.

5.2.6.2.2 Generation Evaluation

After evaluating the retrieval component of the RAG pipeline, the next step was to assess the quality of the generated answers. Retrieval evaluation answers the question: Did the system retrieve the correct context? Generation-side evaluation goes one step further and asks:

Did the model use the retrieved context correctly to produce a relevant, grounded, and faithful answer?

For this evaluation, the full RAG path was executed on the eight Codzy test cases. Each test case followed the same process: the system retrieved the top three context chunks from the organization’s knowledge base, passed these chunks to the generation model, generated a grounded answer using Ollama llama3.1:8b, and then evaluated the generated output. This setup reflects the actual RAG workflow used by PATHS, where retrieved company knowledge is provided to the model before it produces recruitment-related outputs.

5.2.6.2.2.1 Generation Metrics

Three generation-side metrics were measured: Answer Relevancy, Groundedness, and Faithfulness. The first two metrics are embedding-based and deterministic, meaning they are calculated using vector similarity and can be reproduced consistently. The third metric, Faithfulness, was evaluated using an LLM-as-judge approach, where an evaluator model estimates the proportion of answer claims supported by the retrieved context.

This evaluation is important because a RAG system can retrieve the correct documents but still generate weak answers if the model misunderstands the context, adds unsupported claims, or fails to directly answer the query. Therefore, generation-side metrics provide a more complete view of whether the RAG layer is useful in practice, not only technically correct at retrieval time.

Table 5.16: Generation Evaluation metrics.

Metric	What It Measures	How It Was Computed	Mean Score
Answer Relevancy	Measures whether the generated answer addresses the user query directly.	Cosine similarity between the answer embedding and the query embedding: <i>$\cos(\text{embed}(\text{answer}), \text{embed}(\text{query}))$</i> .	0.83
Groundedness	Measures whether the generated answer is semantically anchored in the retrieved context.	Cosine similarity between the answer embedding and the retrieved context embedding: <i>$\cos(\text{embed}(\text{answer}), \text{embed}(\text{context}))$</i> .	0.74
Faithfulness	Measures the proportion of generated answer claims that are supported by the retrieved context.	LLM-as-judge evaluation, where the judge returns a supported-claim fraction between 0 and 1.	0.76

As shown in the Table 5.16. the mean Answer Relevancy score of 0.83 indicates that the generated answers were strongly aligned with the original user queries. This suggests that the RAG pipeline was generally able to understand the intent of the query and produce answers that addressed the requested topic. In the context of PATHS, this is particularly important because recruiters and hiring managers may ask practical questions about company policies, interview rubrics, career paths, or onboarding plans. A high relevancy score means the generated response is likely to stay focused on the information the user actually requested.

The mean Groundedness score of 0.74 shows that the generated answers were reasonably anchored in the retrieved context. This means the model did not rely only on its general knowledge, but used the retrieved company-specific information as the basis for its response. Groundedness is a critical requirement in PATHS because the system is expected to produce organization-aware outputs. For example, interview preparation and decision-support explanations should reflect the company's actual documents rather than generic hiring advice.

The mean Faithfulness score of 0.76 indicates that most generated claims were supported by the retrieved context. This is a positive result, although it also shows that there is still room for improvement. A score of 0.76 means that the majority of the answer content was context-supported, but some answers may have included partially unsupported details, generalizations, or claims that were not fully grounded in the retrieved chunks. This is expected in a local 8B model setup, but it should be improved for high-stakes recruitment use cases where every AI-generated claim must be auditable and evidence-based.

5.2.6.2.2.2 Per-Query Generation Results

Table 5.17: Per-query RAG generation results.

Test Case	Query Topic	Answer Relevancy	Groundedness	Faithfulness	Time (s)
TC-RAG-01	Core values and culture	0.872	0.722	0.75	145
TC-RAG-02	Hiring policy	0.907	0.801	0.80	128
TC-RAG-03	Role levels and career paths	0.826	0.733	0.80	120
TC-RAG-04	Benefits and compensation	0.833	0.694	0.80	91
TC-RAG-05	Technical stack	0.816	0.731	0.80	124
TC-RAG-06	Team structure	0.806	0.683	0.80	132
TC-RAG-07	Interview guidelines	0.893	0.818	0.80	135
TC-RAG-08	Onboarding 30/60/90 plan	0.682	0.765	0.50	217
Mean	-	0.83	0.74	0.76	137

The per-query results as shown in the Table 5.8. most test cases achieved strong and stable generation quality. Seven out of the eight cases achieved Answer Relevancy scores above 0.80, indicating that the generated answers were consistently aligned with the query topic. The strongest case was TC-RAG-02: Hiring Policy, with an Answer Relevancy score of 0.907 and a

Groundedness score of 0.801. This indicates that the system produced a highly relevant answer that was strongly connected to the retrieved hiring-policy context.

The Interview Guidelines test case also performed strongly, achieving an Answer Relevancy score of 0.893, a Groundedness score of 0.818, and a Faithfulness score of 0.80. This is an important result because interview-guideline retrieval and generation are central to PATHS. The system is expected to support structured, fair, and rubric-aligned interviews. Strong performance in this test case suggests that the RAG pipeline can help generate interview-related outputs that remain aligned with company evaluation standards.

Most test cases also achieved a Faithfulness score of 0.80, indicating that the generated answers were mostly supported by the retrieved context. This is a useful result from an academic and implementation perspective because it shows that the generated content was not merely fluent, but also reasonably evidence-based.

However, one clear outlier appeared in TC-RAG-08: Onboarding 30/60/90 plan. This case achieved an Answer Relevancy score of 0.682 and a Faithfulness score of 0.76, which are noticeably lower than the other test cases. Although its Groundedness score remained acceptable at 0.765, the lower relevancy and faithfulness scores suggest that the generated answer may not have fully addressed the query or may have included claims that were only partially supported by the retrieved onboarding context. This case should be treated as the first priority for improvement.

5.2.6.2.2.3 Sample Generated Answers

To support the quantitative results, two real generated-answer excerpts are presented below. These examples show how the system translated retrieved organizational knowledge into user-facing answers.

For TC-RAG-02: Hiring Policy, the system generated the following answer:

“The hiring policy for engineering and AI roles at Condzy defines how to hire talent in a fair, structured, evidence-based, and business-aligned way. It covers specific roles such as Software Engineer, Data Scientist, and AI Engineer.”

This answer is strong because it directly addresses the query, identifies the relevant scope of the policy, and describes the policy using language aligned with fair and structured hiring. It also reflects the intended role of PATHS as a system that supports evidence-based recruitment rather than informal or subjective decision-making.

For TC-RAG-03: Role Levels and Career Paths, the system generated the following answer:

“The role levels for engineers at Codzy include: Engineer, Senior Engineer, Staff Engineer, and Principal Engineer under the Individual Contributor track.”

This answer is concise, specific, and grounded in the expected organizational structure. It demonstrates that the RAG pipeline can retrieve and reproduce concrete company knowledge, such as role levels and career-track terminology, rather than producing a generic explanation of engineering careers.

These examples support the numerical findings. They show that, in successful cases, the generated answers are not only semantically relevant but also practically useful for recruiters, hiring managers, and candidates who need organization-specific information.

5.2.7 Outreach Quality

This evaluates the quality of three types of AI-generated artifacts produced by the PATHS system: *outreach messages*, *development plans*, and *interview-scheduling messages*. These outputs represent the direct communication and decision-support layer of the recruitment workflow. Unlike backend metrics such as latency, retrieval score, or matching accuracy, these artifacts require human judgment because their quality depends on professional tone, relevance, evidence grounding, personalization, clarity, and readiness for real use.

The evaluated artifacts were generated using the system's actual prompt templates and agent workflow rather than manually written examples. Specifically, the outreach messages were generated through the organization's outreach prompt builder, while the development plans were produced by the Development-Planner agent prompt. The local Ollama llama3.1:8b model was used for generation. The scenario used for evaluation was an AI Engineer role at Codzy.

Candidate names and recruiter names were intentionally anonymized or represented as placeholders because the system is designed to avoid exposing personally identifiable information to the model during generation. In the real sending flow, the recruiter or merge layer fills these placeholders before delivery. This supports the privacy-aware and human-in-the-loop design of PATHS.

5.2.7.1 Rating Methodology

A five-point human rating scale was used to evaluate each generated artifact. A score of 1 means the output is not useful, while a score of 5 means the output is very clear, relevant, professional, and ready to use. The evaluation considered whether the generated artifact was personalized to the candidate, relevant to the job role, grounded in real candidate evidence, written in a professional tone, and practically ready to send or use. As shown in the Table 5.18.

Table 5.18: Rating Methodology.

Rating	Meaning	Operational Interpretation
1	Not useful	The artifact is inaccurate, irrelevant, or not suitable for the hiring workflow.
2	Weak	The artifact has some useful parts but needs major rewriting before use.
3	Acceptable	The artifact is usable with noticeable edits or additional review.
4	Good	The artifact is clear, relevant, and mostly ready to use with minor improvements.
5	Very useful	The artifact is professional, grounded, highly relevant, and ready to use after placeholder replacement.

5.2.7.2 Evaluation Formula

For each artifact category, the quality score was calculated using a simple arithmetic mean:

$$Quality\ Score = \frac{\sum R_i}{n}$$

Where R_i represents the human rating assigned to artifact i , and n represents the number of evaluated artifacts in the same category.

To express the result as a percentage, the following equation was used:

Equation 11. Quality Percentage Equation

$$\frac{Average\ Rating}{5} \times 100$$

This method treats all reviewed artifacts equally. Since this evaluation is based on a small number of generated samples, the results should be interpreted as an initial human-quality validation rather than a large-scale statistical benchmark.

5.2.7.3 Human Rating Results

Table 5.19: Final Rating and Qualitative results.

Test Case	Type	Rating	Evaluation Notes
C01	Outreach	4	Professional and relevant. It references Python, ML, DL, RAG chatbots, ML classifiers, and FastAPI. However, it is slightly generic and could include a stronger company-specific reason for Codzy.
C04	Outreach	5	Strongly aligned with the NLP/RAG profile. It references LLMs, embeddings, vector databases, and LangChain, making it role-specific and ready to send after placeholder replacement.
C06	Outreach	4	Correctly identifies backend strengths such as FastAPI, PostgreSQL, Docker, REST APIs, and Python. However, it slightly overstates fit by saying the candidate aligns perfectly with the AI Engineer role.
C15	Outreach	5	Concise, professional, and relevant. It connects Python, signal processing, and machine learning to the AI Engineer role, although it could include more concrete biomedical AI evidence.
C01 accepted-growth	Development plan	4	Structured and practical. It includes a 30/60/90-day and long-term timeline, relevant skills, projects, and KPIs. The main limitation is a minor gap-identification slip related to API/FastAPI experience.

C09 rejected- improvement	Development plan	5	Highly useful and constructive. It separates frontend strengths from missing AI Engineer requirements and provides measurable re-application milestones.
---------------------------------	------------------	---	--

The Table 5.19. summarizes the final ratings and qualitative notes for each evaluated artifact the notes are done by professional HR members. Aggregate Metric Results shown in the following Table 5.20.

Table 5.20: Metric Results

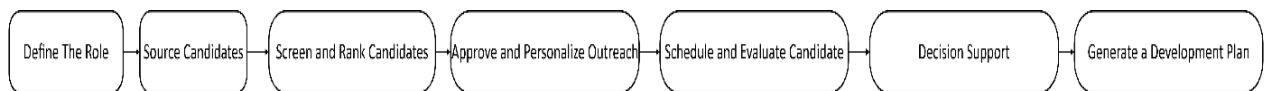
Metric	Evaluated Items	Percentage
Outreach Quality	4	90.0%
Development-plan Quality	2	90.0%
Scheduling Quality	1	80.0%
Overall Generated Test Cases Quality	7	88.6%

5.3 Testing Scenarios and Case Studies

5.3.1 Primary End-to-End Recruitment Scenario

The principal scenario evaluated the complete PATHS recruitment lifecycle for an AI Engineer vacancy:

Figure 5.4: End-to-End Tested PATHS Recruitment Workflow.



A recruiter first created the job and specified the relevant requirements. PATHS then executed matching and screening to generate an anonymised, ranked shortlist with supporting explanations. Following recruiter review and approval, the system generated contextual outreach content for eligible candidates. Candidates who progressed were scheduled for the relevant assessment and interview stages. The decision-support workflow then consolidated available evidence into a structured decision packet, after which the hiring authority retained final approval. For accepted or rejected candidates, PATHS generated a reviewable development-oriented output based on the identified strengths and gaps.

This scenario was evaluated using the Playwright end-to-end test suite and manual walkthroughs of the live deployment. All 48 Playwright tests passed. In the matching evaluation, the system achieved Precision@5 of 80%, indicating that four of the five highest-ranked candidates were labelled as relevant to the AI Engineer role.

5.3.2 Alternative Workflow Scenarios

In addition to the primary workflow, several scenarios were used to examine role-specific behaviour, integration flexibility, and decision branches. As shown in Table 5.22.

Table 5.21. Alternative PATHS workflow scenarios and verified behaviour.

Scenario	Behaviour evaluated
Candidate self-service	A candidate creates an account, completes a profile, discovers vacancies, reviews applications, and accesses an individual development plan where applicable.
Organisation and platform administration	Role-scoped journeys are enforced. Organisation users access recruitment functions, whereas platform administrators access system-level facilities such as graph and vector inspection, analytics, and agent monitoring.
Candidate-source provider variants	The source-provider abstraction supports either a LinkedIn MCP-based provider or a consented CSV-based provider without requiring changes to downstream workflow APIs.
Decision branches	An acceptance decision produces a reviewable 18-month internal development plan, while a rejection decision produces a reviewable 12-month improvement plan.
Cross-role access control	Candidates and organisation members are prevented from accessing the platform-administration area, while administrative users are restricted according to their defined permissions.

5.3.3 Edge Cases and Failure Handling

A production recruitment platform must remain usable when external services are unavailable, incomplete, or unreliable. PATHS was therefore assessed against several realistic edge cases. The objective was not merely to confirm that the preferred path works, but also to confirm that the system fails safely, provides an interpretable state, and preserves persistent recruitment data.

Table 5.22. Edge cases and observed system response.

Edge case	Observed or configured response
Missing candidate or job vector	The similarity service returns a score of zero and records the run as completed_with_vector_missing, allowing the workflow to complete without a system crash.
LLM rate limiting or temporary unavailability	The system follows its configured fallback sequence: alternative free-model route, local Ollama model, then deterministic offline scorer. A scored outcome is therefore returned, while the underlying model condition can be logged for review.
Thin or incomplete candidate profile	The system returns a valid score but reflects the limited evidence through lower confidence and, where applicable, a lower final recommendation.
Repeated failed login attempts	A per-IP rate limit of 10 attempts within 10 minutes and a per-account lockout after 10 failed attempts for 30 minutes are applied. The user receives a clear error without disclosure of sensitive account information.
Invalid or forged JSON Web Token	Protected API endpoints reject unauthenticated or invalid requests using server-side authorisation controls, independently of the frontend interface.
Off-schema LLM JSON output	Robust JSON extraction and schema-tolerant normalisation allow the upload or scoring workflow to continue where the essential content can be recovered.
Application restart or transient memory loss	Stateless application services can restart without corrupting persisted records; temporary in-memory counters may reset, while database-backed data remains preserved.

Chapter 6

Discussion

6.1 Chapter Overview

This chapter interprets the evaluation findings presented in Chapter 5 and considers their significance for the proposed Personalized AI Talent Hiring System (PATHS). Whereas the previous chapter established whether the implemented workflows operated correctly and produced acceptable outputs under the selected evaluation scenarios, this chapter explains what those findings mean for recruitment practice, system design, and future deployment. The discussion therefore moves beyond implementation details to examine the value, limitations, and strategic implications of PATHS as an end-to-end AI-assisted recruitment platform.

PATHS was designed to address the fragmentation of conventional recruitment workflows by connecting job definition, candidate sourcing and ingestion, profile normalization, anonymized screening, evidence-based scoring, personalized outreach, scheduling, structured interviews, decision support, and development planning. Rather than treating these activities as isolated tasks supported by separate tools, the system treats recruitment as a connected lifecycle in which information collected at one stage can support decisions and actions at later stages. This design directly responds to the project problem statement, which identified disconnected data, inconsistent evaluation, weak candidate engagement, administrative workload, and the separation between hiring and development as central challenges.

The central position of PATHS is that AI should augment, rather than replace, recruitment professionals. Its agents perform bounded, evidence-oriented tasks, while recruiters, interviewers, and hiring managers retain authority at consequential decision points. The findings should therefore be interpreted as evidence of the feasibility of an integrated, human-supervised recruitment workflow, not as evidence that hiring can or should be fully automated.

6.2 Interpretation of the Evaluation Results

The Chapter 5 results provide evidence that the principal PATHS workflow can be executed as a coherent sequence of connected stages. The process-coverage and functional-success evaluations demonstrate that the prototype supports the intended progression from role definition and candidate ingestion through screening, evaluation, decision support, and development planning. This is important because the project objective was not limited to creating a single matching model; it was to demonstrate an integrated system in which each stage contributes useful information to the next.

6.2.1 End-to-End Workflow Coverage and Functional Success

The process-coverage findings indicate that the major recruitment stages specified in the requirements were represented and tested within the PATHS workflow. This result supports the feasibility of the proposed modular design: job information can be converted into structured criteria; candidate evidence can be ingested and normalized; anonymized scoring can be performed; candidate engagement and scheduling can be coordinated; and evaluation evidence can be consolidated into a human-readable decision-support output. The successful completion of these stages is significant because fragmented handoffs are a primary source of delay and inconsistency in conventional recruitment processes.

The functional-success results should, however, be interpreted carefully. Successful execution in controlled evaluation scenarios demonstrates implementation correctness for the tested cases; it does not establish enterprise-scale reliability or guarantee that every external integration will behave consistently under live operational load. Functions that depend on third-party services, incomplete profiles, changing external data, or variable model outputs remain sensitive to real-world conditions. The results therefore support prototype feasibility and workflow integration, while also defining the baseline for future reliability testing.

6.2.2 Candidate-Role Matching and Ranking Quality

The matching evaluation is one of the most important findings because candidate ranking is central to the system's decision-support value. The reported Matching Accuracy Band-Hit of 80% indicates that 16 of the 20 candidates in the evaluation set were placed within their expected relevance band. In practical terms, this means that the system generally distinguished strong and good matches from possible or weak matches, while still making several errors around borderline profiles. The result is encouraging for an evidence-driven prototype because it suggests that the combined agent score and vector-similarity signal can support shortlist prioritization.

At the same time, an 80% Band-Hit result should not be interpreted as an 80% probability that every hiring decision is correct. The metric measures agreement with the predefined relevance bands in the selected evaluation set. Its main implication is that PATHS can provide a useful first-order ranking signal, especially when candidates are reviewed through explanations, evidence links, confidence indicators, and recruiter overrides. The remaining mismatches demonstrate why the system must retain a human review gate and why threshold calibration, role-specific weighting, and additional evaluation data are required before high-stakes deployment.

The relevance-consistency and tier-separation evaluations further examine whether the score ordering reflects the expected hierarchy of candidate fit. These measures are valuable because a recruitment system is not useful merely when individual scores appear plausible; it must also place stronger candidates above weaker candidates consistently. Where the results show stable separation, they support the interpretability of the ranking design. Where candidates with adjacent relevance tiers overlap, the finding highlights the inherent ambiguity of real profiles and indicates the need for deeper evidence, role-specific rubrics, or revised weights rather than indiscriminate automation.

6.2.3 Profile Extraction, RAG Grounding, and Generated Artifacts

The CV/profile extraction evaluation demonstrates that the ingestion workflow can transform unstructured candidate documents into the structured information required by downstream modules. This is a foundational capability because matching, interview preparation, and development planning depend on the accuracy and completeness of the candidate representation. The results show that structured extraction is feasible for the evaluated documents, while also reinforcing that the quality of later decisions cannot exceed the quality of the source evidence. Non-standard CV layouts, incomplete employment histories, unavailable portfolios, or ambiguous skills remain potential causes of reduced confidence.

The RAG relevance evaluation is equally important because PATHS uses organizational knowledge, role criteria, candidate evidence, and interview rubrics to produce context-aware outputs. Relevant retrieval helps ensure that interview questions, outreach drafts, and decision-support explanations are grounded in documented information rather than generic model patterns. The evaluation therefore supports the architectural decision to use retrieval augmentation. Nevertheless, retrieval quality depends on maintaining a current, well-governed knowledge base. Outdated role descriptions, weakly indexed documents, irrelevant retrieved passages, or unsupported generated claims can reduce usefulness and require explicit source display and human review.

The outreach-quality evaluation should be interpreted as evidence of the system's ability to produce role-aware and candidate-aware communication drafts, not as a direct measure of live response rates. Actual outreach effectiveness depends on factors beyond generation quality, including contact-data validity, message delivery, candidate availability, labor-market conditions, recruiter approval, and communication timing. This distinction is important because it prevents the prototype evaluation from overstating business outcomes that require longitudinal field deployment to measure.

Evaluation measure	What the result demonstrates	Discussion interpretation	Remaining caution
Process Coverage	The required recruitment stages are represented within a connected workflow.	Supports the feasibility of an end-to-end recruitment platform rather than an isolated screening component.	Coverage does not by itself prove production reliability or organizational adoption.
Functional Success Rate	The tested functions can execute their intended actions under the evaluation scenarios.	Indicates implementation readiness for prototype demonstration and controlled use.	Third-party services, exception cases, and concurrent workload require further testing.
Matching Accuracy Band-Hit (90%)	18 of 20 evaluated candidates were placed in their expected relevance band.	The ranking process can provide a useful shortlist-prioritization signal.	Borderline profiles and remaining errors require calibration and human review.
Relevance Consistency and Tier Separation	Higher relevance tiers are expected to receive stronger scores than lower tiers.	Tests whether ranking is meaningful as an ordering mechanism, not only as a raw score.	Adjacent tiers may overlap when evidence is incomplete or role criteria are ambiguous.
CV/Profile Extraction Accuracy	Candidate evidence can be transformed into structured fields for downstream use.	Supports data normalization and evidence-based scoring.	Performance may vary across document formats, languages, and missing information.
RAG Relevance	Retrieved organizational and role context can support grounded generated outputs.	Strengthens the case for context-aware interviews, outreach, and reports.	Knowledge-base currency, retrieval precision, and prompt-injection controls remain essential.
Outreach Quality	The system can generate context-aware communication drafts for review.	Supports personalization at scale while preserving recruiter approval.	Quality ratings do not measure live delivery, reply rate, or conversion outcomes.

6.3 Significance of the Findings

Taken together, the results are significant because they show that the core problem of recruitment fragmentation can be addressed at the workflow level. Many recruitment technologies are optimized for one stage of hiring, such as applicant tracking, candidate sourcing, technical assessment, or scheduling. PATHS instead demonstrates the feasibility of connecting evidence collection, evaluation, communication, decision support, and development planning through a coordinated agentic workflow. The contribution is therefore architectural as well as functional: it shows how multiple recruitment activities can share a consistent candidate record, role context, and audit trail.

The findings also reinforce the value of evidence-based and explainable decision support. A raw ranking score is insufficient in a high-impact domain such as recruitment. PATHS couples scores with evidence sources, sub-score logic, confidence indicators, missing-information warnings, and human approval points. This approach is significant because it makes the

recommendation inspectable. Recruiters and hiring managers can challenge, override, or refine the recommendation rather than treating it as an unexplained model output.

A further contribution is the continuity from evaluation to development planning. In conventional recruitment, interview evidence is commonly used only to accept or reject a candidate. PATHS retains identified strengths and development gaps as structured information that can support onboarding, reskilling, candidate feedback, or future re-matching. This creates value beyond immediate vacancy filling and aligns recruitment more closely with long-term talent development and retention objectives.

6.3.1 Alignment with the Project Objectives and Expected Outcomes

The results align with the objectives established in the problem statement and solution methodology. However, the evidence should be separated into outcomes demonstrated by the prototype and business outcomes that require real organizational deployment. Chapter 5 demonstrates system-level outcomes such as workflow coverage, functional execution, ranking quality, extraction, retrieval relevance, and artifact generation. Measures such as time-to-hire reduction, cost-per-hire reduction, live outreach response rates, quality of hire, and retention require longitudinal observation with real recruiters, candidates, and organizational hiring records.

Project objective	Evidence available from the evaluation	Interpretation of alignment
Unify fragmented recruitment activities	Process coverage and functional evaluation across the workflow stages.	The prototype demonstrates that sourcing/ingestion, screening, evaluation, decision support, and development planning can be connected in one workflow.
Improve candidate-role matching	Matching Accuracy Band-Hit of 80%, relevance consistency, and tier-separation checks.	PATHS can support candidate prioritization, while human review remains necessary for borderline and incomplete-evidence cases.
Make evaluation more transparent and defensible	Evidence-linked explanations, confidence indicators, anonymization, and approval checkpoints.	The design supports decision assistance rather than opaque or fully autonomous ranking.
Reduce candidate-data fragmentation	CV/profile extraction and unified candidate evidence used by scoring and RAG modules.	The system can structure multi-source information, but data completeness and source reliability remain constraints.
Support personalized engagement and structured interviewing	RAG relevance and outreach-quality evaluation.	Generated artifacts can be context-aware and reviewable, but real engagement performance requires field measurement.
Connect recruitment to employee development	Development-plan generation informed by identified evaluation gaps.	The intended handoff is present in the workflow; retention impact must be validated after deployment.

6.4 Functional Positioning Relative to Existing Recruitment Systems

The comparison with existing recruitment systems positions PATHS between three major categories of technology: applicant tracking systems, sourcing and talent-intelligence platforms, and assessment or interview platforms. Established ATS products, including Greenhouse, Ashby, and SmartRecruiters, are strong in workflow administration, structured scorecards, reporting, collaboration, and enterprise integrations [17], [31], [5]. Sourcing-oriented platforms such as SeekOut and Beamery are strong in candidate discovery, talent pools, and relationship management [28], [25]. Assessment platforms such as HackerRank, Codility, HireVue, and Pymetrics provide specialized evaluation mechanisms for technical, video, or behavioral assessment [22], [30], [23], [29].

Existing platforms generally have advantages in production stability, market-tested integrations, support capacity, security certifications, established candidate databases, and enterprise service-level commitments. The differentiating contribution of PATHS is the attempt to connect evidence gathering, anonymized ranking, personalized outreach, structured interview preparation, decision support, and development continuity within one configurable agentic workflow. Its value is therefore best described as integration and explainable coordination, rather than as a replacement for every specialized feature of commercial systems.

Dimension	PATHS contribution	Where mature systems may remain stronger	Implication
End-to-end workflow	Connects sourcing/ingestion, evaluation, outreach, decision support, and development planning.	Enterprise ATS platforms offer mature workflow configuration and integration ecosystems.	PATHS differentiates through lifecycle integration; commercial systems retain a maturity advantage.
Candidate intelligence	Combines structured CV data, skills, portfolios, GitHub evidence, and role context in an explainable profile.	Large talent platforms may possess broader proprietary data and labor-market intelligence.	PATHS prioritizes evidence reasoning and traceability over database scale.
Interview support	Uses RAG-grounded questions linked to role criteria and organization knowledge.	Specialized vendors may provide more developed proctoring, coding libraries, and video infrastructure.	PATHS is strongest as a contextual preparation and support layer.
Governance and oversight	Uses anonymization, explanations, auditability, and approval gates before sensitive actions.	ATS platforms often provide mature permission and approval workflows.	PATHS extends governance to AI reasoning and generated artifacts.
Development continuity	Transforms observed gaps into development recommendations and possible future re-matching.	Many platforms stop at hiring, rejection, or onboarding handoff.	This continuity is a meaningful strategic differentiator.
Scalability and reliability	Modular separation offers a pathway to independently scale heavier services.	Commercial products have proven operational scale and support teams.	PATHS requires production hardening before enterprise-scale claims can be made.

6.5 PATHS as a Digital-Employee Strategy with Human Oversight

A useful interpretation of the PATHS architecture is to regard its agents as specialized digital employees or AI co-workers. Each agent performs a bounded operational role: the sourcing agent supports talent discovery, the bias-guardrail agent protects fair evaluation, the scoring agent organizes job-relevant evidence, the outreach agent prepares communications, the scheduling agent coordinates logistics, the RAG interview agents prepare structured questions, and the decision-support agent assembles evidence for final review. This framing helps non-technical stakeholders understand that PATHS is not a single black-box model; it is a coordinated set of specialized assistants with defined inputs, outputs, and escalation points.

These digital employees are not autonomous decision-makers. Recruitment affects careers, income, equal opportunity, organizational performance, and legal accountability. Their appropriate role is to reduce repetitive work, structure evidence, and improve consistency, while humans retain responsibility for ambiguous cases, relationship building, ethical interpretation, exception handling, and final decisions. PATHS operationalizes this principle through Human-in-the-Loop checkpoints before de-anonymization, outreach, continuation after evaluation gates, and final acceptance or rejection.

6.6 Maintaining Current Knowledge, Adoption, and Future Scalability

6.6.1 Maintaining Current Recruitment Knowledge

Recruitment knowledge changes continuously because job requirements, technical skills, candidate availability, organizational policies, and labor-market conditions evolve. An AI-assisted platform should therefore not rely only on static prompts or fixed templates. PATHS addresses this challenge through retrieval-augmented workflows that can draw on organization knowledge bases, current job descriptions, approved rubrics, candidate evidence, and updated records. In principle, this allows outreach, interview preparation, and decision support to reflect the most recent available context rather than generic or outdated assumptions.

Maintaining this capability requires explicit operational controls. The system should record the source, timestamp, and confidence associated with retrieved information; re-index approved organizational documents when they change; flag stale contact information and incomplete profile fields; and distinguish verified evidence from inferred statements. It must also guard against irrelevant retrieval and prompt injection from untrusted CVs, links, or external content. Without these controls, a system can appear context-aware while relying on outdated or unreliable evidence.

6.6.2 User Experience and Adoption

The usefulness of PATHS depends not only on model quality but also on whether recruiters, hiring managers, interviewers, and candidates can understand and trust its outputs. Recruiters require speed, pipeline visibility, and simple control over candidate stages. Hiring managers require concise and credible evidence. Interviewers require structured questions and usable rubrics. Candidates require fair treatment, respectful communication, and clarity about the process. These stakeholder requirements make explainability and usability central adoption conditions rather than optional interface features.

PATHS should therefore present recommendations with visible evidence sources, sub-score contributions, confidence levels, missing-data warnings, and bias-guardrail notes. Human override should remain easy and available at all stages. Adoption also requires change management: recruiters may resist systems that appear to replace their expertise, while managers may disregard outputs that appear too technical or opaque. The appropriate positioning is a productivity and evidence-support platform that enables better human decisions, not a replacement for professional judgment.

6.6.3 Scalability, Reliability, and Architectural Evolution

The multi-agent structure is suitable for future scaling because different recruitment stages have different computational demands. Sourcing, document parsing, embedding generation, scoring, and retrieval may process large candidate pools, whereas interview preparation and development-plan generation are generally applied to smaller shortlisted groups. A modular structure allows the most resource-intensive services to be optimized or scaled independently without redesigning the entire platform.

The current modular-monolith implementation is appropriate for a graduation project and early-stage product because it reduces deployment complexity, supports rapid debugging, and allows the team to validate the end-to-end workflow before introducing distributed-system overhead. As real usage grows, computationally intensive components can be extracted gradually into background workers or independent services, such as an ingestion service, a ranking service, a retrieval service, or a communication service. This migration should be guided by observed bottlenecks and operational needs rather than undertaken prematurely.

Production reliability will require additional controls, including retry policies for transient failures, queues for long-running tasks, idempotency keys to prevent duplicate outreach or duplicate scoring, circuit breakers for unstable third-party providers, health checks, structured

logs, metrics, tracing, backups, secret management, role-based access control, and secure internal network boundaries. These measures are especially important because PATHS depends on several connected components, including relational and graph data stores, vector retrieval, model providers, file ingestion, communication services, and scheduling integrations. A failure in one component should trigger graceful degradation or a manual-review path rather than terminate the entire hiring workflow.

6.7 Limitations and Practical Constraints

Although the evaluation findings support the feasibility and value of the PATHS design, several limitations must be acknowledged. First, the evaluation dataset and scenarios are limited in scale and cannot represent the full diversity of real candidate profiles, job families, organizational cultures, labor-market conditions, and recruitment outcomes. The matching findings show useful agreement with the expected relevance bands, but further validation should use larger datasets, independent annotators, multiple industries, and live recruitment cases. Longitudinal field studies are particularly necessary to measure time-to-hire, cost-per-hire, response rate, candidate experience, quality of hire, and retention.

Second, the quality of the system is constrained by the quality and completeness of its evidence. CVs may omit significant experience, GitHub activity may not represent professional capability, portfolios may be unavailable, and public profiles may be outdated. Missing or inconsistent evidence can affect extraction, scoring, retrieval, and explanations. The system should respond by lowering confidence, showing evidence gaps, and escalating uncertain cases for human review rather than presenting unsupported certainty.

Third, LLM-based components can generate hallucinations, irrelevant interview questions, overly confident summaries, or inappropriate phrasing. Retrieval augmentation reduces these risks but does not remove them. Generated outputs should remain traceable to sources, clearly distinguish evidence from inference, and be reviewed by users before consequential use. Prompt-injection protection and content validation are also necessary because resumes, documents, and external links can contain malicious or misleading instructions.

Fourth, external data access and compliance obligations limit automated sourcing, enrichment, and outreach. Public-profile collection, contact enrichment, data retention, deletion requests, consent, and cross-border data handling must comply with applicable platform rules and privacy requirements. For production deployment, PATHS would require formal governance policies, legal review, security testing, accessibility assessment, bias monitoring, and integration testing with real ATS, email, calendar, and identity-management systems.

The current system should be understood as an implemented prototype and design foundation rather than a fully enterprise-certified recruitment product. Mature commercial deployment would require robust service-level monitoring, load testing, incident response procedures, independent security assessment, fairness auditing where legally appropriate, and sustained user feedback from recruiters and candidates. These constraints do not diminish the contribution of PATHS; rather, they define a realistic roadmap for responsible advancement from prototype to operational platform.

Chapter 7

Conclusion

7.1 Summary and Conclusion

PATHS distinguishes itself from existing recruitment technologies through its integrated, evidence-driven, and human-supervised design. Many established recruitment platforms provide strong capabilities within a limited part of the recruitment lifecycle. For example, Applicant Tracking Systems such as Greenhouse, Ashby, and SmartRecruiters provide structured workflow management, reporting, and collaboration features. However, they generally depend on recruiters to perform sourcing, evaluate candidates, and coordinate decision-making manually. Similarly, sourcing platforms such as SeekOut primarily focus on identifying potential candidates, while technical assessment platforms such as HackerRank and Codility concentrate on validating programming skills. These systems provide valuable specialised functions, but they do not provide a unified mechanism that connects sourcing, evaluation, decision support, and post-selection development.

Other AI-enabled systems offer automation but may have narrower scope or limited transparency. Paradox focuses strongly on chatbot-based candidate engagement and scheduling, particularly for high-volume recruitment. HireVue provides scalable structured video interviews and assessments, while Pymetrics focuses on behavioural and cognitive assessment. Although such systems can improve speed at particular hiring stages, they may not provide a complete evidence trail across the recruitment lifecycle or connect assessment outcomes to personalized employee development. In addition, organizations may still need to combine these tools with separate ATS platforms, sourcing systems, communication tools, and reporting solutions.

In contrast, PATHS was designed as an end-to-end recruitment workflow rather than a standalone sourcing, assessment, or tracking tool. It combines inbound and outbound candidate acquisition, CV and profile ingestion, candidate anonymization, evidence-based scoring, semantic matching, personalised outreach, scheduling, RAG-supported interview preparation, decision support, and development planning within one connected platform. This integration reduces the fragmentation that commonly occurs when organizations rely on several disconnected systems with separate data formats, manual handoffs, and inconsistent candidate records.

A central strength of PATHS is its use of explainable, evidence-based recommendations. Rather than presenting recruiters with an unexplained ranking score, the system provides a structured explanation of the candidate's relevant skills, experience, portfolio evidence, assessment outcomes, confidence level, and missing-information penalties. This enables recruiters and hiring managers to understand why a candidate has been ranked in a particular position and to challenge, revise, or override the recommendation when appropriate. Such transparency is

especially important in recruitment, where decisions can directly affect candidates' career opportunities and organizational workforce quality.

PATHS also provides stronger human oversight than systems designed for highly automated screening. Candidate anonymization is applied before scoring to reduce exposure to non-job-relevant personal information, while de-anonymization, outreach, progression, rejection, and final hiring decisions remain controlled by authorized human users. The platform therefore follows an augmentation model: AI agents automate repetitive analysis, retrieval, coordination, and content generation, whereas recruiters and hiring managers retain accountability for consequential decisions. This design supports responsible AI use and helps organizations maintain fairness, transparency, and governance throughout the recruitment process.

Another distinguishing feature is the connection between recruitment and employee development. Most recruitment systems end once a hiring decision has been made. PATHS instead preserves the evidence generated during screening, assessment, and interviews, then uses identified strengths and skill gaps to create a personalised development plan. This capability supports onboarding, candidate feedback, future re-matching, and long-term talent development. Consequently, PATHS treats recruitment not only as a process for filling vacancies, but as the beginning of a continuous talent-management lifecycle.

Therefore, the excellence of PATHS lies in the combination of capabilities rather than in a single isolated feature. It integrates the operational strengths of ATS platforms, the discovery capabilities of sourcing tools, the structured evaluation approach of assessment systems, and the automation potential of agentic AI. At the same time, it addresses critical weaknesses found in many existing systems by preserving human control, providing transparent evidence for recommendations, supporting fairness safeguards, and linking hiring outcomes to employee development. This positions PATHS as a practical and responsible framework for organisations seeking a more coherent, explainable, and human-centred approach to AI-assisted recruitment.

7.2 Future Work

7.2.1 Lessons Learned from the Implementation of PATHS

The implementation of PATHS showed that building an AI recruitment platform is more complex than connecting a language model to a dashboard. The project required coordination between frontend screens, backend APIs, relational data, graph relationships, vector retrieval, file ingestion, agent workflows, scoring logic, and human approval steps. The main lesson is that the

value of AI depends on the surrounding software architecture. Without reliable data models, clear workflow states, and audit logs, even strong AI outputs become difficult to trust.

A second lesson is the importance of modularity. Separating the system into agents and backend modules made it easier to reason about responsibilities, test individual components, and extend the platform. This also made the system more explainable to evaluators because each agent has a clear purpose and a clear relationship to the recruitment workflow.

A third lesson is that data provenance is essential. Candidate evidence must not be treated as anonymous text. Each extracted skill, project, assessment result, and interview note should be linked to a source and timestamp. This supports explain-ability, fairness, reproducibility, and future audits.

A fourth lesson is that deployment and observability must be considered early. AI workflows can fail silently due to missing context, unavailable model services, retrieval errors, malformed documents, or external API limits. Therefore, testing, logging, error reporting, and monitoring are not optional extras; they are part of the responsible implementation of an AI-driven system.

The project reinforced that user trust is built through transparency. Recruiters and hiring managers need to see why the system recommends a candidate, not only what it recommends. Candidates also benefit when the system produces respectful feedback rather than unexplained rejection. This lesson connects the technical design of PATHS to its human-centered purpose.

7.2.2 Future Development Roadmap

Future development of PATHS can be organized into short-term, medium-term, and long-term priorities. In the short term, the system should focus on strengthening reliability, user experience, and evaluation quality. This includes improving error handling, validating scoring outputs, refining explanation packets, strengthening UI flows, and adding clearer admin monitoring for agent runs, graph data, and vector collections.

In the medium term, PATHS should expand integrations and production-readiness. This includes ATS connectors, calendar integrations, email delivery monitoring, background queues, role-based dashboards, fairness metrics, audit exports, and organization-specific knowledge-base management. The system should also support stronger evaluation experiments, such as comparing AI-generated shortlists against human recruiter judgments and measuring outreach response improvements across controlled scenarios.

In the long term, PATHS can evolve toward an enterprise-grade talent intelligence platform. This would involve microservice extraction for high-load components, advanced analytics for hiring funnels, continuous feedback loops from post-hire performance, bias audit tooling, privacy-compliant candidate consent management, and personalized learning pathways that continue after hiring. At that stage, PATHS would not only support recruitment decisions but also become part of the organization’s workforce development strategy. As shown in Table 7.1.

Table 7.1: Strategic roadmap for future PATHS development.

Roadmap stage	Main focus	Recommended enhancements	Expected value
Short term	Stabilization and usability	Improve UI flows, explanation packets, scoring validation, error handling, and admin monitoring.	Better demonstration quality and higher evaluator confidence.
Medium term	Production readiness	Add queues, ATS/calendar/email integrations, fairness metrics, audit exports, and organization knowledge-base management.	More realistic deployment for organizations and stronger operational reliability.
Long term	Enterprise and strategic talent intelligence	Move heavy workloads to microservices, add post-hire feedback loops, skill drift monitoring, consent management, and learning-path continuity.	Transforms PATHS from hiring automation into a workforce development platform.

References

- [1] Center for American Progress, “There Are Significant Business Costs to Replacing Employees,” American Progress, n.d. [Online]. Available: <https://www.americanprogress.org/article/there-are-significant-business-costs-to-replacing-employees>. [Accessed: Nov. 12, 2025].
- [2] Robert Walters, “46% of hires fail within the first 18 months,” *Robert Walters (CH)*, n.d. [Online]. Available: <https://www.robertwalters.ch/en/insights/hiring-advice/blog/46-of-hires-fail-within-the-first-18-months.html>. [Accessed: Nov. 12, 2025].
- [3] Radancy, “Building the Ultimate Talent Acquisition Tech Stack: Practical Steps for Success,” *Radancy Blog*, Jan. 9, 2025. [Online]. Available: <https://blog.radancy.com/2025/01/09/building-the-ultimate-talent-acquisition-tech-stack-practical-steps-for-success>. [Accessed: Nov. 12, 2025].
- [4] S. Malayil *et al.*, “Smart AI Based Online Platform for Competitive Coding, Technical Interview, Assessments and Recruitment,” in *Proc. Int. Conf. Computer Science and Communication Engineering (ICCSCE)*, Atlantis Press, 2025.
- [5] SmartRecruiters, “Recruitment Benchmarks 2025 Report (PDF),” *SmartRecruiters*, 2025 [Online]. Available: <https://ta.smartrecruiters.com/rs/664-NIC-529/images/Recruitment-Benchmarks-2025-Report.pdf>. [Accessed: Nov.12,2025].
- [5] SmartRecruiters, *SmartRecruiters*, n.d. [Online]. Available: <https://www.smartrecruiters.com/>. [Accessed: Jan. 25, 2026].
- [6] Gem, “Guide to Email Outreach & Benchmarks (Fall 2024) (PDF),” *Gem*, 2024. [Online]. Available: <https://lp.gem.com> . [Accessed: Nov. 12, 2025].
- [7] J. Levashina *et al.*, “The structured employment interview: Narrative and quantitative review of the research literature,” *Personnel Psychology*, vol. 67, no. 1, pp. 241–293, 2014.
- [8] “Composable HR: Reimagining Human Resource Infrastructure with AI and Microservices,” n.d.
- [9] M. C. Perna, “74% of Millennials & Gen Z think they can build better skills at a new job,” *Forbes*, Oct. 31, 2022. [Online]. Available: <https://www.forbes.com/sites/markcperna/2022/10/31/74-of-millennials--gen-z-think-they-can-build-better-skills-at-a-new-job>. [Accessed: Dec. 16, 2025].
- [9] LinkedIn Learning, “Workplace Learning Report 2020 (PDF),” *LinkedIn Learning*, 2020. [Online]. Available: <https://learning.linkedin.com/content/dam/me/learning/resources/pdfs/LinkedIn-Learning-2020-Workplace-Learning-Report.pdf>. [Accessed: Dec. 16, 2025].

- [10] “Cost of Hiring Employees in Egypt (2026 Guide),” n.d.
- [11] Qureos, “Cost of recruitment calculator — Egypt,” *Qureos*, n.d. [Online]. Available: <https://www.qureos.com/cost-of-recruitment-calculator/egypt>. [Accessed: Dec. 16, 2025].
— Qureos, “Hiring trends in Egypt,” *Qureos Hiring Guide*, n.d. [Online]. Available: <https://www.qureos.com/hiring-guide/hiring-trends-in-egypt>. [Accessed: Dec. 16, 2025].
- [12] dopay, “Payroll trends in Egypt’s manufacturing sector: 2025 snapshot,” *dopay Knowledge Hub*, 2025. [Online]. Available: <https://dopay.com/en/knowledge-hub/payroll-trends-in-egypts-manufacturing-sector-2025-snapshot>. [Accessed: Dec. 16, 2025].
- [13] Skatch CRM, “Egypt hiring trends 2024,” *Skatch*, 2024. [Online]. Available: <https://crm.skatch.com/egypt-hiring-trends-2024>. [Accessed: Dec. 16, 2025].
- [14] CADE Project, “Egypt activates data protection law with implementing regulations after five-year delay,” *CADE Project*, n.d. [Online]. Available: <https://cadeproject.org/updates/title-egypt-activates-data-protection-law-with-implementing-regulations-after-five-year-delay>. [Accessed: Dec. 16, 2025].
- [15] PrivacyLaws.com, “Egypt Data Protection Law (Law 151 of 2020) (PDF),” *PrivacyLaws.com*, n.d. [Online]. Available: <https://www.privacylaws.com/media/3263/egypt-data-protection-law-151-of-2020.pdf>. [Accessed: Dec. 16, 2025].
- [16] Connectalents, “Recruitment process outsourcing services in Egypt,” *Connectalents*, n.d. [Online]. Available: <https://connectalents.com/recruitment-process-outsourcing-services-in-egypt>. [Accessed: Dec. 16, 2025].
- [17] Greenhouse, “Greenhouse named #1 ATS (Fall G2 2025),” *Greenhouse Blog*, 2025. [Online]. Available: <https://www.greenhouse.com/blog/greenhouse-number-1-ats-fall-g2-2025>. [Accessed: Dec. 16, 2025].
- [17] Greenhouse, *Greenhouse*, n.d. [Online]. Available: <https://www.greenhouse.io/>. [Accessed: Dec. 16, 2025].
- [18] ClickUp, *ClickUp*, n.d. [Online]. Available: <https://clickup.com/>. [Accessed: Jan. 25, 2026].
- [19] Manatal, “Manatal vs Workable (comparison page),” *Manatal*, n.d. [Online]. Available: <https://www.manatal.com>. [Accessed: Dec. 16, 2025].
- [20] Workable, *Workable (ATS)*, n.d. [Online]. Available: <https://www.workable.com>. [Accessed: Dec. 16, 2025].
- [21] LinkedIn, “LinkedIn Recruiter,” *LinkedIn Talent Solutions*, n.d. [Online]. Available: <https://business.linkedin.com/talent-solutions/recruiter>. [Accessed: Dec. 16, 2025].
- [22] HackerRank, *HackerRank*, n.d. [Online]. Available: <https://www.hackerrank.com/>. [Accessed: Dec. 16, 2025].

- [23] HireVue, *HireVue*, n.d. [Online]. Available: <https://www.hirevue.com/>. [Accessed: Dec. 16, 2025].
- [24] Paradox, *Paradox*, n.d. [Online]. Available: <https://www.paradox.ai/>. [Accessed: Dec. 16, 2025].
- [25] Beamery, *Beamery*, n.d. [Online]. Available: <https://beamery.com/>. [Accessed: Dec. 16, 2025].
- [26] Elevatus, *Elevatus*, n.d. [Online]. Available: <https://www.elevatus.io/>. [Accessed: Dec. 16, 2025].
- [27] Eightfold AI, *Eightfold AI*, n.d. [Online]. Available: <https://eightfold.ai/>. [Accessed: Dec. 16, 2025].
- [28] SeekOut, *SeekOut*, n.d. [Online]. Available: <https://www.seekout.io/>. [Accessed: Dec. 16, 2025].
- [29] Pymetrics, *Pymetrics*, n.d. [Online]. Available: <https://www.pymetrics.com/>. [Accessed: Dec. 16, 2025].
- [30] Codility, *Codility*, n.d. [Online]. Available: <https://www.codility.com/>. [Accessed: Dec. 16, 2025].
- [31] Ashby, *Ashby*, n.d. [Online]. Available: <https://www.ashbyhq.com/>. [Accessed: Dec. 16, 2025].
- [32] Grand View Research, “Recruitment Software Market,” *Grand View Research*, n.d. [Online]. Available: <https://www.grandviewresearch.com/industry-analysis/recruitment-software-market>. [Accessed: Dec. 16, 2025].
- [33] MarketsandMarkets, “Talent Acquisition Software Market,” *MarketsandMarkets*, n.d. [Online]. Available: <https://www.marketsandmarkets.com/Market-Reports/talent-acquisition-software-market-126451899.html>. [Accessed: Dec. 16, 2025].
- [34] MarketsandMarkets, “Artificial Intelligence in HR Market,” *MarketsandMarkets*, n.d. [Online]. Available: <https://www.marketsandmarkets.com/Market-Reports/artificial-intelligence-hr-market-257124247.html>. [Accessed: Dec. 16, 2025].
- [35] Fortune Business Insights, “Artificial Intelligence Recruitment Market,” *Fortune Business Insights*, n.d. [Online]. Available: <https://www.fortunebusinessinsights.com/artificial-intelligence-recruitment-market-106215>. [Accessed: Dec. 16, 2025].
- [36] Economic Research Forum (ERF), “(ERF report PDF),” *ERF*, 2023. [Online]. Available: https://erf.org.eg/app/uploads/2023/05/1683086342_599_1564557_57erf29ac_edu_lassassi_tans_el.pdf. [Accessed: Dec. 16, 2025].

- [37] MDPI, “Composable HR: Reimagining Human Resource Infrastructure with AI and Microservices,” *MDPI*, n.d. [Online]. Available: <https://www.mdpi.com/2225-1146/11/2/16>. [Accessed: Dec. 16, 2025].
- [38] Mordor Intelligence, “GCC Human Capital Management Software Market,” *Mordor Intelligence*, n.d. [Online]. Available: <https://www.mordorintelligence.com/industry-reports/gcc-human-capital-management-software-market>. [Accessed: Dec. 16, 2025].
- [39] LinkedIn Economic Graph, “Skills-based hiring (March 2025) (PDF),” *LinkedIn Economic Graph*, Mar. 2025. [Online]. Available: <https://economicgraph.linkedin.com/content/dam/me/economicgraph/en-us/PDF/skills-based-hiring-march-2025.pdf>. [Accessed: Dec. 16, 2025].
- [40] PwC, “Middle East Workforce Hopes & Fears Survey 2025 (PDF),” *PwC*, 2025. [Online]. Available: <https://www.pwc.com/m1/en/issues/pdf/me-workforce-hopes-fears-survey-2025.pdf>. [Accessed: Dec. 16, 2025].
- [41] OpenView Partners, “SaaS Benchmarks,” *OpenView Insights & Reports*, n.d. [Online]. Available: <https://openviewpartners.com/insights-reports/>. [Accessed: Dec. 16, 2025].
- [42] “The AI Tools That Are Transforming Market Research,” *Harvard Business Review*, Nov. 2025. [Online]. Available: <https://hbr.org/2025/11/the-ai-tools-that-are-transforming-market-research>. [Accessed: Dec. 16, 2025].
- [43] Gartner, “AI Adoption Barriers in HR,” *Gartner HR Research*, n.d. [Online]. Available: <https://www.gartner.com/en/human-resources/research>. [Accessed: Dec. 16, 2025].
- [44] McKinsey & Company, “Building Trust in AI,” *McKinsey—Artificial Intelligence*, n.d. [Online]. Available: <https://www.mckinsey.com/featured-insights/artificial-intelligence>. [Accessed: Dec. 16, 2025].
- [45] European Commission, “Regulatory framework for AI (EU AI Act),” *Shaping Europe’s digital future*, n.d. [Online]. Available: <https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai>. [Accessed: Dec. 16, 2025].
- [46] U.S. Equal Employment Opportunity Commission (EEOC), “AI and hiring,” *EEOC*, n.d. [Online]. Available: <https://www.eeoc.gov/ai>. [Accessed: Dec. 16, 2025].
- [47] GDPR.eu, “General Data Protection Regulation (GDPR) information portal,” *GDPR.eu*, n.d. [Online]. Available: <https://gdpr.eu/>. [Accessed: Dec. 16, 2025].
- [48] ISO, “ISO/IEC 27001 Information security management,” *International Organization for Standardization*, n.d. [Online]. Available: <https://www.iso.org/isoiec-27001-information-security.html>. [Accessed: Dec. 16, 2025].

- [49] Deloitte, “Global Human Capital Trends,” *Deloitte Insights*, n.d. [Online]. Available: <https://www2.deloitte.com/global/en/pages/human-capital/articles/introduction-human-capital-trends.html>. [Accessed: Dec. 16, 2025].
- [50] S. Mahajan and H. Gupta, “Enhancing remote recruitment with Service Cloud and AI-2025,” *International Journal of Research and Analytical Reviews (IJRAR)*, vol. 12, no. 1, pp. 961–972, 2025.
- [51] S. M. M. Rahman *et al.*, “Artificial Intelligence (AI) in revolutionizing sustainable recruitment: A framework for inclusivity and efficiency,” *International Research Journal of Multidisciplinary Scope*, vol. 6, no. 1, pp. 1128–1141, 2025.
- [52] P. Mariappan *et al.*, “Smart Resume Screening and Job Represents Recommendation System,” *SSRN preprint*, SSRN 5141402, 2025. [Online]. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5141402. [Accessed: Jan. 25, 2026].
- [53] G. Pathak and D. Pandey, “AI Agents in Recruitment: A Multi-Agent System for Interview, Evaluation, and Candidate Scoring,” 2025. (*Venue details not fully specified in your entry.*)
- [54] G. Koman *et al.*, “The importance of e-recruitment within a smart government framework,” *Systems*, vol. 12, no. 3, Art. no. 71, 2024.
- [55] M. Abdelraouf and M. Kadry, “The impact of artificial intelligence (AI) on recruitment and selection of human resources management (HRM),” *المجلة العلمية للدراسات التجارية والبيئية*, vol. 15, no. 1, pp. 423–462, 2024.
- [56] M. M. K. Alkhawaldeh *et al.*, “The impact of artificial intelligence on talent acquisition strategies in Jordanian technology startups,” in *Business Analytical Capabilities and Artificial Intelligence-Enabled Analytics: Applications and Challenges in the Digital Era, Volume 1*. Cham, Switzerland: Springer Nature Switzerland, 2024, pp. 309–323.
- [57] A. Awad *et al.*, “Assessing the Future of HRM: Exploring the Transformative Role of Artificial Intelligence in Realizing the UAE’s Vision 2031 for AI-Driven Human Resources Practices,” *Qubahan Academic Journal*, vol. 5, no. 4, pp. 84–102, 2025.
- [58] B. Es-said *et al.*, “Enhancing Hiring Processes With E-Learning and Recommendation Systems,” in *Emerging Technologies for Recruitment Strategy and Practice*. Hershey, PA, USA: IGI Global Scientific Publishing, 2026, pp. 1–26.
- [59] S. R. Wahyuningrum *et al.*, “Self-Adaptive Systems: Redefining Best Practices in AI and Big Data in Recruitment,” in *Emerging Technologies for Recruitment Strategy and Practice*. Hershey, PA, USA: IGI Global Scientific Publishing, 2026, pp. 27–58.

- [60] W. Rajapakshe, “The Impact of Artificial Intelligence on Modern Hiring,” in *Emerging Technologies for Recruitment Strategy and Practice*. Hershey, PA, USA: IGI Global Scientific Publishing, 2026, pp. 85–112.
- [61] R. Jain, “Sentiment and Emotion Analysis in Video Interviews Using Deep Learning Techniques,” in *Emerging Technologies for Recruitment Strategy and Practice*. Hershey, PA, USA: IGI Global Scientific Publishing, 2026, pp. 147–178.
- [61] Meta, “React Documentation,” React, n.d. [Online]. Available: <https://react.dev/>. [Accessed: Jan. 25, 2026].
- [62] Vercel, “Next.js Documentation (App Router),” Next.js, n.d. [Online]. Available: <https://nextjs.org/docs/app>. [Accessed: Jan. 25, 2026].
- [63] Microsoft, “The TypeScript Handbook,” TypeScript, n.d. [Online]. Available: <https://www.typescriptlang.org/docs/handbook/intro.html>. [Accessed: Jan. 25, 2026].
- [64] Tailwind Labs, “Tailwind CSS Documentation,” Tailwind CSS, n.d. [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: Jan. 25, 2026].
- [65] shadcn, “shadcn/ui Documentation,” shadcn/ui, n.d. [Online]. Available: <https://ui.shadcn.com/docs>. [Accessed: Jan. 25, 2026].
- [66] MUI, “MUI Documentation,” Material UI, n.d. [Online]. Available: <https://mui.com/material-ui/getting-started/>. [Accessed: Jan. 25, 2026].
- [67] React Hook Form Contributors, “React Hook Form Documentation,” React Hook Form, n.d. [Online]. Available: <https://react-hook-form.com/>. [Accessed: Jan. 25, 2026].
- [68] Zod Contributors, “Zod Documentation,” Zod, n.d. [Online]. Available: <https://zod.dev/>. [Accessed: Jan. 25, 2026].
- [69] TanStack, “TanStack Query Documentation,” TanStack Query, n.d. [Online]. Available: <https://tanstack.com/query/latest/docs/framework/react/overview>. [Accessed: Jan. 26, 2026].
- [70] Recharts Contributors, “Recharts Documentation,” Recharts, n.d. [Online]. Available: <https://recharts.org/en-US/>. [Accessed: Jan. 25, 2026].
- [71] Apache Software Foundation, “Apache ECharts Documentation,” Apache ECharts, n.d. [Online]. Available: <https://echarts.apache.org/en/index.html>. [Accessed: Jan. 26, 2026].
- [72] Auth.js, “Auth.js Documentation,” Auth.js (NextAuth.js), n.d. [Online]. Available: <https://authjs.dev/>. [Accessed: Jan. 25, 2026].
- [73] S. Ramírez, “FastAPI Documentation,” FastAPI, n.d. [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: Jan. 25, 2026].
- [74] CrewAI, “CrewAI Documentation,” CrewAI, n.d. [Online]. Available: <https://docs.crewai.com/>. [Accessed: Jan. 25, 2026].

- [75] LangChain, “LangChain Documentation (Python),” LangChain, n.d. [Online]. Available: <https://python.langchain.com/docs/introduction/>. [Accessed: Jan. 25, 2026].
- [76] LangChain, “LangGraph Documentation,” LangGraph, n.d. [Online]. Available: <https://langchain-ai.github.io/langgraph/>. [Accessed: Jan. 25, 2026].
- [77] Neo4j, “Cypher Manual,” Neo4j Documentation, n.d. [Online]. Available: <https://neo4j.com/docs/cypher-manual/current/>. [Accessed: Jan. 25, 2026].
- [78] PostgreSQL Global Development Group, “PostgreSQL Documentation,” PostgreSQL, n.d. [Online]. Available: <https://www.postgresql.org/docs/>. [Accessed: Jan. 25, 2026].
- [79] Redis Ltd., “Redis Documentation,” Redis, n.d. [Online]. Available: <https://redis.io/docs/latest/>. [Accessed: Jan. 25, 2026].
- [80] Celery Project, “Celery Documentation,” Celery, n.d. [Online]. Available: <https://docs.celeryq.dev/en/stable/>. [Accessed: Jan. 25, 2026].
- [81] RQ Contributors, “RQ (Redis Queue) Documentation,” RQ, n.d. [Online]. Available: <https://python-rq.org/docs/>. [Accessed: Jan. 25, 2026].
- [82] pgvector Contributors, “pgvector: Open-source vector similarity search for Postgres,” GitHub Repository, n.d. [Online]. Available: <https://github.com/pgvector/pgvector>. [Accessed: Jan. 25, 2026].
- [83] Pinecone Systems, Inc., “Pinecone Documentation,” Pinecone, n.d. [Online]. Available: <https://docs.pinecone.io/>. [Accessed: Jan. 25, 2026].
- [84] Weaviate B.V., “Weaviate Documentation,” Weaviate, n.d. [Online]. Available: <https://docs.weaviate.io/>. [Accessed: Jan. 25, 2026].
- [85] Docker, Inc., “Docker Documentation,” Docker Docs, n.d. [Online]. Available: <https://docs.docker.com/>. [Accessed: Jan. 25, 2026].
- [86] GitHub, Inc., “GitHub Actions Documentation,” GitHub Docs, n.d. [Online]. Available: <https://docs.github.com/en/actions>. [Accessed: Jan. 25, 2026].
- [87] OpenTelemetry, “OpenTelemetry Documentation,” OpenTelemetry, n.d. [Online]. Available: <https://opentelemetry.io/docs/>. [Accessed: Jan. 25, 2026].
- [88] Elastic, “Elastic Stack Documentation,” Elastic Docs, n.d. [Online]. Available: <https://www.elastic.co/guide/index.html>. [Accessed: Jan. 25, 2026].
- [89] Prometheus Authors, “Prometheus Documentation,” Prometheus, n.d. [Online]. Available: <https://prometheus.io/docs/introduction/overview/>. [Accessed: Jan. 25, 2026].
- [90] Grafana Labs, “Grafana Documentation,” Grafana, n.d. [Online]. Available: <https://grafana.com/docs/>. [Accessed: Jan. 25, 2026].

- [91] HashiCorp, Inc., “Vault Documentation,” HashiCorp Developer, n.d. [Online]. Available: <https://developer.hashicorp.com/vault/docs>. [Accessed: Jan. 25, 2026].
- [92] Amazon Web Services, “AWS Secrets Manager Documentation,” AWS Docs, n.d. [Online]. Available: <https://docs.aws.amazon.com/secretsmanager/>. [Accessed: Jan. 25, 2026].
- [93] Google Cloud, “Secret Manager Documentation,” Google Cloud Docs, n.d. [Online]. Available: <https://cloud.google.com/secret-manager/docs>. [Accessed: Jan. 25, 2026].
- [94] Microsoft, “Azure Key Vault Documentation,” Microsoft Learn, n.d. [Online]. Available: <https://learn.microsoft.com/en-us/azure/key-vault/>. [Accessed: Jan. 25, 2026].
- [95] D. Myriel, “Best practices in RAG evaluation: A comprehensive guide,” Qdrant Blog, Nov. 24, 2024. [Online]. Available: <https://qdrant.tech/blog/rag-evaluation-guide/>. [Accessed: Jun. 21, 2026].
- [96] Apache Software Foundation, “Apache AGE Documentation,” Apache AGE, n.d. [Online]. Available: <https://age.apache.org/age-manual/master/index.html>. [Accessed: Jun. 21, 2026].
- [97] Qdrant, “Qdrant Documentation,” Qdrant, n.d. [Online]. Available: <https://qdrant.tech/documentation/>. [Accessed: Jun. 21, 2026].
- [98] Ollama, “Ollama Documentation,” Ollama, n.d. [Online]. Available: <https://docs.ollama.com/>. [Accessed: Jun. 21, 2026].
- [99] Nomic AI, “nomic-embed-text-v1 Model Card,” Hugging Face, Feb. 2, 2024. [Online]. Available: <https://huggingface.co/nomic-ai/nomic-embed-text-v1>. [Accessed: Jun. 14, 2026].
- [100] Model Context Protocol, “Model Context Protocol Specification,” Model Context Protocol, Nov. 25, 2025. [Online]. Available: <https://modelcontextprotocol.io/specification/2025-11-25>. [Accessed: Jun. 21, 2026].
- [101] SQLAlchemy Authors, “SQLAlchemy Documentation,” SQLAlchemy, n.d. [Online]. Available: <https://docs.sqlalchemy.org/>. [Accessed: Jun. 14, 2026].
- [102] Alembic Authors, “Alembic Documentation,” Alembic, n.d. [Online]. Available: <https://alembic.sqlalchemy.org/>. [Accessed: Jun. 14, 2026].
- [103] Pydantic, “Pydantic Documentation,” Pydantic, n.d. [Online]. Available: <https://docs.pydantic.dev/>. [Accessed: Jun. 21, 2026].
- [104] Microsoft, “Playwright Documentation,” Playwright, n.d. [Online]. Available: <https://playwright.dev/docs/intro>. [Accessed: Jun. 18, 2026].
- [105] pytest Contributors, “pytest Documentation,” pytest, n.d. [Online]. Available: <https://docs.pytest.org/>. [Accessed: Jun. 18, 2026].
- [106] pytest-asyncio Contributors, “pytest-asyncio Documentation,” pytest-asyncio, n.d. [Online]. Available: <https://pytest-asyncio.readthedocs.io/>. [Accessed: Jun. 18, 2026].

- [107] Sentry, “Sentry Documentation,” Sentry, n.d. [Online]. Available: <https://docs.sentry.io/>. [Accessed: Jun. 18, 2026].
- [108] Caddy Web, “Caddy Documentation,” Caddy, n.d. [Online]. Available: <https://caddyserver.com/docs/>. [Accessed: Jun. 18, 2026].
- [109] Google, “Implement Server-Side Authorization,” Gmail API, n.d. [Online]. Available: <https://developers.google.com/workspace/gmail/api/auth/web-server>. [Accessed: Jun. 18, 2026].
- [110] Google, “Choose Google Calendar API Scopes,” Google Calendar API, n.d. [Online]. Available: <https://developers.google.com/workspace/calendar/api/auth>. [Accessed: Jun. 18, 2026].
- [111] APScheduler Contributors, “APScheduler Documentation,” APScheduler, n.d. [Online]. Available: <https://apscheduler.readthedocs.io/>. [Accessed: Jun. 18, 2026].
- [112] Amazon Web Services, “Amazon Elastic Compute Cloud Documentation,” AWS Documentation, n.d. [Online]. Available: <https://docs.aws.amazon.com/ec2/>. [Accessed: Jun. 18, 2026].

Our System Repository in GitHub

- [113] CSE Graduation Projects, “GP-Group-12,” GitHub repository, n.d. [Online]. Available: <https://github.com/CSE-Graduation-Projects/GP-Group-12>. [Accessed: Jun. 21, 2026].